

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA



**Detección de vocalizaciones de primates
neotropicales mediante cajas tiempo–frecuencia
para la pre-anotación asistida de grabaciones de
monitoreo acústico pasivo**

Tesis para obtener el título profesional de
Ingeniero Informático que presenta:

Fernando Nelson Candia Aroni

Asesor:

Dr. Edwin Rafael Villanueva Talavera

Lima, 2026

Índice general

1. Generalidades	2
1.1. Problemática	2
1.1.1. Definición del problema	2
1.1.2. El material y su anotación	3
1.1.3. Magnitud del problema	5
1.1.4. Relevancia	6
1.2. Árbol de problemas	7
1.2.1. Problema central	8
1.2.2. Causas	9
1.2.3. Efectos	11
1.3. Brecha de investigación	12
1.3.1. Soluciones existentes	13
1.3.2. Vacío	14
1.3.3. Alcance de la tesis	16
1.4. Objetivos	16
1.4.1. Objetivo general	16
1.4.2. Objetivos específicos	17
1.5. Resultados esperados	18
1.5.1. OE1: Protocolo y conjunto de datos curado	19
1.5.2. OE2: Detector y evaluación	21
1.5.3. OE3: Tiempo de revisión y análisis de errores	24
1.6. Trazabilidad	27
1.7. Métodos y procedimientos	27
1.7.1. Metodología CRISP-DM	27
1.7.2. Herramientas	32
1.7.3. Procedimientos y reproducibilidad	34
2. Marco Referencial	36
2.1. Marco Teórico	36
2.1.1. Formulación del problema de detección	36
2.1.2. El espectrograma y la asimetría de sus ejes	39
2.1.3. Detección de objetos sobre audio	39
2.2. Marco Conceptual	40

2.2.1.	Paisaje sonoro	41
2.2.2.	Eventos acústicos	42
2.2.3.	Representaciones tiempo–frecuencia	42
2.2.4.	Detección de eventos sonoros	44
2.2.5.	Clasificación jerárquica	46
2.2.6.	Anotaciones inconsistentes	48
3.	Estado del Arte	50
3.1.	Metodología de la revisión	50
3.1.1.	Objetivos y preguntas de revisión	50
3.1.2.	Estrategia de búsqueda	51
3.1.3.	Criterios de inclusión y exclusión	52
3.2.	Resultados de la revisión	52
3.2.1.	P1: Estrategias de detección tiempo–frecuencia	53
3.2.2.	P2: Clasificación jerárquica fina	54
3.2.3.	P3: Mitigación de ruido y variabilidad entre sensores	55
3.2.4.	P4: Conjuntos de datos y aumento de datos	55
3.2.5.	P5: Transferencia de aprendizaje	56
3.3.	Síntesis	56
4.	Curación del Conjunto de Datos (OE1)	58
4.1.	Introducción	58
4.2.	El material recibido	59
4.2.1.	Estructura y formato	59
4.2.2.	Inventario	61
4.2.3.	Defectos detectados	61
4.3.	Protocolo de curación y estandarización documentado (RE1.1)	63
4.3.1.	Esquema del protocolo	64
4.3.2.	Reglas	65
4.3.3.	Verificación de RE1.1	69
4.3.4.	Fuera de alcance	70
4.4.	Conjunto acústico curado y particionado (RE1.2)	71
4.4.1.	Selección de clases	72
4.4.2.	Partición por grabación	75
4.4.3.	Ventaneo	77
4.4.4.	Representación de entrada	81
4.4.5.	Entregables y verificación	84
4.5.	Ficha del conjunto derivado (RE1.3)	87
4.5.1.	Propósito y autoría	88
4.5.2.	Composición	88
4.5.3.	Geometría de los eventos	89
4.5.4.	Recolección (capa primaria)	93
4.5.5.	Transformaciones (capa derivada)	94

4.5.6.	Diccionario de datos	95
4.5.7.	Usos	96
4.5.8.	Versión del conjunto	96
4.5.9.	Limitaciones	97
4.5.10.	Verificación de RE1.3	98
4.6.	Discusión	99
5.	Detección y Evaluación del Modelo (OE2)	101
5.1.	Introducción	101
5.2.	Diseño experimental, arquitectura propuesta y líneas base (RE2.1) . . .	102
5.2.1.	La tarea	102
5.2.2.	Arquitectura propuesta: AST-Deformable-DETR	103
5.2.3.	Líneas base	110
5.2.4.	Diseño experimental	112
5.2.5.	Verificación de RE2.1	115
5.3.	Pipeline reproducible de entrenamiento, evaluación e inferencia (RE2.2)	116
5.3.1.	Dependencias declaradas	117
5.3.2.	Las cuatro etapas y su interfaz	118
5.3.3.	Artefactos de una corrida y autonomía del <i>checkpoint</i>	119
5.3.4.	Costo de las corridas	120
5.3.5.	Reproducibilidad	121
5.3.6.	Verificación de RE2.2	122
5.4.	Informe comparativo de resultados (RE2.3)	123
5.4.1.	Lo que se compara	123
5.4.2.	Desempeño al punto de operación	124
5.4.3.	Detección, encuadre y clasificación	125
5.4.4.	Cobertura frente a carga de revisión	127
5.4.5.	Lectura por clase	129
5.4.6.	Ejemplos cualitativos	130
5.4.7.	Selección del modelo y qué establece la comparación	131
5.4.8.	Costo de inferencia	132
5.4.9.	Verificación de RE2.3	132
5.5.	Modelo final cargable y demostración funcional (RE2.4)	133
5.5.1.	El modelo final	134
5.5.2.	Formato de salida	134
5.5.3.	Demostración	135
5.5.4.	Verificación de RE2.4	137
5.6.	Código y modelo disponibles en un repositorio (RE2.5)	138
5.6.1.	Estructura	138
5.6.2.	Qué contiene y qué no	140
5.6.3.	Verificación de RE2.5	140
5.7.	Discusión	140

6. Tiempo de Revisión y Análisis de Errores (OE3)	145
6.1. Introducción	145
6.2. Metodología de validación comparativa (RE3.1)	146
6.3. Medición comparativa del tiempo de revisión (RE3.2)	146
6.4. Análisis cualitativo de errores y hallazgos (RE3.3)	146
6.5. Discusión	146
7. Conclusiones	147
7.1. Implicaciones de los resultados	147
7.2. Cumplimiento de los resultados esperados	148
7.3. Cumplimiento de los objetivos específicos	148
7.4. Cumplimiento del objetivo general	149
7.5. La propuesta frente al problema central	149
7.6. Trabajo futuro	149
Referencias	151
A. Plan del Proyecto	156
A.1. Justificación y viabilidad	156
A.1.1. Justificación	156
A.1.2. Beneficios esperados	156
A.1.3. Viabilidad técnica	157
A.1.4. Viabilidad temporal	157
A.1.5. Viabilidad económica	157
A.2. Definición y alcance del proyecto	157
A.2.1. Alcance	157
A.2.2. Entregables y criterios de aceptación	158
A.2.3. Exclusiones	158
A.2.4. Limitaciones y restricciones	160
A.3. Planificación del proyecto	160
A.3.1. Estructura de desglose del trabajo	160
A.3.2. Cronograma	162
A.3.3. Estimación de costos	164
A.3.4. Lista de recursos	166
A.3.5. Gestión de riesgos	167
B. Medios de Verificación	169

Índice de figuras

1.1. Anotación tiempo–frecuencia en Raven	4
1.2. Árbol de problemas	8
1.3. Tres formas de salida	13
1.4. CRISP-DM aplicado a este proyecto	28
1.5. Criterio de IoU	32
2.1. Cadena de la señal	41
2.2. Compromiso de la STFT	43
2.3. Escala mel	44
2.4. Mel de potencia y log-mel sobre la misma ventana	45
4.1. Tabla de selección de Raven	59
4.2. Anotación en Raven Pro	60
4.3. Valores en la columna de especie	61
4.4. Valores en la columna de tipo de llamada	61
4.5. Avisos de audio dañado	62
4.6. Corrupción leve	62
4.7. Corrupción severa	62
4.8. Reglas del protocolo	64
4.9. De las anotaciones curadas al caché	72
4.10. Anotaciones por par	73
4.11. Anidamiento frase–sílabas	75
4.12. Reparto goloso por grabación	76
4.13. Cajas por clase y partición	78
4.14. Asignación de eventos a ventanas	78
4.15. Cajas anotadas por ventana	81
4.16. Normalización de una caja	83
4.17. La ventana más densa	83
4.18. Llamada larga, cortada en ambos bordes	84
4.19. Ventana vacía	84
4.20. Capas del conjunto	88
4.21. Galería de llamadas	91
4.22. Duración frente a ancho de banda	92
4.23. Etiquetas con dos formas	93

4.24. Distribución temporal	93
5.1. Arquitectura propuesta	103
5.2. Pirámide multiescala	105
5.3. Duración de las clases frente a la pirámide	106
5.4. Una capa del decodificador	107
5.5. Muestreo deformable de una cabeza sobre un nivel	107
5.6. Emparejamiento húngaro	108
5.7. Líneas base	111
5.8. Protocolo de comparación	113
5.9. Pipeline del proyecto	117
5.10. Una época del entrenador	119
5.11. Curvas de entrenamiento	121
5.12. Distribución del IoU	126
5.13. Matriz de confusión	126
5.14. Cobertura frente a carga	127
5.15. Ejemplos cualitativos	131
5.16. Flujo de inferencia	135
5.17. Demostración sobre una grabación	136
5.18. La tabla generada abierta en Raven	137
5.19. Árbol del repositorio	139
A.1. EDT en árbol	162

Índice de tablas

1.1. Material recibido por especie	4
1.2. Disponibilidad en Xeno-canto	9
1.3. Vacío de la investigación	15
1.4. IOV de OE1	20
1.5. IOV de OE2	23
1.6. IOV de OE3	25
1.7. Taxonomía de discrepancias	26
1.8. Trazabilidad del planteamiento	27
1.9. Métricas de evaluación	31
1.10. Stack técnico: pipeline	33
1.11. Stack técnico: desarrollo	34
2.1. Arquitecturas frente a requisitos	46
2.2. Repertorio por especie	47
3.1. Cadenas de búsqueda	51
3.2. Criterios de inclusión y exclusión	52
3.3. Trabajos en que se apoya la revisión	53
4.1. Especies del corpus	59
4.2. Columnas de las tablas crudas	60
4.3. Mapa de sinónimos	66
4.4. Vocabulario cerrado	67
4.5. Pares marcados para revisión	67
4.6. Filas que toca cada regla	69
4.7. Valores distintos tras cada etapa	70
4.8. Pares excluidos del experimento	74
4.9. Composición de las particiones	77
4.10. Las 25 clases del experimento	79
4.11. Efecto del ventaneo en las clases largas	80
4.12. Parámetros del espectrograma	82
4.13. Contenido del caché	85
4.14. Composición por especie	89
4.15. Duración y banda por clase	90
4.16. Transformaciones aplicadas	94

4.17. Columnas de las tablas curadas	95
4.18. Estructuras del caché	96
4.19. Verificación de RE1.3	99
5.1. Niveles de la pirámide	105
5.2. Hiperparámetros de la propuesta	109
5.3. Parámetros por bloque	109
5.4. Configuraciones comparadas	111
5.5. Recetas de entrenamiento	112
5.6. Métricas por eje	114
5.7. Verificación de RE2.1	115
5.8. Claves de un checkpoint	120
5.9. Costo de las corridas	121
5.10. Verificación de RE2.2	122
5.11. Resultados al punto de operación	124
5.12. Detección, encuadre y clasificación	125
5.13. Cobertura a carga igual	128
5.14. Recall por clase	129
5.15. Verificación de RE2.3	133
5.16. Columnas de la tabla de salida	135
5.17. Primeras filas de la tabla de salida	136
5.18. Ubicación de cada elemento del indicador	139
7.1. Cumplimiento de los resultados esperados	148
A.1. Criterios de aceptación	159
A.2. EDT del proyecto	161
A.3. Cronograma por entrega	163
A.4. Costos de equipos	164
A.5. Capital intelectual	165
A.6. Costos de software y servicios	165
A.7. Costos indirectos	166
A.8. Resumen de costos	166
A.9. Riesgos del proyecto	168
B.1. Contenido de la carpeta de medios de verificación	170

Capítulo 1

Generalidades

1.1. Problemática

1.1.1. Definición del problema

Los primates de la Amazonía son difíciles de seguir. Viven en un bosque de vegetación densa, y muchas de sus especies son crípticas o poco abundantes, de modo que los encuentros directos son escasos: son las condiciones que hacen del bosque tropical uno de los ambientes más difíciles para el monitoreo de fauna (Zwerts et al., 2021). El monitoreo acústico pasivo (PAM, por sus siglas en inglés) sortea esa dificultad: grabadoras autónomas instaladas en el bosque registran el sonido de forma continua durante semanas o meses, sin intervenir en el ecosistema (Gibb et al., 2018).

Esta tesis trabaja con el material de un programa de ese tipo. En el Tambopata Research Center, en la Reserva Nacional Tambopata (Madre de Dios), el 8 Primates Project, una iniciativa de ciencia ciudadana de Rainforest Expeditions que dirige Mark Bowler, estudia las ocho especies de primates del río Tambopata (Wired Amazon, s.f.): el tamarino de Weddell (*Leontocebus weddelli*), el tití de Toppin (*Plecturocebus toppini*), el capuchino de cabeza chata (*Cebus cuscinus*), el mono araña peruano (*Ateles chamek*), el mono nocturno (*Aotus azarae*), el capuchino de cabeza grande (*Sapajus macrocephalus*), el mono aullador rojo boliviano (*Alouatta sara*) y el mono ardilla boliviano (*Saimiri boliviensis*), ocho de las 42 especies de primates que se registran en el Perú (Pacheco et al., 2021). Con grabadoras desplegadas en el bosque, el proyecto reúne bibliotecas de llamadas de cada especie y las usa para detectar a los monos y para establecer

un programa de monitoreo con pobladores locales (Wired Amazon, s.f.). En adelante, *equipo de investigación* designa al de ese proyecto.

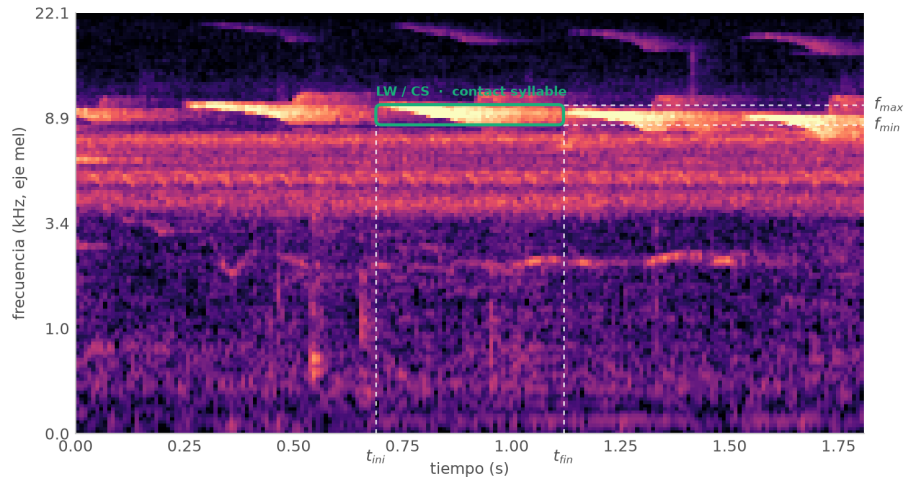
Para que las grabaciones sirvan a ese fin, cada vocalización tiene que encontrarse dentro del audio, delimitarse en el tiempo y en la frecuencia, y recibir la especie y el tipo de llamada. Esa tarea, la *anotación*, la hace a mano un especialista que conoce el repertorio de cada especie, y es el paso más lento de todo el proceso. El problema que aborda esta tesis se enuncia así:

La anotación de las vocalizaciones de primates neotropicales en grabaciones de monitoreo acústico pasivo exige más tiempo de especialista del que está disponible, y por ello avanza mucho más lento que la recolección del audio.

1.1.2. El material y su anotación

El material de partida se produce en dos pasos. Primero, grabadoras AudioMoth, dispositivos de bajo costo y de código abierto (Hill et al., 2018), registran el sonido del bosque en archivos WAV a 44,1 kHz. Después, un especialista abre cada grabación en Raven Pro, el programa de análisis de sonido del Laboratorio de Ornitología de Cornell con que trabaja el equipo (K. Lisa Yang Center for Conservation Bioacoustics, s.f.), e inspecciona su espectrograma: la imagen que muestra la energía del sonido según el tiempo, en el eje horizontal, y la frecuencia, en el vertical. Sobre esa imagen dibuja un rectángulo alrededor de cada vocalización y le asigna dos etiquetas, la especie y el tipo de llamada, escritas como códigos: dos letras para la especie, que la Tabla 1.1 asocia a su nombre, y dos o tres para el tipo de llamada. La Figura 1.1 muestra una de esas anotaciones: una sílaba de contacto del tamarino de Weddell (LW/CS), delimitada por sus instantes de inicio y fin (t_{ini} y t_{fin}) y por su frecuencia mínima y máxima ($f_{\text{mín}}$ y $f_{\text{máx}}$).

Esa caja con sus dos etiquetas es la unidad de información que produce el análisis; el documento llama *anotación* tanto a la tarea como a su producto. Sus cuatro coordenadas resumen la duración, el ancho de banda y la frecuencia central de la vocalización, información que el analista usa directamente (Escobar-Amado et al., 2024). El tipo de llamada, por su parte, conecta la anotación con el comportamiento: el repertorio de estas especies incluye llamadas de contacto, alarmas aéreas y terrestres, duetos,

Figura 1.1*Anotación tiempo-frecuencia en Raven*

llamadas de alimento y vocalizaciones agresivas, de modo que saber qué tipo de llamada se emitió informa más que saber que la especie estuvo presente.

La Tabla 1.1 resume, por especie, el material que el equipo entregó para esta tesis: cuántas grabaciones hay y cuántas horas suman, cuántas de ellas tienen anotaciones, cuántos minutos ocupan las vocalizaciones anotadas y qué fracción del tiempo grabado representan.

Tabla 1.1*Material recibido por especie*

Especie (código)	Grabado		Anotado		
	N.º	Horas	N.º	Evento (min)	Fracción (%)
Mono aullador (AS)	808	7,57	778	324,6	71,5
Capuchino de cabeza grande (SM)	659	6,13	582	16,6	4,5
Mono araña peruano (AC)	561	6,07	384	30,7	8,4
Tamarino de Weddell (LW)	530	5,28	410	47,5	15,0
Tití de Toppin (PT)	375	3,35	320	94,3	47,0
Mono ardilla boliviano (SB)	290	3,08	256	12,2	6,6
Mono nocturno (AA)	117	1,10	82	2,5	3,8
Capuchino de cabeza chata (CC)	16	0,11	7	0,1	1,2
Total	3 356	32,69	2 819	528,5	26,9

En total son 3 356 grabaciones y 32,69 horas, de las cuales 2 819 grabaciones (26,34 horas) tienen anotaciones, y las vocalizaciones anotadas ocupan 528,5 minutos, el 26,9 % de lo grabado. Dos rasgos de la tabla importan para el problema. El primero es qué

representa: no es todo lo que el proyecto grabó, sino la selección que el equipo organizó por especie para anotarla, con un criterio que el material no documenta, y por eso la mayor parte está anotada. Su tamaño lo confirma: 32,69 horas equivalen a menos de un día y medio de registro continuo de una sola grabadora, de modo que incluso este material, el único anotado de que se dispone para estas especies, es una muestra mínima de lo que el PAM puede producir. El segundo es el desequilibrio entre especies: la fracción del tiempo grabado que ocupan las vocalizaciones anotadas varía en más de un orden de magnitud, desde el mono aullador, cuyos aullidos son largos y continuos, hasta el capuchino de cabeza chata, del que apenas hay material. Ese desequilibrio, que responde tanto a la duración de las llamadas de cada especie como a lo que se llegó a grabar y a anotar, condiciona la construcción del conjunto de datos y la manera de evaluar un detector.

1.1.3. Magnitud del problema

El problema existe porque grabar se volvió barato y analizar no. Del lado de la recolección, una grabadora AudioMoth tiene un costo de fabricación de unos 43 dólares estadounidenses (Hill et al., 2018), y los sensores de bajo costo y de código abierto permiten desplegar redes de muchos micrófonos a escala (Gibb et al., 2018); el costo de grabar y de almacenar cayó lo suficiente como para hacer viable la captura continua (Stowell, 2022). La consecuencia se mide en horas. Una sola grabadora en registro continuo produce 24 horas de audio por día, unos 7,6 GB en el formato del material de esta tesis (44,1 kHz, 16 bits, un canal), y una red modesta de diez grabadoras acumula 7 200 horas en un mes. Los proyectos reales operan a esa escala: el detector de aullidos de lobo HOWLish se construyó sobre 50 137 horas de paisaje sonoro, en las que se anotaron a mano 841 eventos de aullido (Campos et al., 2025). Y el uso del método crece: una revisión sistemática del PAM terrestre registró un aumento de más de quince veces en el número de publicaciones entre 1992 y 2018 (Sugai et al., 2019).

Del lado del análisis, la revisión sigue siendo mayoritariamente manual. El análisis bioacústico de vocalizaciones se realiza sobre todo mediante la inspección manual de las grabaciones, un proceso muy subjetivo y que consume mucho tiempo (Romero-Mujalli et al., 2021), y más de la mitad de los estudios de PAM terrestre revisados por Sugai et al. (2019) analizó el audio a mano. Romero-Mujalli et al. (2021) lo resumen al señalar

que hoy el factor que limita la investigación con vocalizaciones animales es el análisis de las grabaciones, y no la técnica para obtenerlas.

El costo de esa revisión se hace visible cuando se mide lo que ahorra asistirle. Con detección automática, el análisis de un año de grabaciones del mono aullador negro y dorado requirió un tiempo equivalente al 2 % de la duración grabada (Pérez-Granados & Schuchmann, 2021). En los casos que comparan directamente con la revisión manual, depurar los falsos positivos de un sistema semiautomático para cuatro especies de primates costó entre el 3 y el 5 % del tiempo del procesamiento humano (Heinicke et al., 2015), HOWLish redujo 15 veces el tiempo del operador (Campos et al., 2025) y un detector de llamadas de langur redujo en un 87 % la carga de revisión manual (Hong et al., 2026). Leídas a la inversa, esas cifras indican que la revisión sin asistencia costaba, según el caso, entre unas ocho y unas treinta veces más tiempo. Y todas corresponden a detectar una o pocas especies: la anotación que requiere el dominio de esta tesis, con caja y tipo de llamada para varias especies, exige más decisiones por evento, y recolectar etiquetas que delimitan cada evento ya es, de por sí, lo que más tiempo consume en la detección de eventos sonoros (Koga et al., 2024).

El tercer factor es quién puede hacer ese trabajo. Asignar el tipo de llamada exige conocer el repertorio de cada especie, y el tiempo de analistas con esa formación es escaso: la falta de tiempo-persona de analistas entrenados es el cuello de botella habitual de la bioacústica (Stowell, 2022), y recolectar anotaciones es el principal obstáculo para construir métodos automáticos (Nguyen Hong Duc et al., 2021). La respuesta habitual es repartir la anotación entre varias personas (Koga et al., 2024), lo que alivia el volumen a costa de introducir criterios distintos en un mismo conjunto de datos, un efecto que reaparece entre las causas del problema.

1.1.4. Relevancia

Reducir el tiempo de anotación importa, en primer lugar, por lo que está en juego para las especies. Los primates son un componente esencial de la biodiversidad tropical; alrededor del 60 % de sus especies está amenazado de extinción y el 75 % tiene poblaciones en declive (Estrada et al., 2017). Entre las ocho especies de esta tesis, el mono araña peruano figura como En Peligro en la Lista Roja de la Unión Internacional para la Conservación de la Naturaleza: se estima que su población se redujo al menos a

la mitad en los últimos 45 años por la caza y la pérdida de hábitat, y la ganadería y la deforestación están extendidas en el sureste del Perú (Alves et al., 2021). Conservar poblaciones así exige seguirlas en el tiempo, y ese seguimiento es la base de las decisiones de conservación, con la dificultad añadida que imponen los bosques tropicales (Zwerts et al., 2021).

En segundo lugar, el PAM es una herramienta adecuada para ese seguimiento. Los sensores acústicos no son invasivos, suelen ser omnidireccionales y ofrecen un área de detección mayor y menos restricciones taxonómicas que las cámaras trampa; permiten muestrear durante periodos largos con poco esfuerzo manual, lo que aumenta la probabilidad de detectar especies raras o poco vocales (Gibb et al., 2018). En primates, la detección automática sobre grabaciones de PAM ya se ha usado para estimar la ocupación de varias especies en un bosque de Costa de Marfil (Kalan et al., 2015) y para describir los patrones de vocalización de monos aulladores en Brasil (do Nascimento et al., 2021; Pérez-Granados & Schuchmann, 2021). Aun así, el PAM rara vez se ha aplicado al monitoreo de primates (Pérez-Granados & Schuchmann, 2021), y sus ventajas solo se materializan si el análisis acompaña a la recolección: los programas de monitoreo deben ser estandarizables, escalables y financieramente sostenibles (Gibb et al., 2018), condiciones que un análisis enteramente manual no cumple a la escala que el PAM hace posible.

En tercer lugar, el problema es relevante para la informática. Convertir una grabación en anotaciones con la forma que usa el especialista exige resolver en un solo sistema la localización de cada vocalización en tiempo y en frecuencia y su clasificación fina por especie y tipo de llamada, es decir, llevar la detección de objetos de la visión por computadora a una señal que no es una imagen natural. Y lo es para el equipo de investigación, que es quien sostiene el monitoreo: cada hora de especialista que se ahorra en la anotación queda disponible para la interpretación ecológica de los datos.

1.2. Árbol de problemas

El árbol de problemas ordena en relaciones de causa y efecto la evidencia de la sección anterior. Se construye con el heurístico de Veselý (2008): el problema central ocupa el centro, sus causas se ubican debajo y sus efectos encima, y cada flecha se lee como “conduce a”. El árbol no cuantifica esas relaciones; su función es explicar por

qué ocurre el problema y qué provoca, y servir de puente hacia los objetivos, que se derivan de él invirtiendo su lógica. La Figura 1.2 lo resume, y las subsecciones que siguen desarrollan cada elemento.

Figura 1.2
Árbol de problemas



La figura se lee de abajo hacia arriba. Las dos causas confluyen en el problema central: una está en las anotaciones que existen y la otra en el propio audio. La primera alimenta además a la segunda, porque sin anotaciones utilizables no puede entrenarse un detector que alivie la búsqueda. Del problema salen los efectos, que se encadenan hasta uno final sobre la conservación de las especies.

1.2.1. Problema central

El problema central es el enunciado en la sección 1.1.1, que la figura reproduce al centro. Describe una sola situación negativa, medida en una sola magnitud: el tiempo

de especialista que exige anotar una hora de audio, frente al que hay disponible. No enuncia la ausencia de una solución particular, como sería afirmar que falta un detector automático: el detector es una de las vías posibles para reducir ese tiempo, y su conveniencia es justamente lo que esta tesis debe demostrar. Las causas que siguen explican por qué persiste la distancia documentada en la sección 1.1.3, aunque la automatización ya existe en otros dominios.

1.2.2. Causas

C1: Las únicas anotaciones disponibles tienen ruido, sesgo e inconsistencias

Un método automático se construye y se evalúa con ejemplos anotados, y para estas especies casi no hay de dónde tomarlos. La Tabla 1.2 lo muestra para Xeno-canto, el repositorio abierto de sonidos de fauna más utilizado: se buscó cada una de las ocho especies por su nombre científico y se contaron las grabaciones en que figura en primer plano, es decir, como sujeto principal.

Tabla 1.2
Disponibilidad en Xeno-canto

Espece	Grabaciones (primer plano)	Duración total
<i>Alouatta sara</i>	8	41:14
<i>Plecturocebus toppini</i>	13	19:20
<i>Ateles chamek</i>	4	03:42
<i>Saimiri boliviensis</i>	4	02:28
<i>Aotus azarae</i>	3	02:18
<i>Cebus cuscinus</i>	0	N/D
<i>Leontocebus weddelli</i>	0	N/D
<i>Sapajus apella macrocephalus</i>	0	N/D
Total	32	69:02

Nota. Consulta realizada el 19 de agosto de 2026 sobre el nombre científico de cada especie (Vellinga, 2026). Duración en minutos y segundos.

En total hay 32 grabaciones y poco más de una hora de audio; tres de las especies no tienen ninguna, y ninguna de las grabaciones disponibles incluye anotaciones tiempo-frecuencia. El material del equipo de investigación es, por tanto, el único conjunto anotado de que se dispone (Tabla 1.1), y sus defectos no pueden sortearse recurriendo a otro: hay que corregirlos. Son de tres clases.

Inconsistencias. El material se anotó a mano, por varias personas y sin un vocabulario cerrado, de modo que sus 20 082 cajas llevan 19 grafías distintas de especie para solo ocho especies, y 63 grafías distintas de tipo de llamada. Parte de esa inconsistencia es de escritura, y se corrige normalizando las etiquetas; otra parte es de criterio, y es propia de la tarea. La anotación manual es subjetiva y genera errores específicos de cada anotador (Nguyen Hong Duc et al., 2021). Para primates ese desacuerdo no se ha medido, pero sí en tareas equivalentes, que también consisten en delimitar eventos sobre el espectrograma y etiquetarlos: al anotar llamadas de ballena azul, dos analistas coincidieron en menos del 50 % de los casos (Leroy et al., 2018), y con veinte anotadores por clip las etiquetas varían tanto en el tipo de evento como en sus tiempos de inicio y fin (Koga et al., 2024). A ello se suma que la propia unidad acústica carece de una definición estable: las unidades pueden anidarse unas dentro de otras, sus definiciones varían entre investigadores y sus rasgos cambian de forma graduada, mientras que los métodos analíticos asumen categorías discretas (Kershenbaum et al., 2016).

Ruido. No todas las cajas son ejemplos válidos: hay filas que marcan ruido de fondo en lugar de una vocalización, cajas sin duración o sin ancho de banda y grabaciones dañadas (sección 4.2.3). La referencia, además, es incompleta. Un mismo analista no reproduce sus propias anotaciones (Leroy et al., 2018), de modo que una vocalización real puede quedar sin caja, y la borrosidad de las fronteras de los eventos desplaza las cajas aun cuando el evento es el mismo (Z. Zhu & Sato, 2025). Por eso una predicción que no coincide con ninguna caja anotada puede ser un error del detector o una omisión de la referencia, y distinguir ambos casos exige el juicio de un especialista.

Sesgo. Lo anotado no es una muestra equilibrada del paisaje sonoro. Las grabaciones se eligieron para anotarlas, no por muestreo, y el desequilibrio entre especies que muestra la Tabla 1.1 pasa intacto al conjunto de datos. Un modelo entrenado con ese material aprende el desequilibrio si nadie lo declara ni lo controla. Sin corregir lo corregible y documentar lo demás, no hay contra qué aprender ni contra qué medir (Koga et al., 2024).

C1 conduce al problema central por dos vías. La directa es que lo ya anotado no puede reutilizarse sin volver a pasar por el especialista: solo quien conoce el repertorio puede decidir qué quiso decir una grafía ambigua o si una caja es un error. En este material, 643 anotaciones quedan en esa situación después de corregir todo lo que

admite una regla (sección 4.3). Además, repartir la anotación entre más personas, la respuesta habitual a la falta de tiempo (sección 1.1.3), añade criterios que luego hay que reconciliar, de modo que el tiempo que se gana en volumen se pierde en revisión. La indirecta pasa por C2: sin una referencia consistente no puede entrenarse ni evaluarse un detector que alivie la búsqueda, como indica la flecha punteada de la Figura 1.2.

C2: Las vocalizaciones son escasas y breves dentro de horas de audio

La segunda causa está en el propio material. Las especies de esta tesis son poco abundantes y crípticas (sección 1.1.1), y sus vocalizaciones ocupan una fracción pequeña de lo grabado: en cinco de las ocho especies, las vocalizaciones anotadas no llegan al 10 % del tiempo de grabación (Tabla 1.1). Es la situación habitual del PAM de primates: las grabaciones continuas contienen relativamente pocas llamadas, y breves (Heinicke et al., 2015). Para no perderlas, el especialista recorre la grabación entera, y el costo de esa búsqueda crece con las horas grabadas, no con las vocalizaciones encontradas: en el sistema para primates de Heinicke et al. (2015), escuchar y verificar 179 horas de grabación tomó unas 360 horas de trabajo.

Los detectores que alivian esa búsqueda en otros dominios no lo hacen en este. Los clasificadores generales de bioacústica se entrenaron sobre todo con aves, como BirdNET (Kahl et al., 2021); su desempeño cae en los taxones con pocos datos anotados (Dubus et al., 2026), y los pocos detectores contruidos para primates se limitan a una especie o a unas pocas (Pérez-Granados & Schuchmann, 2021). Además, las llamadas tenues, transitorias o parcialmente enmascaradas son las que los métodos automáticos pierden con más frecuencia (Gibb et al., 2018). Esa carencia es, en parte, la vía indirecta de C1 descrita arriba. Y aun donde existen detectores, los sistemas más difundidos entregan una probabilidad por segmento de duración fija, y los que delimitan eventos lo hacen solo sobre el eje temporal (Ebberts et al., 2024; Kahl et al., 2021); ninguno devuelve la banda de frecuencia ni el tipo de llamada. Una salida así indica dónde mirar, pero el especialista tiene que volver a dibujar cada caja y a etiquetarla.

1.2.3. Efectos

Los efectos se organizan por actor afectado, siguiendo el criterio que Veselý (2008) ilustra en su tercer ejemplo: E1 recae sobre el equipo que monitorea, E2 sobre el

conocimiento que se obtiene de los datos y E3, efecto final que se deriva de los dos anteriores, sobre la conservación de las especies.

E1: El audio recolectado se acumula sin analizar

Cuando el volumen de datos excede con mucho las horas-persona disponibles (Stowell, 2022), detectar las señales de interés en los paisajes sonoros registrados se convierte en un obstáculo logístico que limita la escala de los estudios (Campos et al., 2025). El esfuerzo y el gasto del despliegue quedan inmovilizados en material sin procesar, la información llega con retraso, y la porción que llega a anotarse responde a la disponibilidad del especialista antes que a un criterio de muestreo.

E2: El análisis se limita a pocas especies y a sus llamadas más evidentes

Lo que sí se analiza tiende a ser lo más fácil de encontrar. Los estudios de PAM con detección automática en primates se concentran en una especie y en su vocalización más larga y potente, el aullido de los monos aulladores, cuyo buen resultado sus propios autores atribuyen a esas características (Pérez-Granados & Schuchmann, 2021). El resto del repertorio, con sus llamadas de contacto y de alarma, y las especies menos vocales quedan fuera del análisis, y con ellos la información que el PAM podía aportar sobre el comportamiento de cada especie.

E3: La conservación decide con información tardía o incompleta

Efecto final, derivado de E1 y E2. Cuando el análisis no acompaña a la recolección, condición de la que dependen las ventajas del PAM (sección 1.1.4), lo raro se pierde en el audio sin revisar (E1) y la imagen de cada especie se reduce a sus llamadas más evidentes (E2). Para especies con poblaciones en declive, esa información tardía o incompleta es la base sobre la que se decide su conservación (Zwerts et al., 2021).

1.3. Brecha de investigación

Las dos causas tienen también una lectura en términos de investigación: para este dominio faltan datos utilizables y faltan herramientas que alivien la búsqueda. Esta sección lo examina con más detalle: ordena primero las soluciones existentes según la forma de su salida, ubica después el vacío en el cruce de dos ejes, el taxón y la tarea, y

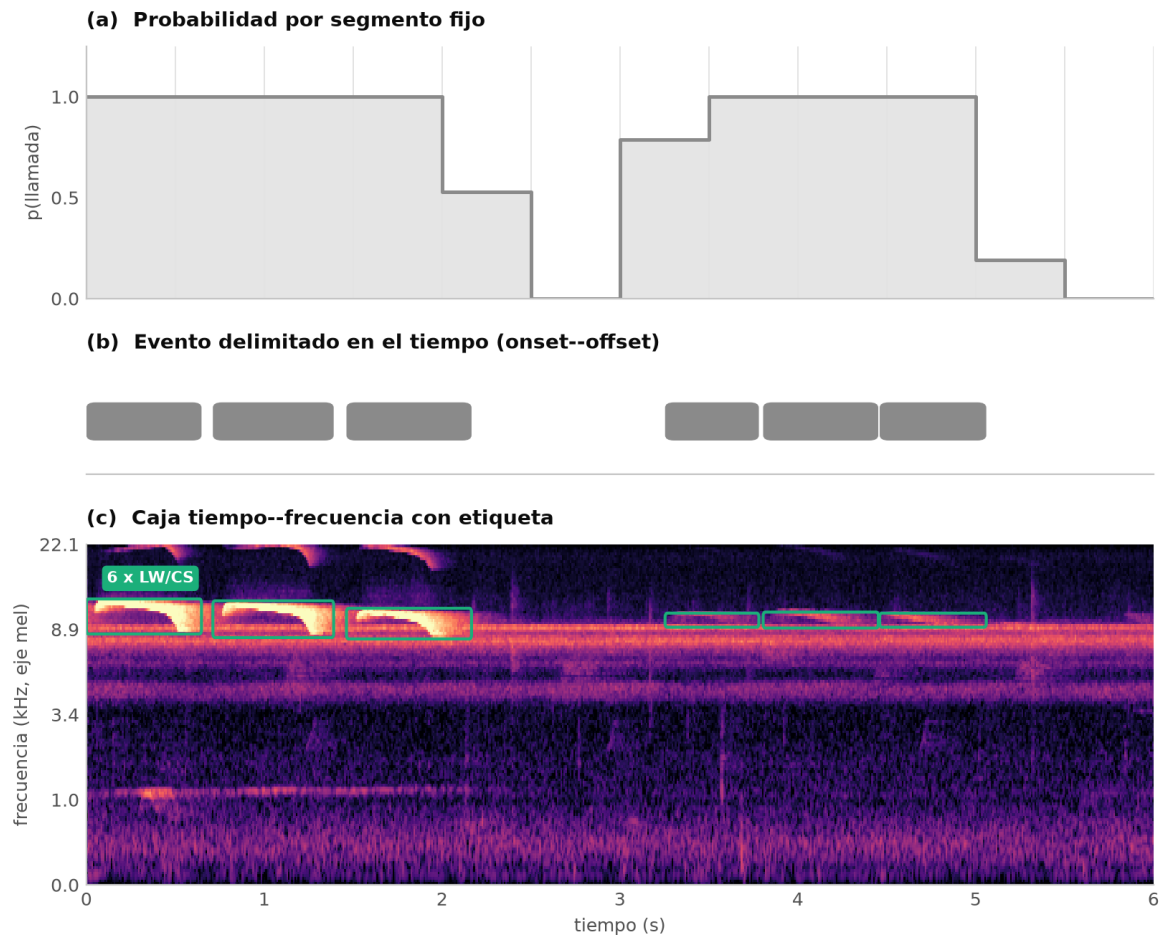
termina delimitando la parte de ese vacío que aborda esta tesis. La revisión sistemática de la literatura del Capítulo 3 confirma esta lectura.

1.3.1. Soluciones existentes

Como adelantó la causa C2, los trabajos previos se distinguen mejor por la forma que toma su salida que por su arquitectura, porque esa forma decide si el resultado puede sustituir la anotación del especialista o solo complementarla. La Figura 1.3 muestra las tres formas posibles sobre una misma secuencia de seis sílabas de contacto del tamarino de Weddell: (a) una probabilidad de llamada por segmento fijo, (b) un intervalo de inicio y fin por evento y (c) una caja tiempo–frecuencia con su etiqueta. Solo la tercera coincide con la anotación de la Figura 1.1.

Figura 1.3

Tres formas de salida



La forma (a) es la de los sistemas más difundidos. BirdNET identifica cientos de especies de aves puntuando segmentos de 3 s (Kahl et al., 2021), y HOWLish detecta

aullidos de lobo sobre ejemplos de 0,96 s cuyas predicciones promedia con una ventana móvil (Campos et al., 2025). La forma (b) delimita eventos, pero solo en el tiempo: los *sound event bounding boxes* formalizan cada predicción como una cuádrupla de clase, inicio, fin y confianza (Ebberts et al., 2024), y YOHO convierte la detección en una regresión de los puntos de inicio y fin de cada clase (Venkatesh et al., 2022). En estas dos formas se ubican los detectores construidos para primates: el sistema semiautomático evaluado en el Parque Nacional Taï, en Costa de Marfil (Heinicke et al., 2015; Kalan et al., 2015), los reconocedores de aullidos de monos aulladores en Brasil (do Nascimento et al., 2021; Pérez-Granados & Schuchmann, 2021) y la red que detecta las llamadas del langur de cabeza blanca en China (Hong et al., 2026). Su objetivo es establecer la presencia de una especie o de su llamada característica, no anotar cada vocalización con su banda de frecuencia y su tipo.

La forma (c) trata el espectrograma como una imagen y localiza cada vocalización con los métodos de detección de objetos de la visión por computadora (Pham et al., 2018). Así se han detectado vocalizaciones de foca barbuda con YOLOv5 (Escobar-Amado et al., 2024) y llamadas de aves migratorias nocturnas con un detector de dos etapas, entrenado sobre anotaciones tiempo–frecuencia que recolectaron decenas de aficionados (Airale et al., 2025). En primates, el antecedente más cercano que se identificó es el uso de DeepSqueak, un programa de detección de cajas desarrollado para roedores, sobre vocalizaciones del lémur ratón gris grabadas en cautiverio, con una clasificación posterior del tipo de llamada (Romero-Mujalli et al., 2021).

1.3.2. Vacío

El vacío tiene dos ejes, y esta tesis se ubica en su cruce (Tabla 1.3). Las filas de la tabla separan los taxones según los datos anotados de que disponen; las columnas separan las dos clases de salida que importan para la anotación: la que indica dónde hay una llamada, por segmento o por intervalo temporal, y la que entrega la caja tiempo–frecuencia con su etiqueta. Cada celda cita los trabajos que la ocupan.

La tabla deja una sola celda sin trabajos: la de los primates neotropicales con salida tiempo–frecuencia. Los antecedentes en primates neotropicales detectan la presencia de una llamada, y los trabajos que entregan cajas lo hacen para otros taxones o para primates grabados en cautiverio.

Tabla 1.3
Vacío de la investigación

	Presencia por segmento o intervalo temporal	Caja tiempo–frecuencia con su etiqueta
Aves y otros taxones con datos anotados	BirdNET (Kahl et al., 2021); lobos (Campos et al., 2025); SEBB y YOHO (Ebbers et al., 2024; Venkatesh et al., 2022)	Aves migratorias (Airale et al., 2025); foca barbuda (Escobar-Amado et al., 2024)
Primates fuera del Neotrópico	Taï, Costa de Marfil (Heinicke et al., 2015; Kalan et al., 2015); langur de cabeza blanca (Hong et al., 2026)	Lémur ratón gris, en cautiverio (Romero-Mujalli et al., 2021)
Primates neotropicales	Monos aulladores (do Nascimento et al., 2021; Pérez-Granados & Schuchmann, 2021)	Vacío: esta tesis

Eje del taxón. La causa C1 ya documentó que para primates neotropicales no existe un conjunto público con anotaciones tiempo–frecuencia (Tabla 1.2). El contraste con las aves muestra que el obstáculo es el taxón y no la región.

De la Reserva Amazónica de Inkaterra, en Madre de Dios, la misma región de las grabaciones de esta tesis, se publicó un conjunto de 21 grabaciones de una hora de paisaje sonoro con 14 798 cajas tiempo–frecuencia de 132 especies de aves (Hopping et al., 2022), y de dos localidades de Colombia, otro de 73,62 horas con 15 372 anotaciones tiempo–frecuencia de 168 especies (Ruiz et al., 2026). Aun así, los autores de este último advierten que los conjuntos con anotación temporal precisa siguen siendo escasos en los paisajes sonoros tropicales (Ruiz et al., 2026), y la investigación con PAM terrestre se ha concentrado en murciélagos y en regiones templadas del hemisferio norte (Sugai et al., 2019). Si las aves de la Amazonía apenas cuentan con datos de este tipo, los primates no cuentan con ninguno.

Eje de la tarea. El vacío tampoco es solo de datos. La salida que necesita el especialista combina dos tareas que la literatura considera poco exploradas: la detección de cajas tiempo–frecuencia, un área en gran medida inexplorada pese a su utilidad para el analista (Z. Zhu & Sato, 2025), y la clasificación fina del tipo de llamada, que en bioacústica se reconoce como un desafío abierto antes que como una capacidad demostrada (Nolasco et al., 2023). A ellas se suma una tercera carencia. El ahorro que

aporta la asistencia se ha medido para detectar una o pocas especies (sección 1.1.3), pero ninguno de los trabajos revisados lo mide para la anotación completa, con caja, especie y tipo de llamada: los que entregan cajas tiempo–frecuencia reportan solo métricas del modelo (Airale et al., 2025; Escobar-Amado et al., 2024; Z. Zhu & Sato, 2025).

1.3.3. Alcance de la tesis

Esta tesis se ubica en el cruce de ambos ejes: responde a las dos causas del árbol y mide su efecto sobre el problema central. La propuesta de solución es un sistema de pre-anotación asistida, que el objetivo general define. Su alcance se limita a las ocho especies del material entregado, a grabaciones ya recolectadas, sin trabajo de campo, y a la pre-anotación: el especialista conserva la decisión final sobre cada caja. Así delimitado, el proyecto abarca el ciclo completo de un sistema de aprendizaje automático, desde la construcción de sus datos hasta la medición de su efecto sobre el trabajo humano, y cada una de esas etapas se formula a continuación como un objetivo.

1.4. Objetivos

Los objetivos convierten el árbol de problemas en un plan de trabajo: el objetivo general responde al problema central; OE1 y OE2, a una causa cada uno, y OE3 mide el efecto sobre el problema central.

1.4.1. Objetivo general

OG. Desarrollar y evaluar un sistema de pre-anotación asistida que proponga cajas tiempo–frecuencia con especie y tipo de llamada sobre grabaciones de PAM de primates amazónicos, con el fin de reducir el tiempo de revisión experta.

El objetivo general ataca el problema central en la magnitud que lo define, el tiempo de especialista. Un sistema de *pre-anotación asistida* propone anotaciones para que el especialista las acepte, las corrija o las descarte en su propia herramienta, en lugar de dibujarlas desde cero. Las propuestas tienen la misma forma que la anotación manual de la Figura 1.1, una caja tiempo–frecuencia con especie y tipo de llamada, porque solo así pueden sustituirla en lugar de complementarla (sección 1.3.1). Los primates amazónicos del enunciado son las ocho especies de la sección 1.1.1. Y el criterio de

éxito no es el desempeño del modelo, que es un medio, sino la reducción del tiempo de revisión experta, que es la magnitud del problema.

El encuadre asistido cuenta con respaldo en la literatura. La inteligencia artificial no reemplaza la experiencia: conforme estos sistemas se estandarizan, asumen el papel de pares expertos con quienes se consulta y debate, sin desplazar el rol del experto ni el del trabajo colaborativo (Stowell, 2022). Y tiene precedentes cuantificados, entre ellos el del sistema semiautomático para primates de Taï, cuyos ahorros de tiempo se revisaron en la sección 1.1.3.

El enunciado fija además un compromiso de diseño: priorizar la cobertura, es decir, que el sistema encuentre la mayor parte de las vocalizaciones, aunque eso suponga proponer también falsos positivos. En un flujo asistido, perder una vocalización cuesta más que descartar una propuesta errónea, porque el especialista revisa la salida de todos modos, y un detector puede resultar útil aun con una precisión muy baja: HOWLish resulta útil operando con una precisión de 0,006 (Campos et al., 2025). La contrapartida es que la carga que genera ese sesgo debe medirse, y de eso se ocupa OE3.

1.4.2. Objetivos específicos

Los tres objetivos específicos se encadenan. El conjunto de datos que produce OE1 es el material con que se entrena y se mide el detector de OE2, y el detector de OE2 es el sistema cuyo efecto sobre el trabajo del especialista mide OE3.

OE1. Definir un protocolo de curación y estandarización, y aplicarlo sobre las anotaciones entregadas por el equipo de investigación para construir un conjunto de datos consistente de cajas tiempo–frecuencia etiquetadas por especie y tipo de llamada.

OE2. Implementar y evaluar un detector que reciba un espectrograma y devuelva cajas tiempo–frecuencia con especie, tipo de llamada y confianza, comparándolo con una línea base establecida.

OE3. Cuantificar la reducción del tiempo de revisión que aporta el sistema y caracterizar sus errores mediante revisión experta de una muestra de predicciones.

Los dos primeros responden a una causa del árbol cada uno; el tercero, al problema central. Su propósito se entiende desde ese vínculo.

OE1 frente a C1. Un detector solo puede aprender y evaluarse contra anotaciones que sigan un criterio único, y las únicas disponibles tienen ruido, sesgo e inconsistencias (causa C1). Un *protocolo de curación y estandarización* es un conjunto documentado de reglas que corrige lo corregible sin volver a anotar el audio: unifica la escritura de las etiquetas sobre un vocabulario cerrado de pares de especie y tipo de llamada, descarta el ruido y las cajas inválidas y deja registro de cada decisión. Lo que no puede corregirse, como el desacuerdo de criterio entre anotadores o el desbalance entre especies, lo separa y lo deja documentado. Aplicarlo produce el conjunto de datos sobre el que trabajan OE2 y OE3.

OE2 frente a C2. El detector ataca la búsqueda que describe la causa C2: en lugar de recorrer la grabación entera, el especialista revisa las propuestas que el detector encuentra. Es el componente que convierte el espectrograma de una grabación en propuestas de anotación con la forma que usa el especialista: cajas tiempo–frecuencia con especie, tipo de llamada y una *confianza*, el valor entre 0 y 1 con que el modelo puntúa cada propuesta y que permite ordenarlas o filtrarlas. Compararlo con una *línea base establecida*, un detector de eficacia ya documentada en la literatura, entrenado y medido sobre los mismos datos, es lo que permite saber si la arquitectura propuesta aporta algo frente a lo que ya existe.

OE3 frente al problema central. OE3 no responde a una causa: mide si la propuesta reduce la magnitud que define el problema, el tiempo de especialista. Cronometra la revisión del mismo tipo de material con y sin las propuestas del detector, el ahorro que la literatura ha medido para detectar una especie pero no para la anotación completa (sección 1.3.2). Y vuelve sobre la causa C1: someter una muestra de predicciones al juicio de un especialista distingue los errores genuinos de las vocalizaciones que la referencia omitió, de modo que mide también la calidad de las anotaciones. Es el objetivo que responde, con una cifra, si el sistema cumple el objetivo general.

1.5. Resultados esperados

Cada objetivo específico se compromete con resultados esperados (RE): productos tangibles que existirán al terminar el proyecto y que pueden inspeccionarse. Para cada uno, la tabla correspondiente fija el *medio de verificación*, es decir, dónde puede comprobarse que el producto existe, y el *indicador objetivamente verificable* (IOV), la

condición observable que debe cumplir para darse por logrado. Los términos técnicos de cada tabla se definen en los párrafos que la preceden, y después de cada tabla se explica qué aporta cada resultado a su objetivo. Los capítulos 4, 5 y 6 desarrollan un objetivo cada uno y verifican sus resultados contra estas tablas.

1.5.1. OE1: Protocolo y conjunto de datos curado

OE1 parte de la causa C1: las anotaciones que entregó el equipo de investigación son las únicas que existen para estas especies, pero no pueden usarse tal como llegan para entrenar ni para evaluar un detector. La tesis no vuelve a anotar el audio. Transforma ese material en un conjunto utilizable y se compromete con tres productos, que la Tabla 1.4 detalla: las reglas de esa transformación (RE1.1), el conjunto que resulta de aplicarlas (RE1.2) y el documento que lo describe (RE1.3). Las grabaciones y las anotaciones originales siguen siendo del equipo; la contribución de la tesis es esa capa derivada.

Cada regla del protocolo (RE1.1) responde a un defecto concreto del material recibido, y el indicador las nombra por ese defecto. La *normalización* unifica los nombres de las columnas y la escritura de las etiquetas, y el *mapa de sinónimos* reúne en una sola clase las distintas grafías con que se escribió. Las filas de *ruido*, que marcan sonido de fondo en lugar de una vocalización, y las *cajas degeneradas*, con duración o ancho de banda nulos, se descartan. Los *pares imposibles*, combinaciones de especie y tipo de llamada que no existen en el repertorio de la especie, se corrigen. El recorte al *límite de Nyquist*, la frecuencia más alta que una grabación puede representar (la mitad de su frecuencia de muestreo: 22,05 kHz a 44,1 kHz), elimina la parte de una caja que excede lo grabado. Y los *registros fuera de vocabulario*, cajas cuya etiqueta no corresponde a ninguna clase admitida, se marcan para revisión en lugar de descartarse.

El conjunto (RE1.2) se entrega *particionado*, es decir, dividido en tres subconjuntos sin elementos comunes: el de entrenamiento, con el que el modelo ajusta sus parámetros; el de validación, con el que se eligen sus configuraciones, y el de prueba, que se reserva para medir el desempeño final. Las proporciones 60/22,5/17,5 indican el porcentaje del material que va a cada uno. La división se hace por archivo de grabación porque el modelo no recibe grabaciones enteras sino *ventanas*, fragmentos de 3 s tomados cada 1,5 s y por tanto solapados: dos ventanas vecinas de una misma grabación son casi

iguales, y repartirlas entre entrenamiento y prueba inflaría el resultado. Cada ventana se guarda como un espectrograma log-mel, la representación del audio que recibe el modelo (sección 2.2.3), en tres archivos `.pt`, uno por subconjunto, y el archivo `meta.json` registra los parámetros con que se generaron. Por último, la *ficha del conjunto* (RE1.3), conocida en inglés como *datasheet*, es el documento que describe la composición, el origen, el proceso de construcción, los usos recomendados y las limitaciones de un conjunto de datos (Gebru et al., 2021).

Tabla 1.4
IOV de OE1

RE	Resultado	Medio de verificación	Indicador objetivamente verificable (IOV)
RE1.1	Protocolo de curación y estandarización documentado.	Documento del protocolo.	Define la normalización de columnas y etiquetas, el mapa de sinónimos, el descarte de ruido y cajas degeneradas, la corrección de pares imposibles, el recorte al límite de Nyquist y el tratamiento de registros fuera de vocabulario.
RE1.2	Conjunto acústico curado y particionado para el experimento.	Directorio <code>data/processed/</code> : etiquetas, metadatos y tres archivos <code>.pt</code> .	Los archivos procesados cargan sin error; <code>meta.json</code> registra la semilla, el criterio de selección, las exclusiones, la etiqueta usada y la normalización; el código divide por archivo de grabación con proporciones 60/22,5/17,5 y no por ventanas aisladas.
RE1.3	Ficha del conjunto derivado.	Capítulo de curación, README y metadatos del conjunto.	Describe las ocho especies, el vocabulario, la composición, el desbalance, el diccionario de campos, el ventaneo de 3 s con salto de 1,5 s, la normalización log-mel, las exclusiones y las limitaciones conocidas.

Aporte de RE1.1. El protocolo hace explícita y repetible la estandarización que exige la causa C1: al quedar escrito como un conjunto de reglas, puede aplicarse de nuevo cada vez que el equipo entregue más anotaciones, y cualquier tercero puede comprobar qué se hizo con cada fila.

Aporte de RE1.2. El conjunto curado y particionado es el insumo directo de OE2 y la referencia contra la que OE3 compara. Que la partición sea por grabación hace

que el desempeño se mida sobre grabaciones que el modelo no vio al entrenar, que es la situación en que el equipo lo usará: sobre grabaciones nuevas.

Aporte de RE1.3. La ficha cumple dos funciones. Permite que el equipo de investigación, o un tercero, use el conjunto sin depender del autor. Y declara sus límites (el desbalance entre especies, los sesgos de lo que se anotó, las clases que quedaron fuera), que son también los límites de lo que OE2 y OE3 pueden afirmar.

1.5.2. OE2: Detector y evaluación

OE2 responde a la causa C2: construye el detector que encuentra las vocalizaciones en lugar del especialista, y lo evalúa. Se compromete con cinco productos que siguen el orden del trabajo: diseñar el detector (RE2.1), construir el *pipeline* que lo entrena y lo aplica (RE2.2), compararlo con las líneas base (RE2.3), entregar el modelo final en el formato de Raven (RE2.4) y publicar el código y el modelo (RE2.5). La Tabla 1.5 los detalla, con términos que conviene definir. Un *detector* es un modelo que recibe el espectrograma de una ventana y devuelve un conjunto de cajas, cada una con su clase y su confianza. La arquitectura propuesta, AST-Deformable-DETR (RE2.1), combina dos piezas que el Capítulo 2 describe: un extractor de características preentrenado con audio, el *Audio Spectrogram Transformer* (AST), y una cabeza de detección de la familia DETR (*DEtection TRansformer*), que predice directamente un conjunto de cajas. Su *dimensión*, su número de *consultas* y su número de *niveles* son sus hiperparámetros principales: el tamaño de su representación interna, el máximo de cajas que puede proponer por ventana y el número de escalas con que analiza el espectrograma. El *pipeline* (RE2.2) es la secuencia de programas que lleva de las anotaciones a un modelo entrenado, y de este a las predicciones sobre una grabación nueva; un *checkpoint* (RE2.4) es el archivo que guarda los pesos de un modelo entrenado, y `labels.json`, la lista de clases que esos pesos predicen.

Las métricas de RE2.3 comparan las predicciones con las anotaciones. Una predicción cuenta como *verdadero positivo* cuando su caja se superpone lo suficiente con una caja anotada de la misma clase, y esa superposición se mide con el IoU (*intersection over union*): el área común de las dos cajas dividida por el área que cubren entre ambas, que vale 1 si coinciden y 0 si no se tocan. Sobre esos aciertos se calculan el *recall*, la fracción de vocalizaciones anotadas que el detector encuentra; la *precisión*, la fracción de sus

predicciones que son correctas, y F1, que combina ambas. La precisión media (AP) resume la relación entre precisión y *recall* a lo largo de todos los umbrales de confianza (Lin et al., 2014), y se reporta con dos exigencias de superposición, IoU 0,25 y 0,5. La matriz de confusión cuenta, para cada clase anotada, a qué clase la asignó el detector. Por último, una *tabla de selección* es el archivo de texto con que Raven guarda y abre anotaciones, y es el formato en que el modelo final entrega sus predicciones (RE2.4).

Aporte de RE2.1. El documento de diseño fija qué se compara y con qué regla antes de entrenar, para que la comparación no pueda ajustarse a posteriori al resultado. Parte de formular la anotación como detección de objetos sobre el espectrograma, la tercera forma de salida de la Figura 1.3, y fija como referencia una línea base establecida sobre espectrogramas (González et al., 2025).

Aporte de RE2.2. El *pipeline* reproducible permite regenerar cada cifra del informe a partir de los datos y del código, y es también lo que el equipo de investigación necesita para aplicar el sistema a grabaciones nuevas sin depender del autor.

Aporte de RE2.3. El informe comparativo dice cuánto acierta el detector y dónde falla, y separa tres ejes porque cada fallo le cuesta al especialista una corrección distinta: una vocalización no detectada lo obliga a dibujar la caja completa, una caja mal encuadrada solo a ajustarla y una etiqueta errónea solo a cambiarla. Exigir un IoU bajo responde a la borrosidad de las fronteras señalada en la causa C1, y priorizar el *recall*, al compromiso de cobertura del objetivo general.

Aporte de RE2.4. El modelo final, capaz de escribir sus predicciones como tabla de selección de Raven, es lo que convierte el detector en pre-anotación: el especialista abre la tabla sobre la grabación, en su propia herramienta, y recibe propuestas con los mismos campos que su anotación: tiempo, frecuencia, especie, tipo de llamada y confianza.

Aporte de RE2.5. El repositorio permite localizar el código y el modelo final, que es la condición para que un tercero compruebe RE2.1 a RE2.4 y para que el equipo siga usando el sistema cuando la tesis termine.

Tabla 1.5
IOV de OE2

RE	Resultado	Medio de verificación	Indicador objetivamente verificable (IOV)
RE2.1	Diseño experimental, arquitectura propuesta y línea base.	Documento de diseño y módulo de arquitecturas.	Especifica la arquitectura AST-Deformable-DETR implementada con todos sus hiperparámetros, y define antes de entrenar la comparación con las líneas base declaradas: configuraciones, partición, métricas y regla de selección.
RE2.2	Pipeline reproducible de entrenamiento, evaluación e inferencia.	Scripts de datos, entrenamiento, evaluación e inferencia; módulos de <code>src/pipelines/</code> ; <code>pyproject.toml</code> y manual técnico.	El pipeline genera los splits procesados, entrena desde un conjunto de configuración, evalúa un checkpoint y ejecuta inferencia sobre un WAV; las dependencias y comandos de ejecución están declarados.
RE2.3	Informe comparativo de resultados.	Informe de experimentación y archivos de métricas.	Reporta, como mínimo, recall, precisión, F1, IoU medio, AP a IoU 0,25 y 0,5, recall por clase y matriz de confusión; separa detección, encuadre y clasificación y justifica la selección del modelo.
RE2.4	Modelo final cargable y demostración funcional.	Checkpoint <code>.pth</code> , <code>labels.json</code> y ejecución de <code>src/infer.py</code> .	El checkpoint se carga sin claves incompatibles y la inferencia sobre un WAV produce un archivo tabulado con las columnas de Raven, incluidas coordenadas tiempo–frecuencia, especie, tipo de llamada y puntuación.
RE2.5	Código y modelo disponibles en un repositorio.	Repositorio de control de versiones.	La URL del repositorio permite localizar el código de curación, entrenamiento, evaluación e inferencia, junto con las instrucciones de ejecución y los artefactos del modelo final que se hayan decidido publicar.

1.5.3. OE3: Tiempo de revisión y análisis de errores

OE3 mide el efecto del sistema sobre el problema central, el tiempo del especialista, y sobre la calidad de la referencia (causa C1). Se compromete con tres productos: la metodología de la medición, fijada antes de medir (RE3.1); la medición del tiempo de revisión (RE3.2), y el análisis de los errores (RE3.3). La Tabla 1.6 los detalla. El *punto de operación* (RE3.1) es la configuración con que se usa el detector: el umbral de confianza por debajo del cual una propuesta no se muestra al revisor, que se elige en validación con la regla de la sección 1.7, y la supresión de no máximos (NMS), que, cuando varias propuestas se superponen por encima de un IoU de 0,3, conserva solo la de mayor confianza. Las dos *condiciones* de RE3.2 son la manual, en la que el especialista anota desde cero, y la asistida, en la que revisa las propuestas del detector; el tiempo de cada una se expresa en minutos por hora de audio, para que materiales de distinta duración sean comparables, y el *orden de revisión* se declara porque quien ya revisó un material lo anota más rápido la segunda vez. En RE3.3, las predicciones *de alta confianza* son las que superan con holgura el punto de operación, y sus errores se clasifican en tres categorías: *evento no anotado*, una vocalización real que la anotación de referencia omitió; *ruido*, una predicción sobre un sonido que no es una vocalización de las especies estudiadas, y *error de encuadre*, una vocalización correctamente detectada cuya caja no se ajusta a la anotada.

De los tres resultados, RE3.2 es el que responde por el objetivo general; los otros dos lo hacen interpretable. RE3.1 impide ajustar la medición al resultado, y RE3.3 dice de qué están hechos los errores que el especialista corrige.

Aporte de RE3.1. La metodología, redactada y fechada antes de ejecutar la evaluación, fija el material de prueba, el punto de operación, el umbral de emparejamiento, las métricas y el protocolo con que el revisor resuelve las discrepancias, siguiendo el marco de tubería para la identificación automatizada de sonidos de Gibb et al. (2018). Fijarla de antemano impide elegir a posteriori el umbral o la métrica que mejor favorezcan al resultado ya obtenido.

Aporte de RE3.2. La medición comparativa es el resultado que responde al objetivo general. Compara minutos por hora de audio en ambas condiciones, sobre material comparable y con el orden de revisión declarado, y suma a la condición asistida el tiempo de procesamiento del modelo, que también forma parte de su costo. El indicador

Tabla 1.6
IOV de OE3

RE	Resultado	Medio de verificación	Indicador objetivamente verificable (IOV)
RE3.1	Metodología de validación comparativa.	Documento de metodología y protocolo de revisión experta.	Especifica el subconjunto de prueba, el punto de operación elegido en validación y el umbral de IoU para verdadero positivo, fijados antes de medir, las métricas, el emparejamiento y el procedimiento para revisar discrepancias y eventos potencialmente omitidos.
RE3.2	Medición comparativa del tiempo de revisión.	Informe de validación, registros de tiempo y medición de inferencia.	Reporta minutos por hora de audio en condición manual y asistida, describe el material comparable y el orden de revisión, y presenta el tiempo de procesamiento del modelo por hora de audio. No fija un porcentaje de ahorro antes de medirlo.
RE3.3	Análisis cualitativo de errores y hallazgos.	Informe de análisis y espectrogramas anotados.	Clasifica una muestra de predicciones de alta confianza en evento no anotado, ruido y error de encuadre, e incluye ejemplos visuales de aciertos, errores comunes y detecciones correctas ausentes de la anotación de referencia.

no fija de antemano un porcentaje de ahorro: se mide y se reporta con sus condiciones, y los ahorros revisados en la sección 1.1.3 son su punto de comparación. La medición se acompaña de magnitudes que se derivan de la propia salida del sistema, como las propuestas por hora de audio y la fracción de audio sin ninguna propuesta, que explican de dónde sale el ahorro, al modo de la reducción de 22 veces del volumen que un operador debía procesar en HOWLish (Campos et al., 2025).

Aporte de RE3.3. El análisis de errores resuelve la dificultad que señala la causa C1: saber cuánto del error aparente es del modelo y cuánto de una referencia incompleta. La clasificación se apoya en la relación geométrica entre la caja predicha y las cajas anotadas, que resume la Tabla 1.7.

La última columna de la tabla indica a qué categoría de RE3.3 corresponde cada tipo. Los tipos A y B quedan determinados por la geometría y son errores de encuadre;

Tabla 1.7
Taxonomía de discrepancias

Tipo	Discrepancia	Definición	¿Sin experto?	Categoría en RE3.3
A	Encuadre	Solapa con una anotación de la misma clase, pero con IoU inferior al umbral de emparejamiento.	Sí	Error de encuadre
B	Unidad	Varias predicciones sobre una misma anotación, o una predicción sobre varias anotaciones: división y fusión.	Sí	Error de encuadre
C	Clase	Encuadre correcto sobre una anotación, con etiqueta de especie o de tipo de llamada incorrecta.	Sí	Error de clase, que se reporta aparte
D	Región sin anotación	No solapa con ninguna caja anotada. Puede ser un evento omitido por la referencia o un falso positivo genuino.	No	Evento no anotado o ruido, según el revisor

el tipo C, una etiqueta equivocada sobre una caja correcta, no pertenece a ninguna de las tres categorías comprometidas y se reporta aparte, y el tipo D, las predicciones que no se superponen con ninguna anotación, es el que el revisor experto resuelve entre evento no anotado y ruido. Los dos modos de error que documenta Z. Zhu y Sato (2025), una caja anotada detectada como varias y varias detectadas como una, corresponden al tipo B, y la borrosidad de las fronteras que ese mismo trabajo describe es lo que hace necesario el tipo A.

Limitación declarada. La resolución de las predicciones de alta confianza situadas en regiones sin anotación (tipo D) entre eventos omitidos por la referencia y falsos positivos genuinos depende del criterio experto, y este trabajo lo obtiene sobre una muestra, no sobre el total. Lo que quede fuera de esa muestra se caracteriza geométricamente y se reporta como una cantidad acotada; el tamaño de la muestra y la disponibilidad del equipo de investigación son, por tanto, una limitación del alcance de RE3.3, y como tal se declaran.

1.6. Trazabilidad

La Tabla 1.8 reúne la cadena que recorre este capítulo, del problema a los productos comprometidos: cada causa del árbol se atiende con un objetivo específico, OE3 mide el efecto sobre el problema central, cada objetivo se concreta en sus resultados esperados y el objetivo general responde al problema central. Los efectos E1 a E3 no se atacan de forma directa: son lo que se espera aliviar si el objetivo general se cumple. La tabla no reemplaza la justificación de cada vínculo, que está en las secciones 1.2.2, 1.4 y 1.5; la resume para que pueda leerse de un vistazo.

Tabla 1.8
Trazabilidad del planteamiento

Elemento del árbol	Objetivo	Resultados	Aporte
C1: las anotaciones tienen ruido, sesgo e inconsistencias	OE1	RE1.1 a RE1.3	Conjunto curado, particionado y documentado
C2: las vocalizaciones son escasas y breves en horas de audio	OE2	RE2.1 a RE2.5	Detector que propone cajas con especie, tipo de llamada y confianza, en el formato de Raven
Problema central (magnitud) y C1 (calidad de la referencia)	OE3	RE3.1 a RE3.3	Tiempo de revisión manual frente a asistido, y errores clasificados por un especialista
Problema central	OG	Todos	Sistema de pre-anotación asistida, evaluado por el tiempo de revisión experta

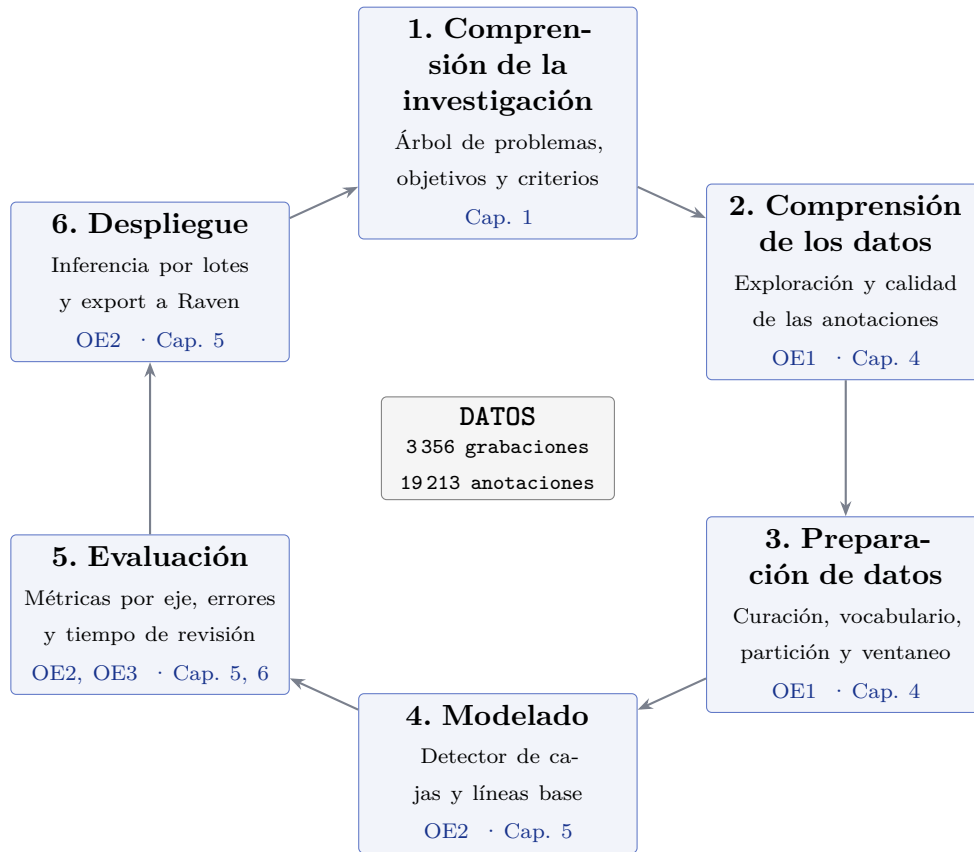
Ninguna causa queda sin objetivo, y ningún objetivo queda sin un elemento del árbol al que responder. Y aunque los once resultados contribuyen al objetivo general, uno solo responde por él: RE3.2, la medición del tiempo de revisión con y sin la asistencia del sistema.

1.7. Métodos y procedimientos

1.7.1. Metodología CRISP-DM

El desarrollo se organiza siguiendo CRISP-DM (*Cross-Industry Standard Process for Data Mining*), un marco iterativo, independiente de la industria y de la herramienta, que proporciona una hoja de ruta general para proyectos de minería de datos (Chapman

Figura 1.4
CRISP-DM aplicado a este proyecto



et al., 2000). Su estructura cíclica, desde la comprensión del problema hasta el despliegue de la solución, se ajusta a un proyecto de investigación aplicada en el que la comprensión de los datos, la experimentación con modelos y la evaluación continua se realimentan entre sí. La Figura 1.4 resume la correspondencia entre las seis fases, los objetivos específicos y los capítulos de este documento.

Las fases 2 y 3 corresponden a OE1; la 4 y la 6, a OE2, y la 5 se reparte entre OE2 y OE3. Los párrafos siguientes describen cada fase.

Fase 1: Comprensión del negocio o de la investigación.

Convierte el conocimiento del dominio y los objetivos del proyecto en una definición formal del problema de minería de datos (Chapman et al., 2000). Esa traducción es el contenido de las secciones anteriores de este capítulo: el problema se analiza con un árbol de problemas, y de él derivan el objetivo general, los objetivos específicos y los resultados esperados. Los criterios de éxito son a la vez técnicos, relacionados con el desempeño en detección, encuadre y clasificación, y operativos, vinculados con el

tiempo de revisión experta que es la magnitud del problema, como recomienda el marco para asegurar que el resultado sea útil y no solo correcto.

Fase 2: Comprensión de los datos.

Parte del material descrito en la sección 1.1.2: una carpeta por especie con las grabaciones y, junto a cada una, la tabla de selección exportada de Raven Pro, en la que cada fila registra las coordenadas de una caja, su amplitud, su calidad y las etiquetas de especie y de tipo de llamada. La fase comprende el análisis exploratorio (espectrogramas, distribución de clases, duraciones y rangos de frecuencia) y la verificación de calidad: audios corruptos, errores tipográficos y etiquetas inconsistentes.

Fase 3: Preparación de datos.

Comprende la limpieza y normalización de etiquetas sobre un vocabulario cerrado, la selección de clases y la partición por grabación, el preprocesamiento del audio, el cálculo de la representación tiempo–frecuencia y la segmentación de las grabaciones largas en ventanas de duración fija con solapamiento. La representación es el espectrograma log-mel, por la razón que da la sección 2.2.3. El aumento de datos se aplica al entrenar: variación de ganancia, mezcla con fondo del propio entorno, corrimiento temporal y pegado de llamadas de otras ventanas, más una perturbación leve de las cajas.

Fase 4: Modelado.

La forma de la salida no está en discusión: el sistema debe emitir la tupla completa de la ecuación (2.1), y esa exigencia excluye de antemano toda arquitectura cuya cabeza produzca una probabilidad por trama o por segmento. Lo que se compara, por tanto, no son familias de arquitectura sino configuraciones que sí satisfacen la exigencia: la arquitectura propuesta sobre un extractor de audio con cabeza de detección, y tres líneas base de detector sobre espectrogramas: uno de dos etapas con propuestas de región, uno denso de una etapa (González et al., 2025) y uno de conjuntos con extractor preentrenado en imágenes.

La regla de decisión se fija antes de entrenar: gana la configuración con mayor valor de la métrica principal definida en la Fase 5, medida sobre la partición de *validación*; la partición de prueba se reserva para reportar el desempeño de la configuración ya elegida y no participa de la elección. La implementación se realiza en PyTorch con

código modular, separando la definición de la arquitectura, el bucle de entrenamiento y las utilidades.

Fase 5: Evaluación.

Los modelos entrenados se evalúan sobre la partición de prueba y los resultados se leen contra los criterios de éxito de la Fase 1. El análisis no se agota en una cifra agregada: identifica qué clases se detectan mejor y peor, qué tipos de error predominan, entre ellos la confusión entre especies acústicamente próximas, los falsos positivos por ruido y los errores de localización en tiempo o en frecuencia, y acompaña las cifras con ejemplos visualizados sobre el espectrograma.

Reportar varias métricas sin jerarquía permitiría elegir a posteriori la que favorezca al modelo preferido. Para evitarlo, la Tabla 1.9 asigna a cada una un rol fijado de antemano. Las métricas son las definidas al presentar los resultados esperados de OE2 (sección 1.5).

La **métrica principal** es el *recall* a nivel de evento, medido al punto de operación que la partición de validación elige para cada configuración: el umbral de confianza de mayor *recall* entre los que alcanzan una precisión de al menos 0,70. El *recall* manda por el compromiso de cobertura del objetivo general. Se acompaña siempre de su intervalo de confianza por remuestreo de grabaciones, y dos configuraciones cuyos intervalos se solapan se declaran no separadas por el experimento.

Un **umbral de confianza fijo y compartido** entre configuraciones no sirve para esta comparación: la confianza que emite cada cabeza depende de su función de pérdida y de cómo normaliza la puntuación, de modo que un mismo número cae en puntos distintos de cada curva y mediría calibración tanto como detección. Por eso cada modelo trae el suyo, elegido en validación, y viaja junto a su *checkpoint*.

El **criterio de selección de *checkpoint*** dentro de una misma corrida es la mAP a IoU 0,3 sobre validación: al no depender del umbral, elige la época que mejor detecta y no la mejor calibrada. Es también la **métrica secundaria** de la comparación entre configuraciones, y la que decide cuando la primaria no separa. El cociente **FP/TP al punto de operación** completa la Tabla 1.9 porque es la magnitud que conecta esta fase con OE3.

El acierto se decide a IoU 0,3 y el encuadre se puntúa aparte. Emparejar a 0,3 responde a si la llamada se *encontró*, que es lo que decide si el revisor la ve; exigir

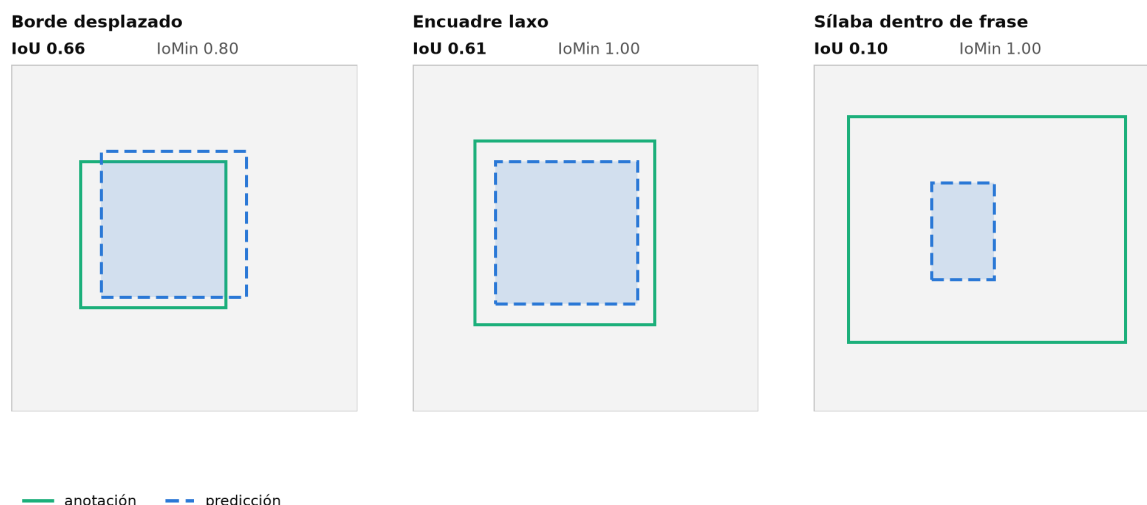
Tabla 1.9
Métricas de evaluación

Métrica	Definición	Rol
<i>Recall</i> por evento, al punto de operación	Proporción de eventos anotados recuperados por alguna predicción cuyo solapamiento con la caja anotada supera IoU 0,3, al umbral que la validación elige para ese modelo con precisión $\geq 0,70$. Se reporta con intervalo de confianza (IC).	Principal
mAP a IoU 0,3	Precisión media sobre las clases de detección, integrando toda la curva precisión–recall: no depende del umbral y por tanto no premia la calibración.	Secundaria y selección de <i>checkpoint</i>
FP/TP al punto de operación	Falsos positivos por cada verdadero positivo: la carga de revisión que el experto asume por cada acierto.	Costo de revisión (OE3)
IoU mediano del encuadre	Solapamiento entre la caja predicha y la anotada, sobre los eventos ya emparejados. Mide encuadre, no detección.	Encuadre
AP a IoU 0,25 y a IoU 0,5	La misma precisión media leída a un umbral de solapamiento laxo y a uno estricto: la distancia entre ambas mide cuánto se pierde al exigir encuadre.	Encuadre
Acierto sobre pares emparejados	Exactitud de la etiqueta de especie y tipo de llamada, condicionada a que la detección sea correcta.	Clasificación
Desempeño por clase	<i>Recall</i> individual para cada par de especie y tipo de llamada, con la causa atribuida a cada caída.	Diagnóstico

0,5 en el mismo paso mezclaría esa pregunta con la de si la caja quedó ajustada, cuando las fronteras del evento son borrosas incluso entre anotadores (Z. Zhu & Sato, 2025). El encuadre se puntúa después, sobre los pares ya emparejados, con el IoU mediano y con la AP a varios umbrales. La Figura 1.5 muestra por qué conviene separar ambas preguntas: compara tres predicciones con la anotación correspondiente, y junto al IoU da el IoMin, la intersección dividida por el área de la caja menor.

Los dos primeros casos, un borde desplazado y un encuadre laxo, superan los dos umbrales aunque la caja predicha no coincida con la anotada. El tercero, una sílaba predicha dentro de una frase anotada, queda muy por debajo de ambos aunque la predicción esté íntegramente contenida en la anotación (IoMin 1,00): es el anidamiento

Figura 1.5
Criterio de IoU



de unidades descrito en la causa C1, que ningún umbral de IoU resuelve y que por eso se trata en la construcción del conjunto de datos.

El punto de operación con que OE3 cronometra la revisión asistida sale de esta misma regla: es el umbral que la validación elige para el modelo seleccionado, y queda fijado antes de medir el tiempo.

Fase 6: Despliegue.

En un proyecto de investigación el despliegue consiste en aplicar el modelo final a un volumen de grabaciones que no participaron del entrenamiento ni de la evaluación, y en producir con ellas tablas de anotación automáticas que el analista pueda abrir directamente en Raven. El artefacto es un *pipeline* de inferencia que recibe un directorio de audio, aplica exactamente el mismo preprocesamiento del entrenamiento, obtiene las predicciones crudas, ejecuta el posprocesamiento y escribe la tabla de selección en el formato esperado.

1.7.2. Herramientas

Cada herramienta se justifica por el requisito que satisface y por la alternativa que se descarta, no por sus virtudes generales; donde no hay alternativa real, se dice. La Tabla 1.10 recoge las que intervienen en el recorrido del dato, desde la lectura del audio hasta la tabla que se abre en Raven.

Tabla 1.10

Stack técnico: pipeline

Herramienta	Función en el <i>pipeline</i>	Por qué esta y no la alternativa
Python 3.12	Lenguaje de todo el sistema	R y MATLAB carecen de implementaciones de referencia de detectores de objetos, que es el componente que no se va a reimplementar
Pandas	Consolidación, limpieza y particionado de las anotaciones tabulares	Procesar los archivos de Raven con scripts <i>ad hoc</i> no deja registro reproducible de qué filas se descartaron y por qué
NumPy	Representación en memoria del audio y de las cajas antes de pasar a tensores	Sin alternativa práctica: es la interfaz común entre SoundFile, PyTorch y Matplotlib
SoundFile y soxr	Lectura por bloques de las grabaciones y remuestreo a 44,1 kHz	Permiten leer solo la ventana pedida de un archivo de una hora, en lugar de cargarlo entero en memoria
torchaudio	Cálculo del espectrograma mel que consume el modelo	Librosa devuelve arreglos de NumPy y obligaría a convertir en cada lote; aquí la transformada entrega tensores que van directamente al caché y al modelo
PyTorch	Definición, entrenamiento y evaluación del detector	Las implementaciones de referencia de detectores están mayoritariamente en PyTorch, y portarlas a otro marco no se justifica
Transformers	Carga del <i>backbone</i> AST preentrenado en AudioSet	Entrenar un transformador de espectrogramas desde cero excede los datos disponibles; la biblioteca aporta los pesos y la interpolación del <i>embedding</i> posicional
Raven Pro / Lite	Origen de las anotaciones y destino de las predicciones exportadas	Sin alternativa: es la herramienta del analista, y su formato de tabla de selección es un requisito de RE2.4, no una elección de diseño

De todas, solo Raven viene impuesta por el problema: su tabla de selección es el formato en que el analista recibe las predicciones. Las demás responden sobre todo a dos criterios: dejar registro reproducible (Pandas) y no hacer trabajo redundante sobre el dato, es decir, leer solo la ventana pedida (SoundFile y soxr), no convertir en cada lote (torchaudio) y reutilizar implementaciones y pesos de referencia (PyTorch y Transformers). La Tabla 1.11 reúne lo que no toca el dato pero fija el entorno y el estado del código con que se obtuvo cada resultado.

Tabla 1.11

Stack técnico: desarrollo

Herramienta	Función	Por qué esta y no la alternativa
uv	Entorno virtual, resolución de dependencias y archivo de bloqueo	Instalar con <i>pip</i> sin bloqueo deja libres las versiones transitivas, y sin ellas el entorno de una corrida antigua no se reconstruye igual
Matplotlib	Figuras del análisis exploratorio y de los resultados	Sin alternativa que aporte algo distinto dentro del mismo ecosistema
Ruff, ty y pytest	Formato, análisis estático y pruebas del código	Las pruebas cubren la conversión entre sistemas de coordenadas de caja, donde un error silencioso desplazaría todas las métricas sin fallar
Git	Versionado del código y de la configuración de cada experimento	Sin alternativa: es la condición para asociar un resultado reportado a un estado exacto del código

El criterio común a ambas es la reproducibilidad: el conjunto de datos y cada cifra del documento pueden regenerarse a partir del material primario, y la subsección siguiente describe los procedimientos que lo garantizan.

1.7.3. Procedimientos y reproducibilidad

Las fases anteriores describen qué se hace con los datos y por qué. Esta subsección recoge únicamente lo operativo que no pertenece a ninguna fase en particular porque atraviesa todas: cómo se custodia el material, cómo se registran los experimentos y qué garantiza que un número reportado pueda volver a obtenerse.

Custodia del material primario. Los audios y sus anotaciones se mantienen bajo una estructura de directorios fija que no se edita nunca a mano. Toda transformación

se produce por script a partir de ella y escribe en un directorio derivado distinto. Esa separación es la que permite reconstruir el conjunto de trabajo desde cero cada vez que el protocolo de curación cambia, y la que impide que una corrección puntual quede sin registro.

Registro de experimentos. Cada corrida queda asociada a un identificador de configuración y a un *commit* del repositorio. El bucle de entrenamiento registra los hiperparámetros, guarda *checkpoints* de forma periódica y evalúa sobre validación a intervalos fijos. Los scripts de evaluación son independientes de los de entrenamiento: cargan un *checkpoint* existente y recalculan las métricas, de modo que reproducir una cifra publicada no exige reentrenar.

Respaldo y portabilidad. El código y la configuración se sincronizan en un repositorio remoto, y el entorno de ejecución se declara en un archivo de dependencias versionado, para que la falla del equipo de desarrollo o la ejecución en una máquina distinta no supongan rehacer trabajo. La gestión de la bibliografía se centraliza en Zotero a lo largo de todo el proceso.

La planificación que ordena todo lo anterior en el tiempo, mediante la estructura de desglose del trabajo, el cronograma, el presupuesto y la gestión de riesgos, se recoge en el Anexo A, fuera del cuerpo, porque no aporta al argumento científico y en el hilo del documento lo interrumpiría.

Capítulo 2

Marco Referencial

El Capítulo 1 planteó el problema y los objetivos en términos del dominio: grabaciones, anotaciones y tiempo de especialista. Este capítulo fija los conceptos que esos objetivos dan por sentados. El marco teórico formaliza la tarea como detección de eventos sobre el espectrograma y precisa qué significa predecir una caja tiempo–frecuencia. El marco conceptual define los términos que los capítulos de resultados usan sin volver a explicarlos: paisaje sonoro, evento acústico, representación tiempo–frecuencia, detección de eventos sonoros, clasificación jerárquica y anotación inconsistente.

2.1. Marco Teórico

2.1.1. Formulación del problema de detección

El problema se plantea en dos niveles. El nivel de la *grabación* es el que le interesa al analista y define el producto; el nivel de la *ventana* es el que consume el modelo y define su entrada y su salida. Separarlos es necesario porque ninguna arquitectura actual procesa una hora de audio de una sola vez, y confundirlos oscurece qué predice realmente un detector.

Nivel de grabación

El sistema recibe un conjunto de grabaciones $G = \{g_1, g_2, \dots, g_M\}$. Cada grabación g_k es una señal de audio $x(t)$ en formato WAV, muestreada a $F_s = 44,1$ kHz, de duración D_k que puede llegar a la hora. Para cada una el sistema debe devolver una lista de eventos $E_k = \{e_1, e_2, \dots, e_{N_k}\}$, cuyo cardinal N_k no se conoce de antemano y donde cada evento es la tupla

$$e_i = \left(t_{\text{ini}}^{(i)}, t_{\text{fin}}^{(i)}, f_{\text{mín}}^{(i)}, f_{\text{máx}}^{(i)}, y^{(i)}, p^{(i)} \right). \quad (2.1)$$

$(t_{\text{ini}}, t_{\text{fin}})$ Instantes de inicio y fin del evento, en segundos desde el comienzo de la grabación, con $0 \leq t_{\text{ini}} < t_{\text{fin}} \leq D_k$.

$(f_{\text{mín}}, f_{\text{máx}})$ Banda de frecuencia que ocupa el evento, en hercios, con $0 \leq f_{\text{mín}} < f_{\text{máx}} \leq F_s/2$. Es la componente que distingue esta formulación de la detección de eventos sonoros puramente temporal.

y Etiqueta tomada de un vocabulario cerrado $Y = \{y_1, \dots, y_L\}$ cuyos elementos son pares de especie y tipo de llamada. La sección 2.2.5 justifica por qué la etiqueta es un par plano y no dos niveles predichos por separado; las proyecciones $\pi_{\text{esp}}(y)$ y $\pi_{\text{tipo}}(y)$ recuperan cada nivel cuando se lo necesita.

p Confianza en $[0, 1]$ asignada por el modelo, que ordena las predicciones y define el punto de operación de la revisión asistida.

Las cuatro primeras componentes definen una caja rectangular sobre el plano tiempo–frecuencia, la misma que el especialista dibuja en Raven (Figura 1.1); las dos restantes la etiquetan y la puntúan.

Nivel de ventana

La grabación se recorta en ventanas de duración fija T_w , tomadas cada H segundos con $H < T_w$, de modo que ningún evento quede siempre partido por un borde. Cada ventana se convierte en un espectrograma mel $X \in \mathbb{R}^{F \times T}$, con F bandas de frecuencia y T tramas temporales, y ese tensor es la entrada del modelo.

El detector es una función de parámetros θ que a cada ventana le asigna un conjunto de tamaño fijo

$$f_{\theta}(X) = \left\{ (\hat{b}_q, \hat{\pi}_q) \right\}_{q=1}^Q, \quad \hat{b}_q \in [0, 1]^4, \quad \hat{\pi}_q \in [0, 1]^{L+1}, \quad (2.2)$$

donde Q es un número de consultas fijado por la arquitectura, independiente del contenido de la ventana y mayor que la cantidad de eventos que cabe esperar en ella. Cada consulta produce una caja $\hat{b}_q = (c_x, c_y, w, h)$ en coordenadas normalizadas al recuadro de la ventana (centro, ancho y alto, los cuatro en $[0, 1]$) y una distribución $\hat{\pi}_q$ sobre $Y \cup \{\emptyset\}$, en la que $\emptyset$ es la clase “sin objeto” que absorbe las consultas sobrantes.

El número de eventos no se predice: resulta de cuántas consultas eligen una clase de Y por encima del umbral de confianza.

Esta es la formulación de *predicción de conjuntos*: la salida no es una rejilla puntuada ni una secuencia, sino un conjunto sin orden, y durante el entrenamiento cada anotación de la ventana se asigna a exactamente una consulta mediante un emparejamiento biyectivo, mientras que el resto se supervisa hacia $\emptyset$. De ahí que dentro de la ventana no haya anclas ni supresión de no máximos.

Del marco normalizado al marco físico

Lo que emite el modelo no son segundos ni hercios sino fracciones de la ventana, y la conversión no es simétrica entre los dos ejes. En tiempo es afín: si la ventana empieza en t_0 ,

$$t_{\text{ini}} = t_0 + \left(c_x - \frac{w}{2}\right)T_w, \quad t_{\text{fin}} = t_0 + \left(c_x + \frac{w}{2}\right)T_w. \quad (2.3)$$

En frecuencia no lo es, porque el eje vertical del espectrograma está en escala mel. Con $m(\cdot)$ la función que convierte hercios a mel y $[f_{\text{lo}}, f_{\text{hi}}]$ los extremos del banco de filtros, la coordenada normalizada $u \in [0, 1]$ vuelve a hercios mediante

$$\varphi(u) = m^{-1}\left(m(f_{\text{lo}}) + u[m(f_{\text{hi}}) - m(f_{\text{lo}})]\right), \quad (2.4)$$

y entonces $f_{\text{mín}} = \varphi(c_y - h/2)$ y $f_{\text{máx}} = \varphi(c_y + h/2)$. La consecuencia práctica es que una caja de altura constante en el marco normalizado cubre una banda mucho más ancha en la parte alta del espectro que en la baja.

De las ventanas a la grabación

Como las ventanas se solapan, una misma vocalización puede predecirse en dos o tres de ellas, y una vocalización más larga que T_w solo puede predecirse por partes. Las predicciones de toda la grabación se llevan al marco común, se agrupan por clase y se depuran por supresión de no máximos, de modo que la lista E_k contenga un evento por vocalización. Ese es el punto en el que el nivel de ventana devuelve el control al nivel de grabación, y también el punto en el que el ventaneo deja su huella sobre los eventos más largos que la ventana.

2.1.2. El espectrograma y la asimetría de sus ejes

La señal de audio es unidimensional, pero los eventos de interés se distinguen por cómo se distribuye su energía entre frecuencias a lo largo del tiempo. La transformada de Fourier de tiempo corto (STFT) recupera esa estructura: divide la señal en tramas solapadas, aplica una ventana a cada una y calcula su espectro. La magnitud del resultado es el espectrograma, una matriz $X \in \mathbb{R}^{F \times T}$ cuyos parámetros, como el tamaño de ventana, el salto entre ventanas y el tipo de ventana, fijan el compromiso entre resolución temporal y resolución frecuencial.

Interesa aquí una asimetría del espectrograma que conviene explicitar porque condiciona el diseño del detector. Tratarlo como una imagen es útil, pero sus dos ejes no son equivalentes: una traslación en tiempo preserva el significado del patrón, mientras que una traslación en frecuencia lo cambia: un mismo contorno desplazado media octava es otra vocalización, o ninguna. Los patrones espectro-temporales de los eventos sonoros dependen de la banda de frecuencia en que ocurren, de modo que la convolución bidimensional estándar, equivariante a traslación en ambos ejes, no es adecuada para el eje frecuencial (Nam et al., 2022). La consecuencia práctica es que el modelo debe conservar de forma explícita la posición en frecuencia en lugar de descartarla.

2.1.3. Detección de objetos sobre audio

Una vez que el sonido se representa como una matriz tiempo-frecuencia, la tarea de la ecuación (2.1) coincide con la de la detección de objetos en visión por computadora: afirmar que una clase está presente y localizarla, localización que se representa típicamente mediante una caja delimitadora (Lin et al., 2014). La analogía no es superficial. Los eventos acústicos se manifiestan como patrones bidimensionales de tiempo y frecuencia, con texturas y estructuras geométricas propias, análogos a los objetos que aparecen en imágenes naturales (Pham et al., 2018), y el encuadre trasciende la bioacústica hacia el análisis de señales en general (Wood et al., 2022).

Familia R-CNN. Los detectores de dos etapas proponen primero regiones candidatas y luego las clasifican y refinan. Sobre espectrogramas, un detector de dos etapas localiza llamadas de aves migratorias nocturnas (Airale et al., 2025), y DeepSqueak, construido sobre Faster R-CNN, se ha usado con las vocalizaciones de un primate en cautiverio (Romero-Mujalli et al., 2021).

Familia YOLO. Los detectores de una etapa, cuyo representante más difundido es YOLO (*You Only Look Once*), predicen clase y caja en una sola pasada sobre una rejilla, lo que los hace más rápidos y más simples de entrenar. Su uso sobre espectrogramas está reportado como línea base competitiva (González et al., 2025), y YOLOv5 se ha aplicado a vocalizaciones de foca barbuda (Escobar-Amado et al., 2024). La misma idea de una sola pasada inspira a YOHO, que la aplica al eje temporal (Venkatesh et al., 2022).

Familia DETR y transformadores. Los detectores basados en atención eliminan las anclas y la supresión de no máximos, y formulan la detección como la predicción directa de un conjunto de objetos emparejado con las anotaciones. Su adaptación al audio es reciente: SoundDet trata el audio crudo como una señal espacio-temporal y detecta eventos móviles sobre ella (Y. He et al., 2021), y Z. Zhu y Sato (2025) combinan un transformador de espectrograma autosupervisado con un decodificador deformable de esta familia para detectar cajas tiempo–frecuencia, el precedente directo de la arquitectura propuesta en esta tesis.

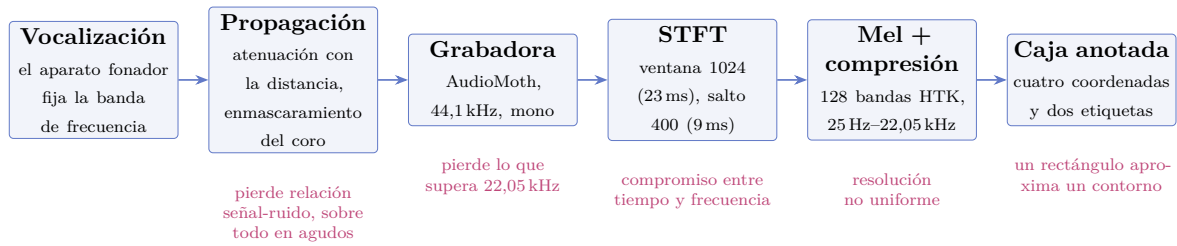
Cajas frente a máscaras. Una alternativa a la caja es la segmentación semántica, que asigna una etiqueta a cada celda tiempo–frecuencia y describe con más fidelidad los contornos de una vocalización (Jin et al., 2022). Esta tesis opta por la caja por dos razones: es la forma en que el analista anota en Raven, y por tanto la única para la que existe una referencia disponible; y es la que se exporta sin pérdida a una tabla de selección. Una máscara exigiría reanotar el corpus completo.

2.2. Marco Conceptual

Los conceptos que siguen no se presentan por área temática sino siguiendo el recorrido de la señal, desde que el animal vocaliza hasta que el analista recibe una etiqueta revisable. La Figura 2.1 dibuja ese recorrido como una cadena de seis eslabones.

Debajo de cada eslabón figura lo que se pierde en él: la propagación resta relación señal-ruido, sobre todo en los agudos; la grabadora descarta lo que supera 22,05 kHz; la STFT obliga a elegir entre resolución temporal y frecuencial; la escala mel reparte la resolución de forma no uniforme, y la caja aproxima con un rectángulo un contorno que no lo es. Cada pérdida crea la necesidad del eslabón siguiente, y por eso las subsecciones siguen el orden en que aparecen los problemas.

Figura 2.1
Cadena de la señal



2.2.1. Paisaje sonoro

La bioacústica estudia el sonido en los seres vivos, y esta tesis la aplica a través del monitoreo acústico pasivo descrito en la sección 1.1.1. Lo que la grabadora registra no es la vocalización sino la suma de todo lo que llega al micrófono, el paisaje sonoro del ecosistema (Blumstein et al., 2011), y en un entorno como la Amazonía esa suma tiene tres componentes:

Biofonía. Los sonidos de origen biológico. Incluye la señal de interés, las vocalizaciones de primates, y también el coro de insectos, anfibios y aves, que ocupa las mismas bandas durante horas seguidas.

Geofonía. Los sonidos de origen físico: viento, lluvia, movimiento de la vegetación. Su energía se reparte sobre bandas amplias y su carácter es o bien cuasi-estacionario o bien de ráfaga.

Antropofonía. Los sonidos de origen humano: motosierras, disparos, motores. No solo interfieren con la señal, sino que son en sí mismos un indicador de presión sobre el ecosistema (Blumstein et al., 2011).

A ello se suma la atenuación: el animal está a una distancia desconocida del sensor, de modo que la misma vocalización llega unas veces nítida y otras apenas por encima del fondo. El archivo resultante contiene, entonces, una señal mezclada, atenuada y de relación señal-ruido variable dentro de una misma grabación. La pregunta que abre el eslabón siguiente es qué hay que buscar dentro de esa mezcla, es decir, qué constituye exactamente un evento.

2.2.2. Eventos acústicos

La vocalización se produce por el flujo de aire desde los pulmones, que hace vibrar las cuerdas vocales en la laringe y actúa como fuente sonora primaria; el tracto vocal supralaríngeo, formado por la faringe y las cavidades oral y nasal, filtra esa fuente y fija sus características finales: frecuencia fundamental, distribución de armónicos, amplitud, duración y timbre. Es la teoría fuente-filtro, que la investigación sobre comunicación vocal de mamíferos aplica de forma general (Taylor & Reby, 2010).

La consecuencia de esa estructura fuente-filtro es la que interesa aquí. La frecuencia fundamental queda determinada por la geometría y la tensión de las cuerdas vocales, y las resonancias del tracto vocal concentran la energía en un conjunto acotado de bandas. Un evento no es, por tanto, un instante con una etiqueta: es una región del plano tiempo-frecuencia con un límite inferior y un límite superior de frecuencia que dependen de la anatomía de quien vocaliza. La banda que ocupa una especie es una propiedad física de su aparato fonador, no una convención de dibujo del anotador. Esa es la razón por la que la caja tiempo-frecuencia de la ecuación (2.1) es la representación adecuada del evento, y no un rectángulo elegido por comodidad: los cuatro números que la definen tienen referente físico.

La misma estructura explica dos hechos que reaparecen más adelante. El primero es que distintas especies tienden a separarse por banda, lo que hace de la frecuencia una característica discriminante y no un dato accesorio. El segundo es que la estructura de las vocalizaciones de los primates no humanos está muy conservada dentro de cada especie, pero admite ajustes dentro de esos límites, como los rasgos propios de cada grupo (Fischer & Hammerschmidt, 2020); esa variabilidad amplía el rango de formas que el detector debe reconocer como la misma clase.

El evento existe, entonces, como región bidimensional. El problema es que la señal grabada es unidimensional: hay que recuperar esa estructura antes de poder buscarla.

2.2.3. Representaciones tiempo-frecuencia

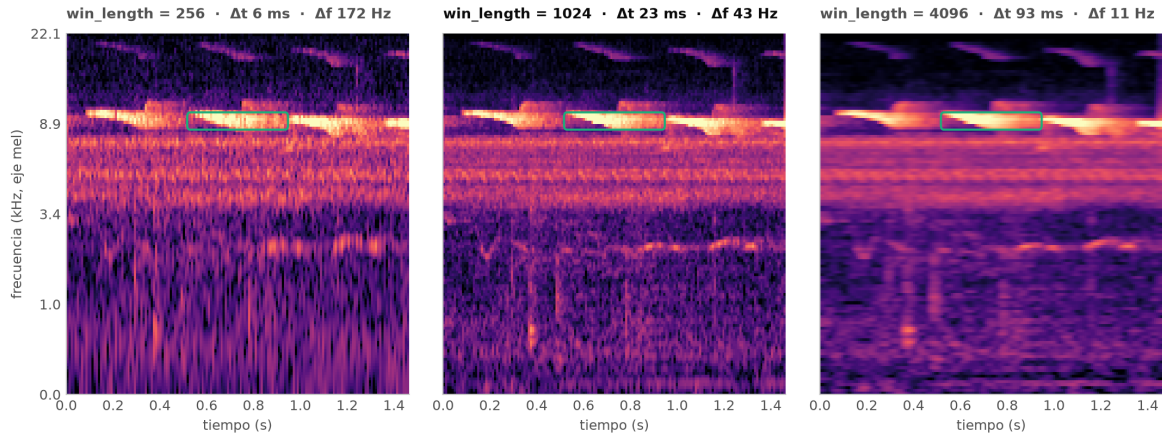
La STFT, descrita en la sección 2.1.2, recupera la dimensión que falta. Sobre la matriz que produce se aplican dos transformaciones habituales: la escala mel, que reagrupa las bandas según una progresión aproximadamente logarítmica, y la compresión logarítmica de la magnitud a decibelios. El resultado, el espectrograma log-mel, es la

entrada estándar en detección de eventos sonoros, y es la transformación que convierte un problema de audio en uno de análisis de imágenes.

Ninguno de esos pasos es gratuito. El primero es la longitud de la trama de la STFT, que fija un compromiso entre las dos resoluciones: la Figura 2.2 pinta la misma secuencia de sílabas del tamarino de Weddell con tres longitudes de trama.

Figura 2.2

Compromiso de la STFT



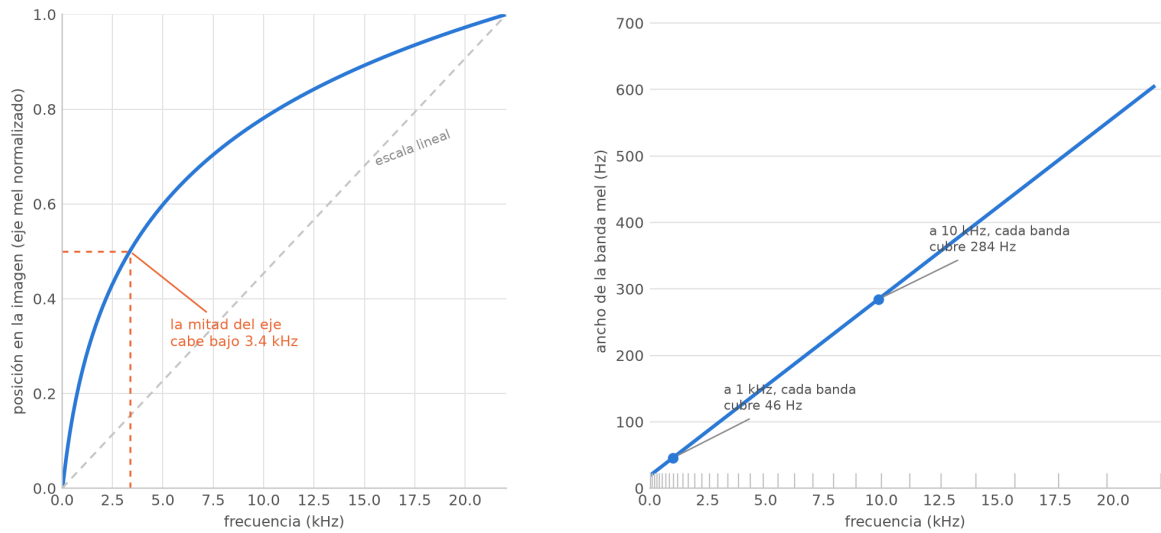
Con la trama más corta (256 muestras, 6 ms) cada sílaba queda nítida en el tiempo pero se emborrona en frecuencia, con una resolución de 172 Hz; con la más larga (4096 muestras, 93 ms) ocurre lo contrario, y los bordes de inicio y fin se difuminan. La trama intermedia de 1024 muestras, la que usa esta tesis, conserva a la vez el contorno de la sílaba y sus fronteras.

El segundo paso es la escala mel, que reparte la resolución de forma no uniforme entre graves y agudos (Figura 2.3).

La mitad del eje vertical queda por debajo de 3,4 kHz, y el ancho de cada banda crece con la frecuencia: 46 Hz a 1 kHz y 284 Hz a 10 kHz. Para este corpus la consecuencia es concreta: las llamadas agudas, como las de los tamarinos, quedan comprimidas en pocas bandas, y una caja de altura fija en el marco normalizado cubre en los agudos una banda mucho más ancha que en los graves (ecuación 2.4).

La elección de la representación no es un trámite. La compresión logarítmica no es la única opción: la normalización de energía por canal (PCEN) combina control adaptativo de ganancia y compresión de rango dinámico, y en detección bioacústica de campo fue el contribuyente principal a la mejora de precisión sobre la línea base log-mel (Lostanlen et al., 2019). Su ventaja depende, sin embargo, de una condición del

Figura 2.3
Escala mel



ruido: proviene de estimar y descontar un fondo que varía lentamente respecto de la escala temporal del evento. El coro de insectos amazónico cumple esa condición, pues es cuasi-estacionario durante minutos, pero la lluvia y las ráfagas de viento no la cumplen, y sobre ellas el control de ganancia puede atenuar la propia vocalización junto con el fondo.

Esta tesis adopta el log-mel. El extractor de la arquitectura propuesta, el *Audio Spectrogram Transformer* (AST), se preentrenó sobre espectrogramas log-mel (Gong et al., 2021), y ese preentrenamiento solo se aprovecha si la entrada conserva la misma representación; las líneas base, preentrenadas en imágenes, reciben el mismo log-mel escalado como imagen. La Figura 2.4 muestra por qué la compresión es imprescindible: pinta la misma ventana sin comprimir y en log-mel.

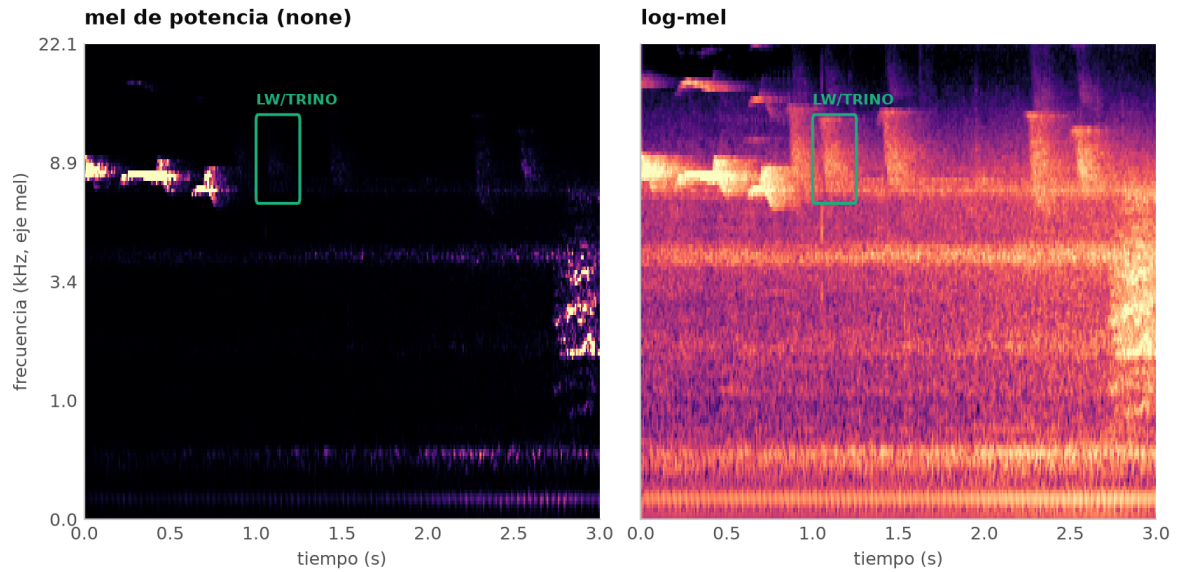
La caja marca un trino del tamarino de Weddell (LW/TRINO). Sin compresión, el mel de potencia solo muestra los sonidos más intensos y el trino casi desaparece; el log-mel lo hace visible, pero también el coro de fondo, que ocupa bandas enteras durante toda la ventana. Queda así una imagen en la que el evento es visible; falta el mecanismo que lo localice dentro de ella.

2.2.4. Detección de eventos sonoros

La detección de eventos sonoros (SED) consiste en identificar la presencia de un evento y localizar sus límites sobre la representación tiempo-frecuencia. La distinción

Figura 2.4

Mel de potencia y log-mel sobre la misma ventana



decisiva dentro de esta tarea es la *forma de la salida*, con la que la sección 1.3.1 ordenó las soluciones existentes (Figura 1.3). Expresada en los términos de la ecuación (2.1), la gradación es la siguiente: la salida por segmento fijo cuantiza t_{ini} y t_{fin} al tamaño del segmento y no estima la banda de frecuencia; la salida temporal estima t_{ini} y t_{fin} , pero descarta $f_{\text{mín}}$ y $f_{\text{máx}}$, que al tratar los eventos acústicos se establecieron como magnitudes físicas del aparato fonador, y solo la salida tiempo–frecuencia emite la tupla completa. Es la única que coincide con lo que el analista produce a mano en Raven y, por tanto, la única que puede sustituir su insumo en lugar de complementarlo. Esa propiedad explica por qué los detectores existentes, que no emiten la tupla completa, no alivian la búsqueda que describe la causa C2 del árbol de problemas.

El problema impone seis exigencias, y cada familia de la literatura responde a una de ellas. Presentarlas en orden cronológico no dice nada; presentarlas por la exigencia que satisfacen permite decidir, y así las ordena la Tabla 2.1. Las vocalizaciones de primates se organizan en secuencias cuya identidad depende del orden y del espaciado de sus unidades, lo que explica por qué el contexto largo aparece como exigencia propia y no como un refinamiento del reconocimiento local.

El criterio de descarte es la forma de la salida, no el desempeño reportado. Una arquitectura solo es candidata a arquitectura final si su cabeza puede emitir $f_{\text{mín}}$ y $f_{\text{máx}}$. Bajo esa regla, las cuatro primeras familias de la Tabla 2.1 quedan descartadas *como sistema completo*, porque su formulación habitual termina en una probabilidad por

Tabla 2.1*Arquitecturas frente a requisitos*

Exigencia	Familia	Qué aporta y dónde se queda corta
Reconocer un patrón espectro-temporal local	Redes convolucionales	Aplican filtros sobre vecindades del espectrograma y reconocen patrones locales, pero por sí solas no sostienen el contexto de una secuencia ni emiten cajas
Sostener el contexto a lo largo de una frase	Convolucionales recurrentes	Añaden capas recurrentes sobre las características convolucionales para sostener el contexto temporal; su salida sigue siendo una probabilidad por trama
Ganar profundidad sin degradación	Conexiones residuales	Permiten entrenar redes muy profundas sin pérdida de desempeño (K. He et al., 2016)
Modelar dependencias a larga distancia sin recurrencia	Transformadores	La autoatención relaciona en paralelo todas las posiciones del espectrograma; el <i>Audio Spectrogram Transformer</i> lleva esa idea al audio, pero produce representaciones de una sola escala (Z. Zhu & Sato, 2025)
Preservar la posición en frecuencia	Ninguna de las anteriores por sí sola	La convolución bidimensional estándar es equivariante a traslación en ambos ejes, propiedad deseable en tiempo y errónea en frecuencia; corregirlo exige adaptar el filtro a la banda (Nam et al., 2022)
Emitir $f_{\min}$ y $f_{\max}$	Cabezas de detección de objetos	Única salida que coincide con la tupla de la ecuación (2.1); es la familia descrita en la sección 2.1.3

trama o por segmento; ninguna queda descartada *como extractor de características*, que es el papel en el que siguen siendo pertinentes. La cabeza que sí satisface la exigencia es la de los detectores de objetos descritos en la sección 2.1.3, y por eso la arquitectura de esta tesis combina un extractor de audio con una cabeza de detección, en lugar de adoptar una de las cuatro familias tal como se emplean en la literatura de SED.

Localizado el evento, queda el problema de nombrarlo.

2.2.5. Clasificación jerárquica

La etiqueta que el analista necesita contiene dos niveles anidados, la especie y, dentro de su repertorio, el tipo de llamada. Las unidades acústicas se agrupan en notas, trinos, sílabas, frases, motivos y cantos, y pueden estar jerárquicamente anidadas: una

secuencia de unidades puede considerarse ella misma una unidad y ser reemplazada por una etiqueta única (Kershenbaum et al., 2016).

El primer nivel se define sobre las ocho especies de la Tabla 1.1, identificadas por su código de dos letras.

El segundo nivel de la etiqueta se define dentro de cada especie, sobre el vocabulario de la Tabla 2.2. Los repertorios están desbalanceados, de modo que los 41 pares posibles no se reparten de forma pareja entre las ocho especies, y esa asimetría es la que condiciona la elección del esquema de clasificación.

Tabla 2.2
Repertorio por especie

Especie	Tipos de llamada (código: nombre)	N.º
AA <i>Aotus azarae</i>	gc: <i>gulp</i> ; hm: <i>hoot</i> ; sc: <i>squeak</i>	3
AC <i>Ateles chamek</i>	chc: <i>chitter</i> ; gc: <i>growl</i> ; cc: <i>contact</i> ; whc: <i>whinnie</i> ; sc: <i>squeak</i> ; bc: <i>bark</i>	6
AS <i>Alouatta sara</i>	hc: <i>howl</i> ; bc: <i>bark</i>	2
CC <i>Cebus cuscinus</i>	cc: <i>contact</i>	1
LW <i>Leontocebus weddelli</i>	cc: <i>contact</i> ; cs: <i>contact syllable</i> ; aa: <i>aerial alarm</i> ; ta: <i>terrestrial alarm</i> ; sqc: <i>squeal</i> ; phc: <i>phee</i> ; tr: <i>trino</i> ; tf: <i>trino fast</i> ; tt: <i>trino transition</i> ; tj [†] sin denominación; vc: <i>visual contact</i>	11
PT <i>Plecturocebus toppini</i>	dc: <i>duet</i> ; ac: <i>alarm</i> ; bp [†] <i>bellow phrase</i> ; pp [†] <i>pant phrase</i> ; sqc [†] <i>squeal</i>	5
SB <i>Saimiri boliviensis</i>	pcc: <i>peep contact</i> ; ppc: <i>play peep</i> ; lpc: <i>long peep</i> ; spc: <i>spit</i> ; sc: <i>shriek</i>	5
SM <i>Sapajus macrocephalus</i>	cc: <i>contact</i> ; acc: <i>aggressive contact</i> ; sc: <i>squeal</i> ; pc: <i>purrr</i> ; whc: <i>whistle</i> ; hic: <i>hip</i> ; fc: <i>food</i> ; fs: <i>food syllable</i>	8
Total		41

[†] Categorías sin nombre establecido en la literatura del repertorio, acuñadas por el equipo de anotación. El Capítulo 4 documenta cuáles de ellas sobreviven al vocabulario cerrado y cuáles se descartan por ser unidades de nivel de frase.

Un mismo código puede designar llamadas distintas en especies distintas: *sc* es *squeak* en AA y AC, *shriek* en SB y *squeal* en SM. Por eso la unidad de etiquetado en

el resto del documento es siempre el par, escrito **especie/tipo**, y nunca el código de llamada por sí solo.

Ante esa estructura caben tres esquemas. La *clasificación plana* crea una clase por cada par y trata la jerarquía como inexistente. El enfoque *local por nodo padre* entrena un clasificador de especie y, debajo, un clasificador de tipo por cada especie. El *modelo global jerárquico* aprende la estructura completa de una vez. Esta tesis adopta la clasificación plana sobre un vocabulario cerrado de pares: los esquemas jerárquicos exigen un número mínimo de ejemplos en cada nodo hijo, y el desbalance del corpus deja varios pares con demasiados pocos como para sostener un clasificador propio. El Capítulo 4 documenta esa composición y cierra el vocabulario en consecuencia.

Queda un supuesto implícito en todo lo anterior: que la etiqueta de referencia contra la cual se entrena y se mide es un dato fijo. No lo es.

2.2.6. Anotaciones inconsistentes

Un sistema supervisado se entrena y se evalúa contra anotaciones humanas, de modo que las propiedades de esas anotaciones forman parte del planteamiento y no del apartado de limitaciones. Tres conceptos las describen.

Etiqueta fuerte y etiqueta débil. Una etiqueta *débil* declara que una clase está presente en algún lugar de la grabación, sin decir dónde; es lo que ofrecen los repositorios grandes de audio de fauna. Una etiqueta *fuerte* delimita el evento: aporta sus fronteras además de su clase. El material de esta tesis es de etiqueta fuerte y bidimensional, pues delimita también en frecuencia, lo que lo hace más informativo y, a la vez, más costoso de producir y más expuesto a desacuerdo, porque hay más decisiones que tomar por evento.

Acuerdo entre anotadores y consigo mismo. El acuerdo *inter-anotador* mide cuánto coinciden dos analistas que anotan el mismo audio; el *intra-anotador*, cuánto coincide un mismo analista consigo mismo al anotar dos veces el mismo segmento. Ambos son bajos en bioacústica, con las magnitudes que reunió el análisis de la causa C1 (sección 1.2.2).

Frontera borrosa del evento. El desacuerdo anterior no se reparte de forma uniforme: se concentra en los bordes. Una vocalización no empieza ni termina en un instante definido, sino que emerge del fondo y se desvanece en él, y lo mismo ocurre en

frecuencia con los armónicos superiores. En detección de cajas tiempo–frecuencia se observan desplazamientos relativamente grandes entre la caja predicha y la anotada, de lo que se concluye que las fronteras de los eventos son borrosas y difíciles de determinar (Z. Zhu & Sato, 2025).

Estos tres conceptos dan forma conceptual a la causa C1 del árbol de problemas: a la heterogeneidad de las anotaciones y a la referencia incompleta que impide medir los errores sin un especialista. También fundamentan las dos decisiones de evaluación que fijó la sección 1.7.1: emparejar con un IoU bajo evita medir el acuerdo con un borde que el propio anotador no reproduce, y priorizar el *recall* evita descartar como error lo que puede ser un evento válido ausente de una referencia incompleta.

Capítulo 3

Estado del Arte

Definida la tarea en el Capítulo 2, este capítulo revisa cómo la ha abordado la literatura reciente. Primero declara la metodología de la revisión (preguntas, búsqueda y criterios de selección); después organiza los resultados por pregunta, para comprobar si el vacío que identificó la sección 1.3.2 se sostiene frente a una búsqueda sistemática.

3.1. Metodología de la revisión

3.1.1. Objetivos y preguntas de revisión

La revisión busca identificar y evaluar el estado del arte en técnicas de aprendizaje profundo aplicables a una solución automatizada que produzca anotaciones detalladas, que incluyen localización tiempo–frecuencia, identificación de especie y clasificación del tipo de vocalización, sobre grabaciones de monitoreo acústico pasivo de primates amazónicos. Se organiza en torno a cinco preguntas:

- P1.** ¿Qué estrategias de aprendizaje profundo han obtenido mejores resultados en la detección tiempo–frecuencia de vocalizaciones en grabaciones de monitoreo acústico pasivo capturadas en entornos ruidosos?
- P2.** ¿Qué estrategias han obtenido mejores resultados en la clasificación jerárquica fina de los eventos detectados, identificando de forma simultánea la especie y el tipo específico de vocalización?
- P3.** ¿Qué técnicas de procesamiento resultan más efectivas para mitigar el ruido ambiental y la variabilidad propia de sensores de bajo costo como AudioMoth?

- P4.** ¿Qué conjuntos de datos públicos existen con vocalizaciones etiquetadas, qué formatos de anotación emplean, y qué estrategias de aumento de datos compensan mejor la escasez y el desbalance de clases?
- P5.** ¿Cómo se adaptan mediante transferencia de aprendizaje los modelos preentrenados en audio general o en bioacústica de aves a las tareas de detección (P1) y clasificación jerárquica (P2)?

3.1.2. Estrategia de búsqueda

La búsqueda se ejecutó sobre dos bases indexadas. Scopus cubre publicaciones seriadas de más de cinco mil editores en 140 países e incorpora una herramienta de búsqueda asistida que agrupa los artículos relacionados con una consulta. Web of Science aporta herramientas de visualización sobre la relevancia y las tendencias de las revistas dentro de sus categorías temáticas. La Tabla 3.1 recoge las cadenas empleadas y el número de documentos recuperado en cada motor.

Tabla 3.1
Cadenas de búsqueda

Motor	Cadena de búsqueda	Docs.
Scopus	TITLE-ABS-KEY (<i>deep learning</i> OR <i>neural network*</i> OR <i>artificial intelligence</i> OR <i>transformer*</i> OR <i>machine learning</i>) AND TITLE-ABS-KEY (<i>sound event detection</i> OR <i>acoustic event detection</i> OR <i>audio event localization</i> OR <i>bioacoustic detection</i>) AND TITLE-ABS-KEY (<i>biodiversity</i> OR <i>wildlife</i> OR <i>animal vocalizations</i> OR <i>primates</i> OR <i>anthropogenic noise</i> OR <i>forest monitoring</i> OR <i>ecological monitoring</i>)	33
Web of Science	TS = (mismos tres bloques de términos, unidos por AND)	6
Total		39

Nota. Los tres bloques combinan técnica, tarea y dominio de aplicación. El asterisco es el comodín de truncamiento de ambos motores.

Scopus aporta la mayor parte de los documentos. El total es reducido porque la cadena exige a la vez los tres bloques de términos, y los criterios de cribado lo acotan todavía más.

3.1.3. Criterios de inclusión y exclusión

Los resultados se cribaron con los criterios de la Tabla 3.2.

Tabla 3.2

Criterios de inclusión y exclusión

Criterio	Inclusión	Exclusión
Temporalidad	Publicaciones de 2020 en adelante	Anteriores a 2020
Calidad	Revistas Q1 o conferencias de alto impacto	Revistas no indexadas o de cuartiles inferiores
Enfoque	Técnicas de aprendizaje profundo	Aprendizaje automático clásico sin componente profundo
Análisis de señal	Análisis tiempo-frecuencia	Análisis exclusivamente temporal o frecuencial
Dominio	Bioacústica, con preferencia por mamíferos y primates	Reconocimiento de voz humana sin transferencia al dominio
Entorno	Entornos naturales ruidosos, preferentemente tropicales	Entornos exclusivamente de laboratorio
Detección	Métodos que localizan el evento en tiempo y frecuencia	Métodos que clasifican la grabación completa
Clasificación	Clasificación multinivel de especie y tipo de vocalización	Clasificadores binarios o de un solo nivel
Desbalance	Técnicas para desbalance de clases y escasez de datos	Métodos que exigen grandes volúmenes etiquetados
Evaluación	Métricas completas de precisión, <i>recall</i> y F1	Trabajos sin evaluación cuantitativa
Idioma	Inglés o español	Otros idiomas

Dos de ellos hacen el trabajo principal de filtrado: el criterio de detección descarta los trabajos que solo clasifican la grabación completa sin localizar nada dentro de ella, y el de clasificación descarta los clasificadores binarios o de un solo nivel taxonómico. Son precisamente los dos ejes en los que se sitúa el vacío descrito en la sección 1.3.2.

3.2. Resultados de la revisión

Los resultados se apoyan en los trabajos de la Tabla 3.3, que indica a qué pregunta de revisión responde cada uno.

Siete de los nueve trabajos son de 2020 en adelante, como exige el criterio de temporalidad de la Tabla 3.2. Los dos anteriores se incorporan fuera de la búsqueda,

Tabla 3.3*Trabajos en que se apoya la revisión*

Trabajo	Publicación	Preguntas
K. He et al. (2016) ^a	Conferencia (CVPR)	P1
Lostanlen et al. (2019) ^a	Revista (<i>PLOS ONE</i>)	P1, P2, P3, P4
Politis et al. (2020)	Taller (DCASE)	P1, P2, P4
Bonet-Solá & Alsina-Pagès (2021)	Revista (<i>Sensors</i>)	P1
Kahl et al. (2021)	Revista (<i>Ecological Informatics</i>)	P1, P2, P4
Meedeniya et al. (2023)	Revista (<i>ACM Computing Surveys</i>)	Contexto general
Nolasco et al. (2023)	Revista (<i>Ecological Informatics</i>)	P1, P2, P4, P5
Kath et al. (2024)	Revista (<i>Ecological Informatics</i>)	P2, P5
Z. Zhu & Sato (2025)	Taller (DCASE)	P1

Nota. ^a Anterior a 2020: no cumple el criterio de temporalidad de la Tabla 3.2, no proviene de la búsqueda y se cita como antecedente.

como antecedentes sin los cuales no se leen los demás: la arquitectura residual, base de BirdNET, y el estudio de redes de sensores en que se apoya la discusión de PCEN como alternativa al log-mel.

Ninguno de los trabajos recuperados evalúa arquitecturas de aprendizaje profundo para la detección y clasificación de vocalizaciones de primates en el contexto amazónico. Los resultados que siguen se apoyan, por tanto, en la extrapolación desde dominios bioacústicos adyacentes pero más desarrollados, sobre todo la detección de aves y la localización de eventos sonoros, que enfrentan desafíos análogos de ruido ambiental, variabilidad entre sensores y escasez de datos etiquetados (Lostanlen et al., 2019; Meedeniya et al., 2023).

3.2.1. P1: Estrategias de detección tiempo–frecuencia

Las redes convolucionales son la base de los sistemas que operan sobre espectrogramas. Un modelo convolucional de arquitectura relativamente simple ya supera de forma clara a métodos tradicionales como el flujo espectral o los detectores basados en energía, aunque adolece de falta de robustez y de generalización ante la variabilidad del ruido (Lostanlen et al., 2019). Las redes convolucionales recurrentes añaden capas recurrentes para modelar dependencias temporales más largas y fueron la arquitectura predominante

en las ediciones iniciales de los desafíos de localización y detección de eventos sonoros (Politis et al., 2020). Las variantes residuales facilitan el entrenamiento de redes más profundas sin degradación (K. He et al., 2016), y son la base de BirdNET (Kahl et al., 2021).

Los transformadores aparecen como innovación arquitectónica en detección bioacústica de pocos ejemplos, si bien la mayoría de sistemas exitosos de ese desafío siguieron usando redes convolucionales (Nolasco et al., 2023). En detección con salida bidimensional, el trabajo relevante sustituye los *backbones* preentrenados en imágenes por representaciones nativas de audio y añade una pirámide multiescala, precisamente porque las arquitecturas de tipo transformador de espectrograma producen representaciones de una sola escala (Z. Zhu & Sato, 2025).

Del lado de la representación de entrada, la ventaja de PCEN sobre el log-mel que describe la sección 2.2.3 (Lostanlen et al., 2019) se repite en el desafío de detección con pocos ejemplos, donde fue la representación mayoritaria entre los sistemas exitosos (Nolasco et al., 2023). La elección de la representación resultó, en esos trabajos, tan determinante como la elección de la arquitectura (Bonet-Solá & Alsina-Pagès, 2021).

3.2.2. P2: Clasificación jerárquica fina

La revisión no identificó estrategias diseñadas y evaluadas explícitamente para la clasificación jerárquica fina que asigna de forma simultánea la especie y el tipo de vocalización. La literatura recuperada se concentra en tres tareas vecinas pero distintas: la detección de eventos, que localiza sin etiquetar en detalle (Lostanlen et al., 2019; Politis et al., 2020); la clasificación de especie, multiclase o multietiqueta, que identifica qué especies están presentes, como BirdNET, que reconoce 984 especies de aves (Kahl et al., 2021), o los clasificadores lineales que Kath et al. (2024) entrenan sobre *embeddings* para anuros; y la clasificación de eventos ya detectados en un conjunto plano de clases predefinidas (Politis et al., 2020).

La necesidad de distinguir tipos de llamada o sílabas se reconoce como un desafío de clasificación de grano fino en bioacústica, pero se enuncia como problema abierto y no como capacidad demostrada (Nolasco et al., 2023). Este es, en la revisión, el hueco más nítido y el que sostiene el eje de tarea de la sección 1.3.2.

3.2.3. P3: Mitigación de ruido y variabilidad entre sensores

Ningún trabajo recuperado prueba técnicas específicamente sobre ruido amazónico, pero varios abordan el ruido ambiental heterogéneo y la variabilidad entre sensores en despliegues de monitoreo acústico pasivo. Dos resultados son directamente aplicables.

El primero es la normalización de energía por canal como preprocesamiento del espectrograma: al combinar compresión de rango dinámico y control adaptativo de ganancia, suprime el ruido estacionario y realza los eventos transitorios, y con sus parámetros ajustados a entornos exteriores ruidosos produce distribuciones de magnitud consistentes y cuasi-gaussianas entre sensores distintos (Lostanlen et al., 2019). El segundo son las redes adaptativas al contexto, que procesan en una rama auxiliar estadísticas espectrales resumidas a largo plazo, mediante percentiles del espectrograma, y adaptan el modelo a las condiciones de cada punto de monitoreo, mejorando la robustez frente a la variabilidad espacial del ruido (Lostanlen et al., 2019). Ambas técnicas resultaron complementarias: su combinación fue lo que produjo detección robusta a escala de red de sensores.

3.2.4. P4: Conjuntos de datos y aumento de datos

Los conjuntos públicos disponibles se concentran de forma abrumadora en aves: BirdVox (Lostanlen et al., 2018), Xeno-canto (Vellinga, 2026) y la Macaulay Library (Cornell Lab of Ornithology, s.f.). Fuera de ese taxón, la revisión identificó AnuraSet para anuros (Cañas et al., 2023), la base de Watkins para mamíferos marinos (Sayigh et al., 2016) y los conjuntos de los desafíos DCASE (*Detection and Classification of Acoustic Scenes and Events*) para sonidos generales (Nolasco et al., 2023; Politis et al., 2020). No se encontró ningún conjunto público a gran escala etiquetado específicamente para vocalizaciones de primates, lo que confirma por una segunda vía el eje de dominio de la sección 1.3.2.

En cuanto a formatos, los eventos localizados en tiempo se distribuyen típicamente como archivos delimitados con tiempo de inicio, tiempo de fin y etiqueta de clase (Nolasco et al., 2023); en tareas de localización se añade la dirección de llegada (Politis et al., 2020); y en repositorios grandes como Xeno-canto lo habitual son etiquetas débiles de presencia o ausencia a nivel de grabación completa (Kahl et al., 2021). Ninguno de esos formatos registra la banda de frecuencia del evento.

Frente a la escasez y al desbalance, las técnicas reportadas son la adición de ruido de fondo realista, el desplazamiento de tono, el estiramiento temporal, el enmascaramiento de bloques de tiempo o frecuencia sobre el espectrograma y el entrenamiento con mezcla de ejemplos (Kahl et al., 2021; Lostanlen et al., 2019; Politis et al., 2020). El sobremuestreo de clases minoritarias es la respuesta habitual al desbalance, mientras que el aprendizaje sensible al costo fue probado sin mejora apreciable (Kahl et al., 2021). Cuando los ejemplos etiquetados son muy escasos, las redes prototípicas permiten adaptarse a clases nuevas con apenas cinco ejemplos (Nolasco et al., 2023).

3.2.5. P5: Transferencia de aprendizaje

La transferencia de aprendizaje es la estrategia central cuando los datos etiquetados son limitados (Kath et al., 2024). El hallazgo más útil de la revisión es que la proximidad del dominio de origen importa más que el tamaño del modelo: BirdNET, entrenado en aves, transfiere mejor a la clasificación de anuros que modelos entrenados en audio general o en imágenes (Kath et al., 2024). Eso lo posiciona como candidato razonable para transferir a primates.

Sobre el método de adaptación, la revisión distingue dos rutas. La extracción de características usa el modelo preentrenado como extractor fijo y entrena un clasificador simple sobre sus *embeddings*; la penúltima capa de BirdNET resultó particularmente efectiva en esa función (Kath et al., 2024). El ajuste fino reentrena parcial o totalmente los pesos con los pocos datos de la tarea objetivo, y exige cuidado para no sobreajustar (Nolasco et al., 2023). Ninguno de los trabajos documenta la adaptación de estos modelos a la detección con salida bidimensional ni a la clasificación jerárquica de primates.

3.3. Síntesis

Leídas en conjunto, las respuestas a las cinco preguntas son las siguientes.

Detección y robustez (P1, P3). No se identificó ningún estudio que aplique y evalúe modelos de aprendizaje profundo a la detección tiempo–frecuencia de primates amazónicos. La literatura de dominios vecinos sí ofrece estrategias trasladables: las arquitecturas convolucionales y convolucionales recurrentes como base (Lostanlen et al., 2019; Politis et al., 2020), las variantes residuales y los transformadores para secuencias largas (K. He et al., 2016; Nolasco et al., 2023), y, frente a la variabilidad del ruido, la

normalización de energía por canal junto con las redes adaptativas al contexto, cuya combinación resultó la más efectiva a escala de red de sensores (Lostanlen et al., 2019).

Clasificación jerárquica (P2). No se hallaron estrategias explícitamente diseñadas y evaluadas para asignar de forma conjunta especie y tipo de vocalización. Existen enfoques de clasificación multietiqueta sobre *embeddings* preentrenados (Kath et al., 2024), pero la tarea jerárquica específica sigue requiriendo desarrollo propio.

Datos y anotaciones (P4). Se confirma la ausencia de conjuntos públicos a gran escala etiquetados para primates y adecuados para aprendizaje profundo: el esfuerzo de la comunidad se concentra en aves. Los formatos de anotación de uso común registran tiempo de inicio, tiempo de fin y clase (Nolasco et al., 2023), y descartan la banda de frecuencia, lo que significa que incluso los conjuntos disponibles no servirían como referencia para la tarea de esta tesis sin reanotación.

Transferencia (P5). La transferencia desde dominios cercanos es la respuesta más viable a la escasez de datos etiquetados, con evidencia de transferencia exitosa de aves a otros taxones (Kath et al., 2024; Nolasco et al., 2023). Queda abierto cómo aplicarla cuando la salida deseada no es una etiqueta por segmento sino una caja tiempo–frecuencia, que es exactamente el problema que aborda el Capítulo 5.

Los dos ejes del vacío descrito en la sección 1.3.2 quedan confirmados de forma independiente por esta revisión: no hay trabajo publicado sobre primates neotropicales en este contexto (eje de dominio) y no hay estrategias establecidas para la salida bidimensional con etiquetado jerárquico (eje de tarea). El cruce de ambos ejes es la posición que ocupa esta tesis.

Capítulo 4

Curación del Conjunto de Datos (OE1)

4.1. Introducción

Con el problema, los conceptos y el vacío establecidos en los capítulos anteriores, este y los dos capítulos siguientes presentan los resultados, uno por objetivo específico. Este capítulo presenta los alcanzados para el primero:

OE1. Definir un protocolo de curación y estandarización, y aplicarlo sobre las anotaciones entregadas por el equipo de investigación para construir un conjunto de datos consistente de cajas tiempo–frecuencia etiquetadas por especie y tipo de llamada.

Los tres resultados comprometidos en la Tabla 1.4 ocupan una sección cada uno: el protocolo documentado (RE1.1, sección 4.3), el conjunto curado y particionado (RE1.2, sección 4.4) y la ficha del conjunto derivado (RE1.3, sección 4.5). Cada sección describe el resultado, cómo se alcanzó con los métodos y procedimientos del Capítulo 1, cómo reproducirlo, y la verificación y las mediciones con que se comprueba su indicador. La sección 4.6 discute los tres en conjunto.

Antes de los resultados, la sección 4.2 caracteriza el material tal como se recibe: es el diagnóstico que justifica cada regla del protocolo.

4.2. El material recibido

4.2.1. Estructura y formato

El material llega en una carpeta por especie, nombrada <nombre_común>__<CÓDIGO>, con las grabaciones WAV a 44,1 kHz capturadas con dispositivos AudioMoth desplegados en el bosque y, junto a cada una, la tabla de selección que el analista exportó desde Raven Pro. La Tabla 4.1 lista las ocho especies y sus códigos. El código de la carpeta es un dato estructural, fijado una vez por sesión de campo, y el protocolo lo trata como la fuente de verdad de la especie (sección 4.3.2).

Tabla 4.1
Especies del corpus

Cód.	Especie	Carpeta
AA	<i>Aotus azarae</i>	night_monkey__AA
AC	<i>Ateles chamek</i>	peruvian_spider_monkey__AC
AS	<i>Alouatta sara</i>	howler_monkey__AS
CC	<i>Cebus cuscinus</i>	shock_headed_capuchin_monkey__CC
LW	<i>Leontocebus weddelli</i>	weddells_saddleBack_tamarin__LW
PT	<i>Plecturocebus toppini</i>	toppins_titi_monkey__PT
SB	<i>Saimiri boliviensis</i>	bolivian_squirrel_monkey__SB
SM	<i>Sapajus macrocephalus</i>	large_headed_capuchin__SM

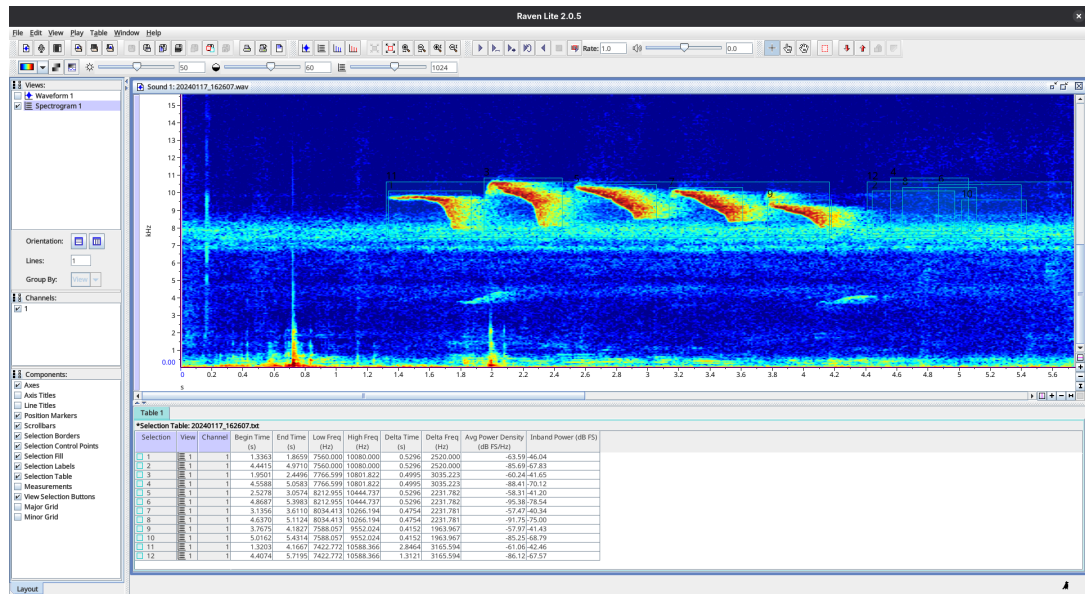
Cada tabla de selección es un archivo de texto delimitado por tabulaciones, con una fila por caja, que Raven lee y escribe directamente (Figura 4.1).

Figura 4.1
Tabla de selección de Raven

Selection	View	Channel	Begin Time (s)	End Time (s)	Low Freq (Hz)	High Freq (Hz)	Inband Power (dB FS)	Species	Call type	Rating	Reference
1	Spectrogram	1	1.336330093	1.865888854	7560.000	10080.000	-46.04	LW	CS	C	
2	Spectrogram	1	4.441470101	4.971028862	7560.000	10080.000	-67.83	noise			1
3	Spectrogram	1	1.950136839	2.449607034	7766.599	10801.822	-41.65	LW	CS	B	
4	Spectrogram	1	4.558815508	5.058285703	7766.599	10801.822	-70.12	noise			3
5	Spectrogram	1	2.527837305	3.057396066	8212.955	10444.737	-41.20	LW	CS	B	
6	Spectrogram	1	4.868727738	5.398286499	8212.955	10444.737	-78.54	noise			5
7	Spectrogram	1	3.135626338	3.611025680	8034.413	10266.194	-40.34	LW	CS	B	
8	Spectrogram	1	4.637045780	5.112445122	8034.413	10266.194	-75.00	noise			7
9	Spectrogram	1	3.767486223	4.182708433	7588.057	9552.024	-41.43	LW	CS	B	
10	Spectrogram	1	5.016161711	5.431383921	7588.057	9552.024	-68.79	noise			9
11	Spectrogram	1	1.320278575	4.166656916	7422.772	10588.366	-42.46	LW	CC	B	
12	Spectrogram	1	4.407365444	5.719457935	7422.772	10588.366	-67.57	noise			11

Esa fila no se teclea: Raven deriva las cuatro coordenadas de la región rectangular que el analista dibuja sobre el espectrograma. La Figura 4.2 muestra la operación desde el otro lado, con cada caja pintada sobre la grabación.

Figura 4.2
Anotación en Raven Pro



Las columnas de especie y tipo de llamada, en cambio, sí se escriben a mano, y son las que concentran los defectos que el protocolo corrige. En el corpus aparecen 14 encabezados distintos, y no todos sobreviven: la Tabla 4.2 recoge el destino de cada uno.

Tabla 4.2
Columnas de las tablas crudas

Columna de Raven	En cleaned/	Tratamiento
Begin Time (s)	begin_time_s	Se conserva; define el lado izquierdo de la caja
End Time (s)	end_time_s	Se conserva; lado derecho
Low Freq (Hz)	low_freq_hz	Se conserva; lado inferior
High Freq (Hz)	high_freq_hz	Se conserva, recortada a Nyquist
Species	species	Se reemplaza por el código de la carpeta
Call type	call_type	Se normaliza y se mapea al vocabulario
Selection, View, Channel		Estado interno de Raven; se descartan
Begin File, File Offset (s), Reference		Referencias al archivo; se descartan
Inband Power (dB FS), Rating		No se usan; no llegan a cleaned/

Sobreviven seis columnas: las cuatro coordenadas de la caja y las dos etiquetas. Todo lo demás describe la sesión de trabajo en Raven o el archivo de origen, no el evento.

4.2.2. Inventario

El corpus de primates tiene 3 356 grabaciones y 2 835 tablas de anotación. De ellas, 2 823 tienen una grabación homónima junto a ellas, 12 no la tienen y 4 no pueden leerse por estar vacías o truncadas. Las tablas emparejadas suman 20 082 filas crudas, que son el punto de partida de todo lo que sigue.

4.2.3. Defectos detectados

El análisis exploratorio del material reveló cinco problemas que impiden usarlo directamente para entrenar.

Inconsistencias de etiquetado. Una misma clase aparece escrita de varias formas. La columna de tipo de llamada trae 63 valores distintos escritos a mano y la de especie, 19 (Figuras 4.3 y 4.4). Entre ellos conviven mayúsculas y minúsculas, espacios finales, separadores y errores de escritura: CS, CS_, CS_A, contact syllable, noise|, noises, _BC y bC designan tres clases, no ocho.

Figura 4.3

Valores en la columna de especie

```
print("\nEspecies únicas en el dataset:")
print("-----")
for i, specie in enumerate(raw_annotations["specie"].unique(), 1):
    print(f"{i}. {specie}")
```

✓ 0.0s

Especies únicas en el dataset:

1. CC
2. PT
3. nan
4. LW
5. 8
6. -
7. PTD
8. LW
9. L
10. }
11. AS
12. AC
13. AC
14. SM
15. AA
16. AN
17. SN
18. #
19. AS-----AS
20. SB

Figura 4.4

Valores en la columna de tipo de llamada

```
print("Tipos de llamadas únicas en el dataset:")
print("-----")
for i, call in enumerate(raw_annotations["call_type"].unique(), 1):
    print(f"{i}. {call}")
```

✓ 0.0s

Tipos de llamadas únicas en el dataset:

1. CC
2. nan
3. BP
4. PP
5. AC
6. DC
7. SQC
8. D
9. CHC
10. CS
11. noise
12. AA
13. A
14. B
15. TA
16. C
17. CS
18. CS A
19. noise
20. contact call
21. contact syllable
22. TT
23. TR
24. VC
25. TJ
26. TC
27. noise|
28. noises

Integridad de los archivos. La carga automatizada de las grabaciones emite advertencias sobre archivos de audio dañados (Figura 4.5), que detienen el proceso si no

se manejan de forma explícita. La corrupción se manifiesta como bloques de ceros en la señal y admite grados: la Figura 4.6 muestra un caso leve y la Figura 4.7 uno en el que la mayor parte del archivo es inutilizable. El origen probable son cortes durante la transferencia o fallos del medio de almacenamiento.

Figura 4.5
Avisos de audio dañado

```
Procesando 2674 archivos de audio...
=====
⚠ ADVERTENCIA en 20240117_060129.wav: El espectrograma contiene ceros.
⚠ ADVERTENCIA en 20240117_163024.wav: El espectrograma contiene ceros.
⚠ ADVERTENCIA en 20240207_102952.wav: El espectrograma contiene ceros.
⚠ ADVERTENCIA en 20240214_102327.wav: Anotaciones exceden duración del audio (7.65s > 6.87s)
⚠ ADVERTENCIA en 20240214_102452.wav: Anotaciones exceden duración del audio (5.92s > 5.13s)
⚠ ADVERTENCIA en 20240214_102634.wav: Anotaciones exceden duración del audio (13.15s > 13.11s)
⚠ ADVERTENCIA en 20240415_114228.wav: El espectrograma contiene ceros.
⚠ ADVERTENCIA en 20240429_104951.wav: El espectrograma contiene ceros.
⚠ ADVERTENCIA en 20240504_080326.wav: El espectrograma contiene ceros.
⚠ ADVERTENCIA en 20240510_065700.wav: El espectrograma contiene ceros.
⚠ ADVERTENCIA en 20240510_071046.wav: El espectrograma contiene ceros.
⚠ ADVERTENCIA en 20240516_081806.wav: El espectrograma contiene ceros.
⚠ ADVERTENCIA en 20240516_081815.wav: El espectrograma contiene ceros.
⚠ ADVERTENCIA en 20240530_110626.wav: El espectrograma contiene ceros.
⚠ ADVERTENCIA en 20240621_074433.wav: Anotaciones exceden duración del audio (30.69s > 27.25s)
⚠ ADVERTENCIA en 20240805_091140.wav: El espectrograma contiene ceros.
⚠ ADVERTENCIA en 20240805_091302.wav: El espectrograma contiene ceros.
⚠ ADVERTENCIA en 20240809_154436.wav: El espectrograma contiene ceros.
⚠ ADVERTENCIA en 20240809_154640.wav: El espectrograma contiene ceros.
⚠ ADVERTENCIA en 20240809_154724.wav: El espectrograma contiene ceros.
⚠ ADVERTENCIA en 20240809_154800.wav: El espectrograma contiene ceros.
⚠ ADVERTENCIA en 20240813_092636.wav: El espectrograma contiene ceros.
⚠ ADVERTENCIA en 20240815_170844.wav: El espectrograma contiene ceros.
```

Cada aviso nombra el archivo afectado, pero no dice cuánto de la señal se perdió; eso es lo que muestran los dos grados de corrupción.

Figura 4.6
Corrupción leve

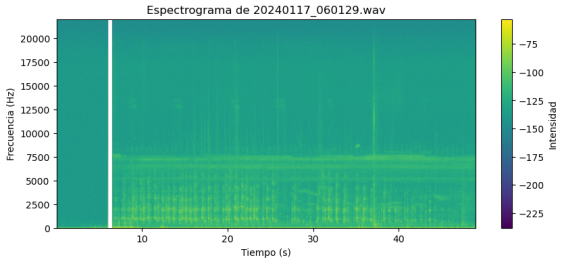
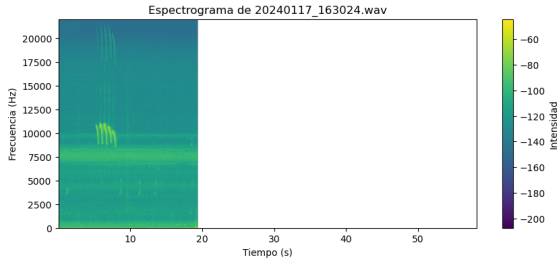


Figura 4.7
Corrupción severa



Discrepancias entre especie escrita y carpeta. La etiqueta de especie de la anotación no siempre coincide con el código de la carpeta contenedora: en 879 filas la columna viene vacía y en 129 trae un valor distinto. Hay que decidir cuál de las dos fuentes prevalece antes de consolidar nada.

Frecuencias por encima de Nyquist. 8 cajas declaran una frecuencia superior mayor que el límite de Nyquist de la señal (22 050 Hz a 44,1 kHz de muestreo), un valor que no puede corresponder a energía observable en la grabación.

Variabilidad en la unidad de análisis. La granularidad de las anotaciones no es uniforme: en unos casos el analista etiquetó sílabas individuales y en otros delimitó la secuencia completa con una sola caja. No es un error de anotación sino la manifestación directa del anidamiento jerárquico de las unidades acústicas (Kershenbaum et al., 2016) descrito en el Capítulo 2; la sección 4.4.1 lo cuantifica y decide qué hacer con él.

4.3. Protocolo de curación y estandarización documentado (RE1.1)

Resultado. Un protocolo documentado que convierte las anotaciones entregadas en el conjunto consistente que persigue OE1. Define la normalización de columnas y etiquetas, el mapa de sinónimos, el descarte de ruido y de cajas degeneradas, la corrección de pares imposibles, el recorte al límite de Nyquist y el tratamiento de los registros fuera de vocabulario, que son los siete elementos que exige su indicador (Tabla 1.4).

Cómo se alcanzó. Aplicando a las etiquetas la Fase 3 de CRISP-DM (sección 1.7.1), a partir del diagnóstico de la Fase 2 que hace la sección 4.2. Como exige la custodia del material primario (sección 1.7.3), el protocolo es un programa, no una edición manual del material recibido. La distinción importa: un protocolo ejecutable vuelve a aplicarse sobre los datos primarios cada vez que una regla cambia, y deja registro de qué descarta y por qué; una corrección a mano no es reproducible ni auditable. El guion `src/prepare_annotations.py` recorre `data/raw/`, aplica las reglas de `src/data/annotations.py` en el orden de la Figura 4.8 y escribe `data/cleaned/`.

Cómo reproducirlo. Con las tablas crudas en `data/raw/` y el código del repositorio, `src/prepare_annotations.py` regenera `data/cleaned/` completo. Las reglas están fijadas en el código y el *script* no tiene parámetros que alteren el resultado, de modo que dos ejecuciones sobre el mismo material producen las mismas tablas.

Medio de verificación. Esta sección, que es el documento del protocolo; su implementación en los dos archivos citados, publicados en el repositorio del proyecto (<https://github.com/nhrot-fc/tesis-primate>); y el conteo de lo que toca cada regla sobre

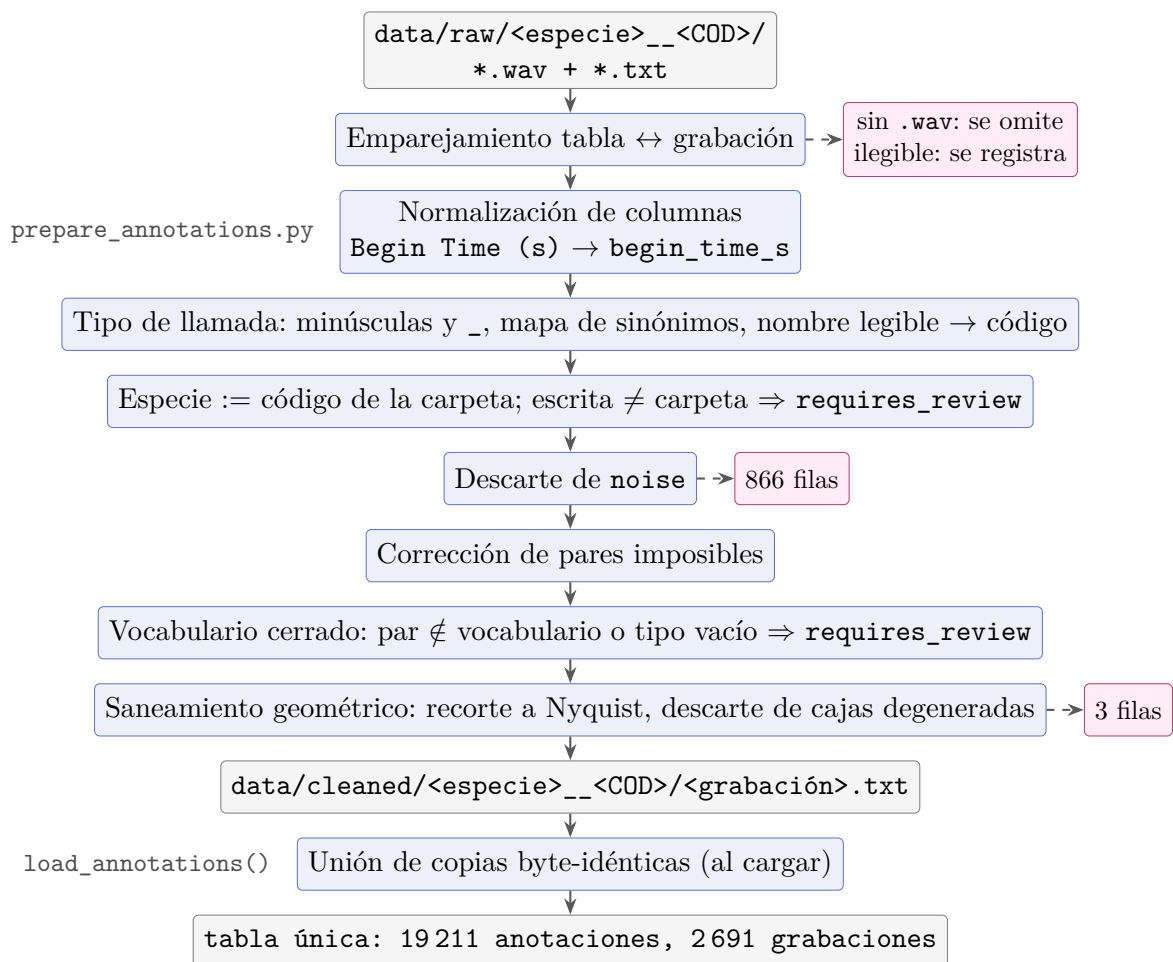
las 20 082 filas crudas, en la sección 4.3.3. Las tablas del protocolo, tal como las aplica el código, están además en la carpeta RE1.1 de los medios de verificación (Anexo B).

4.3.1. Esquema del protocolo

La Figura 4.8 recorre el protocolo de la entrada a la salida. Las reglas se aplican en el orden mostrado y cada una tiene su apartado más abajo.

Figura 4.8

Reglas del protocolo



Las tres salidas punteadas de la derecha son los únicos puntos en que el protocolo elimina algo: tablas sin grabación, filas de ruido y cajas degeneradas. Todo lo demás se corrige o se marca para revisión. La unión de copias, al pie, es la única regla que se aplica al cargar `cleaned/` y no al escribirlo, porque necesita ver todas las carpetas a la vez.

4.3.2. Reglas

Emparejamiento con la grabación. Solo se procesa la tabla que tiene una grabación homónima junto a ella. El nombre del `.wav` se obtiene del de la tabla quitando el sufijo `.Table.1.selections` que Raven añade, y tolerando un espacio final. Una tabla huérfana no aporta nada entrenable y su presencia suele indicar un error de transferencia: se omite y se lista en el registro. Una tabla ilegible se captura sin detener el proceso y también se registra.

Normalización de columnas. Los encabezados se convierten a `snake_case` programático (`Begin Time (s)` $\rightarrow$ `begin_time_s`). Un solo paso absorbe las variantes de mayúsculas y espacios, y hace que el resto del código dependa de un nombre estable. Las columnas de estado interno de Raven se descartan: describen la sesión de trabajo, no el evento.

Normalización del tipo de llamada. Sobre la columna manual se aplican, en este orden, dos reglas. La *normalización tipográfica* reduce toda etiqueta a minúsculas y a caracteres alfanuméricos separados por guion bajo; el paso absorbe mayúsculas, espacios y signos sin necesidad de enumerarlos, y en 52 filas el valor cambia más allá de la mayúscula. El *mapa de sinónimos* (Tabla 4.3, 10 entradas) unifica lo que la tipografía no captura por ser un error de escritura o una abreviatura alternativa; a él se suma, por especie, la traducción del nombre legible al código (`contact syllable` $\rightarrow$ `cs`), de modo que las tablas escritas en prosa converjan con las escritas en código. Toca 105 filas.

Asignación de especie por directorio. La columna `Species` se reemplaza por el código de la carpeta contenedora. El criterio no es arbitrario: el directorio se fija una vez por sesión de campo, mientras que la columna se teclea en cada fila, y es donde se acumulan los errores de la sección 4.2.3. Pero una llamada de otra especie anotada dentro de una carpeta, si es real, no debe recibir la especie de la carpeta: la fila conserva lo escrito en `written_species` y se marca `requires_review`, de modo que ni entra al experimento con la especie equivocada ni se descarta en silencio. La unión de copias resuelve el caso en que esa llamada sí está anotada en su propia carpeta.

Descarte de ruido. Las filas cuyo tipo de llamada normalizado es `noise` (866) marcan la ausencia de evento, no un evento, y se eliminan. El fondo que el detector

Tabla 4.3*Mapa de sinónimos*

Escrito	Código
noises	noise
cs_a	cs
whinnie	whc
tca	ta
tac	ta
contact	cc
chcj	chc
php	phc
sqr	sqc
tc	tr

debe aprender a no proponer entra al conjunto por otra vía (las ventanas vacías de la sección 4.4.3) y no por estas filas. Las 180 filas sin tipo de llamada, en cambio, son cajas que el analista dibujó sobre un evento: no se eliminan, se marcan para revisión.

Corrección de pares imposibles. Un conjunto pequeño de correcciones explícitas resuelve pares que no existen en el repertorio de la especie y cuya intención es inequívoca: aa/hc pasa a aa/hm, porque *Aotus azarae* no tiene llamada de aullido pero sí de ululato (23 filas). Toda corrección de este tipo se declara en código (MANUAL_FIXES); ninguna se aplica a mano sobre los datos.

Vocabulario cerrado y marca de revisión. El vocabulario admisible se declara como el conjunto de pares (especie, tipo de llamada) válidos del dominio (41 pares, Tabla 4.4), no como aquello que aparezca en los datos. Cerrar el vocabulario responde a que las definiciones de unidad y las etiquetas semánticas varían ampliamente entre investigadores, incluso dentro de un mismo grupo taxonómico (Kershenbaum et al., 2016): sin una lista fija, la misma señal recibe etiquetas distintas según quién la anote.

Toda fila que no pueda asignarse con certeza a un par del vocabulario se marca con la bandera `requires_review` en lugar de eliminarse. Son tres los motivos: el par (especie de la carpeta, tipo) no está en el vocabulario (336 filas), el tipo de llamada viene vacío (180) o la especie escrita difiere de la carpeta (129). En `data/cleaned/` quedan así 643 anotaciones marcadas, el 3,3 % del total, repartidas en 45 pares distintos; los 32 pares que el vocabulario no reconoce, con 516 anotaciones, son los de la Tabla 4.5.

Tabla 4.4
Vocabulario cerrado

Esp.	Tipos	Códigos de tipo de llamada
AA	3	gc (gulp call), hm (hoot call), sc (squeak call)
AC	6	chc (chitter call), gc (growl call), cc (contact call), whc (whinnie call), sc (squeak call), bc (bark call)
AS	2	hc (howl call), bc (bark call)
CC	1	cc (contact call)
LW	11	cc (contact call), cs (contact syllable), aa (aerial alarm call), ta (terrestrial alarm call), sqc (squeal call), phc (phee call), tr (trino call), tf (trino fast call), tj (unofficial tj call), tt (trino transition call), vc (visual contact call)
PT	5	dc (duet call), ac (alarm call), bp (unofficial bellow phrase), pp (unofficial pant phrase), sqc (unofficial squeal call)
SB	5	pcc (peep contact call), ppc (play peep call), lpc (long peep call), spc (spit call), sc (shriek call)
SM	8	cc (contact call), acc (aggressive contact call), sc (squeal call), pc (purr call), whc (whistle call), hic (hip call), fc (food call), fs (food syllable)

Tabla 4.5
Pares marcados para revisión

Par	Filas	Par	Filas
as/cp	91	sb/phc	3
as/pp	85	as/pc	3
as/ip	73	sm/chc	3
as/	53	lw/c	3
sm/	47	cc/	2
pt/	30	sb/cc	1
ac/	21	sb/hic	1
lw/a	20	as/chc	1
lw/b	15	sm/spc	1
lw/	15	aa/hf	1
sb/	11	aa/	1
sb/pcs	8	pt/d	1
sm/hc	8	pt/chc	1
aa/sqc	6	lw/t	1
as/cc	4	lw/chc	1
sm/sic	4	lw/lw	1

La diferencia con descartarlas es deliberada, y la razón es que **no hay forma de distinguir un error de escritura de una subdivisión legítima del repertorio**. Entre los pares fuera de vocabulario conviven las dos cosas: variantes tipográficas que el mapa de sinónimos no previó, y códigos que parecen subdivisiones no documentadas de un tipo de llamada ya conocido: **as/cp**, **as/pp** e **as/ip** sobre *Alouatta sara* son el caso más visible. Decidir cuál es cuál exigiría una referencia del repertorio vocal que no acompaña a las anotaciones, de modo que el protocolo no decide: marca y conserva. **cleaned/** los guarda con la bandera y RE1.2 los deja fuera del experimento (no entran como clase, y toda ventana que los pisa se descarta para no enseñarlos tampoco como fondo), pero la evidencia queda intacta para cuando el vocabulario se amplíe. El capítulo asume, y así lo declara la sección 4.5.9, que el material llega sucio en la columna de tipo de llamada: es una limitación del conjunto y, a la vez, el margen más claro de mejora para una versión futura del detector.

Saneamiento geométrico. Las coordenadas de la caja se validan con independencia de la etiqueta. La frecuencia superior se *recorta a Nyquist* (22 050 Hz): por encima de la mitad de la frecuencia de muestreo no hay energía observable en una señal a 44,1 kHz, de modo que un valor mayor (8 filas) es un error de anotación y no una propiedad del evento. Se *descartan las cajas degeneradas*, de duración inferior a 10 ms o de ancho de banda nulo o negativo (3 filas): una caja sin área no define una región y no puede emparejarse con ninguna predicción. Y se calculan la *duración* y el *ancho de banda* de cada caja, sobre los que se construye el análisis de geometría de la sección 4.5.3.

Unión de copias de una misma grabación. Al cargar **cleaned/** se detecta que una misma grabación está copiada, byte a byte, en dos o tres carpetas de especie, y que cada copia lleva solo las anotaciones de la especie de su carpeta. Tratarlas como grabaciones distintas deja la llamada de la otra especie sin anotar en cada copia, da supervisión contradictoria sobre el mismo audio y permite que caiga en dos particiones distintas, con la consiguiente fuga entre entrenamiento y prueba. La regla identifica las copias por tamaño y huella MD5 del primer megabyte, las funde en una sola grabación con la unión de sus anotaciones y descarta las anotaciones repetidas. En el corpus hay 128 copias, que reducen las grabaciones anotadas a 2 691.

4.3.3. Verificación de RE1.1

La Tabla 4.6 es la medición que verifica este resultado: cuenta, para cada regla del indicador, cuántas filas toca sobre las 20 082 crudas.

Tabla 4.6

Filas que toca cada regla

Regla	Filas	Tratamiento
Tablas sin grabación homónima	12	se omiten
Tablas ilegibles	4	se omiten y se registran
Filas crudas en tablas con audio	20 082	punto de partida
Tipo de llamada con espacios o signos de más	52	se normaliza
Tipo de llamada por sinónimo o nombre legible	105	mapa explícito
Especie vacía	879	la de la carpeta
Especie escrita distinta de la carpeta	129	la de la carpeta + requires_review
Tipo de llamada vacío	180	requires_review
Tipo de llamada noise	866	se descarta
Par imposible corregido	23	aa/hc → aa/hm
Frecuencia superior por encima de Nyquist	8	se recorta a 22 050 Hz
Caja degenerada (< 10 ms o sin banda)	3	se descarta
Par fuera de vocabulario	336	requires_review
Copias del mismo audio en otra carpeta	128	se unen a su original
Anotaciones repetidas entre copias	0	se descartan
Anotaciones conservadas	19 213	→ data/cleaned/
De ellas, marcadas requires_review	643	fuera del experimento (RE1.2)

El protocolo conserva 19 213 anotaciones, el 95,7 %, y casi todo lo que descarta es ruido marcado como tal por el propio analista: las demás reglas corrigen o marcan, pero no eliminan. La Tabla 4.7 mide el efecto de la normalización desde el otro lado: cuántos valores distintos quedan en las dos columnas manuales tras cada etapa.

El tipo de llamada pasa de 63 grafías a 31 códigos del vocabulario de la Tabla 4.4, y la especie converge a ocho valores porque la fija la carpeta.

La auditoría de la Tabla 4.6 vuelve a aplicar el protocolo sobre **data/raw/** y llega a 19 213 anotaciones, 2 más que las 19 211 que quedan en **data/cleaned/**. La diferencia está localizada y es instructiva: la auditoría trabaja sobre la tabla completa en memoria, que conserva las columnas de medición de Raven, mientras que **cleaned/** guarda solo

Tabla 4.7*Valores distintos tras cada etapa*

Etapas	Especie	Tipo de llamada
crudo	19	63
tipografía	15	55
sinónimos	8	42
vocabulario	8	31

las ocho columnas del protocolo (la especie de la carpeta y la escrita, el tipo de llamada, los cuatro bordes de la caja y la marca de revisión); dos pares de anotaciones de una misma grabación son idénticas en esas ocho columnas y solo se distinguen por **Inband Power**, de modo que la unión de copias las funde en una. Que ambas cifras se puedan comparar y que su diferencia se pueda explicar fila por fila es, en sí mismo, la propiedad que persigue un protocolo ejecutable.

Cada pasada deja un registro de lo que hizo: cuántas grabaciones y anotaciones salieron, cuántas quedaron marcadas para revisión, cuántas tablas resultaron ilegibles y cuáles no tenían grabación al lado. La unión de copias ocurre en `data.annotations.load_annotations()`, que es la única puerta por la que el resto del trabajo lee las anotaciones, de modo que ninguna etapa posterior puede ver las copias sin unir.

4.3.4. Fuera de alcance

- No corrige coordenadas más allá del recorte a Nyquist ni desplaza cajas: la geometría dibujada por el analista se respeta tal cual.
- No resuelve el anidamiento frase-sílaba; esa decisión se toma al seleccionar las clases del experimento (sección 4.4.1), porque depende de la forma de salida del detector y no de la calidad de la anotación.
- No mide el acuerdo entre anotadores, que exigiría anotar dos veces una muestra; la ficha lo declara entre sus limitaciones.
- No decide qué hacer con las anotaciones marcadas `requires_review`: las conserva y las deja al equipo de investigación.

4.4. Conjunto acústico curado y particionado (RE1.2)

Resultado. El directorio `data/processed/`: las 25 clases del experimento, la partición por grabación en proporción 60,0 / 22,5 / 17,5 y el caché de 38 713 ventanas de 3 s con 44 983 cajas normalizadas, más los metadatos que permiten reconstruirlo.

Cómo se alcanzó. Con el resto de la Fase 3 de CRISP-DM (sección 1.7.1): selección de clases, partición, ventaneo y representación. `src/prepare_data.py` parte de las 19 211 anotaciones de `data/cleaned/`, aplica las cuatro reglas de selección de clases de la sección 4.4.1, reparte las grabaciones enteras con el algoritmo de la sección 4.4.2, segmenta cada grabación en ventanas (sección 4.4.3) y materializa el espectrograma mel de cada una (sección 4.4.4). La Figura 4.9 resume el recorrido.

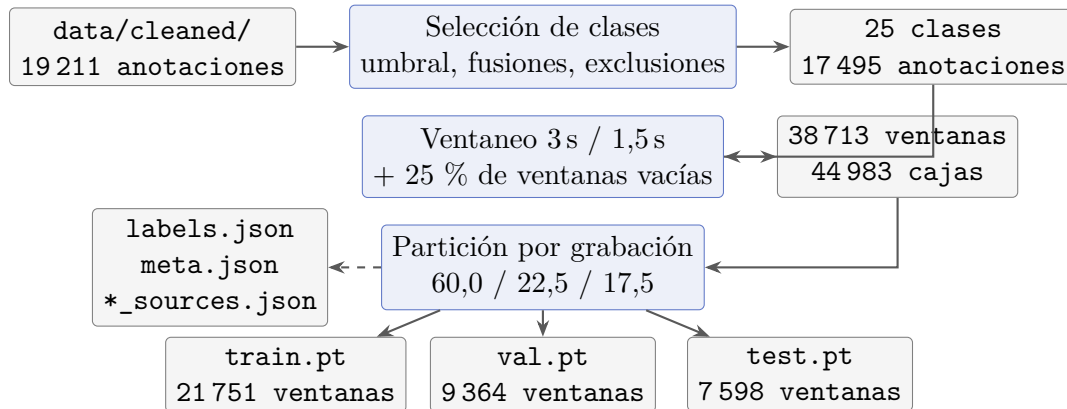
Cómo reproducirlo. `src/prepare_data.py` reconstruye `data/processed/` a partir de `data/cleaned/` con la semilla y los parámetros que registra `meta.json`. La construcción es determinista: la misma entrada, la misma semilla y los mismos parámetros dan las mismas ventanas, en el mismo orden.

Medio de verificación. El directorio `data/processed/`, con tres archivos `.pt`, `labels.json`, `meta.json` y los `*_sources.json`. No se publica, porque las grabaciones pertenecen al equipo de investigación, de modo que el documento reproduce lo necesario para verificarlo sin él: la sección 4.4.5 describe cada archivo, reproduce el `meta.json` íntegro y aplica las tres comprobaciones del indicador. El código que lo construye está en el repositorio del proyecto. La carpeta RE1.2 de los medios de verificación (Anexo B) contiene `meta.json`, `labels.json`, las listas de grabaciones por partición y el registro de la carga del caché.

La figura se lee de izquierda a derecha y de arriba abajo: las 19 211 anotaciones curadas se reducen a 25 clases, se cortan en 38 713 ventanas y se reparten en tres archivos, uno por partición, junto con los metadatos que permiten reconstruirlos. Las subsecciones siguientes desarrollan cada paso.

Figura 4.9

De las anotaciones curadas al caché



4.4.1. Selección de clases

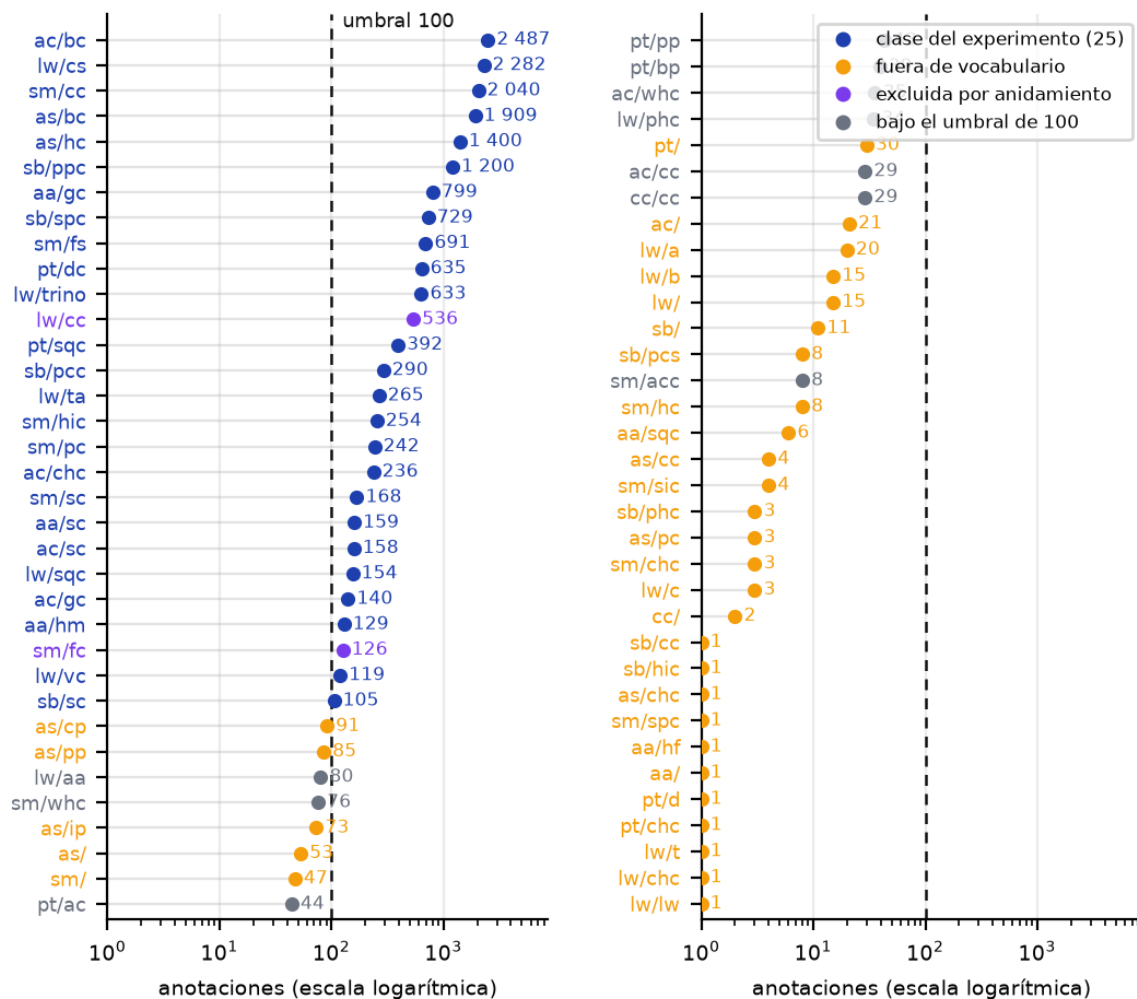
Las anotaciones curadas cubren 37 pares (especie, tipo de llamada) del vocabulario. No todos pueden ser clase de un detector, y el paso de uno a otro sigue cuatro reglas declaradas en `src/prepare_data.py` y registradas en `meta.json`.

1. **Anotaciones en revisión.** Las 643 filas marcadas `requires_review` por RE1.1 no entran como clase, y toda ventana que las pisa se descarta del caché: hasta que el equipo las resuelva no se enseñan ni como caja ni como fondo.
2. **Exclusión por anidamiento.** Se retiran las clases de *frase*, por la razón que desarrolla la sección 4.4.1.
3. **Fusión de variantes.** Los tipos de trino de LW se reúnen en `lw/trino`, y el pío largo `sb/lpc` se une al pío de juego `sb/ppc`. Las variantes comparten banda y duración casi por completo (sección 4.5.3) y por separado no alcanzan el umbral de la regla siguiente.
4. **Umbral de frecuencia.** Se conservan los pares con al menos 100 anotaciones. Por debajo de ese piso una clase no sostiene ni entrenamiento ni evaluación por clase.

El resultado son 25 clases con 17 495 anotaciones, el 93,6 % del material en vocabulario. La Figura 4.10 sitúa el umbral sobre la distribución completa de pares.

La elección del umbral en 100 y no más arriba responde a una tensión que conviene enunciar. Un umbral alto produce un experimento cómodo (pocas clases, todas abundantes) a costa de dejar sin cubrir la mayor parte del repertorio documentado. Bajarlo

Figura 4.10
Anotaciones por par



hasta 100 cubre el 93,6 % del material y hace que cinco especies aporten entre dos y cinco clases en lugar de una, al precio de admitir clases cuyo desempeño estará limitado por su propia escasez y no por la arquitectura. Cualquier resultado por clase debe leerse con esa distinción presente. La Tabla 4.8 lista los 44 pares del vocabulario que quedan fuera, con 1 595 anotaciones.

Entre los 41 pares que no alcanzan el umbral está *cc/cc*, la única llamada de *Cebus cuscinus*, de modo que el experimento cubre 7 de las 8 especies; los demás quedan fuera por anidamiento o por escasez, como indica la última columna.

Sílabas y frases

Una *frase* es una secuencia de *sílabas*, y los expertos anotan ambas: el 94,2 % de las frases *lw/cc* contienen al menos una sílaba *lw/cs*, y el 96,8 % de las frases *sm/fc*

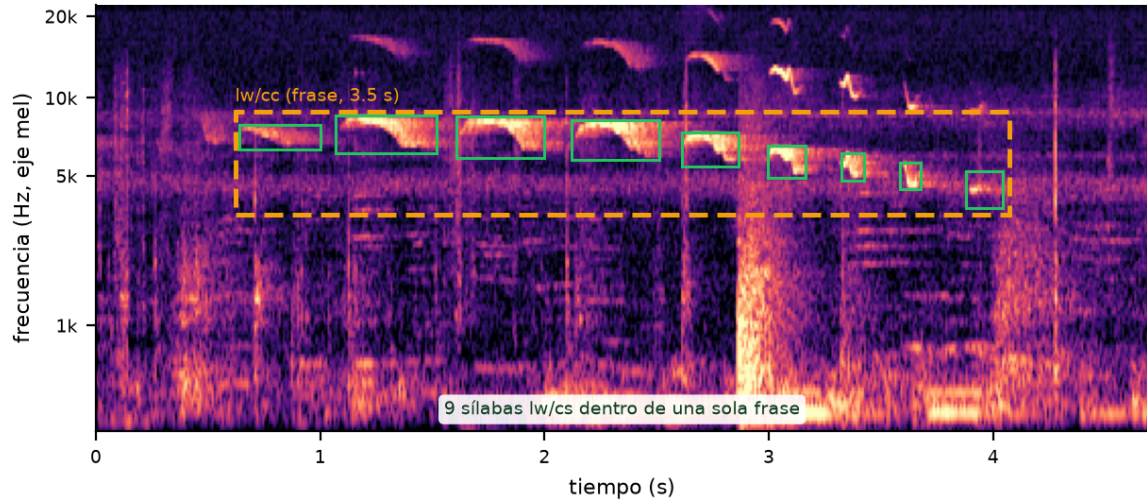
Tabla 4.8*Pares excluidos del experimento*

Par	Tipo de llamada	Anot.	Motivo
lw/cc	contact call	536	excluida (frase)
sm/fc	food call	126	excluida (frase)
as/cp		91	marcada para revisión
as/pp		85	marcada para revisión
lw/aa	aerial alarm call	80	< 100 anotaciones
sm/whc	whistle call	76	< 100 anotaciones
as/ip		73	marcada para revisión
as/		53	marcada para revisión
sm/		47	marcada para revisión
pt/ac	alarm call	44	< 100 anotaciones
pt/pp	unofficial pant phrase	43	< 100 anotaciones
pt/bp	unofficial bellow phrase	39	< 100 anotaciones
ac/whc	whinnie call	35	< 100 anotaciones
lw/phc	phee call	34	< 100 anotaciones
pt/		30	marcada para revisión
ac/cc	contact call	29	< 100 anotaciones
cc/cc	contact call	29	< 100 anotaciones
ac/		21	marcada para revisión
lw/a		20	marcada para revisión
lw/b		15	marcada para revisión
lw/		15	marcada para revisión
sb/		11	marcada para revisión
sb/pcs		8	excluida (frase)
sm/acc	aggressive contact call	8	< 100 anotaciones
sm/hc		8	marcada para revisión
aa/sqc		6	marcada para revisión
as/cc		4	marcada para revisión
sm/sic		4	marcada para revisión
sb/phc		3	marcada para revisión
as/pc		3	marcada para revisión
sm/chc		3	marcada para revisión
lw/c		3	marcada para revisión
cc/		2	marcada para revisión
sb/cc		1	marcada para revisión
sb/hic		1	marcada para revisión
as/chc		1	marcada para revisión
sm/spc		1	marcada para revisión
aa/hf		1	marcada para revisión
aa/		1	marcada para revisión
pt/d		1	marcada para revisión
pt/chc		1	marcada para revisión
lw/t		1	marcada para revisión
lw/chc		1	marcada para revisión
lw/lw		1	marcada para revisión

contienen una sílaba **sm/fs**. El mismo tramo de audio lleva, por tanto, una caja grande y varias pequeñas dentro de ella (Figura 4.11).

Figura 4.11

Anidamiento frase-sílaba



En la figura, la caja grande es una frase **lw/cc** y las pequeñas, las sílabas **lw/cs** que contiene. El detector no puede representar esa relación. Devuelve una lista plana de cajas, y el emparejamiento durante el entrenamiento asocia cada anotación con exactamente una predicción, sin manera de expresar que una caja contiene a otra. Conservar ambos niveles equivaldría a pedirle que informe dos veces el mismo sonido, como si fueran dos eventos superpuestos sin relación. Es la consecuencia operativa del anidamiento jerárquico que Kershenbaum et al. (2016) describen como propiedad general de las unidades acústicas animales: una secuencia de unidades puede ella misma considerarse una unidad. Las clases de frase se excluyen, en consecuencia, antes del entrenamiento.

4.4.2. Partición por grabación

Regla. La unidad de partición es la *grabación*: todas las ventanas de un mismo archivo caen en la misma partición. Repartir ventanas sueltas dejaría dos ventanas solapadas de una misma anotación en particiones distintas, y la prueba mediría memoria en lugar de generalización (Koga et al., 2024).

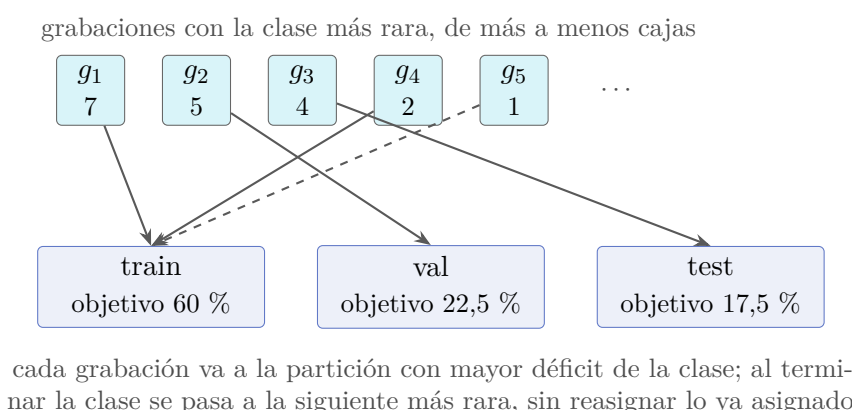
La alternativa no es un descuido hipotético. El prototipo inicial de este trabajo la cometió: barajaba los clips y los repartía por índice, con lo que dos ventanas solapadas

de la misma anotación podían quedar una en entrenamiento y otra en prueba. Sobre esa partición el clasificador alcanzaba un 99 % de exactitud, una cifra que no medía generalización sino memoria.

Algoritmo. La asignación es golosa y estratificada por clase (`split_manifest` en `src/data/manifest.py`); la Figura 4.12 la esquematiza y los pasos que siguen la detallan.

Figura 4.12

Reparto goloso por grabación



1. Se cuenta, por grabación, cuántas cajas de cada clase contiene, y se fija como objetivo de cada partición la fracción 60,0 / 22,5 / 17,5 del total de cada clase.
2. Las clases se recorren **de la más rara a la más común**. Para cada clase, las grabaciones que la contienen y aún no están asignadas se ordenan de más a menos cajas de esa clase, y cada una va a la partición con mayor déficit en esa clase; en caso de empate, a la que tenga menos ventanas respecto de su objetivo.
3. Las grabaciones sin cajas de ninguna clase (las ventanas vacías de la sección 4.4.3) van donde falten ventanas.

Recorrer las clases de la más rara a la más común hace que el reparto de las escasas se decida primero y no quede como residuo del de las abundantes: una clase con un centenar de anotaciones concentradas en pocas grabaciones podría, de otro modo, caer entera en una sola partición. La semilla (42) fija el orden inicial de las grabaciones y hace la partición reproducible.

Composición. La Tabla 4.9 recoge la composición de las tres particiones tras el ventaneo.

Tabla 4.9*Composición de las particiones*

Partición	Grabaciones	Ventanas	Vacías	% vacías	Cajas	GB
Entrenamiento	1 248	21 751	4 608	21,2 %	26 938	3,44
Validación	690	9 364	2 713	29,0 %	10 114	1,48
Prueba	601	7 598	2 357	31,0 %	7 931	1,20
Total	2 539	38 713	9 678	25,0 %	44 983	6,13

Nota. Hay más cajas que anotaciones porque una anotación puede caer en dos ventanas consecutivas: la ventana de 3 s avanza 1,5 s (sección 4.4.3).

1 grabación del conjunto de experimento está dañada y no puede leerse: no entra al caché aunque tenga anotaciones, y por eso el total de grabaciones es menor que las 2 527 que aportan alguna anotación de alguna clase.

La fracción de ventanas vacías difiere entre particiones porque las ventanas se reparten por grabación, no una a una. La Tabla 4.10 desglosa la composición por clase.

La proporción de entrenamiento se conserva clase a clase (última columna), mientras que el desbalance entre clases se mantiene, con 23 veces entre **ac/bc** (2 838 cajas de entrenamiento) y **sb/sc** (126). La Figura 4.13 muestra ambos hechos a la vez.

Cada barra conserva la misma proporción entre entrenamiento, validación y prueba; lo que cambia de una clase a otra es su longitud. Los puntos marcan las anotaciones de cada clase: en las llamadas largas, como **as/hc**, la barra las supera varias veces, porque cada llamada se reparte entre muchas ventanas, efecto que se mide a continuación.

4.4.3. Ventaneo

Regla de asignación. Las grabaciones tienen duración variable y el detector consume entradas de tamaño fijo, de modo que cada grabación se segmenta en ventanas de 3 s que avanzan 1,5 s. Una anotación entra en una ventana si el solapamiento entre ambas cubre al menos la mitad de la *parte visible* del evento, definida como el mínimo entre su duración y la longitud de la ventana (Figura 4.14). Comparar contra la duración completa dejaría inalcanzable a todo evento de más de 6 s, porque no hay posición de la ventana en la que quepa la mitad de un aullido de 30 s; con la parte visible, ese aullido entra en toda ventana que caiga dentro de él y ninguna anotación del experimento queda sin ventana. Al entrar, la caja se recorta a los límites de la ventana.

Figura 4.13
Cajas por clase y partición

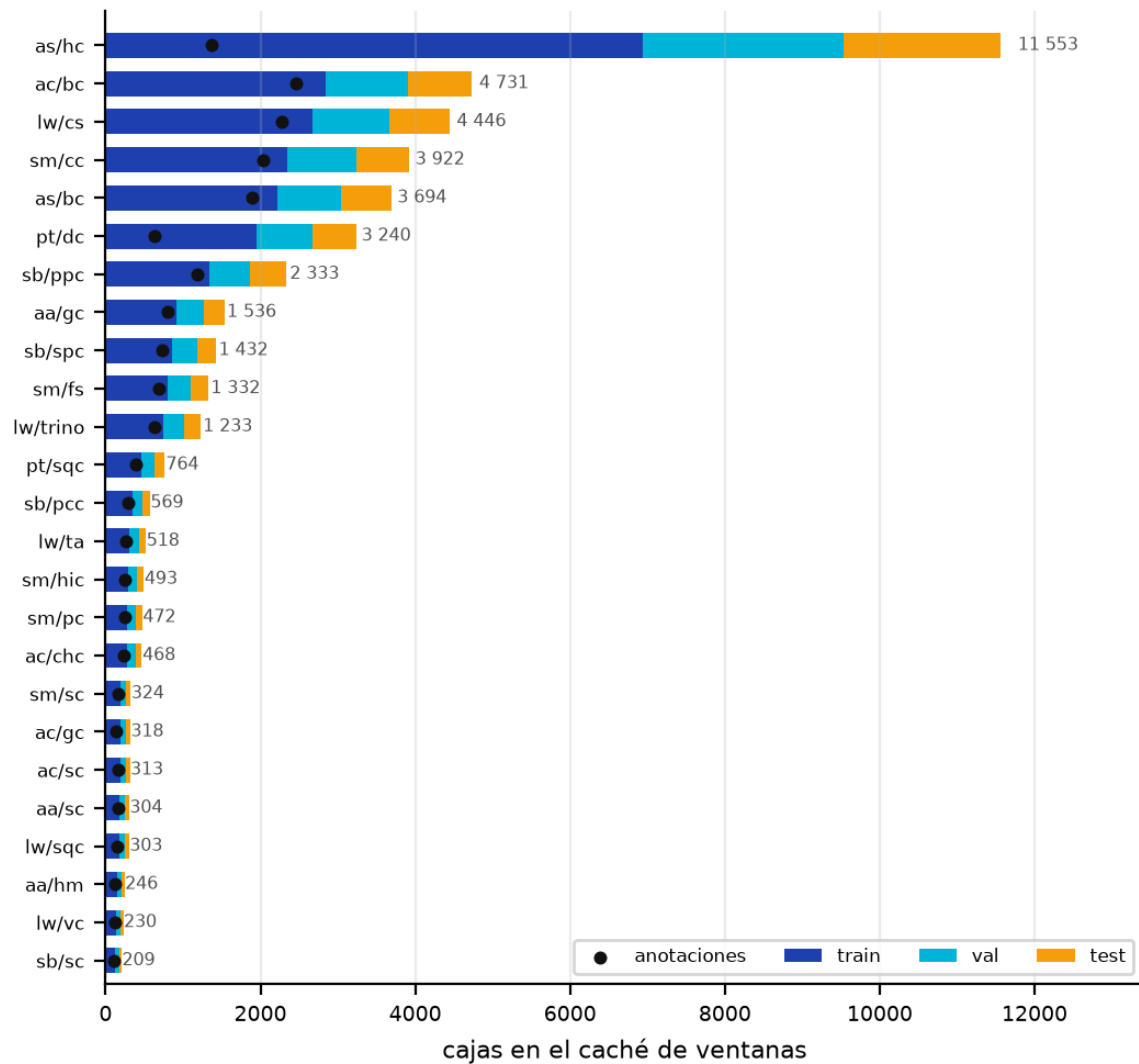


Figura 4.14
Asignación de eventos a ventanas

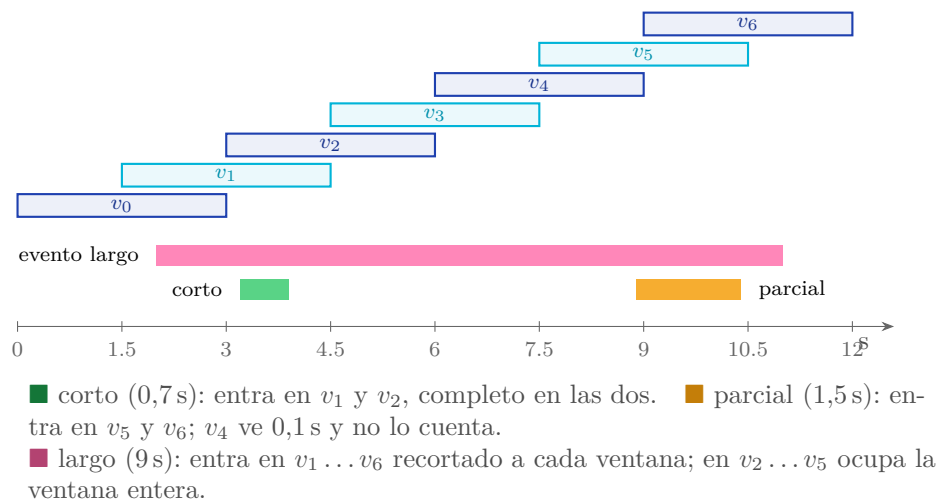


Tabla 4.10
Las 25 clases del experimento

Clase	Tipo de llamada	Anot.	Grab.	Train	Val	Test	Train %
ac/bc	bark call	2 453	137	2 838	1 065	828	60 %
lw/cs	contact syllable	2 279	248	2 667	1 000	779	60 %
sm/cc	contact call	2 031	466	2 353	883	686	60 %
as/bc	bark call	1 894	236	2 216	831	647	60 %
as/hc	howl call	1 375	697	6 933	2 599	2 021	60 %
sb/ppc	play peep call (ppc, lpc)	1 189	191	1 340	529	464	57 %
aa/gc	gulp call	797	58	922	345	269	60 %
sb/spc	spit call	729	166	859	323	250	60 %
sm/fs	food syllable	691	89	799	301	232	60 %
lw/trino	trino (tr, tj, tt, tf)	633	122	739	278	216	60 %
pt/dc	duet call	632	292	1 944	729	567	60 %
pt/sqc	unofficial squeal call	387	13	465	161	138	61 %
sb/pcc	peep contact call	290	95	341	128	100	60 %
lw/ta	terrestrial alarm call	265	80	311	116	91	60 %
sm/hic	hip call	254	61	296	111	86	60 %
sm/pc	purr call	242	83	283	106	83	60 %
ac/chc	chitter call	229	142	281	105	82	60 %
sm/sc	squeal call	163	33	195	73	56	60 %
aa/sc	squeak call	159	33	183	68	53	60 %
ac/sc	squeak call	158	31	188	71	54	60 %
lw/sqc	squeal call	154	43	182	69	52	60 %
ac/gc	growl call	140	65	191	71	56	60 %
aa/hm	hoot call	127	24	148	54	44	60 %
lw/vc	visual contact call	119	44	138	52	40	60 %
sb/sc	shriek call	105	51	126	46	37	60 %
Total		17 495	2 527	26 938	10 114	7 931	60 %

Nota. Las tres columnas de partición cuentan cajas en ventanas del caché, no anotaciones. **as/hc** y **pt/dc** multiplican su material porque cada llamada larga atraviesa varias ventanas (Tabla 4.11).

Los tres casos de la figura resumen la regla: el evento corto entra completo en las dos ventanas que lo contienen, el parcial solo en las que cubren la mitad de su parte visible, y el largo en todas las que caen dentro de él, recortado a cada una.

Efecto sobre los eventos largos. Lo que el ventaneo cobra es la fragmentación, y la Tabla 4.11 la mide. Cada anotación se reparte, en promedio, entre 2,6 ventanas (el salto de 1,5 s hace que casi todo evento breve aparezca en dos), pero las dos clases largas, **as/hc** y **pt/dc**, se reparten entre ocho y cinco. Casi todas sus cajas tocan al menos un borde temporal de la ventana y en la mayoría ocupan la ventana entera: **as/hc** aporta

8,4 cajas por anotación, el 99,6 % de ellas tocando un borde y el 77,5 % ocupando la ventana de lado a lado. Entre las dos aportan el 33 % de las cajas del conjunto.

Tabla 4.11

Efecto del ventaneo en las clases largas

Clase	Anot.	Cajas	Cajas por anotación	Anot. > 3 s	Cajas en el borde	Ventana entera
as/hc	1 375	11 553	8,4	94,7 %	99,6 %	77,5 %
pt/dc	632	3 240	5,1	91,0 %	99,2 %	63,5 %
Resto (23 clases)	15 488	30 190	1,9	0,3 %	12,1 %	0,2 %
Total	17 495	44 983	2,6	11,0 %	40,8 %	24,6 %

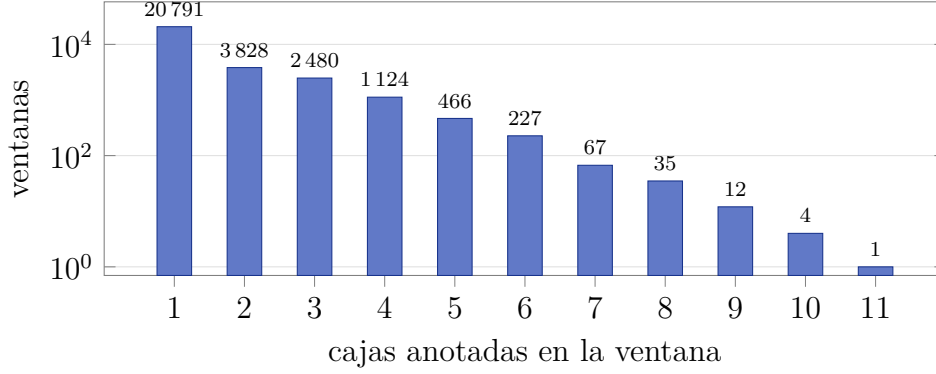
Nota. Una caja está en el borde cuando su lado temporal izquierdo o derecho coincide con el límite de la ventana, es decir, cuando el evento continúa fuera de ella; ocupa la ventana entera cuando toca los dos.

Esto se consigna aquí, y no entre las limitaciones, porque determina la lectura de cualquier resultado por clase en dos direcciones opuestas. Por un lado, las dos clases largas se vuelven las *más fáciles* del conjunto: una caja que ocupa la ventana completa se acierta con casi cualquier predicción grande. Por otro, al aportar el 33 % de las cajas, dominan cualquier cifra agregada: un *recall* global que las incluya mide, en un tercio de su peso, la capacidad de dibujar un rectángulo del tamaño de la ventana. Por eso el promedio por clase debe reportarse junto al agregado, y no en su lugar.

Ventanas vacías. A las ventanas con al menos una anotación se suma un 25 % de ventanas *vacías*, tomadas al azar (semilla 42) de tramos sin ninguna llamada y sin ningún evento a medias: una ventana que ve menos de la mitad de una llamada no cuenta como vacía. Sin ellas el modelo nunca ve fondo puro (selva, lluvia, insectos) y aprende que en toda entrada hay algo que reportar, que es justamente el sesgo que paga la carga de revisión. Son 9 678 ventanas, el 25,0 % del total.

Densidad de eventos por ventana. La Figura 4.15 cuenta cuántas cajas anotadas lleva cada ventana con contenido.

La mediana es 1, el máximo 11 y el 97 % de las ventanas lleva cuatro o menos. Es el máximo, y no la mediana, el que acota cuántas predicciones simultáneas debe poder emitir un detector por ventana.

Figura 4.15*Cajas anotadas por ventana*

4.4.4. Representación de entrada

Espectrograma log-mel. Cada ventana se transforma en un espectrograma mel de potencia con los parámetros de la Tabla 4.12, y el caché guarda ese mel sin comprimir, en `float32`. La compresión logarítmica y la normalización se aplican al leer el caché y no al construirlo, de modo que un mismo caché sirve a todos los modelos. El modelo propuesto las aplica en su primera capa: la compresión logarítmica y una normalización por lote. Los detectores de imagen lo convierten a decibelios y lo escalan a $[0, 1]$ con el rango en dB de la Tabla 4.12, dentro del modelo (Faster R-CNN) o al exportar las imágenes (YOLO y RT-DETR). Los once parámetros fijan el espectrograma de principio a fin: `torchaudio.transforms.MelSpectrogram` instanciada con estos valores lo reproduce.

Tres de los once parámetros responden directamente a la geometría de los eventos: la ventana de 3 s, que contiene entera a casi todas las clases; la trama de 1 024 muestras, más corta que los eventos más breves, y el rango de frecuencia hasta Nyquist, que no trunca las llamadas más agudas.

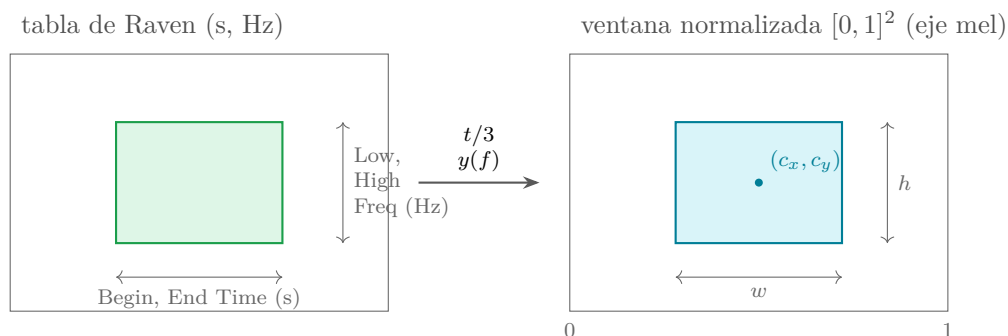
Normalización de las cajas. Las cajas se guardan en formato centro, ancho y alto normalizados al intervalo $[0, 1]$ de la ventana (Figura 4.16): el tiempo se divide por los 3 s de la ventana, y la frecuencia pasa por la misma escala mel que produce la imagen,

$$y(f) = \frac{m(f) - m(f_{\min})}{m(f_{\max}) - m(f_{\min})}, \quad m(f) = 2595 \log_{10} \left(1 + \frac{f}{700} \right).$$

La caja de la izquierda está en segundos y hercios, como la escribe Raven; la de la derecha, en fracciones de la ventana, como la consume el modelo. Gracias a esa escala

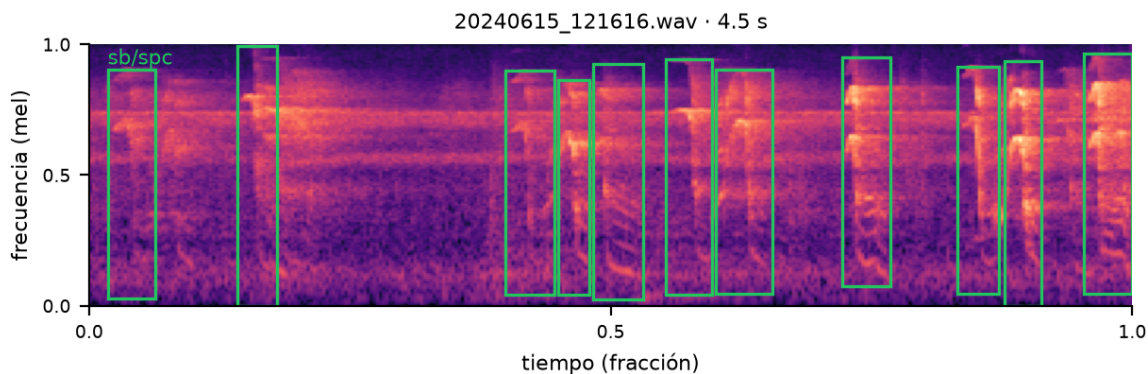
Tabla 4.12*Parámetros del espectrograma*

Parámetro	Valor	Justificación
Frecuencia de muestreo	44 100 Hz	Igual a la de las grabaciones, de modo que esta etapa no remuestrea; su mitad, 22 050 Hz, es el techo de Nyquist que fija el límite superior del rango de frecuencia
Ventana / salto	3,0 s / 1,5 s	Contiene entera a casi todas las clases; el costo es la fragmentación de las dos largas (Tabla 4.11)
Solapamiento mínimo	0,5	Un evento entra si la mitad de su parte visible cae en la ventana; medirlo contra la parte visible es lo que mantiene alcanzables a los eventos de más de 6 s
Longitud de ventana STFT	1 024 (23 ms)	Más corta que los eventos más breves: 34 ms el mínimo, 81 ms la mediana de <code>sm/cc</code>
Función de ventana	Hann	Valor por defecto de <code>torchaudio</code> ; ninguna clase mostró fuga espectral atribuible a la ventana
Puntos de la transformada rápida de Fourier	4 096	Relleno con ceros: densifica la rejilla de frecuencia sin tocar la resolución temporal
Salto STFT	400 (9 ms)	Produce 331 tramas por ventana
Bandas mel	128	Lo que espera el extractor preentrenado del modelo propuesto; 256 no dio ganancia medible
Escala mel	HTK ($q = 2\,595$, $f_{\text{quiebre}} = 700$ Hz)	Convención por defecto de <code>torchaudio</code> ; es la misma fórmula con la que se mapean las cajas
Rango de frecuencia	25 Hz – 22 050 Hz	Un techo de 16 kHz truncaba la mitad de las cajas de <code>sb/spc</code> , cuya frecuencia superior media supera los 16 kHz
Rango en dB	−54,3 a 27,9 dB	Percentiles 1 y 99,9 de $10 \log_{10}(\text{mel} + 10^{-6})$ sobre 2 000 ventanas de entrenamiento; los modelos que pintan el mel como imagen normalizan con él

Figura 4.16*Normalización de una caja*

común, el rectángulo que el modelo predice es el que Raven dibuja, sin reinterpretación, y la inversa $y \mapsto f$ recupera los hercios al exportar las predicciones.

Las tres figuras que siguen muestran ventanas del caché tal como las recibe el modelo, con el mel en decibelios y las cajas ya normalizadas. La Figura 4.17 es la más poblada del conjunto: las cajas se superponen y comparten banda, que es el caso que obliga al detector a separar eventos simultáneos.

Figura 4.17*La ventana más densa*

La Figura 4.18 es el caso contrario: una llamada más larga que la ventana, que entra cortada por los dos bordes. El modelo nunca ve su principio ni su final, y es el efecto que la Tabla 4.11 cuantifica.

La Figura 4.19 no tiene ninguna caja. Las ventanas vacías no se descartan: son las que enseñan al modelo qué no es un evento, y en este conjunto el fondo que contienen es el coro de insectos y las bandas de ruido estable que atraviesan el espectrograma de lado a lado.

Figura 4.18

Llamada larga, cortada en ambos bordes

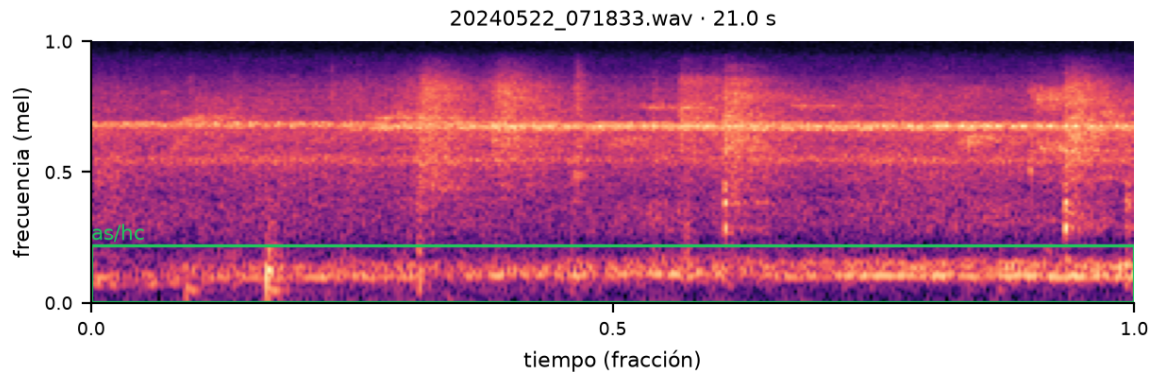
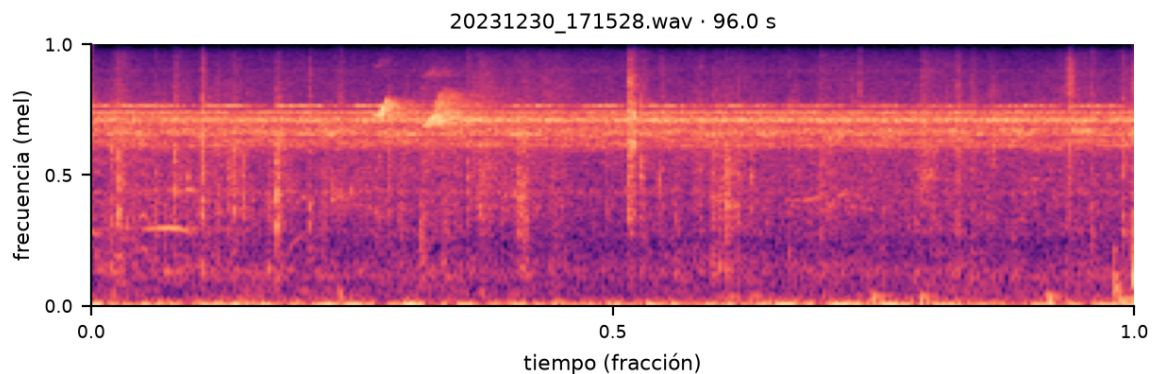


Figura 4.19

Ventana vacía



Las tres ventanas resumen la variedad que el caché entrega al modelo: eventos que se superponen, eventos más largos que la entrada y fondo sin ningún evento.

4.4.5. Entregables y verificación

`data/processed/` contiene los archivos de la Tabla 4.13, en total 6,1 GB. Los tres `.pt` son diccionarios con las mismas tres claves: `images`, un tensor $(N, 1, 128, 331)$ en `float32` con el mel de cada ventana; `boxes`, una lista de N tensores $(n_i, 4)$ con las cajas normalizadas; y `labels`, una lista de N tensores $(n_i,)$ con el índice de clase de cada caja según `labels.json`.

El indicador de RE1.2 pide tres comprobaciones, y las tres se verifican sobre el caché que está en disco:

1. **Los archivos procesados cargan sin error.** Los tres `.pt` se abren con `torch.load(path, mmap=True, weights_only=True)`, que lee cajas y etiquetas sin traer los espectrogramas a memoria, y se comprueban las claves y las

Tabla 4.13
Contenido del caché

Archivo	Contenido
<code>train.pt</code> , <code>val.pt</code> , <code>test.pt</code> <code>labels.json</code>	Ventanas de cada partición: mel, cajas y etiquetas Índice de clase → nombre especie/llamada , las 25 clases en orden alfabético
<code>meta.json</code>	Semilla, criterio de selección, exclusiones y fusiones, proporciones de partición, rango en dB y parámetros del espectrograma
<code>train_sources.json</code> , ...	Grabación y segundo de inicio de cada ventana, para volver al audio

formas: 21 751, 9 364 y 7 598 ventanas de (1, 128, 331). Lo confirman los dos usos que tuvo el caché: la exportación de `train.pt` y `val.pt` a las imágenes con que se entrenó el modelo final no encontró ninguna corrupta, y la evaluación leyó `val.pt` y `test.pt` completos. Los dos registros están en la carpeta RE1.2 del Anexo B.

2. **`meta.json` registra la semilla, el criterio de selección, las exclusiones, la etiqueta usada y la normalización.** El Listado 4.1 lo reproduce íntegro. Cada corrida de entrenamiento lo copia en su `config.json` y en el propio *checkpoint*, de modo que un modelo lleva consigo la descripción del conjunto con el que se entrenó.

3. **El código divide por archivo de grabación y no por ventanas aisladas.** Los `*_sources.json` de las tres particiones no comparten ni una sola grabación: las tres intersecciones son vacías. La partición se decide sobre la grabación entera, de modo que ninguna ventana de una grabación puede caer en un lado y su vecina solapada en otro.

Listado 4.1
`meta.json` del caché actual

```
{
  "seed": 42,
  "min_pair_count": 100,
  "empty_ratio": 0.25,
  "split_ratios": [
    0.6,
    0.225,
```

```

    0.175
],
"excluded_labels": [
    "lw/cc",
    "sm/fc",
    "sb/pcs"
],
"joined_labels": {
    "lw/tr": "lw/trino",
    "lw/tj": "lw/trino",
    "lw/tt": "lw/trino",
    "lw/tf": "lw/trino",
    "sb/lpc": "sb/ppc"
},
"db_range": [
    -54.27337650299072,
    27.8653164958956
],
"params": {
    "clip_len_s": 3.0,
    "clip_hop_s": 1.5,
    "min_overlap": 0.5,
    "pad_seed": 0,
    "target_sr": 44100,
    "n_fft": 4096,
    "win_length": 1024,
    "hop_length": 400,
    "n_mels": 128,
    "f_min": 25.0,
    "f_max": 22050.0,
    "mel_scale": "htk",
    "mel_break_hz": 700.0,
    "mel_scale_q": 2595.0,
    "eps": 1e-06
}

```

```
| }  
| }
```

La construcción es determinista. Con la misma `cleaned/`, la misma semilla y los mismos parámetros se obtienen las mismas ventanas en el mismo orden, de modo que el conjunto no depende de cuándo se construyó. Volver a aplicar hoy las cuatro reglas de selección sobre `data/cleaned/` devuelve 25 clases, las mismas 25 de `labels.json` y en el mismo orden: el conjunto de experimento no depende de cuándo se ejecutó el *pipeline* sino de las reglas y de la semilla, que es lo que exige el indicador.

4.5. Ficha del conjunto derivado (RE1.3)

Resultado. La ficha del conjunto derivado: esta sección, su versión en un solo archivo, que es el `README.md` de la carpeta RE1.3 de los medios de verificación (Anexo B), y los metadatos del caché (`meta.json` y `labels.json`, en la carpeta RE1.2). Describe las 8 especies, el vocabulario, la composición, el desbalance, el diccionario de campos, el ventaneo, la normalización, las exclusiones y las limitaciones conocidas, que son los nueve elementos de su indicador.

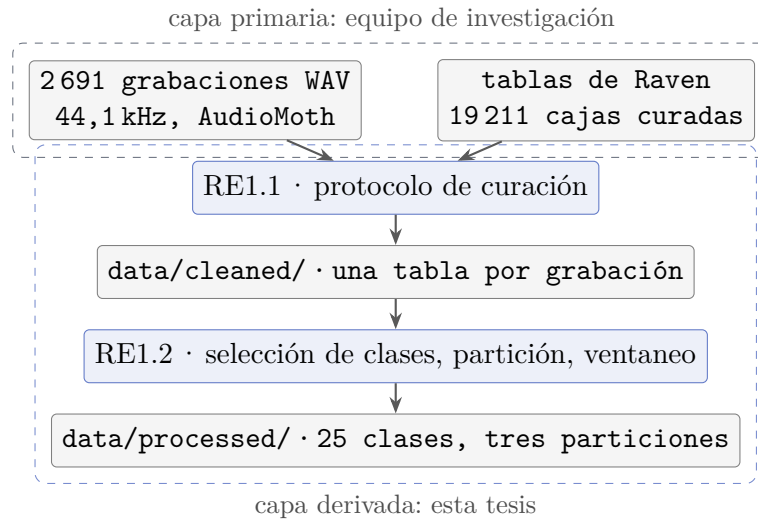
Cómo se alcanzó. Es la descripción de los datos que pide la Fase 2 de CRISP-DM (sección 1.7.1), hecha sobre el conjunto ya construido y con la estructura de las *datasheets* para conjuntos de datos (Gebru et al., 2021): propósito, composición, recolección, transformaciones, usos previstos y limitaciones. Se omite deliberadamente el apartado de distribución y mantenimiento que esa estructura contempla: publicar el conjunto o sostenerlo como recurso de terceros está fuera del alcance de la tesis y es una decisión del equipo de investigación, no del autor. La ficha distingue dos capas de responsabilidad (Figura 4.20), y esa distinción gobierna todo lo que sigue.

Cómo reproducirlo. Ninguna cifra, tabla o figura de la ficha se escribe a mano. El cuaderno del capítulo, publicado en el repositorio junto al documento, las recalcula a partir de `data/raw/`, `data/cleaned/` y `data/processed/`, de modo que la ficha se regenera cada vez que el conjunto cambia.

Medio de verificación. Esta sección es la ficha. Cada uno de los nueve elementos del indicador está en un apartado identificado del capítulo, y la sección 4.5.10 indica

dónde. El README y los metadatos que nombra el medio de verificación están en las carpetas RE1.3 y RE1.2 del Anexo B.

Figura 4.20
Capas del conjunto



La capa primaria, formada por las grabaciones y las tablas de Raven, pertenece al equipo de investigación, y la ficha la describe sin modificarla. La capa derivada, desde la curación hasta el caché, es la contribución de esta tesis, y la ficha la documenta paso a paso.

4.5.1. Propósito y autoría

Finalidad. El conjunto derivado se construyó como material de entrenamiento y evaluación del detector de OE2, sobre vocalizaciones de primates neotropicales grabadas con sensores pasivos de bajo costo. Las anotaciones primarias existían antes de esta tesis, producidas para el estudio del repertorio vocal de cada especie; la capa derivada las convierte en un conjunto consistente, particionado y documentado.

Autoría. Capa primaria: el equipo del 8 Primates Project (Wired Amazon, s.f.), que opera los sensores y anota en Raven. Capa derivada: el autor, con la supervisión del asesor.

4.5.2. Composición

El conjunto derivado tiene dos niveles de instancia. En `cleaned/` la instancia es la *anotación*: una caja tiempo–frecuencia sobre una grabación, con especie y tipo de

llamada. En **processed/** la instancia es la *ventana* de 3 s: un espectrograma mel con cero o más cajas normalizadas.

En total, 2 691 grabaciones anotadas que suman 25,0 horas de audio, 19 211 anotaciones sobre 8 especies y 37 pares del vocabulario; de ellas, 17 495 anotaciones en 25 clases forman el conjunto de experimento, sobre 7 de las 8 especies. Quedan fuera del experimento, pero dentro de **cleaned/**, las 643 anotaciones marcadas para revisión y los pares que no alcanzan el umbral. La Tabla 4.14 lo desglosa por especie.

Tabla 4.14
Composición por especie

Especie	Cód.	Grab.	Horas	Anot.	Pares	Clases	Anot. en experimento
<i>Leontocebus weddelli</i>	LW	410	3,7	4 159	8	5	3 450
<i>Sapajus macrocephalus</i>	SM	582	5,3	3 668	8	5	3 381
<i>Alouatta sara</i>	AS	778	7,2	3 619	2	2	3 269
<i>Ateles chamek</i>	AC	384	3,7	3 106	6	4	2 980
<i>Saimiri boliviensis</i>	SB	256	2,8	2 348	4	4	2 313
<i>Plecturocebus toppini</i>	PT	320	2,8	1 185	5	2	1 019
<i>Aotus azarae</i>	AA	82	0,8	1 095	3	3	1 083
<i>Cebus cuscinus</i>	CC	7	0,0	31	1	0	0
Total		2 691	25,0	19 211	37	25	17 495

Nota. Grabaciones y horas cuentan las que tienen al menos una anotación de la especie; una grabación con dos especies anotadas cuenta en ambas, por lo que las filas suman más que el total. Pares: tipos de llamada del vocabulario con al menos una anotación. Clases: pares que entran al experimento.

Desbalance. La distribución es de cola larga en dos niveles. Entre especies, *Leontocebus weddelli* aporta 8 tipos de llamada y *Cebus cuscinus* uno solo, con tan pocas anotaciones que no llega a ser clase. Dentro de una especie la distribución es tan sesgada como entre especies: los diez pares mayores concentran el 76 % de las anotaciones en vocabulario y la cola llega hasta pares con una sola (Figura 4.10). Entre **ac/bc**, con 2 487 anotaciones, y **sb/sc**, con 105, median más de veinte veces.

4.5.3. Geometría de los eventos

Las llamadas anotadas son extraordinariamente desiguales en tamaño, y ese hecho condiciona todas las decisiones de diseño posteriores. La más larga dura 73,3 s y la más corta 34 ms, una razón de 2 134×; el ancho de banda va de 224 a 21 784 Hz, 97×. La magnitud se aprecia mejor por comparación: en COCO, la colección con la que se

evalúan habitualmente los detectores de objetos (Lin et al., 2014), los objetos mayores superan a los menores en un factor de entre 10 y 100. El problema que enfrenta este detector es, en ese eje, un orden de magnitud más severo que aquel para el que se diseñaron las arquitecturas que hereda, y la consecuencia inmediata es que ninguna longitud de ventana única sirve para todas las clases.

La Tabla 4.15 sitúa cada clase en duración y banda, y hace visibles dos efectos a la vez. Las **especies se separan en buena medida por banda**: *Alouatta sara* ocupa el grave, por debajo de 1,5 kHz, mientras que la banda mediana de **lw/cs** empieza por encima de los 8 kHz. Los **tipos de llamada de una misma especie no se separan**: en LW, los trinos, **lw/vc** y **lw/cs** comparten banda y duración casi por completo.

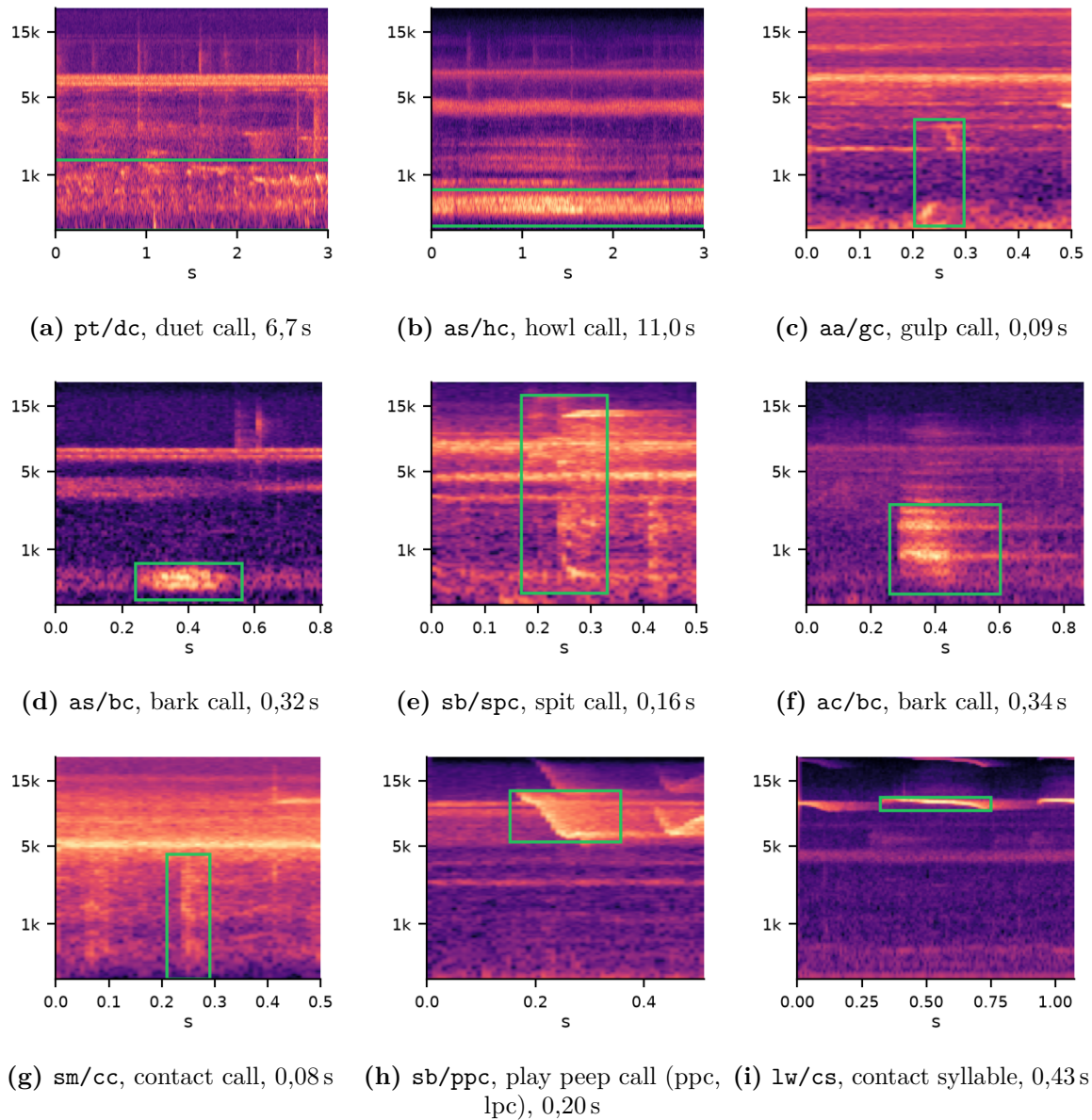
Tabla 4.15

Duración y banda por clase

Clase	Anot.	Duración (s) mediana [RIC]	Banda (Hz)	Ancho de banda (Hz)	CV del ancho
as/hc	1 375	11,0 [7,1–15,8]	47–769	718	0,23
pt/dc	632	6,7 [4,8–9,5]	25–1 474	1 385	0,68
ac/chc	229	1,2 [0,89–1,6]	192–3 378	3 208	0,50
ac/sc	158	1,2 [0,87–1,4]	2 640–10 726	8 135	0,34
ac/gc	140	1,0 [0,66–1,9]	198–1 465	1 276	0,55
lw/sqc	154	0,81 [0,60–1,0]	5 034–12 950	7 753	0,48
sb/pcc	290	0,73 [0,50–1,1]	6 536–11 755	4 955	0,34
sm/sc	163	0,59 [0,35–0,82]	3 381–10 753	6 946	0,36
sm/pc	242	0,55 [0,40–0,82]	2 806–4 435	1 542	0,42
lw/trino	633	0,53 [0,35–0,73]	5 744–11 126	5 459	0,29
lw/cs	2 279	0,43 [0,34–0,50]	8 038–10 215	2 099	0,34
sb/sc	105	0,43 [0,35–0,58]	2 534–15 464	13 243	0,23
lw/vc	119	0,42 [0,33–0,49]	5 438–8 808	3 491	0,40
lw/ta	265	0,42 [0,31–0,55]	1 546–6 880	5 151	0,27
ac/bc	2 453	0,34 [0,24–0,50]	374–2 616	2 234	0,60
as/bc	1 894	0,32 [0,26–0,40]	105–667	566	0,30
pt/sqc	387	0,27 [0,24–0,31]	4 338–5 078	666	0,42
sm/hic	254	0,27 [0,19–0,36]	249–4 927	4 649	0,54
aa/sc	159	0,23 [0,21–0,27]	107–13 147	12 790	0,19
aa/hm	127	0,23 [0,20–0,27]	66–529	452	0,17
sb/ppc	1 189	0,20 [0,15–0,30]	7 010–11 185	3 898	0,47
sm/fs	691	0,19 [0,15–0,26]	1 210–2 438	1 292	0,63
sb/spc	729	0,16 [0,13–0,22]	249–15 931	15 601	0,18
aa/gc	797	0,09 [0,08–0,12]	97–3 074	2 857	0,35
sm/cc	2 031	0,08 [0,06–0,11]	523–3 604	3 064	0,33

Nota. RIC: rango intercuartílico (percentiles 25–75). La banda es [mediana de $f_{\min}$, mediana de $f_{\max}$] por clase; el ancho de banda es la mediana de $f_{\max} - f_{\min}$ anotación por anotación, no la resta de la columna anterior. CV: coeficiente de variación del ancho de banda.

Figura 4.21
Galería de llamadas



Esa disociación tiene una consecuencia metodológica directa: si la geometría distingue especies pero no tipos de llamada, entonces *localizar* y *etiquetar* son problemas de dificultad distinta y no deben puntuarse con una sola cifra.

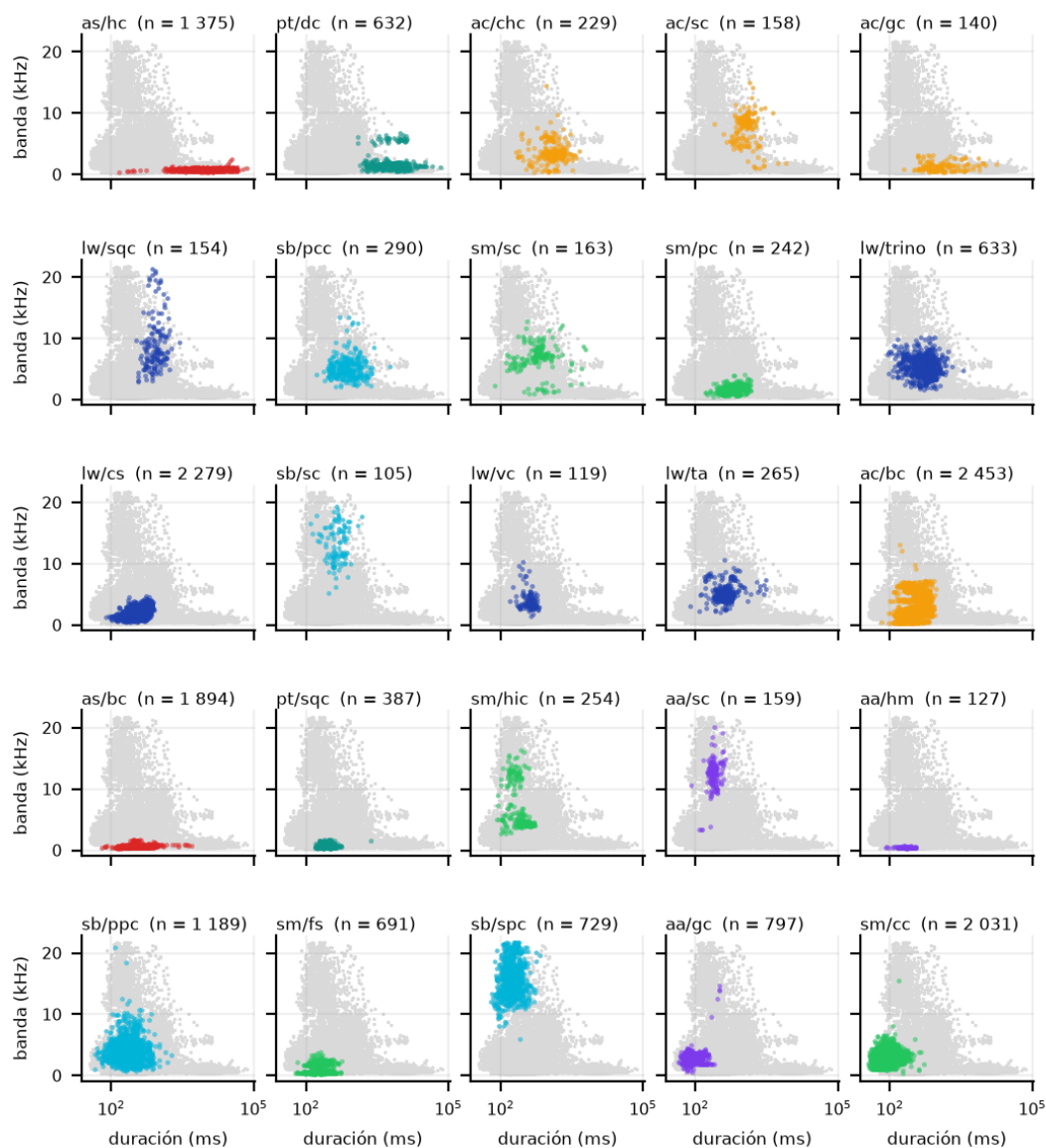
La Figura 4.21 muestra 9 clases sobre el espectrograma: la más anotada de cada especie y las siguientes en volumen, ordenadas por banda del grave al agudo.

Ordenadas así, las llamadas recorren el espectrograma del grave al agudo, y se ve a simple vista lo que la Tabla 4.15 cuantifica: cada especie ocupa su propia banda, mientras que las llamadas de una misma especie se parecen entre sí.

La Figura 4.22 desglosa la misma geometría clase por clase, y revela un problema distinto del desbalance.

Figura 4.22

Duración frente a ancho de banda

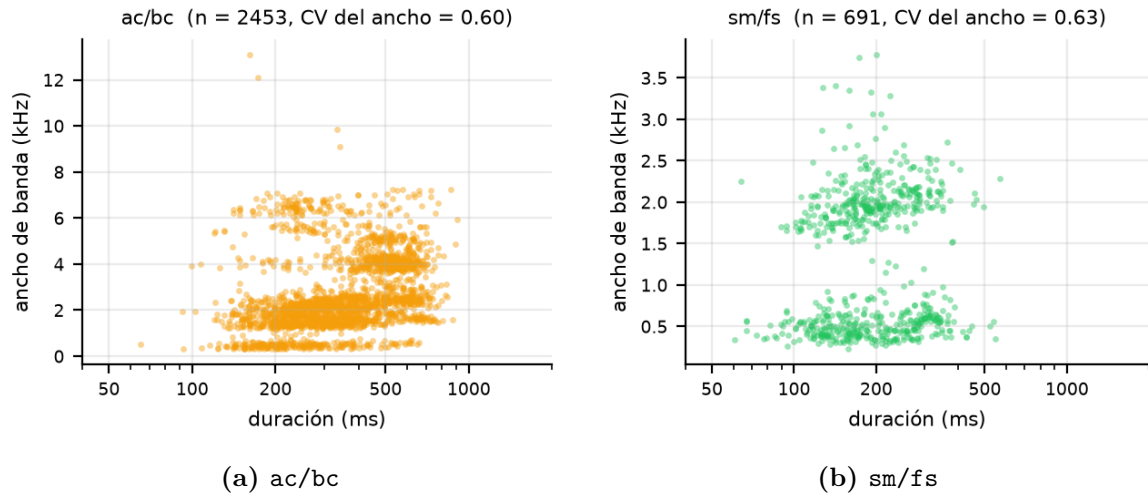


En **ac/bc** y **sm/fs** (coeficientes de variación del ancho de banda 0,60 y 0,63) la nube de duración frente a ancho de banda se parte en dos grupos separados, como se aprecia al aislar esas dos clases (Figura 4.23).

Que una etiqueta cubra dos nubes disjuntas sugiere que agrupa dos variantes de llamada que el vocabulario actual no distingue. Es un hallazgo sobre el esquema de anotación, no sobre el modelo.

Figura 4.23

Etiquetas con dos formas

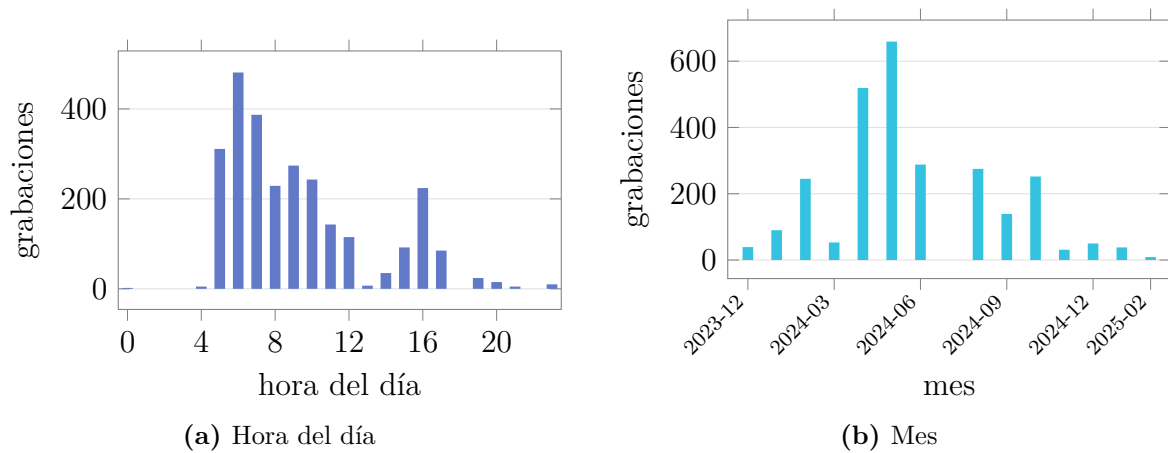


4.5.4. Recolección (capa primaria)

Las grabaciones se capturaron en el Tambopata Research Center, en la Reserva Nacional Tambopata (Madre de Dios, Perú), con dispositivos AudioMoth a 44,1 kHz, en archivos WAV de duración corta: el 93 % dura un minuto o menos, con mediana de 31 s. Los nombres de archivo codifican la fecha y la hora de inicio: el material va del 14/12/2023 al 27/02/2025, con la distribución por hora y por mes de la Figura 4.24. Las anotaciones se crearon en Raven Pro: el analista dibuja una región rectangular sobre el espectrograma, el programa deriva de ella los tiempos y las frecuencias, y las columnas de especie y tipo de llamada se escriben a mano.

Figura 4.24

Distribución temporal



La mayoría de las grabaciones se hicieron entre las cinco y las doce de la mañana, con un segundo bloque a media tarde, y el mes con más grabaciones es 05/2024. El

horario coincide con el de la toma de datos en campo que declara el proyecto, de 5 a 11 y de 16 a 18 horas (Wired Amazon, s.f.). Salvo el sitio y ese horario, el protocolo de campo no acompaña al material entregado, de modo que la ficha describe lo que puede reconstruirse del propio corpus, a partir de los nombres de archivo y de las cabeceras de los WAV, y declara el resto como información de procedencia no disponible (sección 4.5.9).

4.5.5. Transformaciones (capa derivada)

Cada transformación es un programa del repositorio y se documentó en las secciones anteriores; la Tabla 4.16 las enumera para que la ficha se lea sin saltar atrás.

Tabla 4.16

Transformaciones aplicadas

Etap	Qué hace	Dónde
Curación	Normaliza columnas y etiquetas, fija la especie por carpeta, descarta ruido y cajas degeneradas, recorta a Nyquist, marca los pares fuera de vocabulario y une las copias de una misma grabación	§4.3
Selección de clases	Excluye las frases anidadas, fusiona variantes de trino y de pío, exige 100 anotaciones por clase	§4.4.1
Partición	Reparte las grabaciones enteras en train/val/test (60,0 / 22,5 / 17,5), estratificando por clase de la más rara a la más común	§4.4.2
Ventaneo	Corta cada grabación en ventanas de 3 s con salto de 1,5 s; una anotación entra si la ventana cubre la mitad de su parte visible; añade 25 % de ventanas vacías	§4.4.3
Representación	Espectrograma mel de potencia (128 bandas, 25 Hz–22 050 Hz, HTK) y cajas normalizadas (c_x, c_y, w, h) con el eje de frecuencia en escala mel	§4.4.4

Ninguna etapa modifica la capa primaria: todas leen de **data/raw/** o de la etapa anterior y escriben en un directorio distinto, de modo que el conjunto derivado puede reconstruirse desde cero cada vez que una regla cambia.

4.5.6. Diccionario de datos

El conjunto se organiza en tres directorios, uno por etapa, con la estructura que sigue. Las tablas de `cleaned/` tienen las columnas de la Tabla 4.17, y el caché de `processed/`, las estructuras de la Tabla 4.18.

```
data/
raw/<nombre_comun>__<COD>/      capa primaria, sin modificar
    <AAAAMDD_HHMMSS>.wav
    <AAAAMDD_HHMMSS>.Table.1.selections.txt
cleaned/<nombre_comun>__<COD>/  RE1.1: una tabla normalizada por
    grabacion
    <AAAAMDD_HHMMSS>.txt
processed/                      RE1.2: el cache de ventanas
    train.pt  val.pt  test.pt
    labels.json  meta.json
    train_sources.json  val_sources.json  test_sources.json
```

Tabla 4.17

Columnas de las tablas curadas

Columna	Tipo	Significado
species	texto	Código de especie en minúscula (lw); el de la carpeta
written_species	texto	Lo que traía la columna Species , en minúscula
call_type	texto	Código del tipo de llamada normalizado (cs); puede estar fuera del vocabulario
begin_time_s	real	Inicio de la caja, en segundos desde el inicio de la grabación
end_time_s	real	Fin de la caja, en segundos; mayor que begin_time_s + 0,01
low_freq_hz	real	Frecuencia inferior de la caja, en Hz
high_freq_hz	real	Frecuencia superior, en Hz; no mayor que 22050
requires_review	lógico	La fila no pudo asignarse con certeza a un par del vocabulario

Al cargar con `data.annotations.load_annotations()` se añaden `audio_path`, `duration_s` y `bandwidth_hz`, y se unen las copias de una misma grabación (sección 4.3.2).

Tabla 4.18
Estructuras del caché

Clave o archivo	Tipo	Significado
<code>images</code>	<code>float32</code> ($N, 1, 128, 331$)	Mel de potencia de cada ventana, sin comprimir
<code>boxes</code>	lista de ($n_i, 4$)	Cajas (c_x, c_y, w, h) en $[0, 1]$: x es tiempo, y frecuencia en escala mel
<code>labels</code>	lista de ($n_i,$) <code>int64</code>	Índice de clase de cada caja, según <code>labels.json</code>
<code>labels.json</code>	JSON	Índice $\rightarrow$ nombre de clase
<code>meta.json</code>	JSON	Listado 4.1
<code>*_sources.json</code>	JSON	<code>recordings</code> , <code>recording_of_window</code> y <code>clip_start_s</code> : qué grabación y qué segundo origina cada ventana

Las dos tablas son el diccionario de campos que exige el indicador: cada columna o clave tiene su tipo y su significado, y con ellas un tercero puede leer el conjunto sin consultar el código.

4.5.7. Usos

Uso previsto. Entrenar y evaluar detectores de cajas tiempo–frecuencia (OE2) y medir la reducción del tiempo de revisión que aportan (OE3). Las particiones son fijas: validación elige *checkpoints* y umbrales, prueba se mide una vez.

Usos secundarios. Clasificación de ventanas (una etiqueta por ventana, derivable de las cajas), detección agnóstica a la clase y estudio de la geometría del repertorio por especie.

Usos no recomendados. Estimar abundancia o presencia de las especies: las grabaciones se seleccionaron para anotar, no por muestreo del paisaje. Comparar tipos de llamada entre especies como si fueran categorías homólogas: el vocabulario es por especie. Inferir el comportamiento del detector sobre las clases de frase excluidas o sobre las especies sin clase.

4.5.8. Versión del conjunto

La versión del conjunto derivado queda identificada por `meta.json` (que registra la semilla, el criterio de selección, las exclusiones, la etiqueta usada y la normalización) y por la huella SHA-256 de la lista de grabaciones de cada partición. Dos capas derivadas

con el mismo `meta.json` y las mismas huellas son el mismo conjunto. Cuando el equipo entrega anotaciones nuevas o corrige las existentes, el conjunto se vuelve a construir desde `data/raw/` y la versión cambia.

Fuera de alcance. La distribución del conjunto derivado y su mantenimiento como recurso público no forman parte de esta tesis: las grabaciones y las anotaciones primarias pertenecen al equipo de investigación y el trabajo aquí descrito termina en la capa derivada que alimenta a OE2 y OE3.

4.5.9. Limitaciones

- **Anotación de una sola fuente.** No hay medición de acuerdo entre anotadores, de modo que el error de etiquetado residual no puede separarse del error del modelo.
- **La columna de tipo de llamada llega sucia y no se puede sanear sin el repertorio de referencia.** 32 pares quedan fuera del vocabulario cerrado y, entre ellos, no hay forma de separar el error tipográfico de la subdivisión legítima aún no documentada (sección 4.3.2). El protocolo los marca y los conserva en lugar de decidir, pero el conjunto de experimento se construye asumiendo que lo que quedó fuera es ruido. Si parte de ello fuera repertorio real, el vocabulario está incompleto: es la vía de mejora más directa para una versión futura del detector, y no depende de la arquitectura sino de ampliar la referencia.
- **Procedencia de campo incompleta.** Del despliegue solo se conoce el sitio. El material no viene acompañado del protocolo (número y ubicación de los sensores, ganancia, ciclo de trabajo y criterio con que se eligieron los archivos que se anotaron), de modo que la ficha reconstruye lo que se puede leer del propio corpus y nada más. El horario de campo del proyecto explica el sesgo horario que se describe arriba, pero no permite saber cuánto de él se debe también al ritmo de actividad de las especies.
- **Cajas dibujadas con criterio variable en frecuencia.** El eje de frecuencia lo fija el analista a ojo, lo que obliga a emparejar las predicciones con un umbral de IoU tolerante.

- **Fragmentación de las llamadas largas.** *as/hc* y *pt/dc* sobreviven al ventaneo como series de cajas recortadas al borde de la ventana, sin ningún mecanismo que las reagrupe en el evento original (Tabla 4.11). Corregirlo exigiría una segunda longitud de ventana para esas clases, o una agregación que reconstruya el evento completo a partir de sus fragmentos antes de puntuarlo; ninguna de las dos se implementó.
- **Frases excluidas.** Las clases de frase quedan fuera por anidamiento y dejan sin cubrir el nivel superior de la jerarquía vocal.
- **Cobertura parcial del repertorio.** Los tipos de llamada bajo el umbral de 100 anotaciones y los 32 pares fuera de vocabulario no entran al experimento; *Cebus cuscinus* no tiene ninguna clase.
- **Escasez en 11 clases,** que aportan menos de 300 cajas de entrenamiento cada una. Cuando una de ellas obtiene mal *recall*, la causa puede ser el modelo o puede ser que nunca vio material suficiente.
- **Etiquetas que esconden dos formas.** *ac/bc* y *sm/fs* agrupan cada una dos variantes geométricas distintas (Figura 4.23).
- **Copias de grabaciones en la capa primaria.** El mismo audio aparece en varias carpetas de especie, anotado solo para la especie de cada carpeta; la capa derivada las une, pero la llamada de una tercera especie no anotada en ninguna copia sigue sin caja.
- **Sesgo horario y estacional.** La mayoría de las grabaciones son matinales y se concentran en pocos meses (Figura 4.24); no hay garantía de que el fondo acústico de otras horas o estaciones esté representado.
- **Un solo sistema de captura.** Todo el material proviene de AudioMoth; la transferencia a otros sensores no está medida.

4.5.10. Verificación de RE1.3

El indicador de RE1.3 enumera nueve elementos que la ficha debe describir. La Tabla 4.19 indica, para cada uno, el apartado, la tabla o la figura del capítulo donde está documentado.

Tabla 4.19
Verificación de RE1.3

Elemento del indicador	Dónde está documentado
Las ocho especies	Composición por especie (Tabla 4.14)
Vocabulario	Vocabulario cerrado (Tabla 4.4) y pares marcados para revisión (Tabla 4.5)
Composición	Tablas 4.14, 4.9 y 4.10
Desbalance	Composición, párrafo “Desbalance”, y Figuras 4.10 y 4.13
Diccionario de campos	Diccionario de datos (Tablas 4.17 y 4.18)
Ventaneo de 3 s con salto de 1,5 s	Sección 4.4.3, Figura 4.14 y Tabla 4.11
Normalización log-mel	Compresión logarítmica y rango en dB (sección 4.4.4 y Tabla 4.12); clave <code>db_range</code> de <code>meta.json</code> (Listado 4.1)
Exclusiones	Selección de clases (sección 4.4.1) y pares excluidos (Tabla 4.8)
Limitaciones conocidas	Sección 4.5.9

Los nueve elementos están documentados, y todos con cifras generadas a partir del propio conjunto. El README del conjunto que nombra el medio de verificación es el de la carpeta RE1.3 del Anexo B: reúne la ficha en un solo archivo, sigue el orden de los nueve elementos de la tabla y lleva las mismas cifras, generadas por el mismo cuaderno.

4.6. Discusión

Qué se obtuvo. El objetivo específico se cumple con sus tres resultados. El protocolo (RE1.1) convierte 20 082 filas escritas a mano, con 63 grafías distintas de tipo de llamada, en 19 211 anotaciones sobre un vocabulario cerrado de 31 códigos, y deja marcadas (no borradas) las 643 que no puede resolver por sí solo. El conjunto particionado (RE1.2) lleva esas anotaciones a 38 713 ventanas y 44 983 cajas repartidas por grabación en 60,0 / 22,5 / 17,5, sin una sola grabación compartida entre particiones. La ficha (RE1.3) documenta ambas capas y declara doce limitaciones conocidas.

Qué significan. El resultado central no es el tamaño del conjunto sino su *trazabilidad*: cada anotación que entra al experimento puede seguirse hasta la fila cruda de la que salió y hasta la regla que la tocó, y cada cifra de este capítulo se regenera con un comando. Eso es lo que permite atribuir una diferencia de desempeño entre detectores a la arquitectura y no al conjunto. Dos propiedades del conjunto condicionan de antemano

la lectura de esos resultados. La primera es que la geometría separa especies pero no tipos de llamada dentro de una especie (sección 4.5.3), de donde se sigue que localizar y clasificar son problemas de dificultad distinta y que puntuarlos con una sola cifra ocultaría cuál de los dos falla. La segunda es que el ventaneo de 3 s multiplica las dos clases largas hasta el 33 % de las cajas, de modo que cualquier promedio agregado está dominado por las clases más fáciles del conjunto; de ahí la insistencia en el promedio por clase.

Consistencia con la literatura. Las tres decisiones de mayor consecuencia tienen precedente. Cerrar el vocabulario responde a la variabilidad entre investigadores de las definiciones de unidad que documentan Kershenbaum et al. (2016); excluir las clases de frase es la consecuencia operativa del anidamiento jerárquico que ese mismo trabajo describe, y que un detector de lista plana no puede representar; y particionar por archivo de grabación evita que la variación de las etiquetas fuertes se sume a una fuga de datos que inflaría la evaluación (Koga et al., 2024). La forma de la ficha sigue las *datasheets* de Gebru et al. (2021), cuya motivación (que no existe un proceso estandarizado para documentar conjuntos de datos) se verifica en este caso: nada del material recibido venía acompañado de una descripción de su composición o de sus sesgos.

Generalización. El protocolo es transferible en dos niveles distintos. Su *estructura* (normalizar, sanear, cerrar el vocabulario, marcar lo dudoso en lugar de descartarlo y unir copias) no depende de este corpus y se aplica a cualquier material de PAM anotado en Raven por varias personas a lo largo del tiempo. Su *contenido* sí es propio: el mapa de sinónimos, las correcciones de pares imposibles y el vocabulario de 41 pares describen a estas ocho especies y a quienes las anotaron. Otro corpus reutilizaría el programa y reescribiría esas tres tablas. Las decisiones de RE1.2 son más dependientes del material: la ventana de 3 s y el umbral de 100 anotaciones se eligieron para este reparto de duraciones y este volumen, y habría que revisarlos antes de reutilizarlos.

Limitaciones. La sección 4.5.9 las declara todas; tres pesan sobre lo que sigue. La falta de una medición de acuerdo entre anotadores deja sin techo la evaluación, y es la única de las tres que no puede corregirse sin trabajo del equipo de investigación. La fragmentación de las llamadas largas queda documentada pero no resuelta. Y la exclusión de las clases de frase hace que el sistema que esta tesis evalúa detecte sílabas, no secuencias.

Capítulo 5

Detección y Evaluación del Modelo (OE2)

5.1. Introducción

Este capítulo presenta los resultados alcanzados para el segundo objetivo específico:

OE2. Implementar y evaluar un detector que reciba un espectrograma y devuelva cajas tiempo–frecuencia con especie, tipo de llamada y confianza, comparándolo con una línea base establecida.

Los cinco resultados comprometidos en la Tabla 1.5 ocupan una sección cada uno: el diseño experimental, la arquitectura propuesta y las líneas base (RE2.1, sección 5.2), el *pipeline* reproducible de entrenamiento, evaluación e inferencia (RE2.2, sección 5.3), el informe comparativo de resultados (RE2.3, sección 5.4), el modelo final cargable y su demostración (RE2.4, sección 5.5) y la publicación del código y del modelo (RE2.5, sección 5.6). Cada sección sigue la estructura del Capítulo 4: el resultado, cómo se alcanzó, cómo reproducirlo y la verificación de su indicador. La sección 5.7 discute los cinco en conjunto.

El conjunto sobre el que se entrena y se mide es el que produce el Capítulo 4: 21 751 ventanas de entrenamiento, 9 364 de validación y 7 598 de prueba, repartidas por grabación y con 25 clases de especie y tipo de llamada. El costo de revisión que genera el modelo elegido aquí es materia de OE3.

5.2. Diseño experimental, arquitectura propuesta y líneas base (RE2.1)

Resultado. La arquitectura propuesta (un AST-Deformable-DETR de dimensión 128, 100 consultas y 4 niveles, implementada en `src/models/`), sus hiperparámetros, las tres líneas base contra las que se compara, las recetas de entrenamiento y el protocolo de comparación con su regla de decisión fijada antes de correr el experimento.

Cómo se alcanzó. Aplicando la Fase 4 de CRISP-DM (sección 1.7.1), que fija la forma de la salida, las configuraciones que se comparan y la regla de decisión antes de entrenar. El punto de partida es la formulación como detección de objetos sobre el espectrograma (sección 2.1.3) y la representación que fija el Capítulo 4: el modelo recibe exactamente el tensor que el caché almacena y devuelve cajas en el mismo marco normalizado en el que se anotaron. Las tres decisiones que lo separan de un detector de imágenes estándar (extractor preentrenado en audio, pirámide sobre una rejilla de tokens sin jerarquía y cabeza de predicción de conjuntos) responden cada una a una propiedad del material documentada en el capítulo anterior.

Cómo reproducirlo. `src/train.py` instancia la propuesta o cualquiera de las líneas base según la arquitectura que se le indique, con la receta de entrenamiento que fija esta sección, y copia la configuración completa al `config.json` de la corrida. Así, cada modelo de este capítulo se vuelve a construir con los mismos hiperparámetros.

Medio de verificación. Esta sección, que es el documento de diseño, y el módulo de arquitecturas `src/models/` del repositorio del proyecto (<https://github.com/nhrot-fc/tesis-primate>), en el *commit* `b895fe2`; la sección 5.2.5 contrasta cada elemento del indicador con el código. La configuración completa de las cuatro corridas comparadas está en la carpeta RE2.1 de los medios de verificación (Anexo B).

5.2.1. La tarea

La tarea es la del nivel de ventana de la sección 2.1.1. La entrada es una ventana de 3 s convertida en espectrograma mel de potencia $X \in \mathbb{R}^{1 \times 128 \times 331}$ (sección 4.4.4), y la salida, el conjunto de la ecuación (2.2): cajas en el marco normalizado de la ventana, cada una con una de las 25 clases y una confianza.

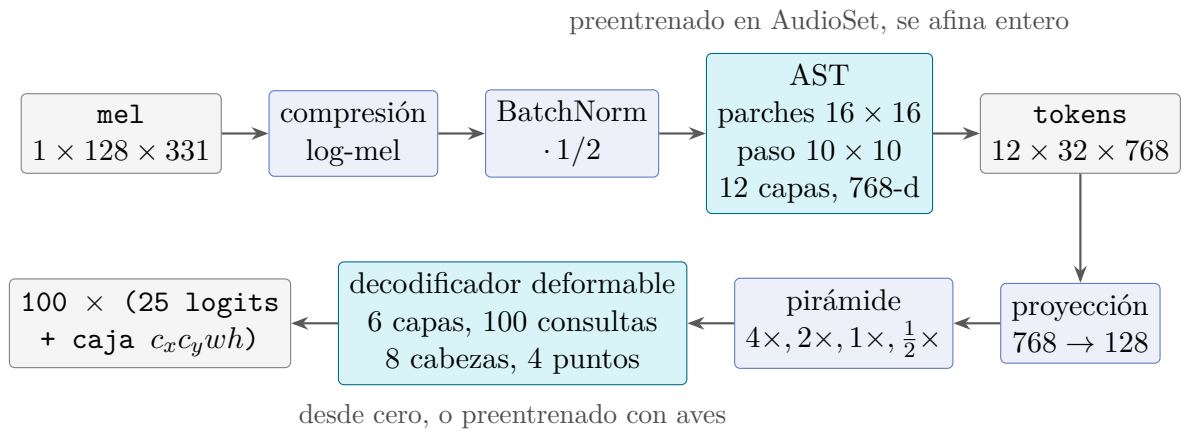
La formulación separa tres preguntas, y el protocolo de la sección 5.2.4 las puntúa por separado: si la llamada se *encontró* (detección), si la caja quedó bien *encuadrada* (localización) y si la etiqueta es *correcta* (clasificación). Mantenerlas separadas no es un refinamiento metodológico: es lo que permite decir, al final del capítulo, dónde está el error.

5.2.2. Arquitectura propuesta: AST-Deformable-DETR

La Figura 5.1 recorre el modelo de la entrada a la salida.

Figura 5.1

Arquitectura propuesta



El mel entra por la izquierda, atraviesa el extractor y vuelve, en la fila inferior, como 100 propuestas de caja con sus logits de clase. Tres decisiones definen ese recorrido.

Un extractor preentrenado en audio, no en imágenes. El AST (*Audio Spectrogram Transformer*) preentrenado en AudioSet (MIT/*ast-finetuned-audioset-10-10-0.4593*) sustituye al *backbone* convolucional de imágenes que traen los detectores estándar. La brecha de modalidad entre imagen y audio justifica esa sustitución (Z. Zhu & Sato, 2025): las estadísticas de un espectrograma (ejes asimétricos, sin invariancia a la traslación en frecuencia) no son las de una fotografía.

Una pirámide multiescala sobre los tokens. El AST produce una única rejilla de 12×32 tokens, es decir, una sola escala, mientras que la duración mediana de las clases va de 81 ms (*sm/cc*) a 11,0 s (*as/hc*), un rango de más de dos órdenes de magnitud (sección 4.5.3). La pirámide corrige que las arquitecturas de tipo *Vision Transformer* (ViT) produzcan representaciones de escala única (Z. Zhu & Sato, 2025), siguiendo la receta de ViTDet: deconvoluciones y una convolución con paso 2 sobre el mismo mapa.

Un decodificador deformable de predicción de conjuntos. La cabeza es el decodificador de Deformable DETR (X. Zhu et al., 2021): 100 consultas aprendidas atienden a unos pocos puntos muestreados alrededor de una caja de referencia, en los 4 niveles, y refinan la caja capa a capa. No hay anclas, y el emparejamiento húngaro durante el entrenamiento fuerza una predicción por evento.

Compresión y normalización de entrada

El caché guarda el mel de potencia sin comprimir, y la primera capa del modelo lo comprime con $\log(\text{mel} + 10^{-6})$: el log-mel, la representación sobre la que se preentrenó el AST (sección 2.2.3). A continuación, una `BatchNorm2d` sin parámetros afines lleva la entrada a media 0 y varianza 1 por lote, y se divide entre 2: es el rango con el que el AST se preentrenó. La corrida que se compara en la sección 5.4 usa `logmel`.

Extractor: AST con paso temporal adaptado

El AST corta el espectrograma en parches de 16×16 y los proyecta a 768 dimensiones. Con el paso original de 10 en ambos ejes, una ventana de 128 bandas por 331 tramas produce 12×32 tokens, es decir unos 94ms de audio por token en el eje temporal. La incrustación posicional del *checkpoint*, entrenada para otra rejilla, se interpola bilinealmente a la nueva. El extractor se afina entero, a la misma tasa que la cabeza: con el extractor congelado la propuesta quedó última en una comparación anterior, y descongelarlo es lo que la pone a la altura de las líneas base.

Pirámide multiescala

Los tokens se proyectan de 768 a 128 dimensiones y se reordenan como un mapa 12×32 . Sobre ese mapa, cuatro ramas producen los 4 niveles de la Figura 5.2.

Cada rama cambia la resolución del mismo mapa: dos deconvoluciones la cuadruplican, una la duplica, la identidad la conserva y una convolución con paso 2 la reduce a la mitad. La Tabla 5.1 recoge el tamaño de celda de cada nivel.

El nivel más fino resuelve 23,4ms por celda, por debajo de la mediana de la clase más corta (`sm/cc`, 81 ms); el más grueso, 187,5 ms, es del orden de las sílabas largas. La Figura 5.3 sitúa las 25 clases frente a esas cuatro resoluciones y frente a la ventana.

Figura 5.2
Pirámide multiescala

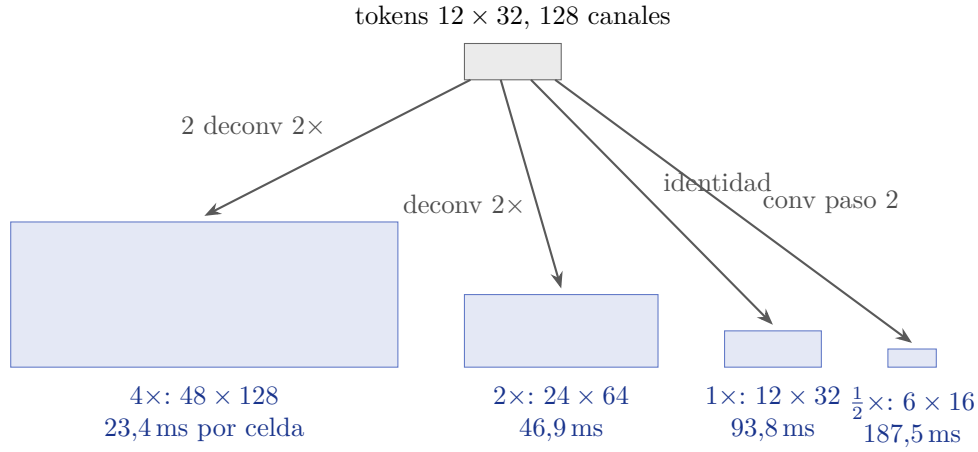


Tabla 5.1
Niveles de la pirámide

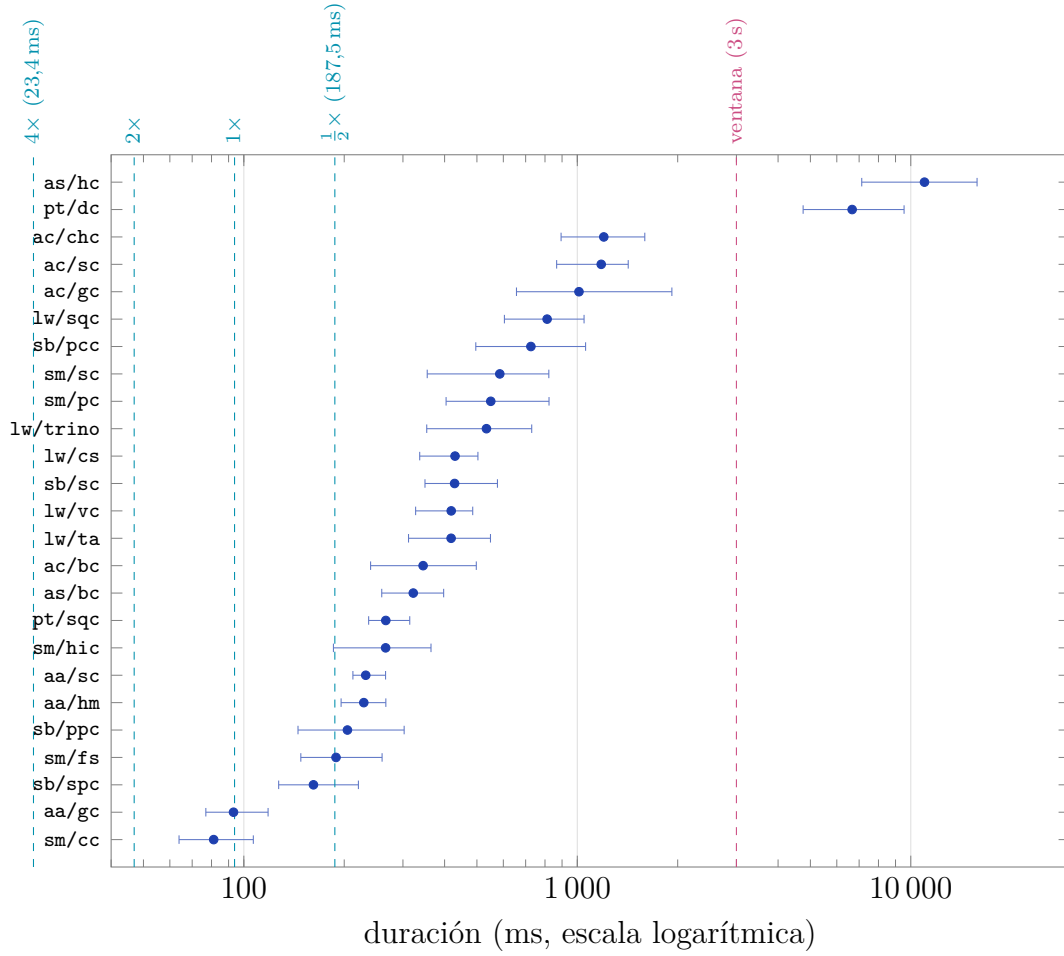
Escala	Mapa (bandas $\times$ tramas)	ms por celda	Bandas mel por celda
4 $\times$	48×128	23,4	2,7
2 $\times$	24×64	46,9	5,3
1 $\times$	12×32	93,8	10,7
$\frac{1}{2}\times$	6×16	187,5	21,3

Las líneas punteadas son las cuatro resoluciones y la ventana. Las duraciones medianas de casi todas las clases caen entre la resolución más fina y la ventana, y solo 2 clases (las dos de ventana) la superan.

Decodificador deformable

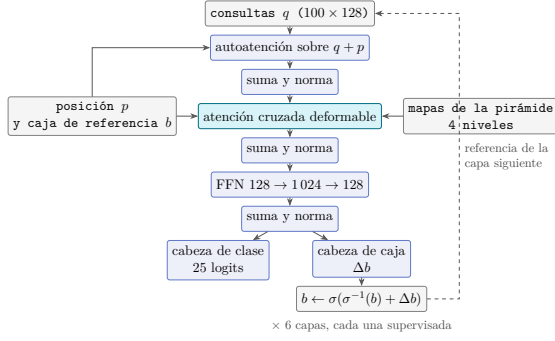
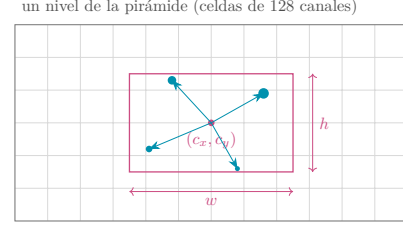
El decodificador tiene 6 capas idénticas (Figura 5.4), de dimensión 128, con 8 cabezas de atención, FFN de 1024 y *dropout* 0,1. Cada una de las 100 consultas lleva dos vectores aprendidos, contenido y posición, y de la posición se deriva un punto de referencia inicial (c_x, c_y) que, con un tamaño inicial de 0,1, forma la caja de referencia de la primera capa. En cada capa ocurre lo siguiente:

1. **Autoatención** entre consultas, con la posición sumada a las claves y consultas en cada capa, para que las consultas no se diluyan y puedan repartirse los eventos entre sí.
2. **Atención cruzada deformable** (Figura 5.5): en lugar de atender a todas las celdas de los cuatro mapas, cada consulta y cada cabeza muestrean 4 puntos por

Figura 5.3*Duración de las clases frente a la pirámide*

nivel, en posiciones que la propia consulta predice como desplazamientos relativos al tamaño de su caja de referencia, y los combinan con pesos que también predice (*softmax* sobre los 4×4 puntos). Los desplazamientos arrancan en una rejilla radial y no en ruido, como en el diseño original (X. Zhu et al., 2021). Es lo que hace tratable atender a un mapa de 48×128 celdas y lo que permite mezclar escalas sin elegir una.

3. **FFN** y normalización residual.
4. **Cabezas por capa.** Una cabeza lineal produce los 25 logits de clase y un perceptrón multicapa (MLP) de tres capas produce un delta de caja que se suma, en el espacio logit, a la caja de referencia; la caja refinada es la referencia de la capa siguiente, sin gradiente entre capas. Cada capa se supervisa por separado (pérdidas auxiliares).

Figura 5.4*Una capa del decodificador***Figura 5.5***Muestreo deformable de una cabeza sobre un nivel*

Para cada consulta y cabeza, la consulta predice 4 desplazamientos Δ y 4 pesos a (el tamaño del punto). El punto muestreado es $c + \Delta \cdot (w, h)/2$ y su valor, la interpolación bilineal del mapa. Se repite en los 4 niveles y los pesos se normalizan sobre los 4×4 puntos.

La figura de la izquierda resume una capa; la de la derecha, el paso que la hace eficiente: cada consulta lee solo 4 puntos por nivel alrededor de su caja, en lugar de todas las celdas de los cuatro mapas.

Emparejamiento y pérdida

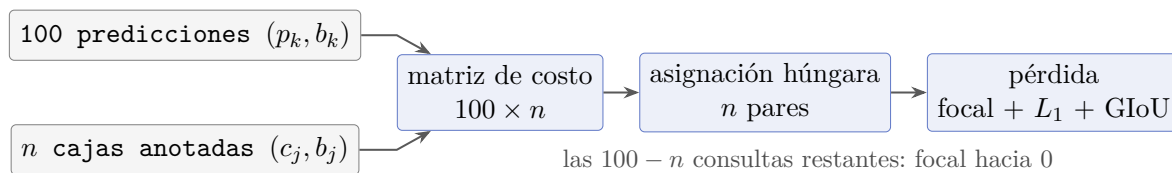
Las 100 predicciones de una ventana y sus n cajas anotadas se emparejan uno a uno con el algoritmo húngaro sobre una matriz de costo (Figura 5.6):

$$C_{kj} = 2,0 C_{\text{focal}}(p_k, c_j) + 5,0 \|b_k - b_j\|_1 - 2,0 \text{GIoU}(b_k, b_j),$$

donde el costo de clase es la pérdida focal positiva menos la negativa que se ahorra (X. Zhu et al., 2021) y GIoU es el IoU generalizado, que sigue dando señal cuando las cajas no se solapan. Sobre los pares emparejados se calcula la pérdida

$$\mathcal{L} = 1,0 \mathcal{L}_{\text{focal}} + 5,0 \mathcal{L}_1 + 2,0 \mathcal{L}_{\text{GIoU}},$$

con la focal ($\alpha = 0,25$, $\gamma = 2,0$) sobre sigmoides independientes por clase y sin canal de “no objeto”: la confianza de una consulta no emparejada se empuja a cero en todas las clases. El modelo puede así emitir una caja de alta confianza con la etiqueta equivocada, que el protocolo mide como acierto de detección y fallo de clasificación, que es exactamente la separación que la sección 5.2.4 necesita. Los sesgos de las cabezas de clase se inicializan en $\log \frac{1-\pi}{\pi}$ con $\pi = 0,01$, para que al inicio casi ninguna consulta dispare.

Figura 5.6*Emparejamiento húngaro*

De las 100 predicciones, solo las n emparejadas con una anotación reciben las tres pérdidas; las demás aprenden a no disparar.

Inferencia

La confianza de cada consulta es el máximo de sus 25 sigmoides y la clase, su argumento; se conservan las cajas con confianza $\geq 0,001$ (el piso del protocolo) y, como para todos los modelos, a lo sumo 100 por ventana. El emparejamiento uno a uno desalienta los duplicados al entrenar, pero 100 consultas sobre una a tres llamadas dejan en inferencia cajas repetidas y anidadas, de modo que se aplica la misma supresión de no máximos por clase que usan las líneas base (IoU 0,3). El presupuesto de 100 consultas se dimensiona sobre el máximo de cajas por ventana del conjunto (11, sección 4.4.3) y no sobre la mediana (1).

Cabeza preentrenada con cantos de aves

La cabeza deformable se entrena desde cero, mientras que las líneas base parten de un detector completo. Para acortar esa asimetría, la configuración que entra a la comparación preentrena la misma arquitectura, con los mismos hiperparámetros, sobre un caché aparte de cantos de aves construido con el ventaneo y el mel del Capítulo 4 (`data/processed_extra/`), y después la afina con primates: el afinado carga en el modelo recién construido todo tensor del preentrenado con el mismo nombre y forma (todo salvo las cabezas de clase, que dependen del número de clases) y sigue la receta de la propuesta.

Hiperparámetros y tamaño

La Tabla 5.2 reúne los hiperparámetros tal como los declara el código.

La mayoría replica los valores de Deformable DETR; los que se ajustaron a este material son el paso temporal del parcheo, la dimensión de la cabeza, los niveles y las consultas, cada uno con su motivo. La Tabla 5.3 muestra cuánto pesa cada bloque.

Tabla 5.2*Hiperparámetros de la propuesta*

Hiperparámetro	Valor	Motivo
Extractor	AST	MIT/ast-finetuned-audioset-10-10-0.4593, afinado entero
Paso temporal de parcheo	10	12×32 tokens, 94 ms por token
Dimensión de la cabeza	128	Cabeza pequeña sobre tokens de 768
Niveles de la pirámide	4	De 23,4 a 187,5 ms por celda
Consultas	100	Muy por encima de 11, el máximo de cajas por ventana
Capas del decodificador	6	Como Deformable DETR
Cabezas / puntos por nivel	8 / 4	Como Deformable DETR
FFN / <i>dropout</i>	1 024 / 0,1	Como Deformable DETR
Focal α , γ	0,25, 2,0	Valores de RetinaNet
Costos de emparejamiento (clase, L_1 , GIoU)	2,0, 5,0, 2,0	X. Zhu et al. (2021)
Pesos de la pérdida (clase, L_1 , GIoU)	1,0, 5,0, 2,0	La clase pesa el doble al asignar que al entrenar
Prior de las cabezas de clase	0,01	Casi nada dispara al inicio
Compresión de entrada	logmel	La del preentrenamiento del AST
NMS por clase	IoU 0,3	El mismo posprocesado que las líneas base

Tabla 5.3*Parámetros por bloque*

Bloque	Parámetros (M)	Fracción
Extractor AST (12 capas, 768 d)	85,55	96,5 %
Front-end y normalización de entrada	0,00	0,0 %
Proyección a la dimensión de la cabeza	0,10	0,1 %
Pirámide multiescala	0,26	0,3 %
Decodificador deformable (6 capas)	2,72	3,1 %
Total	88,64	100 %

Nota. La compresión de entrada no aparece con parámetros porque el log-mel no tiene ninguno.

El reparto importa para leer la comparación: el extractor concentra el 96,5 % de los 88,6 M de parámetros y la cabeza completa cabe en 3,1 M. Cambiar el número de clases o ensanchar la cabeza cuesta poco; lo que cuesta, en memoria y en tiempo, es afinar el AST.

5.2.3. Líneas base

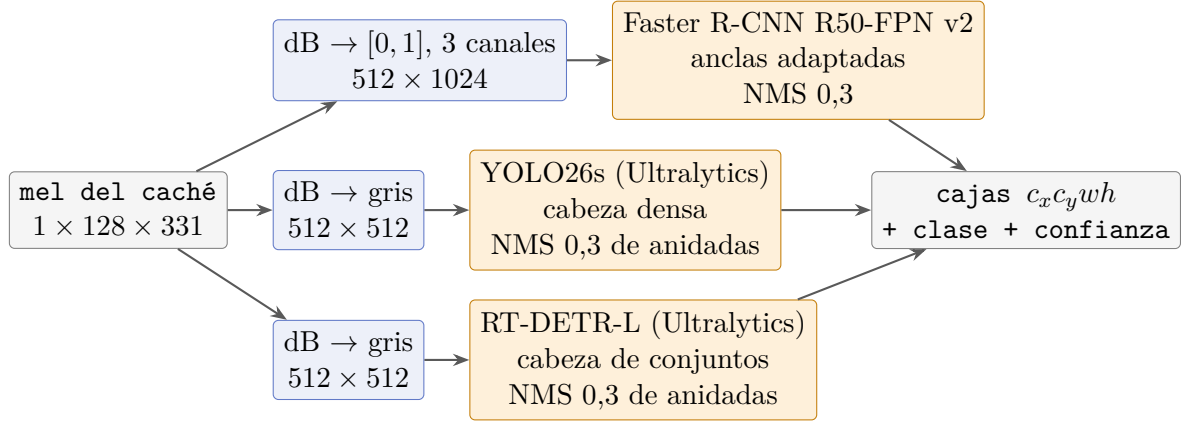
Se declaran tres líneas base y cada una controla una cosa distinta (Figura 5.7). Las tres consumen el mismo mel del caché, pintado como imagen con el rango en dB que registra `meta.json`, y devuelven cajas en el mismo marco normalizado, de modo que comparten datos, particiones y protocolo con la propuesta.

Faster R-CNN R50-FPN v2. El detector de dos etapas de referencia, con *backbone* ResNet-50 y pirámide de características (FPN, *Feature Pyramid Network*) preentrenados en COCO; el mel se reescala a 512×1024 píxeles y se replica en tres canales. Se adaptan las razones de aspecto de las anclas porque las cajas de llamadas van de muy anchas a muy altas, lejos de las razones de los objetos de COCO. Es la línea base sobre espectrogramas con precedente directo en bioacústica (Escobar-Amado et al., 2024).

YOLO26s. Un detector denso de una etapa, pequeño (10,0 M de parámetros), entrenado con la herramienta de Ultralytics desde sus pesos de COCO sobre una imagen gris de 512×512 ; es la línea base de la familia YOLO sobre espectrogramas (González et al., 2025). Las aumentaciones de imagen que no tienen sentido en un espectrograma se apagan (volteos, mosaico, tono y saturación) y quedan un corrimiento pequeño, una escala y brillo.

RT-DETR-L. Un detector de conjuntos con extractor de imagen preentrenado en COCO (32,9 M), también por Ultralytics. Es la línea base que separa las dos hipótesis que la propuesta mezcla: si una cabeza de tipo DETR rinde distinto por ser una cabeza DETR, o por el extractor de audio que la alimenta. Entró al experimento después que las otras dos, de forma exploratoria, y la sección 5.2.5 declara la consecuencia que eso tiene sobre su reproducibilidad.

Figura 5.7
Líneas base



Las tres líneas base comparten la entrada, el mel del caché pintado como imagen de distinto tamaño, y la forma de la salida; difieren en el detector. La Tabla 5.4 reúne las cuatro configuraciones de la comparación, la propuesta y las tres líneas base.

Tabla 5.4
Configuraciones comparadas

Configuración	Extractor	Parám. (M)	Preen- trenado	Qué controla
AST-Def-DETR (propuesta)	AST / AudioSet (afinado); cabeza / aves	88,6	extractor y cabeza	—
Faster R-CNN R50-FPN v2	ResNet-50 / COCO	43,4	detector entero	detector de dos etapas preentrenado en COCO
YOLO26s	CSP / COCO	10,0	detector entero	detector denso de una etapa preentrenado en COCO
RT-DETR-L	HGNetv2 / COCO	32,9	detector entero	cabeza de conjuntos con extractor de imagen preentrenado en COCO

Nota. Parámetros según el registro de cada corrida. En la propuesta y en las dos de Ultralytics se entrena todo; en Faster R-CNN, el tallo y las dos primeras etapas del ResNet quedan congelados (43,2 M entrenables de 43,4 M).

Las cuatro difieren en extractor, tamaño y preentrenamiento, y la última columna dice qué variable controla cada línea base frente a la propuesta.

5.2.4. Diseño experimental

Recetas de entrenamiento

Cada configuración se entrena con la receta de su *preset* (Tabla 5.5). Los modelos del proyecto usan AdamW con un programa OneCycle, recorte del gradiente y las mismas aumentaciones sobre el audio: ganancia aleatoria, mezcla de una ventana vacía a un SNR dado, corrimiento temporal y pegado de llamadas de otras ventanas, además de un *jitter* de las cajas objetivo. Los dos modelos de Ultralytics se entrenan con su propia herramienta y sus aumentaciones de imagen, restringidas como se dijo. **Las recetas no están igualadas, y se declara:** cada marco usa la suya, de modo que la comparación mide sistemas completos y no solo arquitecturas.

Tabla 5.5

Recetas de entrenamiento

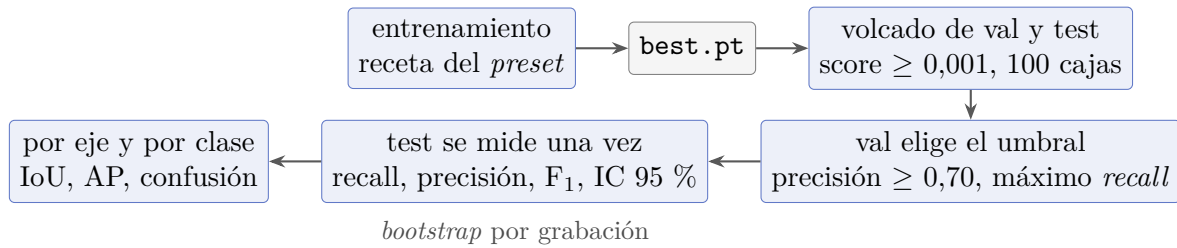
Configuración	Épocas	Lote	lr	Optimizador	Elección de <code>best.pt</code>
AST-Def-DETR (propuesta)	30	8	10^{-4}	AdamW + OneCycle	mAP@0,3 en validación
Faster R-CNN R50-FPN v2	12	4	10^{-4}	AdamW + OneCycle	mAP@0,3 en validación
YOLO26s	30	32	auto	auto (Ultralytics)	aptitud de Ultralytics
RT-DETR-L	30	16	10^{-4}	AdamW	aptitud de Ultralytics

La asimetría de la última columna es una diferencia de receta, no de protocolo: las dos corridas de Ultralytics eligen su *checkpoint* con la aptitud de su propio validador y no con la mAP@0,3 que usan las otras dos, pero el protocolo de comparación vuelve a medir a todos con la misma vara.

Protocolo de comparación

El protocolo vive en `src/evaluation/protocol.py`, se fijó antes de mirar los resultados de prueba y la sección 5.4 lo aplica sin cambios (Figura 5.8). Consta de seis reglas, que la figura ordena en el tiempo y que se detallan a continuación.

1. **Se vuelcan predicciones, no métricas.** De cada `best.pt` se guardan, por ventana de validación y de prueba, todas las cajas con confianza $\geq 0,001$; toda

Figura 5.8*Protocolo de comparación*

cifra sale después de esos volcados. Cambiar una métrica o un umbral no exige volver a pasar los modelos por la GPU.

2. **Mismo presupuesto para todos.** Se conservan las 100 mejores cajas por ventana: torchvision corta en 100 y Ultralytics en 300, y sin techo común la cola de la curva precisión–*recall* sería del marco de trabajo y no del modelo.
3. **Acierto a $\text{IoU} \geq 0,3$.** Se mide si la llamada se *encontró*, no si la caja quedó ajustada: el eje de frecuencia lo dibuja el anotador con criterio variable y las llamadas son cortas. A IoU 0,5 el corrimiento temporal tolerado es un tercio del ancho de la caja; a 0,3 sube a la mitad. Más abajo no: a 0,25 una predicción de banda completa acertaría una parte de las cajas sin encuadrar nada. El encuadre se reporta aparte, a 0,5. La precisión media se calcula con la definición y la interpolación de la curva precisión–*recall* que fija COCO (Lin et al., 2014).
4. **Umbral por modelo, elegido en validación.** De la rejilla 0,01–0,99, el umbral de mayor *recall* entre los que alcanzan una precisión de al menos 0,70. Un umbral fijo compartido mide calibración, no detección: cada cabeza reparte sus confianzas a su manera, y la sección 5.4.4 muestra cuánto cambia la lectura según dónde se mida.
5. **La prueba se mide una vez**, en ese umbral, con intervalos de confianza al 95 % por *bootstrap* de 1 000 remuestreos *de grabaciones*: las ventanas vecinas comparten la mitad del audio y la calidad de una grabación es de la grabación entera. Remuestrear ventanas daría intervalos más angostos de lo que corresponde.
6. **Las clases de ventana van aparte.** Si la mediana del ancho de caja de una clase ocupa al menos el 95 % de la ventana (*as/hc*, *pt/dc*), detectarla es clasificar

la ventana: se reportan, pero no entran en la mAP ni en el promedio por clase. Quedan 23 clases de detección.

Métricas por eje y regla de decisión

La Tabla 5.6 asigna a cada eje la pregunta que responde y la magnitud con que se responde, en la línea que fija la sección 1.7.1.

Tabla 5.6
Métricas por eje

Eje	Pregunta	Magnitud
Detección	¿Se encontró la llamada?	<i>Recall</i> y precisión emparejando sin mirar la clase, a IoU 0,3, al punto de operación
Encuadre	¿La caja está donde el anotador la dibujó?	IoU mediano de esos pares; <i>recall</i> con clase a IoU 0,5; AP a IoU 0,25 y 0,5
Clasificación	¿Se etiquetó bien lo que se encontró?	Fracción de pares con la clase y con la especie correctas; <i>recall</i> y AP por clase; matriz de confusión
Operación	¿Cuánto cuesta revisar?	Umbral elegido, F_1 , FP/TP, propuestas por hora de audio, minutos de GPU por hora de audio

La regla de decisión se declara antes de correr el experimento y no se cambia después:

La métrica **primaria** es el *recall* en prueba al punto de operación de precisión $\geq 0,70$, con su intervalo de confianza; la **secundaria** es la mAP@0,3. Dos configuraciones cuyos intervalos se solapan *no se separan* con este experimento, y se dice así. El modelo que pasa a RE2.4 es el de mayor *recall* primario; si la primaria no separa, decide la secundaria, y en caso de empate también en ella, el más barato en inferencia.

La comparación pone a prueba una hipótesis de diseño: que un extractor preentrenado en audio con una cabeza de conjuntos supera a los detectores preentrenados en las imágenes de COCO cuando todos se entrenan sobre el mismo mel. Con la regla anterior, se confirma si la propuesta queda primera en la métrica primaria o, si esta no separa, en la secundaria. La sección 5.4.7 da el veredicto.

5.2.5. Verificación de RE2.1

El indicador pide que se especifique la arquitectura implementada, con sus hiperparámetros, y que se defina la comparación con las líneas base declaradas. La Tabla 5.7 contrasta cada elemento con el código.

Tabla 5.7
Verificación de RE2.1

Elemento del IOV	Valor	Dónde se comprueba
Arquitectura AST-Deformable-DETR implementada	sí	<code>src/models/deformable_detr.py</code> y <code>backbone.py</code> ; la Figura 5.1 es su recorrido
Todos sus hiperparámetros	88,6 M de parámetros	Tablas 5.2 y 5.3, generadas desde el código y desde el <code>best.pt</code> : dimensión 128, 100 consultas, 4 niveles
Comparación definida antes de entrenar: configuraciones	3 líneas base	Sección 5.2.3 y Tabla 5.4
Partición, métricas y regla de selección	fijadas antes de entrenar	Protocolo de la sección 5.2.4, que aplica el de la sección 1.7

Por qué 100 consultas y 4 niveles. El prototipo con que se planificó el proyecto usaba 64 consultas y 3 niveles, y la implementación final usa 100 y 4. Ninguno de los dos cambios altera la arquitectura:

- **De 64 a 100 consultas.** El presupuesto de consultas acota cuántos eventos puede emitir el modelo en una ventana. El máximo observado en el conjunto es 11 cajas por ventana, de modo que 64 ya bastaba; subirlo a 100 añade 100×128 parámetros de incrustación (una fracción despreciable de los 88,6 M) y deja margen para escenas más densas que las de este corpus. El costo es cuadrático en la autoatención entre consultas, que sobre 100 elementos es irrelevante frente al extractor.
- **De 3 a 4 niveles.** El nivel que se añadió es el $\frac{1}{2} \times$, el más grueso (187,5 ms por celda), y la razón está en la Figura 5.3: con tres niveles la resolución más gruesa habría sido 93,8 ms, por debajo de la duración mediana de buena parte de las clases, y el decodificador habría tenido que componer los eventos largos a partir de celdas finas.

Una cuarta configuración, incorporada después. RT-DETR-L se lanzó de forma exploratoria con la herramienta de Ultralytics, fuera del registro de *presets*, y su receta

se recuperó del `config.json` de la corrida para dejarla en `src/train.py` como `-arch rtdetr`. Comparte el flujo de YOLO26s salvo en dos puntos, que son los que la corrida original fijó a mano: el optimizador, AdamW con tasa inicial 10^{-4} , porque con el que elige Ultralytics por su cuenta la configuración no converge; y el lote, 16 en lugar de 32, porque el decodificador de conjuntos no entra en memoria con el lote de YOLO. Su *checkpoint* se carga en modo estricto y reproduce su volcado caja por caja, de modo que las cifras que aquí se reportan proceden del mismo código que se publica.

5.3. Pipeline reproducible de entrenamiento, evaluación e inferencia (RE2.2)

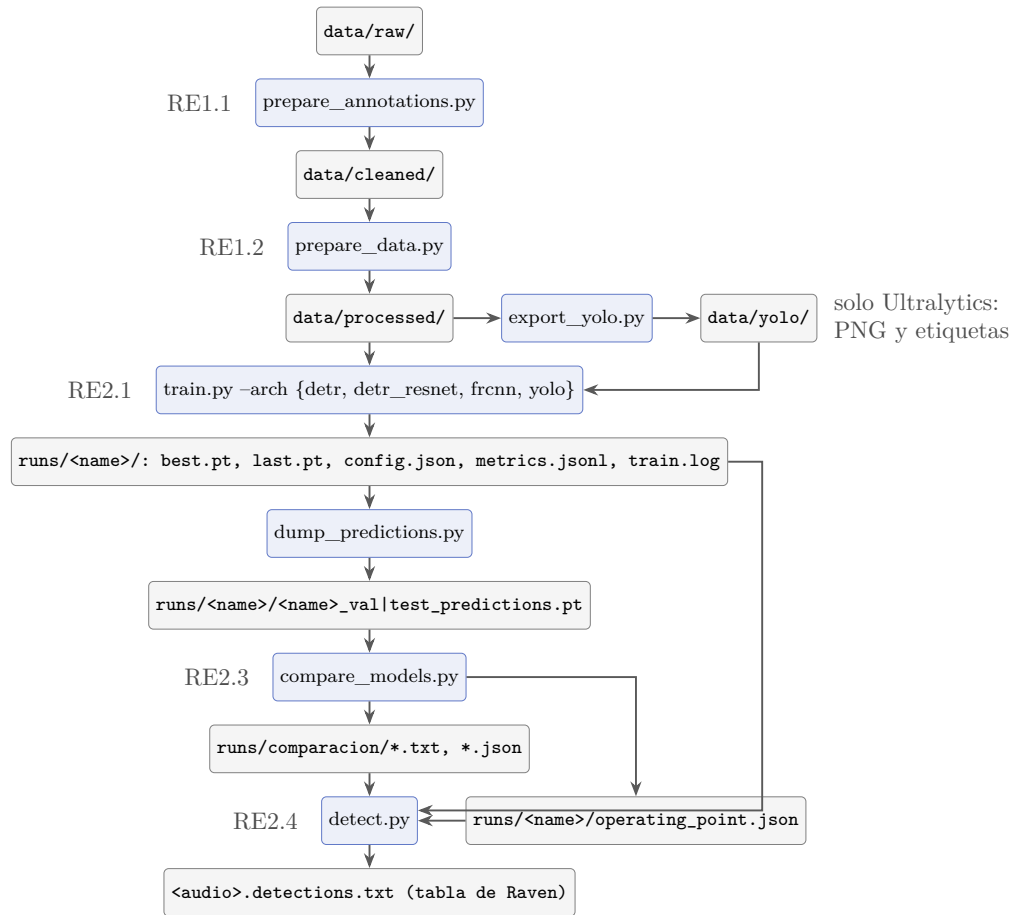
Resultado. Un *pipeline* de ocho puntos de entrada que va de las tablas de Raven a las tablas de Raven: prepara los datos, entrena desde una configuración, evalúa un *checkpoint* y ejecuta inferencia sobre un WAV, con las dependencias y los comandos declarados.

Cómo se alcanzó. Con los procedimientos de la sección 1.7.3: material primario intocado, experimentos registrados y entorno declarado. Cada etapa es un *script* de `src/` con su propio `-help`; ninguna modifica los artefactos de la anterior, de modo que se puede volver a ejecutar cualquiera desde su entrada (Figura 5.9). Las etapas de datos son las del Capítulo 4; las de modelo, las de la sección anterior.

Cómo reproducirlo. Con el entorno que fija `uv.lock`, se ejecutan las cuatro etapas en orden, cada una desde su *script*. La semilla única, la configuración que viaja con cada *checkpoint* y los controles que impiden mezclar versiones del caché hacen que el recorrido se repita; la sección 5.3.5 detalla qué queda fijado y qué no.

Medio de verificación. Los *scripts* de `src/`, `pyproject.toml` y `uv.lock`, publicados en el repositorio del proyecto (<https://github.com/nhrot-fc/tesis-primate>), y esta sección como manual técnico; la sección 5.3.6 recorre el indicador etapa por etapa. El indicador nombra los módulos de `src/pipelines/`, la organización del prototipo; en la implementación final esos módulos son los paquetes `src/data/`, `src/models/`, `src/training/`, `src/evaluation/` y `src/inference/`, con un *script* por etapa. El registro de entrenamiento de cada corrida está en la carpeta RE2.2 de los medios de verificación (Anexo B).

Figura 5.9
Pipeline del proyecto



La figura se lee de arriba abajo: las dos primeras etapas preparan los datos (RE1.1 y RE1.2), la tercera entrena (RE2.1), la cuarta evalúa (RE2.3) y la última escribe la tabla de Raven (RE2.4). El desvío de la derecha es la exportación que solo necesitan los modelos de Ultralytics.

5.3.1. Dependencias declaradas

El entorno está declarado y bloqueado, que es la condición para que el *pipeline* sea reproducible y no solo ejecutable en una máquina. `pyproject.toml` fija Python ≥ 3.12 y las dependencias base (PyTorch, torchvision, torchaudio, pandas y soundfile) y separa en dos extras lo que solo necesita una familia de arquitecturas: `detr` (transformers, timm) para las dos basadas en DETR y `yolo` (ultralytics, opencv) para las dos de Ultralytics. Esa separación tiene una consecuencia práctica: una instalación de inferencia no arrastra las librerías de entrenamiento. `uv.lock` fija las versiones exactas de todo el árbol.

Los pesos preentrenados (el AST de AudioSet, el Deformable DETR de COCO) se resuelven en el primer uso y quedan en caché local, de modo que las corridas posteriores no dependen de la red. El entrenamiento se hizo sobre una GPU de 24 GB; la inferencia no necesita GPU.

5.3.2. Las cuatro etapas y su interfaz

El indicador de este resultado exige que los comandos de ejecución queden declarados. El Listado 5.1 los recoge: es la interfaz del *pipeline*, no un instructivo, y cada línea corresponde a una caja de la Figura 5.9.

Listado 5.1

Interfaz del pipeline, una línea por etapa

```
# 1. datos (capítulo 4)
prepare_annotations.py [--force]      raw/ -> cleaned/
prepare_data.py          [--force]    cleaned/ -> processed/
export_yolo.py           processed/ -> yolo/ (solo Ultralytics
    )

# 2. entrenamiento: --arch elige el preset; --hp y --cfg lo pisan
train.py --arch detr  --name detr_t10_logmel --hp frontend=logmel
train.py --arch frcnn --name frcnn_v3
train.py --arch yolo  --name yolo26s_v3 --hp model=yolo26s
train.py --arch detr  --limit 64 --cfg epochs=1      # prueba de humo
notebooks/pretrain_birds_train.py --name detr_birds_v2
notebooks/pretrain_birds_train.py --finetune <aves>/best.pt --name <
    corrida>

# 3. evaluación: volcar primero, medir después
dump_predictions.py --run <corrida> [--device cuda:1]
compare_models.py runs/*/*_predictions.pt --output <salida>
./evaluation.sh [corridas...]      las dos cosas, solo lo desactualizado

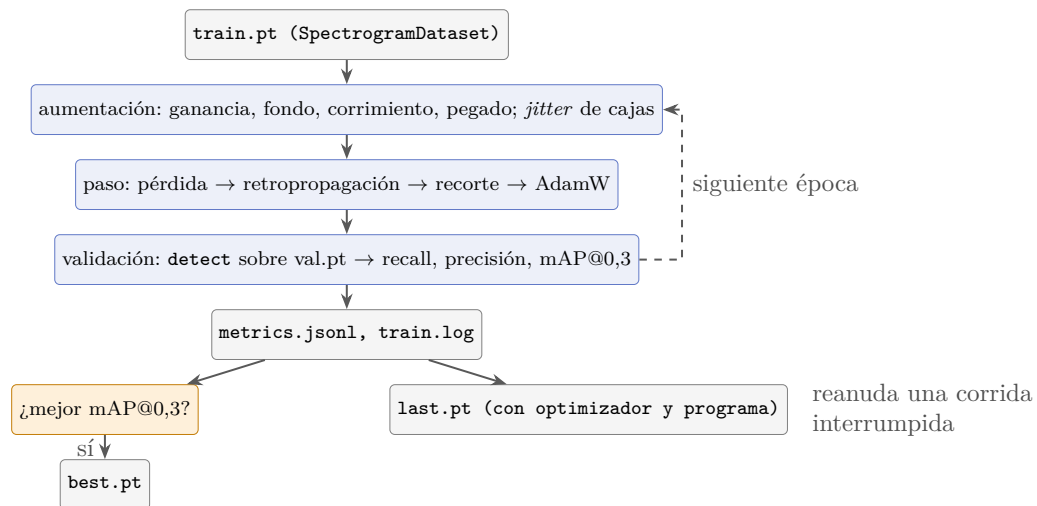
# 4. inferencia
detect.py <audio o carpeta> --model runs/<corrida>/best.pt
```

Una corrida se identifica por su *preset* (la arquitectura del registro `models.registry.ARCHITECTURES` con sus hiperparámetros y su `TrainConfig`) y por lo que se le pisa encima. Cualquier valor que se cambie respecto del *preset* queda escrito en el nombre de la corrida, en su `config.json` y dentro del propio *checkpoint*: es lo que permite, meses después, saber con qué se entrenó un archivo de pesos sin depender de ninguna anotación externa.

El ciclo de entrenamiento (Figura 5.10) es común a las arquitecturas del proyecto; las de Ultralytics delegan en su herramienta con las aumentaciones restringidas de la sección anterior y al terminar exportan su mejor época al formato de *checkpoint* del proyecto, de modo que aguas abajo las cuatro se tratan igual.

Figura 5.10

Una época del entrenador



Cada época aumenta los datos, entrena, valida y registra. El `best.pt` solo se reemplaza cuando mejora la `mAP@0,3` de validación, y el `last.pt` guarda además el estado del optimizador para reanudar una corrida interrumpida.

5.3.3. Artefactos de una corrida y autonomía del *checkpoint*

Una corrida deja su `config.json` con la arquitectura, los hiperparámetros y el `meta.json` del caché con el que se entrenó; su `train.log` y su `metrics.jsonl` con una línea por época; el `best.pt` de la mejor época y el `last.pt` con optimizador y programa para reanudar; después de la evaluación, los dos volcados y su `operating_point.json`.

Un *checkpoint* es autocontenido (Tabla 5.8): `models.registry.load_checkpoint(path)` reconstruye el modelo a partir de `architecture`, `hparams` y el número de clases

de `labels`, carga el `state_dict` en modo estricto (ninguna clave puede faltar ni sobrar) y devuelve el modelo junto con las etiquetas, la configuración y el umbral de operación si existe. Es lo que abren `detect.py` y `dump_predictions.py`, y es la propiedad que RE2.4 verifica.

Tabla 5.8
Claves de un checkpoint

Clave	Contenido	Para qué
<code>architecture</code>	<code>ast_deformable_detr</code>	clave de <code>models.registry.ARCHITECTURES</code> con la que se rearma
<code>hparams</code>	5 claves	hiperparámetros del modelo, los mismos que recibió <code>build_model</code>
<code>labels</code>	25 nombres	las clases, en orden: fija el número de salidas y el nombre de cada una
<code>config</code>	15 claves	la receta de la corrida y el <code>meta.json</code> del caché con el que se entrenó
<code>state_dict</code>	398 tensores, 88,6 M de parámetros	los pesos
<code>epoch</code>	16	la época de la que salió
<code>metrics</code>	3 claves	sus métricas de validación

Las claves de arquitectura, hiperparámetros y etiquetas bastan para reconstruir el modelo sin el código de entrenamiento ni el caché, y eso es lo que permite publicar un único archivo como modelo final.

5.3.4. Costo de las corridas

La Tabla 5.9 registra las cuatro corridas de la comparación: cuántas épocas corrieron de las planificadas, en cuál quedó el *checkpoint* elegido y cuánto costó cada una. El total es de 17,4 horas de GPU, con un reparto muy desigual: 1,0 h la más barata y 6,6 h la propuesta, que es la única que afina un extractor de 88,6 M de parámetros.

La Figura 5.11 muestra las curvas de validación. Las dos corridas del proyecto se miden con la mAP@0,3 del protocolo y las dos de Ultralytics con la mAP@0,5 de su propio validador: no son la misma magnitud y por eso van en paneles separados.

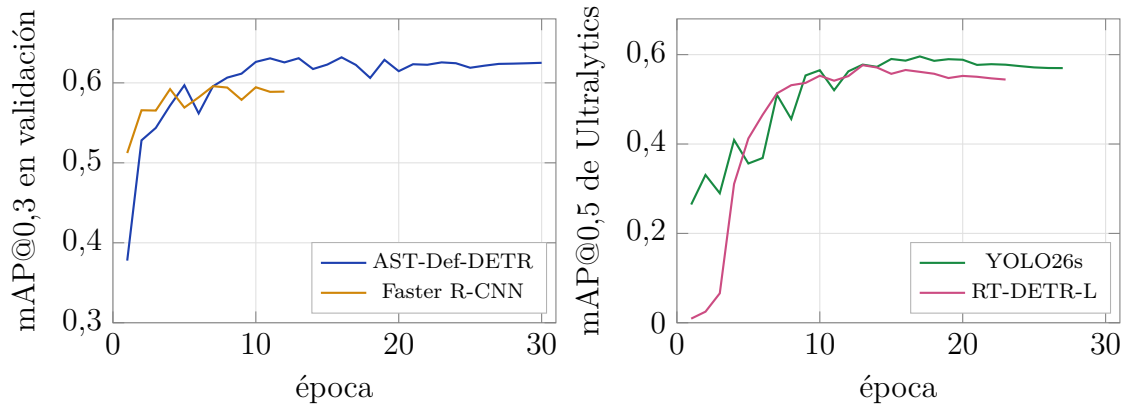
Las cuatro curvas tienen la misma forma: la mayor parte de la mejora ocurre en el primer tercio del programa y después la curva se aplana. La propuesta alcanza su máximo en la época 16 de 30 y Faster R-CNN en la 7 de 12; prolongar el programa no habría cambiado la elección.

Tabla 5.9
Costo de las corridas

Configuración	Épocas	Época elegida	Minutos por época	Horas de GPU
AST-Def-DETR (propuesta)	30 de 30	16	13,2	6,6
Faster R-CNN R50-FPN v2	12 de 12	7	28,1	5,6
YOLO26s	27 de 30	17	2,3	1,0
RT-DETR-L	23 de 30	13	10,9	4,2

Nota. Horas de GPU: diferencia entre la primera y la última marca de tiempo del registro de la corrida, de modo que incluye la validación de cada época. La GPU se compartió con otros procesos: son cotas superiores y no son estrictamente comparables entre corridas. Las dos corridas de Ultralytics pararon antes de las 30 épocas por su criterio de paciencia. La corrida de la propuesta cuenta solo el afinado con primates, no el preentrenamiento con aves.

Figura 5.11
Curvas de entrenamiento



5.3.5. Reproducibilidad

- **Semillas.** Una semilla única (42) fija la partición, las ventanas vacías, el orden de los lotes, las aumentaciones y el *bootstrap* del protocolo; las corridas de Ultralytics reciben la misma y corren en modo determinista.
- **Trazabilidad.** Cada *checkpoint* lleva dentro el `meta.json` del caché con el que se entrenó y sus hiperparámetros; cada corrida, su `config.json` y su registro con marcas de tiempo. Una cifra de este capítulo se rastrea a un *commit* (b895fe2), un `meta.json` y un `best.pt`.
- **Fallo temprano.** `prepare_*` se niegan a pisar salidas, el entrenamiento de Ultralytics se niega a entrenar sobre un export de otra versión del caché,

`compare_models.py` se niega a comparar volcados que no evaluaron las mismas ventanas, y `load_checkpoint` carga en modo estricto.

- **Pruebas de humo.** `-limit N -cfg epochs=1 workers=0` recorre el ciclo completo en minutos; es la prueba que precede a cada corrida larga.
- **Lo que no está fijado.** El orden de las operaciones en GPU no es determinista en todos los núcleos de cuDNN: dos corridas con la misma semilla dan curvas parecidas, no idénticas. El costo de una corrida depende de la carga de la GPU compartida.

5.3.6. Verificación de RE2.2

El indicador enumera cuatro capacidades y dos requisitos de declaración. La Tabla 5.10 las recorre con la evidencia de cada una.

Tabla 5.10
Verificación de RE2.2

Capacidad	Comando	Evidencia
Genera las particiones procesadas	<code>prepare_annotations.py</code> , <code>prepare_data.py</code>	<code>data/processed/</code> con los tres <code>.pt</code> , <code>labels.json</code> y <code>meta.json</code> (sección 4.4.5)
Entrena desde un conjunto de configuración	<code>train.py -arch</code>	Las cuatro corridas de la Tabla 5.9, cada una con su <code>config.json</code>
Evalúa un <i>checkpoint</i>	<code>dump_predictions.py</code> , <code>compare_models.py</code> , <code>evaluation.sh</code>	<code>runs/comparacion/</code> con el umbral, las métricas y el reparto por clase de cada corrida, publicados en la carpeta RE2.3 del Anexo B
Ejecuta inferencia sobre un WAV	<code>detect.py</code>	La demostración de la sección 5.5, con su transcripción
Dependencias declaradas	<code>pyproject.toml</code> , <code>uv.lock</code>	Python ≥ 3.12 , extras <code>detr</code> y <code>yolo</code>
Comandos declarados	Listado 5.1	Los ocho puntos de entrada de la Figura 5.9

Las seis filas tienen su evidencia en el documento o en el repositorio. La única que depende de material no publicado es la primera, porque el caché se construye con las grabaciones del equipo de investigación.

5.4. Informe comparativo de resultados (RE2.3)

Resultado. La comparación de las cuatro configuraciones sobre la misma partición de prueba, con el mismo protocolo, separada en los tres ejes (detección, encuadre y clasificación) y con la selección del modelo justificada por la regla declarada en la sección 5.2.4.

Cómo se alcanzó. Aplicando la Fase 5 de CRISP-DM (sección 1.7.1), con sus métricas y su criterio de emparejamiento. `evaluation.sh` vuelca las predicciones crudas de cada `best.pt` sobre validación y prueba y aplica el protocolo sin volver a pasar los modelos por la GPU. Las cifras de validación eligen el umbral de cada modelo; las de prueba se miden una sola vez con ese umbral.

Cómo reproducirlo. Con los cuatro `best.pt` y el caché de `data/processed/`, `evaluation.sh` vuelve a volcar las predicciones y `compare_models.py` recalcula todas las métricas. El umbral se elige con una regla fija y el *bootstrap* usa la semilla 42, de modo que las cifras salen idénticas. Solo se publica el *checkpoint* del modelo final (RE2.4); los otros tres se obtienen entrenando con sus recetas (RE2.1).

Medio de verificación. Esta sección, que es el informe de experimentación, y los archivos de métricas que produce `compare_models.py` (`comparacion_modelos_v3.txt` y su `.json`), de los que se generan todas las tablas y figuras de la sección; la sección 5.4.9 contrasta el indicador magnitud por magnitud. Los dos archivos de métricas, el punto de operación de cada corrida y la matriz de confusión están en la carpeta RE2.3 de los medios de verificación (Anexo B).

5.4.1. Lo que se compara

Las cuatro configuraciones de la Tabla 5.4 se entrenaron sobre el mismo conjunto derivado de 25 clases, con la misma partición por grabación, y se miden sobre las mismas 7598 ventanas de prueba (7931 cajas anotadas, 3,17 horas de audio). El protocolo comprueba esa igualdad antes de comparar nada y se niega a seguir si dos volcados no evaluaron las mismas ventanas.

5.4.2. Desempeño al punto de operación

La Tabla 5.11 reporta el umbral que la validación eligió para cada modelo y lo que ese umbral rinde en prueba.

Tabla 5.11
Resultados al punto de operación

Configuración	Umbral	Recall (val)	Recall (test)	IC 95 %	Prec.	F ₁	mAP @0,25	mAP @0,3	mAP @0,5
AST-Def-DETR (propuesta)	0,21	0,827	0,797	[0,771; 0,825]	0,669	0,728	0,589	0,577	0,451
Faster R-CNN R50-FPN v2	0,69	0,806	0,781	[0,753; 0,808]	0,680	0,727	0,592	0,574	0,457
YOLO26s	0,11	0,837	0,811	[0,787; 0,835]	0,665	0,731	0,629	0,620	0,521
RT-DETR-L	0,39	0,836	0,808	[0,786; 0,829]	0,673	0,734	0,619	0,601	0,499

Nota. El *recall* y la precisión de esta tabla exigen la etiqueta correcta; la Tabla 5.12 los separa del acierto de detección. Las mAP se promedian sobre las 23 clases de detección, sin las dos de ventana.

El intervalo es de 1 000 remuestreos de grabaciones, no de ventanas.

Lo primero que hay que registrar, porque condiciona todo lo que sigue, es que **la métrica primaria no separa a ninguna pareja de configuraciones**: los cuatro intervalos de confianza se solapan, el *recall* de prueba va de 0,781 a 0,811 (una diferencia de 0,030) y las cuatro F₁ caben en tres milésimas. Por la regla declarada en la sección 5.2.4, eso no se redondea a un ganador: se dice que el experimento no las distingue en esa magnitud, y se pasa a la secundaria.

La métrica secundaria sí ordena, y con un margen de 0,047:

$$\text{YOLO26s (0,620)} > \text{RT-DETR-L (0,601)} > \text{AST-Def-DETR (0,577)} \approx \text{Faster R-CNN (0,574)}$$

La mAP es libre de umbral: integra toda la curva precisión–*recall* en lugar de leer un punto, de modo que no depende de dónde cada cabeza puso su escala de confianza. Que ordene donde el punto de operación no ordena significa que las cuatro configuraciones alcanzan una cobertura equivalente al presupuesto de revisión que fija 0,70 de precisión, pero no con la misma calidad de curva: dos de ellas sostienen mejor la precisión cuando se les pide más cobertura. El mismo orden se mantiene a IoU 0,25 y a IoU 0,5, y la distancia crece al apretar el encuadre (0,521 contra 0,451), lo que apunta a que la diferencia está en cómo encuadran y no en cuánto encuentran.

Los umbrales elegidos son muy distintos entre sí. 0,11 para YOLO26s y 0,69 para Faster R-CNN, con la propuesta en 0,21 y RT-DETR-L en 0,39. Esto es exactamente

lo que la cuarta regla del protocolo anticipaba: un umbral fijo compartido (por ejemplo el 0,5 de fábrica) habría situado a cada modelo en un punto distinto de su curva y habría medido calibración tanto como detección. La sección 5.4.4 desarrolla la consecuencia.

5.4.3. Detección, encuadre y clasificación

La Tabla 5.12 separa los tres ejes. Es la tabla más importante del capítulo, porque es la que dice dónde está el error.

Tabla 5.12

Detección, encuadre y clasificación

Configuración	Detección		Encuadre		Clasificación	
	Recall	Prec.	IoU mediano	Recall @0,5	Clase correcta	Especie correcta
AST-Def-DETR (propuesta)	0,811	0,681	0,766	0,729	0,982	0,994
Faster R-CNN R50-FPN v2	0,793	0,691	0,768	0,716	0,983	0,991
YOLO26s	0,823	0,676	0,792	0,754	0,981	0,992
RT-DETR-L	0,817	0,681	0,797	0,747	0,985	0,996

Nota. Detección: emparejando sin mirar la clase. Encuadre: IoU mediano de esos mismos pares, y *recall* exigiendo la clase a IoU 0,5. Clasificación: de lo que se encontró, qué fracción lleva la clase y la especie anotadas.

La clasificación no es el problema. Entre 0,981 y 0,985 de los pares emparejados llevan la clase correcta, y al menos 0,991 la especie correcta, en las cuatro configuraciones. Distinguir 25 pares de especie y tipo de llamada, con 23 veces más cajas de entrenamiento en la clase más frecuente que en la menos frecuente, cuesta menos de dos puntos una vez que el evento está localizado. La distancia entre el *recall* de detección de la Tabla 5.12, que empareja sin mirar la clase, y el de la Tabla 5.11, que sí exige la etiqueta, mide ese costo: para el modelo seleccionado es de poco más de un punto.

El encuadre sí separa. El IoU mediano va de 0,766 a 0,797, y la fracción de pares que superan IoU 0,5 (el umbral estricto) va de 90,7 % a 92,2 %. La Figura 5.12 muestra la distribución completa: las cuatro tienen la misma forma, una moda ancha entre 0,7 y 0,9, pero las dos configuraciones de Ultralytics la desplazan a la derecha. Esta es la diferencia que la mAP@0,5 amplifica.

La matriz de confusión del modelo seleccionado (Figura 5.13) confirma la lectura por el otro lado.

Figura 5.12

Distribución del IoU

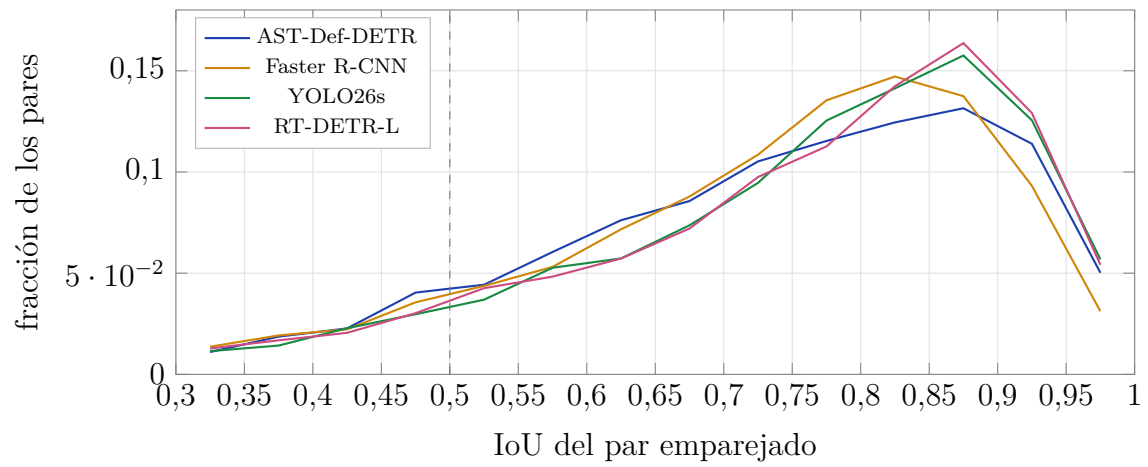
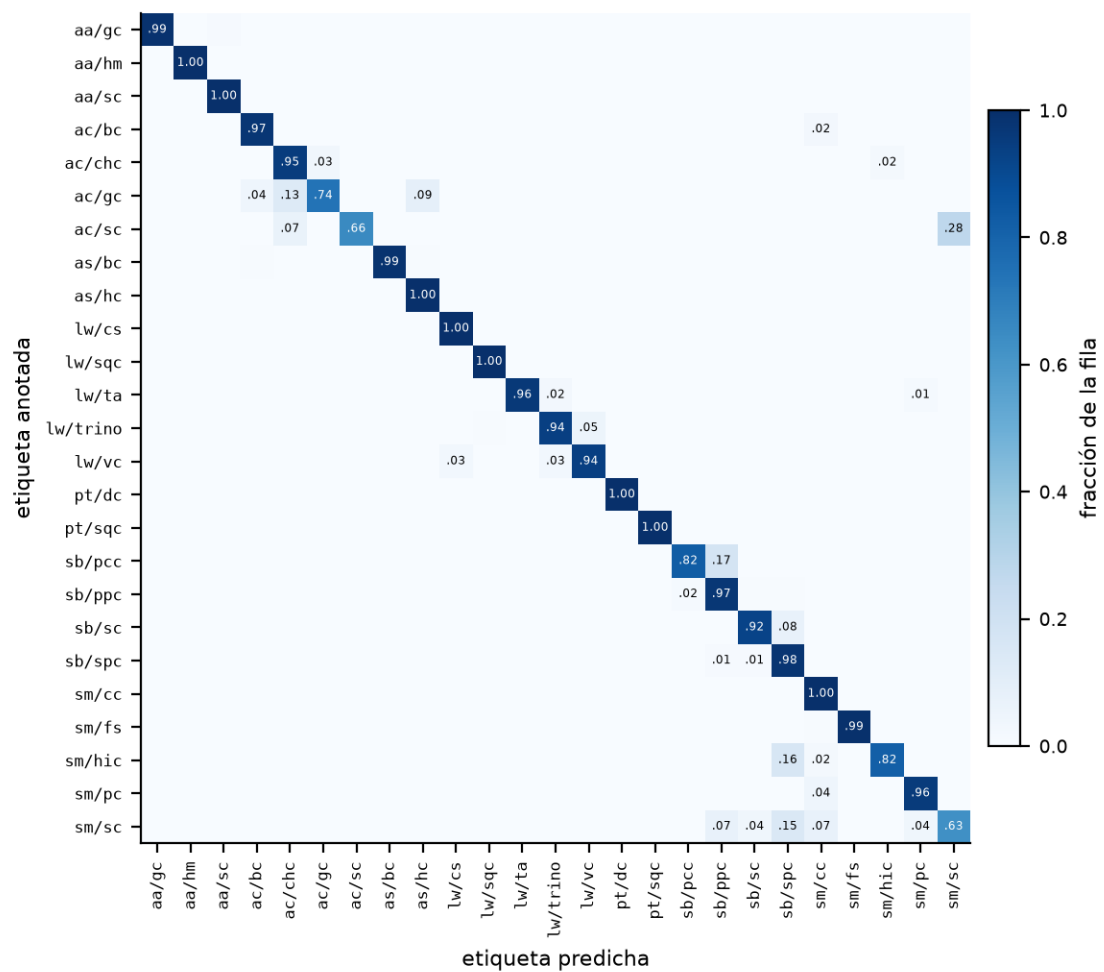


Figura 5.13

Matriz de confusión



Sobre 6 531 pares emparejados, solo el 0,019 de la masa cae fuera de la diagonal, y el 55,4 % de esos errores es entre tipos de llamada de la *misma* especie: el error que anticipaba la geometría del conjunto, que separa especies pero no tipos de llamada dentro de una especie (sección 4.5.3). La confusión más frecuente, *ac/bc* $\rightarrow$ *sm/cc*, suma 17 pares, el 14,0 % de todos los errores de etiqueta.

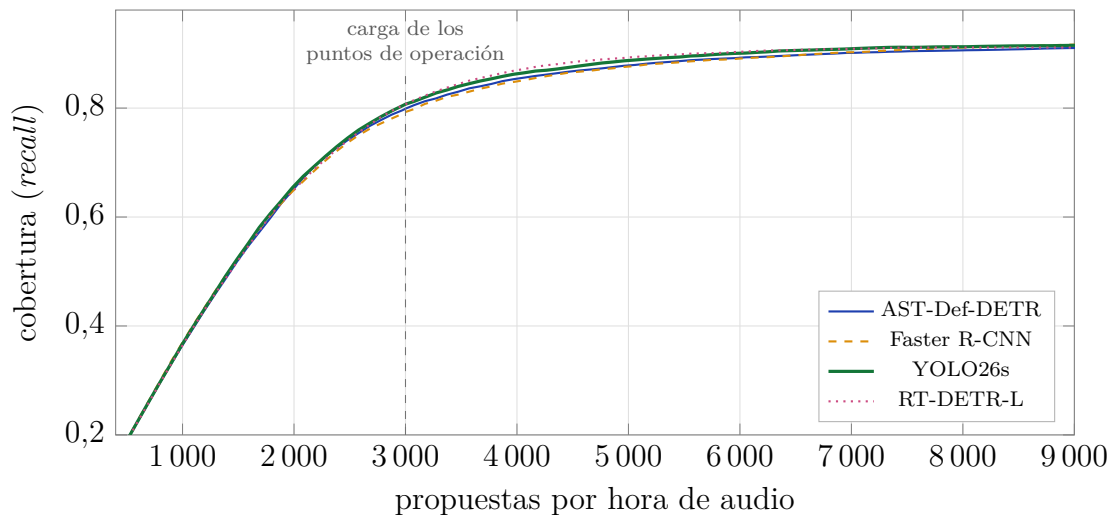
5.4.4. Cobertura frente a carga de revisión

La confianza que emite un detector no es una probabilidad comparable entre arquitecturas: depende de la función de pérdida, del número de propuestas que la cabeza produce y de cómo se normaliza la puntuación. Los cuatro umbrales elegidos (de 0,11 a 0,69) lo muestran sin ambigüedad.

La magnitud que sí es comparable, y además es la que interesa a OE3, es **cuántos eventos recupera cada modelo cuando a todos se les permite proponer la misma cantidad de cajas por hora de audio**. La Figura 5.14 barre el umbral de cada configuración y lee su curva a la misma abscisa; la Tabla 5.13 la tabula en seis presupuestos.

Figura 5.14

Cobertura frente a carga



Cada curva es un modelo: cuanto más alta, más eventos recupera con la misma cantidad de propuestas por hora, y la línea punteada marca la carga de los cuatro puntos de operación. La Tabla 5.13 lee las curvas en seis presupuestos.

El resultado refuerza la lectura de la Tabla 5.11 y añade algo que el punto de operación no dejaba ver.

Tabla 5.13
Cobertura a carga igual

Propuestas por hora	AST-Def- DETR	Faster R-CNN	YOLO26s	RT-DETR-L
1 500	0,522	0,526	0,526	0,521
2 000	0,652	0,650	0,657	0,649
3 000	0,799	0,793	0,807	0,807
4 000	0,854	0,849	0,863	0,869
6 000	0,893	0,891	0,901	0,903
9 000	0,910	0,916	—	0,915
Techo (sin umbral)	0,932	0,953	0,912	0,940
Carga en el techo	133 421	84 093	7 528	109 197

Nota. Cada columna es la curva de un modelo leída a la misma abscisa. El techo es el *recall* con el umbral en el piso del protocolo, esto es, con todo lo que el modelo es capaz de proponer; la última fila es la carga a la que lo alcanza. Un guion significa que el modelo no llega a emitir tantas propuestas por hora.

En el rango de carga que un revisor humano puede absorber, las cuatro configuraciones son indistinguibles. A 3 000 propuestas por hora (aproximadamente la carga de los cuatro puntos de operación, que promedian 2 980) la cobertura va de 0,793 a 0,807. Cuatro arquitecturas con 10,0, 32,9, 43,4 y 88,6 millones de parámetros, con extractores preentrenados en tres corpus distintos y con recetas de entrenamiento que no están igualadas, convergen a la misma cobertura cuando se les da el mismo presupuesto de revisión. *La elección de arquitectura pesa menos, en esta tarea y con este material, que la elección del punto de operación.*

Lo que separa a las curvas es dónde termina cada una. El techo de cobertura (lo que el modelo recupera emitiendo absolutamente todo lo que puede proponer) va de 0,912 (YOLO26s) a 0,953 (Faster R-CNN). Pero el precio de ese techo es abismalmente distinto: Faster R-CNN necesita 84 093 propuestas por hora para alcanzarlo y YOLO26s llega al suyo con 7 528, un orden de magnitud menos. Ninguna de esas cifras es operativa (revisar ochenta mil cajas por hora de audio no es asistencia), pero la comparación sí dice algo: las curvas de los otros tres modelos solo superan a YOLO26s en una región de carga que nadie va a usar.

El umbral de 0,5 de fábrica no habría comparado nada. El umbral que la validación elige para YOLO26s es 0,11, cinco veces menor. Operarlo a 0,5 lo habría situado en la parte conservadora de su curva y a Faster R-CNN (cuyo umbral elegido

es 0,69) en la permisiva, y el orden entre los dos se habría invertido sin que ninguna propiedad de las arquitecturas cambiara. Es la razón por la que el punto de operación de cada modelo debe elegirse barriendo su umbral, y no heredando uno.

5.4.5. Lectura por clase

El agregado está dominado por dos clases: **as/hc**, **pt/dc** aportan el 32,6 % de las cajas de prueba y son, por la fragmentación que documenta la sección 4.4.3, las más fáciles del conjunto, porque sus cajas ocupan la ventana entera. Por eso la Tabla 5.14 reporta también el promedio por clase, que trata igual a una clase de 37 cajas que a una de dos mil.

Tabla 5.14
Recall por clase

Clase	Cajas test	AST-Def-DETR	Faster R-CNN	YOLO26s	RT-DETR-L
as/hc*	2021	0,972	0,980	0,947	0,963
ac/bc	828	0,819	0,806	0,820	0,848
lw/cs	779	0,877	0,807	0,813	0,866
sm/cc	686	0,758	0,845	0,847	0,850
as/bc	647	0,672	0,543	0,617	0,583
pt/dc*	567	0,966	0,961	0,942	0,938
sb/ppc	464	0,552	0,433	0,692	0,603
aa/gc	269	0,770	0,751	0,807	0,743
sb/spc	250	0,660	0,644	0,776	0,640
sm/fs	232	0,530	0,513	0,625	0,664
lw/trino	216	0,690	0,560	0,648	0,671
pt/sqc	138	0,935	0,935	0,993	0,942
sb/pcc	100	0,500	0,630	0,710	0,650
lw/ta	91	0,747	0,901	0,857	0,934
sm/hic	86	0,500	0,500	0,593	0,581
sm/pc	83	0,446	0,578	0,518	0,699
ac/chc	82	0,585	0,732	0,720	0,720
ac/gc	56	0,357	0,304	0,339	0,125
sm/sc	56	0,375	0,196	0,304	0,214
ac/sc	54	0,333	0,333	0,389	0,500
aa/sc	53	0,642	0,792	0,755	0,792
lw/sqc	52	0,865	0,808	0,865	0,827
aa/hm	44	0,818	0,591	0,795	0,568
lw/vc	40	0,575	0,900	0,800	0,700
sb/sc	37	0,622	0,486	0,676	0,568
Promedio por clase		0,636	0,634	0,694	0,665
Agregado (micro)	7931	0,797	0,781	0,811	0,808

Nota. En negrita, el mejor de las cuatro configuraciones en cada clase. Un asterisco marca las dos clases de ventana, que no entran en el promedio por clase.

El promedio por clase separa las configuraciones mucho más que el agregado: 0,694 para YOLO26s, 0,665 para RT-DETR-L, 0,636 para la propuesta y 0,634 para Faster R-

CNN. Es el mismo orden que la mAP, y con una distancia mayor: el modelo seleccionado le saca seis puntos de promedio por clase a la propuesta mientras que en el agregado le saca uno y medio. La ventaja no viene de las clases fáciles, viene de las escasas.

Tres patrones se leen en la tabla.

Hay un suelo que ninguna configuración cruza. `ac/gc`, `sm/sc` quedan por debajo de 0,5 en las cuatro, y son dos de las 8 clases con 60 cajas de prueba o menos. Cuando las cuatro arquitecturas fallan por igual sobre una clase, la causa razonable es el material y no el diseño: son las clases que el umbral de 100 anotaciones dejó entrar por poco y cuyo desempeño la sección 4.5.9 anticipó como no atribuible a la arquitectura.

La escasez no lo explica todo. `lw/ta` (91 cajas) llega a 0,934 y `pt/sqc` (138) a 0,993 con el modelo seleccionado, por encima de clases cinco veces más numerosas. Ambas ocupan bandas de frecuencia estrechas y bien separadas, lo que sugiere que la separabilidad geométrica de la sección 4.5.3 pesa más que el volumen de ejemplos. En la otra dirección, `as/bc` (647 cajas) no pasa de 0,672 en ninguna configuración: es una llamada que convive con la clase de ventana `as/hc` de la misma especie y queda enterrada en el coro.

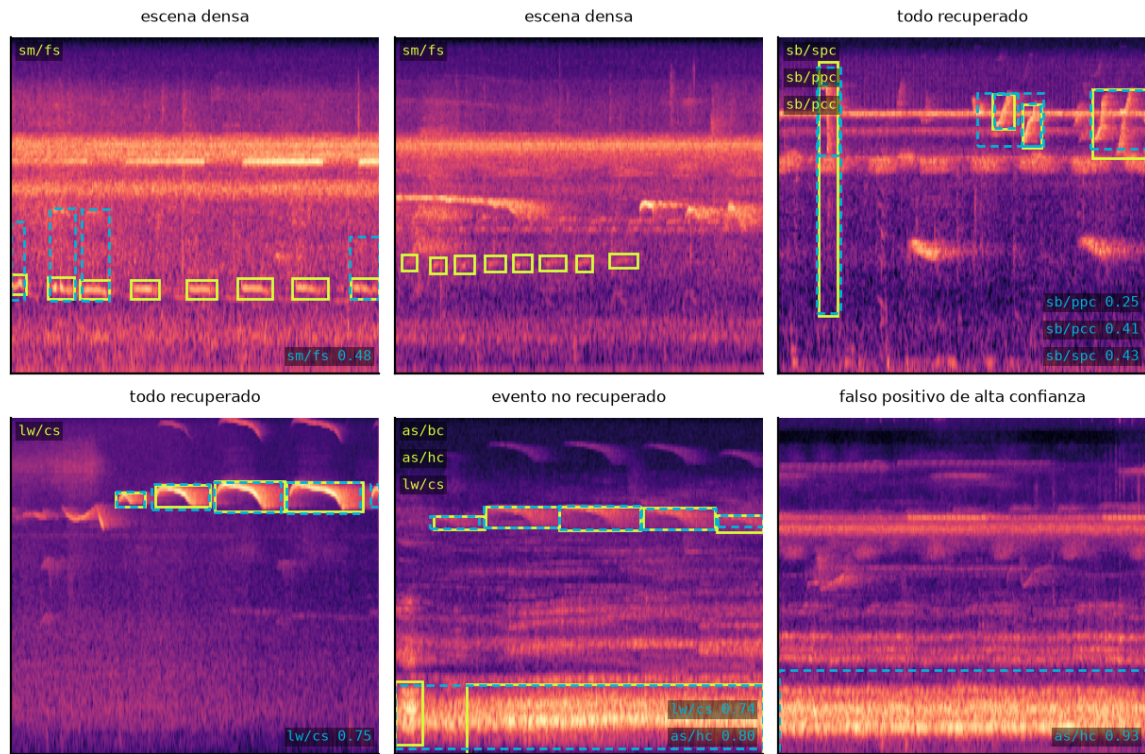
Ninguna configuración gana en todas partes. El modelo seleccionado es el mejor en 9 de las 23 clases de detección, RT-DETR-L en 7, la propuesta en 4 y Faster R-CNN en 3. La propuesta gana justamente donde el contexto largo importa (`as/bc` y `lw/trino`, dos clases que se emiten en secuencia), que es lo que cabría esperar de una cabeza de conjuntos con atención sobre toda la ventana.

5.4.6. Ejemplos cualitativos

La Figura 5.15 muestra seis ventanas de prueba con las anotaciones del equipo y las predicciones del modelo seleccionado a su umbral. Las seis se eligieron por criterio explícito y no por aspecto: dos escenas densas, dos ventanas con todos sus eventos recuperados, una con un evento que el modelo no encontró y una con un falso positivo de alta confianza, todas de grabaciones distintas.

Las dos escenas densas ilustran los dos extremos del comportamiento del modelo sobre el mismo tipo de material. En la primera recupera la mayoría de una serie de llamadas cortas y repetidas, con alguna caja de más; en la segunda, una serie equivalente de la misma clase en otra grabación, no emite ni una sola caja por encima del umbral.

Figura 5.15
Ejemplos cualitativos



La diferencia entre ambas no está en la densidad sino en la relación señal-ruido: la segunda tiene el coro de fondo encima de la banda de la llamada. Es un modo de fallo que conviene caracterizar, porque determina qué fracción del material queda sin asistir.

5.4.7. Selección del modelo y qué establece la comparación

El modelo que pasa a RE2.4 es YOLO26s (corrida yolo26s_v3). La regla declarada lo decide en tres pasos, y conviene dejar escrito cada uno:

1. La métrica primaria (*recall* en prueba con su intervalo) **no separa** a ninguna de las seis parejas posibles. No elige.
2. La secundaria ($mAP@0,3$) ordena las cuatro y pone a YOLO26s primero con 0,620 frente a 0,601 del segundo. Elige.
3. El desempate por costo de inferencia no hizo falta, pero apunta en la misma dirección: el modelo seleccionado es también el más barato (sección 5.4.8).

La hipótesis de diseño no se confirma. La propuesta queda tercera de cuatro en las dos métricas: 0,797 de *recall* frente a 0,811 del modelo seleccionado, y 0,577 de $mAP@0,3$ frente a 0,620. En la primaria la diferencia cae dentro de los intervalos de

confianza; en la secundaria, la propuesta empata con Faster R-CNN y queda por detrás de YOLO26s y de RT-DETR-L. No supera de forma medible a ninguna línea base, que es lo que la hipótesis afirmaba.

RT-DETR-L, que se incluyó para esto (sección 5.2.3), indica de qué componente viene el resultado. Predice un conjunto de cajas igual que la propuesta, pero desde un extractor preentrenado en imágenes, y con 32,9 M de parámetros frente a 88,6 M queda a la par en la primaria (0,808) y por encima en la secundaria (0,601 frente a 0,577). El contraste apunta a que la cabeza de conjuntos no es lo que falla: lo que no aporta una ventaja medible es el preentrenamiento en audio, en esta tarea y con este presupuesto de entrenamiento. Como las recetas de entrenamiento no están igualadas, es una indicación y no una atribución (sección 5.7).

5.4.8. Costo de inferencia

El costo se midió sobre las mismas 512 ventanas de prueba y en la misma GPU (NVIDIA RTX A5500), incluyendo el preprocesamiento de cada arquitectura, la supresión de no máximos y el tope de 100 cajas por ventana. El modelo seleccionado consume 0,12 minutos de GPU por hora de audio; la propuesta, 0,30; RT-DETR-L, 0,32; Faster R-CNN, 0,87. Son 7,3 veces entre los extremos. En términos operativos, el modelo seleccionado procesa el audio unas 501 veces más rápido que en tiempo real y el más lento unas 69 veces: las cuatro son viables para monitoreo acústico pasivo, de modo que el costo de inferencia es un criterio de desempate y no una restricción.

5.4.9. Verificación de RE2.3

El indicador enumera siete magnitudes mínimas, exige separar los tres ejes y justificar la selección del modelo. La Tabla 5.15 las localiza una por una.

Cada fila apunta a una tabla, una figura o una sección de este capítulo, y la última columna da el valor del modelo seleccionado, de modo que el indicador puede comprobarse sin salir del documento.

Tabla 5.15
Verificación de RE2.3

Elemento del IOV	Dónde	Valor del modelo seleccionado
<i>Recall</i>	Tabla 5.11	0,811 [0,787; 0,835]
Precisión	Tabla 5.11	0,665
F ₁	Tabla 5.11	0,731
IoU medio	Tabla 5.12, Figura 5.12	0,792 (mediano)
AP a IoU 0,25	Tabla 5.11	0,629
AP a IoU 0,5	Tabla 5.11	0,521
<i>Recall</i> por clase	Tabla 5.14	0,694 de promedio por clase
Matriz de confusión	Figura 5.13	0,019 fuera de la diagonal
Separa detección, encuadre y clasificación	Sección 5.4.3, Tabla 5.12	los tres ejes, por separado
Justifica la selección del modelo	Sección 5.4.7	regla declarada, aplicada en tres pasos

Nota. Los archivos de métricas son `comparacion_modelos_v3.txt` y su `.json` con las mismas cifras, en la carpeta RE2.3 del Anexo B. Toda magnitud de este capítulo procede de volver a aplicar el protocolo sobre los volcados crudos, no de una cifra transcrita.

5.5. Modelo final cargable y demostración funcional (RE2.4)

Resultado. El *checkpoint* del modelo seleccionado, con su vocabulario y su punto de operación, y la demostración de que una inferencia sobre un WAV produce una tabla de selección de Raven con coordenadas tiempo–frecuencia, especie, tipo de llamada y puntuación.

Cómo se alcanzó. Es la Fase 6 de CRISP-DM (sección 1.7.1). `detect.py` corta cada grabación en las mismas ventanas de 3 s con salto de 1,5 s del caché, ejecuta el modelo por lotes, aplica el posprocesamiento de la evaluación, funde entre ventanas solapadas las cajas anidadas de la misma clase y escribe la tabla junto a cada audio.

Cómo reproducirlo. Basta el *checkpoint* publicado en la versión v0.2.0 y el código del repositorio: `src/detect.py` acepta un WAV o una carpeta y escribe la tabla junto a cada audio. La demostración de la sección 5.5.3 se obtuvo así sobre una grabación de prueba.

Medio de verificación. El *checkpoint* del modelo final, que lleva dentro su vocabulario, y su punto de operación, publicados en la versión v0.2.0 del reposito-

rio (<https://github.com/nhrot-fc/tesis-primate/releases/tag/v0.2.0>) como el archivo `detector-2.0.0-model-yolo26s_v3.zip`, con su suma de verificación SHA-256; y la tabla de selección que la inferencia produce sobre una grabación de prueba (sección 5.5.3). El indicador nombra un archivo `.pth` y `src/infer.py`, nombres del prototipo: en la implementación final el *checkpoint* es un `.pt` autocontenido y la inferencia la ejecuta `src/detect.py`, con la misma función. La carpeta RE2.4 de los medios de verificación (Anexo B) contiene la comprobación de que el *checkpoint* carga sin claves incompatibles, su suma SHA-256, su punto de operación y la tabla de salida de la demostración.

5.5.1. El modelo final

El modelo que pasa a producción es el ganador de la comparación: `yolo26s_v3`, la época 17 de su corrida, con 10,0 M de parámetros en un *checkpoint* de 40 MB. Su punto de operación, elegido en validación con precisión $\geq 0,70$, es un umbral de confianza de 0,11, al que en prueba recupera 0,811 de las cajas anotadas con precisión 0,665. El umbral viaja junto al *checkpoint* en `operating_point.json` y es el que `detect.py` usa por defecto, de modo que el usuario no tiene que conocerlo.

Carga sin claves incompatibles. `load_checkpoint` reconstruye la arquitectura a partir de las claves `architecture` y `hparams` del archivo, con el número de clases que dicta `labels` (25), y carga el `state_dict` en modo estricto: los 708 tensores del archivo coinciden uno a uno con los del modelo reconstruido, sin faltantes ni sobrantes.

5.5.2. Formato de salida

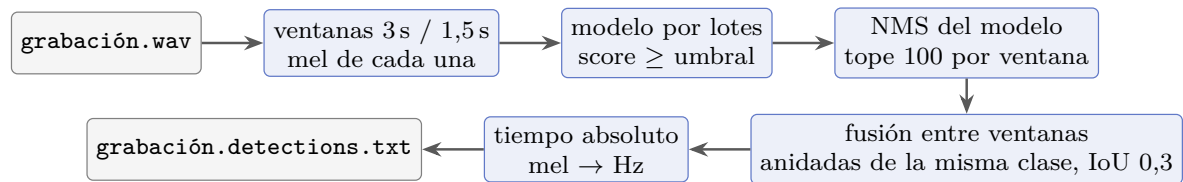
La salida es una *tabla de selección* de Raven Pro: texto delimitado por tabulaciones con las 10 columnas de la Tabla 5.16, que Raven abre sobre la grabación y dibuja como cajas sobre el espectrograma. Es el mismo formato en el que el equipo entrega sus anotaciones (sección 4.2), con dos diferencias: la columna **Score**, que Raven ignora y permite ordenar o filtrar las detecciones, y que **Species** y **Call type** vienen siempre en mayúscula y del vocabulario cerrado. Las coordenadas de frecuencia se obtienen invirtiendo la escala mel con la que se normalizaron las cajas (ecuación 2.4).

Que la salida tenga esta forma no es un detalle de implementación: es la tercera forma de salida de la sección 1.3.1, la única que coincide con la anotación del especialista, y llega en el archivo que el analista ya usa.

Tabla 5.16*Columnas de la tabla de salida*

Columna	Contenido
Selection	Correlativo desde 1, como exige Raven
View, Channel	Spectrogram 1, 1 : la vista y el canal en los que Raven dibuja
Begin Time (s), End Time (s)	Lados temporales de la caja, en segundos desde el inicio del archivo
Low Freq (Hz), High Freq (Hz)	Lados de frecuencia, en hercios
Species	Código de especie en mayúscula (LW)
Call type	Código del tipo de llamada en mayúscula (CS)
Score	Confianza del modelo en $[0, 1]$; solo en tablas del modelo

Las columnas de coordenadas y de etiquetas son las mismas que escribe el analista; **Score** es la única que añade el modelo. La Figura 5.16 resume cómo se llega a ellas desde la grabación.

Figura 5.16*Flujo de inferencia*

Raven Pro | hoja de cálculo

El paso que no existe en la evaluación es la fusión entre ventanas: como las ventanas se solapan, una misma llamada puede detectarse dos veces, y la fusión deja una sola caja antes de pasar a tiempo absoluto y a hercios.

5.5.3. Demostración

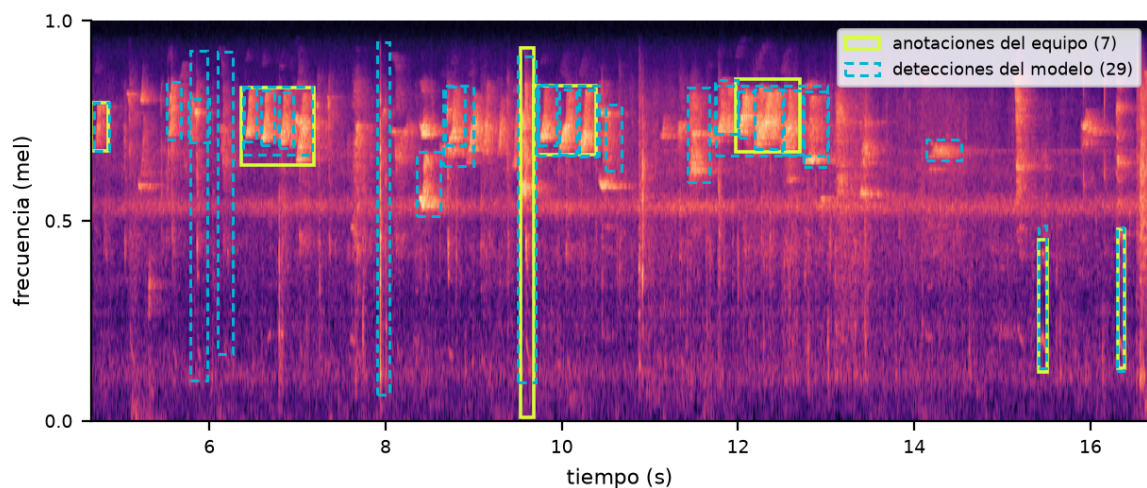
El modelo final se aplicó sobre una grabación de la partición de prueba, que no participó del entrenamiento ni de la selección del *checkpoint*: `20240516_083055.wav`, de 35,2 segundos, con 14 anotaciones del equipo repartidas entre **sb/pcc**, **sb/ppc**, **sb/spc**, **sm/cc**, **sm/sc**. La inferencia corrió en CPU, que es la condición más desfavorable y la que tendría un analista sin GPU, y produjo una tabla con las 10 columnas de la Tabla 5.16 y una fila por detección. La Tabla 5.17 recoge sus primeras filas.

Sobre esa grabación el modelo emite 55 detecciones al umbral 0,11. A $\text{IoU} \geq 0,3$ en coordenadas de la grabación entera, 13 de las 14 anotaciones del equipo tienen una

Tabla 5.17*Primeras filas de la tabla de salida*

Sel.	Begin Time (s)	End Time (s)	Low Freq (Hz)	High Freq (Hz)	Species	Call type	Score
1	0,34	1,15	3 157	4 511	SM	PC	0,32
2	1,51	1,57	584	2 953	SM	CC	0,28
3	2,29	2,45	5 912	9 220	SB	PPC	0,18
4	2,57	3,00	6 218	9 921	SB	PPC	0,51
5	2,92	3,50	3 748	5 094	SM	PC	0,65
6	4,69	4,87	6 677	10 543	SB	PPC	0,52
7	5,51	5,68	7 471	12 723	SB	PPC	0,20
8	5,77	5,99	7 262	10 887	SB	PPC	0,19

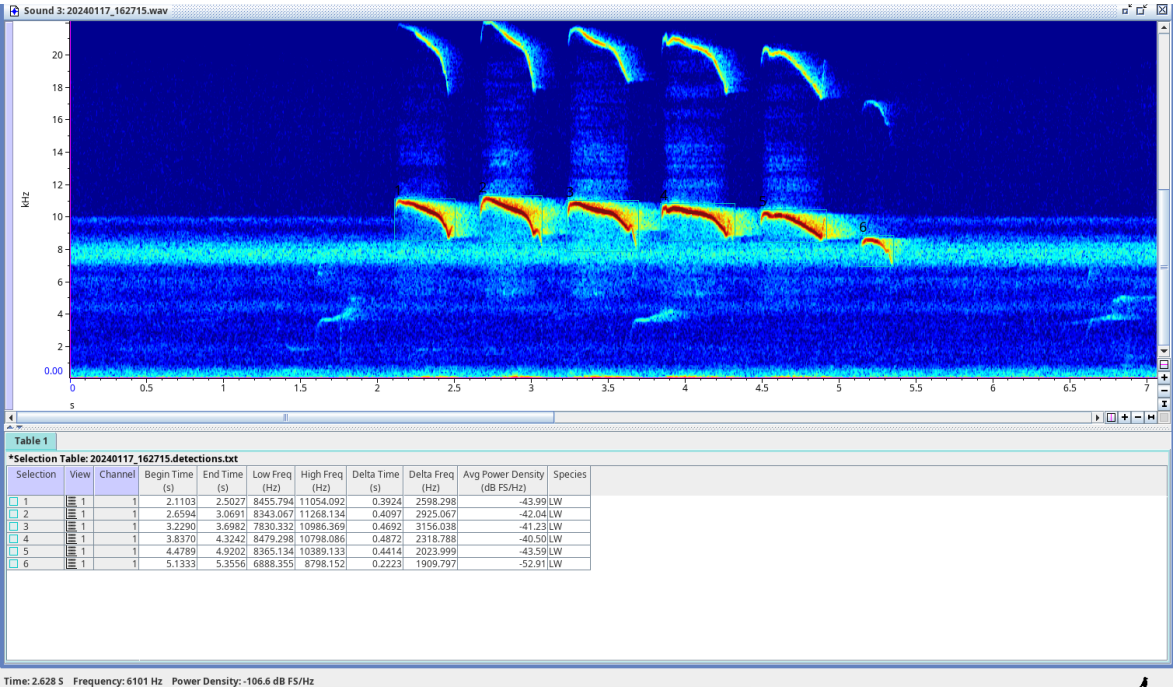
detección encima, 15 de las 55 detecciones caen sobre una anotación y 13 de ellas llevan además la misma etiqueta. La Figura 5.17 superpone las dos capas, tal como se ven en Raven, sobre el tramo de 12 segundos que concentra más anotaciones. **Es un ejemplo, no una medición:** una grabación no es una muestra, y las métricas son las de la sección 5.4.

Figura 5.17*Demostración sobre una grabación*

El archivo se abre en Raven sobre la grabación como una tabla de selección más, sin conversión ni herramienta añadida. La Figura 5.18 lo muestra sobre otra grabación de prueba, 20240117_162715.wav de *Leontocebus weddelli*: Raven Lite carga 20240117_162715.detections.txt como *Selection Table*, dibuja las seis detecciones como cajas en sus coordenadas de tiempo y frecuencia, las numera sobre el espectrograma y arrastra la columna **Species** a la tabla. A las columnas que produjo el modelo

añade las que calcula por su cuenta (Delta Time, Delta Freq, Avg Power Density), igual que con una tabla anotada a mano. Call type y Score viajan en el archivo pero esta versión del programa no las muestra, de modo que el filtrado por confianza se hace fuera de Raven.

Figura 5.18
La tabla generada abierta en Raven



A partir de ahí el trabajo es el habitual: el analista oye cada caja, borra las que no correspondan, corrige las que estén mal encuadradas y guarda la tabla ya revisada. Ese es el circuito cuyo tiempo mide OE3.

5.5.4. Verificación de RE2.4

El indicador exige dos cosas, y las dos quedan demostradas arriba:

1. El *checkpoint* se carga sin claves incompatibles. La carga en modo estricto de la sección 5.5.1 no deja tensores faltantes ni sobrantes.
2. La inferencia sobre un WAV produce un archivo tabulado con las columnas de Raven. La Tabla 5.17 lo muestra: 55 filas con coordenadas tiempo–frecuencia, especie, tipo de llamada y puntuación, en las 10 columnas de la Tabla 5.16. Que esas columnas son las de Raven y no una imitación lo comprueba la Figura 5.18: el archivo se abre en el propio programa, sobre su grabación, y las detecciones quedan dibujadas sobre el espectrograma.

5.6. Código y modelo disponibles en un repositorio (RE2.5)

Resultado. El repositorio público <https://github.com/nhrot-fc/tesis-primate>, que contiene el código de curación, entrenamiento, evaluación e inferencia y las instrucciones de ejecución, y que publica el *checkpoint* del modelo final en su versión v0.2.0.

Cómo se alcanzó. Con el procedimiento de respaldo de la sección 1.7.3, versionando el proyecto desde el inicio y no al entregarlo: 123 *commits* y una versión declarada en `pyproject.toml` (0.2.5), con la trazabilidad que describe la sección 5.3.5.

Cómo reproducirlo. Por dos caminos. Desde el código, clonando el repositorio en el *commit* `b895fe2`, instalando el entorno de `uv.lock` y ejecutando las etapas de la sección 5.3. Sin instalar nada, con el `README` del repositorio, que explica cómo descargar de su página de versiones el programa para Windows y el modelo, y cómo procesar con ellos una grabación.

Medio de verificación. La URL del repositorio y la de su versión v0.2.0 (<https://github.com/nhrot-fc/tesis-primate/releases/tag/v0.2.0>), que publica el modelo final; esta sección es el índice que lleva de cada verbo del indicador al archivo que lo implementa, y la carpeta RE2.5 de los medios de verificación (Anexo B) lo reproduce.

5.6.1. Estructura

El repositorio separa el código del material que no cabe en él. La Figura 5.19 muestra el primer nivel: `src/` contiene el *pipeline* entero y `research/`, este documento con sus figuras. Las dos carpetas sin versionar son las que guardan datos del equipo de investigación y artefactos de decenas de gigabytes, que se reconstruyen con el código que sí está publicado.

El indicador de RE2.5 enumera lo que el repositorio debe permitir hacer. La Tabla 5.18 lleva cada uno de esos verbos al archivo que lo implementa y al resultado esperado que lo documenta.

Cada fila apunta a una ruta del repositorio público, salvo la última, que es un archivo de su versión v0.2.0.

Figura 5.19*Árbol del repositorio*

tesis-primate/	
README.md	que es el proyecto y como se ejecuta
pyproject.toml	dependencias, extras y grupos; uv.lock las fija
evaluation.sh	volcado + comparacion de todas las corridas
Makefile	compila la tesis
src/	el pipeline entero (figura 5.9)
notebooks/	auditoria, curacion asistida, preentrenamiento
con aves	
research/	la tesis: main.tex, capitulos/<nn>/{tex, figures,
notebook}	
docs/	requisitos del sistema y notas técnicas
data/ (no versionado)	raw/, cleaned/, processed/, yolo/
runs/ (no versionado)	corridas: checkpoints, volcados,
comparacion	

Tabla 5.18*Ubicación de cada elemento del indicador*

Qué	Dónde	Documentado en
Curación	src/prepare_annotations.py, src/data/annotations.py, src/data/species.py	RE1.1
Conjunto y partición	src/prepare_data.py, src/data/manifest.py, src/data/cache.py	RE1.2, RE1.3
Arquitecturas	src/models/	RE2.1
Entrenamiento	src/train.py, src/training/	RE2.2
Evaluación	src/dump_predictions.py, src/compare_models.py, src/evaluation/, evaluation.sh	RE2.1, RE2.3
Inferencia	src/detect.py, src/inference/	RE2.4
Instrucciones de ejecución	README.md, pyproject.toml, Listado 5.1	RE2.2
Modelo final	El <i>checkpoint</i> autocontenido de yolo26s_v3 con su operating_point.json, en la versión v0.2.0	Sección 5.5.1

5.6.2. Qué contiene y qué no

El código, completo. Todo el *pipeline* tal como lo describe la sección 5.3, desde la lectura de las tablas de Raven hasta la escritura de la tabla de salida. No depende de nada que no esté declarado en `pyproject.toml` y fijado en `uv.lock`.

El modelo final, no las corridas. El artefacto que se publica es el *checkpoint* de `yolo26s_v3` (autocontenido, con su arquitectura, sus hiperparámetros, su vocabulario y el `meta.json` del caché con el que se entrenó (Tabla 5.8)) junto con su punto de operación. Las corridas completas, con sus `last.pt` y sus volcados, suman decenas de gigabytes y no aportan nada que el *checkpoint* elegido y las cifras de este capítulo no digan ya.

Las grabaciones y las anotaciones primarias, no. Pertenecen al equipo de investigación. El repositorio no las contiene ni las referencia por ubicación, y los pesos preentrenados de terceros (el AST de AudioSet, los de COCO) se resuelven desde su origen y conservan la licencia de su autor.

Fuera de alcance. La distribución del código y del modelo como producto (empaquetado para usuarios finales, versionado para terceros, licenciamiento y mantenimiento) no forma parte de esta tesis. El compromiso de este resultado, tal como lo enuncia su indicador, es que la URL permita localizar el código y los artefactos del modelo final, y eso es lo que la Tabla 5.18 verifica.

5.6.3. Verificación de RE2.5

El indicador pide que la URL permita localizar cuatro cosas, y las cuatro quedan localizadas en la Tabla 5.18: el código de curación, el de entrenamiento, el de evaluación y el de inferencia, cada uno con el resultado esperado que lo documenta; las instrucciones de ejecución, en el Listado 5.1; y los artefactos del modelo final, que son el *checkpoint* de `yolo26s_v3` y su punto de operación.

5.7. Discusión

Qué se obtuvo. Se implementó y se evaluó un detector que recibe un espectrograma y devuelve cajas tiempo–frecuencia con especie, tipo de llamada y confianza, y se lo

comparó con tres líneas base bajo un protocolo declarado antes de correr el experimento. Sobre las 7 598 ventanas de prueba, el modelo seleccionado recupera 0,811 de las 7 931 cajas anotadas $[0,787; 0,835]$ con precisión 0,665, al umbral que la validación eligió; como la métrica primaria no separa a las cuatro configuraciones, lo eligió la secundaria (sección 5.4.7). De los 55 resultados que produce sobre una grabación de 35,2 segundos, el sistema entrega una tabla que el analista abre en su propia herramienta.

Qué significan. Tres lecturas, en orden de importancia para el problema.

La primera es que **el error está en encontrar y encuadrar, no en nombrar**. Una vez localizado el evento, la clase casi nunca falla (sección 5.4.3): la parte del problema que parecía más difícil, con 25 clases desbalanceadas, resultó ser la que el modelo resuelve. Para el uso asistido esto es una buena noticia con una consecuencia concreta: el revisor no tendrá que corregir etiquetas, tendrá que confirmar o descartar cajas, que es una operación mucho más rápida. Y para el diseño de un sistema futuro dice dónde invertir: en la detección y el encuadre, no en la cabeza de clasificación.

La segunda es que **la elección del punto de operación pesa más que la elección de la arquitectura**. A 3 000 propuestas por hora de audio las cuatro configuraciones cubren entre 0,793 y 0,807, pero mover el umbral de un solo modelo lo lleva de 0,52 a 0,91 dentro del mismo rango de carga. Esto reordena la prioridad del trabajo que queda: una vez que se tiene un detector razonable, el margen está en calibrarlo contra el presupuesto de revisión real, que es justamente lo que mide OE3.

La tercera es que **el preentrenamiento en audio no aportó una ventaja medible**. Se esperaba que la propuesta superara a los detectores preentrenados en COCO, y quedó tercera de cuatro (sección 5.4.7). Para el equipo es la lectura favorable: el modelo seleccionado es también el más pequeño (10,0 M de parámetros frente a 88,6 M), el que menos entrenamiento necesitó (1,0 h de GPU frente a 6,6 h) y el más rápido en inferencia. OE2 pide un detector comparado con una línea base, no que gane la propuesta, y la herramienta usa el mejor de los cuatro. Para la investigación es un resultado negativo, y vale como tal porque la regla de decisión se fijó antes de entrenar. La propuesta gana algunas comparaciones parciales (cuatro clases, y las dos clases que se emiten en secuencia), pero elegir las después de ver los resultados equivaldría a cambiar esa regla.

Consistencia con la literatura. Los resultados concuerdan con la línea del Capítulo 3 en tres puntos y discrepan en uno.

Concuerdan en que **la detección bidimensional sobre el espectrograma es una formulación viable en bioacústica**: Escobar-Amado et al. (2024) reportan con Faster R-CNN sobre llamadas de foca, y Airale et al. (2025) y González et al. (2025) con detectores de la familia YOLO, cifras del mismo orden que las de la Tabla 5.11 sobre repertorios de tamaño comparable. Concuerdan también en que **un umbral de IoU bajo es lo razonable cuando las fronteras del evento son borrosas** (Z. Zhu & Sato, 2025): la Figura 5.12 muestra que incluso los pares que el protocolo cuenta como aciertos tienen un IoU mediano de alrededor de 0,78, lejos de lo que se exige en detección sobre imágenes naturales. Y concuerdan en que **la salida debe tener la forma que el analista ya usa**: la diferencia entre entregar una probabilidad por segmento (Kahl et al., 2021) y entregar una caja con especie, tipo de llamada y confianza es lo que permite abrir el resultado en Raven sin ningún paso intermedio.

Discrepan con Z. Zhu y Sato (2025), el precedente directo de esta arquitectura, que con un AST *autosupervisado* supera a Faster R-CNN en un dominio de una sola clase de sonido. Aquí el extractor es supervisado y las clases son 25, desbalanceadas: este trabajo no refuta aquel resultado, muestra que no se traslada sin más a un repertorio de primates.

Generalización. Qué de esto sobrevive fuera de este corpus.

Se generaliza, con bastante confianza, **la separación de los tres ejes como método de evaluación**. Es independiente del taxón y del repertorio, y su valor (saber si un agregado bajo se debe a detección o a clasificación) se vuelve mayor cuanto más clases tiene el problema. Se generaliza también **la lectura de cobertura frente a carga**: cualquier sistema de pre-anotación asistida compite contra un presupuesto de revisión, y el umbral compartido entre arquitecturas es un artefacto de calibración en todas partes, no solo aquí.

Se generaliza con reservas **el hallazgo de que la clasificación es fácil**. Depende de que las clases sean acústicamente separables, y en este corpus lo son en buena medida porque la especie está determinada por la carpeta de origen (sección 4.3.2): un modelo que vea una grabación de una especie no anotada por el equipo no tiene por

qué comportarse igual. Un corpus con varias especies simpátricas anotadas en la misma grabación pondría a prueba ese resultado.

No se generaliza **el orden entre arquitecturas**. Es el resultado de este material, con este presupuesto de entrenamiento y con recetas no igualadas; con diez veces más datos anotados, o con un programa de entrenamiento más largo para la propuesta, el orden podría ser otro. Tampoco se generalizan los umbrales elegidos, que son propiedades de cada *checkpoint* y no de la arquitectura.

Limitaciones. Cinco, ordenadas por cuánto comprometen las conclusiones.

1. **No hay techo de evaluación.** No se midió el acuerdo entre anotadores sobre una muestra del material, de modo que no se sabe cuánto del error que la Tabla 5.11 atribuye al modelo es en realidad desacuerdo entre humanos. Es la limitación más seria de todo el capítulo: sin ella no se puede decir si 0,811 está cerca o lejos de lo alcanzable. OE3 la aborda parcialmente al hacer que un experto revise una muestra de predicciones.
2. **La comparación no aísla componentes.** Las configuraciones usan el optimizador y las aumentaciones de su marco de trabajo, de modo que la comparación mide sistemas completos, que es lo que se declaró, y no permite atribuir la diferencia a un componente. Las dos variantes de la propuesta que lo harían (extractor congelado, y Deformable DETR con su ResNet-50 de COCO entrenado con la misma receta) ya están en el código y quedan como trabajo futuro.
3. **Las clases escasas no se evalúan de forma fiable.** 8 de las 25 clases tienen 60 cajas de prueba o menos, y sobre esas cifras un intervalo de confianza por clase sería tan ancho que no distinguiría nada. Las cifras por clase de la Tabla 5.14 son indicativas en la mitad inferior de la tabla, no concluyentes.
4. **Las dos clases de ventana inflan los agregados.** En as/hc, pt/dc detectar equivale a clasificar la ventana, y aportan el 32,6 % de las cajas de prueba: todo agregado que las incluya, como el *recall* global, sale más alto que el de las clases de evento.
5. **La prueba se midió una vez, por diseño, pero con una sola semilla.** El protocolo protege contra el sobreajuste a la partición de prueba, no contra la

varianza de entrenamiento: no se sabe cuánto se movería cada cifra al reentrenar la misma configuración con otra semilla. Dado que las cuatro caben en 0,030, esa varianza podría ser del mismo orden que las diferencias que se reportan.

Capítulo 6

Tiempo de Revisión y Análisis de Errores (OE3)

6.1. Introducción

Este capítulo corresponde al tercer objetivo específico:

OE3. Cuantificar la reducción del tiempo de revisión que aporta el sistema y caracterizar sus errores mediante revisión experta de una muestra de predicciones.

Según el cronograma (Tabla A.3), sus tres resultados se entregan en E3; en esta entrega el capítulo fija su estructura. Los resultados comprometidos en la Tabla 1.6 ocuparán una sección cada uno: la metodología de validación comparativa (RE3.1, sección 6.2), la medición comparativa del tiempo de revisión (RE3.2, sección 6.3) y el análisis cualitativo de errores y hallazgos (RE3.3, sección 6.4). Cada sección seguirá la estructura de los capítulos 4 y 5, y la sección 6.5 discutirá los tres en conjunto.

El material de entrada es el modelo seleccionado en el Capítulo 5 y la partición de prueba construida en el Capítulo 4: OE3 no entrena ni vuelve a ajustar nada, mide lo que cuesta usar el sistema.

- 6.2. Metodología de validación comparativa
(RE3.1)
- 6.3. Medición comparativa del tiempo de revisión
(RE3.2)
- 6.4. Análisis cualitativo de errores y hallazgos
(RE3.3)
- 6.5. Discusión

Capítulo 7

Conclusiones

7.1. Implicaciones de los resultados

El trabajo deja dos cosas que no existían al empezar. La primera es un conjunto acústico utilizable: 19 213 anotaciones curadas bajo un vocabulario cerrado, de las que 17 495 forman un experimento de 25 clases particionado por grabación, con su ficha y sus limitaciones declaradas. La segunda es la evidencia de que la detección de cajas tiempo–frecuencia sobre espectrograma es viable en este dominio, y de que el cuello de botella no está donde se esperaba.

Esa es la implicación principal del Capítulo 5: la clasificación de especie y tipo de llamada resulta casi resuelta sobre los pares emparejados, en las cuatro configuraciones comparadas. El problema difícil no era distinguir 25 clases con más de un orden de magnitud de desbalance, sino encontrar y encuadrar el evento. Para el equipo de investigación la consecuencia es concreta: la revisión asistida consistirá sobre todo en ajustar cajas, no en corregir etiquetas.

La segunda implicación es metodológica. Al declarar la regla de decisión antes de entrenar, el experimento pudo cerrarse con un resultado que no favorece a la arquitectura propuesta sin que eso lo invalide, y al separar la medición en detección, encuadre y clasificación quedó visible qué parte del problema está resuelta y cuál no. Una sola cifra agregada habría escondido ambas cosas.

7.2. Cumplimiento de los resultados esperados

La Tabla 7.1 contrasta cada resultado esperado del Capítulo 1 con la sección que lo entrega y lo verifica contra su indicador.

Tabla 7.1

Cumplimiento de los resultados esperados

RE	Resultado	Verificado en	Estado
RE1.1	Protocolo de curación documentado	Sección 4.3.3	Cumplido
RE1.2	Conjunto curado y particionado	Sección 4.4.5	Cumplido
RE1.3	Ficha del conjunto derivado	Sección 4.5.10	Cumplido
RE2.1	Diseño, arquitectura y líneas base	Sección 5.2.5	Cumplido
RE2.2	<i>Pipeline</i> reproducible	Sección 5.3.6	Cumplido
RE2.3	Informe comparativo desagregado	Sección 5.4.9	Cumplido
RE2.4	Modelo final y salida a tabla de Raven	Sección 5.5.3	Cumplido
RE2.5	Repositorio con código y modelo	Sección 5.6	Cumplido

Los ocho resultados de OE1 y OE2 se entregan y se verifican contra su indicador. La única desviación respecto de lo comprometido se declara en su sección: los nombres de archivo que citan los indicadores de RE2.2 y RE2.4, heredados del prototipo, corresponden en la implementación final a otros con la misma función.

7.3. Cumplimiento de los objetivos específicos

OE1. Cumplido. El Capítulo 4 entrega el protocolo, el conjunto particionado por grabación y la ficha, y mide cada uno contra su indicador: el protocolo conserva el 95,7 % de las filas crudas, la partición no comparte ninguna grabación entre subconjuntos y la ficha cubre los apartados comprometidos, incluidas las limitaciones que el propio material impone.

OE2. Cumplido. El Capítulo 5 implementa el detector, lo compara con tres líneas base bajo un protocolo único declarado de antemano y selecciona YOLO26s por la métrica secundaria, con 0,811 de *recall* y 0,620 de mAP en la partición de prueba, medida una sola vez. El modelo final se carga y produce una tabla que Raven abre sobre la grabación, y el repositorio contiene el código y el *checkpoint*.

OE3. Se entrega en E3: el cronograma (Tabla A.3) programa sus tres resultados después de esta entrega, y el Capítulo 6 fija su estructura.

7.4. Cumplimiento del objetivo general

El objetivo general se compromete con la reducción del tiempo de revisión experta, y esa magnitud la mide RE3.2. Su veredicto se enuncia en E3, una vez medida; adelantarlo sería fijar la conclusión antes que la evidencia.

7.5. La propuesta frente al problema central

El problema central del Capítulo 1 es que la anotación de las vocalizaciones de primates neotropicales exige más tiempo de especialista del que está disponible, y por ello avanza mucho más lento que la recolección del audio. La alternativa de solución ataca sus dos causas, una por objetivo específico, y con el tercero mide su efecto sobre el problema.

Contra el ruido, el sesgo y las inconsistencias de las únicas anotaciones disponibles (C1) actúa OE1. El vocabulario cerrado, el mapa de sinónimos y la regla que hace de la carpeta la fuente de verdad de la especie convierten un material escrito a mano, con decenas de grafías para la misma clase, en 25 clases con criterio único. No corrige el juicio del anotador, que es irrecuperable sin volver al audio, pero elimina la parte del desacuerdo que era de escritura y no de criterio, y deja marcado y conservado lo que no puede decidir.

Contra la búsqueda manual de vocalizaciones escasas y breves en horas de audio (C2) actúa OE2, y aquí conviene ser preciso sobre lo que el sistema es y lo que no. No sustituye al analista ni pretende hacerlo: la cabeza emite cajas con la forma exacta en que el equipo ya anota, y el analista las recibe en su propia herramienta para aceptarlas, corregirlas o borrarlas. Es pre-anotación asistida, y por eso el compromiso elegido privilegia la cobertura sobre la precisión: perder una vocalización cuesta más que descartar una propuesta falsa, porque el humano revisa la salida de todos modos.

7.6. Trabajo futuro

Lo que queda fuera no es una lista de deseos: son las limitaciones declaradas en las discusiones de los capítulos de objetivo, ordenadas por cuánto comprometen lo que aquí se afirma.

- **Medir el acuerdo entre anotadores.** Es la carencia más seria. Sin ella no hay techo de evaluación, y una parte del error que este trabajo atribuye al modelo podría ser desacuerdo humano.
- **Separar el extractor de la cabeza.** Dos variantes de la propuesta lo harían: con el extractor congelado, y el mismo decodificador sobre el ResNet-50 de COCO con la misma receta. El código ya las admite; sin ellas, la comparación entre configuraciones no puede atribuir la diferencia a un componente.
- **Reconstruir el evento largo.** Las clases de duración mayor que la ventana sobreviven fragmentadas, sin ningún mecanismo que las reagrupe antes de puntuarlas.
- **Cubrir el nivel de frase.** Las clases de frase quedan fuera por anidamiento jerárquico, y con ellas el nivel superior de la jerarquía vocal.
- **Cerrar el vocabulario con el repertorio de referencia.** Los pares fuera de vocabulario mezclan error tipográfico y subdivisión legítima, y no hay forma de separarlos sin una referencia del repertorio vocal de cada especie.

Referencias

- Airale, L., Pajot, A., & Linossier, J. (2025). *NBM: An open dataset for the acoustic monitoring of nocturnal migratory birds in Europe* [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2412.03633>
- Alves, S. L., Ravetta, A. L., Paim, F. P., Mittermeier, R. A., Rabelo, R. M., Wallace, R. B., Messias, M. R., Calouro, A. M., Rylands, A. B., de Melo, F. R., & Boubli, J. P. (2021). *Ateles chamek* (amended version of 2020 assessment). *The IUCN Red List of Threatened Species, 2021*, e.T41547A191685783. <https://doi.org/10.2305/IUCN.UK.2021-1.RLTS.T41547A191685783.en>
- Blumstein, D. T., Mennill, D. J., Clemins, P., Girod, L., Yao, K., Patricelli, G., Deppe, J. L., Krakauer, A. H., Clark, C., Cortopassi, K. A., Hanser, S. F., McCowan, B., Ali, A. M., & Kirschel, A. N. G. (2011). Acoustic monitoring in terrestrial environments using microphone arrays: Applications, technological considerations and prospectus. *Journal of Applied Ecology*, 48(3), 758–767. <https://doi.org/10.1111/j.1365-2664.2011.01993.x>
- Bonet-Solá, D., & Alsina-Pagès, R. M. (2021). A comparative survey of feature extraction and machine learning methods in diverse acoustic environments. *Sensors*, 21(4), 1274. <https://doi.org/10.3390/s21041274>
- Campos, R., Krofel, M., Rio-Maior, H., & Renna, F. (2025). HOWLish: A CNN for automated wolf howl detection. *Remote Sensing in Ecology and Conservation*. <https://doi.org/10.1002/rse2.70024>
- Cañas, J. S., Toro-Gómez, M. P., Sugai, L. S. M., Benítez Restrepo, H. D., Rudas, J., Posso Bautista, B., Toledo, L. F., Dena, S., Domingos, A. H. R., de Souza, F. L., Neckel-Oliveira, S., da Rosa, A., Carvalho-Rocha, V., Bernardy, J. V., Sugai, J. L. M. M., dos Santos, C. E., Bastos, R. P., Llusia, D., & Ulloa, J. S. (2023). A dataset for benchmarking Neotropical anuran calls identification in passive acoustic monitoring. *Scientific Data*, 10(1), Artículo 771. <https://doi.org/10.1038/s41597-023-02666-2>
- Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.
- Cornell Lab of Ornithology. (s.f.). *Macaulay Library* [Base de datos]. Recuperado el 7 de octubre de 2026, de <https://www.macaulaylibrary.org/>
- do Nascimento, L. A., Pérez-Granados, C., & Beard, K. H. (2021). Passive acoustic monitoring and automatic detection of diel patterns and acoustic structure of howler monkey roars. *Diversity*, 13(11), 566. <https://doi.org/10.3390/d13110566>
- Dubus, G., d'Audiffret, T., Auger, C., Cornette, R., Hauptert, S., Kasekendi, I., Katumba, R., Magaldi, H., Pernel, L., Rugonge, H., Sueur, J., Tibesigwa, J. J., & Krief, S. (2026). *DeepForestSound: A multi-species automatic detector for passive acoustic*

- monitoring in African tropical forests, a case study in Kibale National Park* [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2604.08087>
- Ebbbers, J., Germain, F. G., Wichern, G., & Le Roux, J. (2024). *Sound event bounding boxes* [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2406.04212>
- Escobar-Amado, C., Badiey, M., & Wan, L. (2024). Computer vision for bioacoustics: Detection of bearded seal vocalizations in the Chukchi Shelf using YOLOv5. *IEEE Journal of Oceanic Engineering*, 49(1), 133–144. <https://doi.org/10.1109/JOE.2023.3307175>
- Estrada, A., Garber, P. A., Rylands, A. B., Roos, C., Fernandez-Duque, E., Di Fiore, A., Nekaris, K. A.-I., Nijman, V., Heymann, E. W., Lambert, J. E., Rovero, F., Barelli, C., Setchell, J. M., Gillespie, T. R., Mittermeier, R. A., Verde Arregoitia, L., de Guinea, M., Gouveia, S., Dobrovolski, R., . . . Li, B. (2017). Impending extinction crisis of the world’s primates: Why primates matter. *Science Advances*, 3(1), e1600946. <https://doi.org/10.1126/sciadv.1600946>
- Fischer, J., & Hammerschmidt, K. (2020). Towards a new taxonomy of primate vocal production learning. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 375(1789), 20190045. <https://doi.org/10.1098/rstb.2019.0045>
- Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daume III, H., & Crawford, K. (2021). Datasheets for datasets. *Communications of the ACM*, 64(12), 86–92. <https://doi.org/10.1145/3458723>
- Gibb, R., Browning, E., Glover-Kapfer, P., & Jones, K. E. (2018). Emerging opportunities and challenges in passive acoustic monitoring. *Methods in Ecology and Evolution*, 9(5), 1191–1201. <https://doi.org/10.1111/2041-210X.12978>
- Gong, Y., Chung, Y.-A., & Glass, J. (2021). AST: Audio Spectrogram Transformer. *Proceedings of Interspeech 2021*, 571–575. <https://doi.org/10.21437/Interspeech.2021-698>
- González, S. S., Quiros, S. B., & Toledano, D. T. (2025). YOLO-based transfer learning for sound event detection using visual object detection techniques. *Applied Sciences*, 16(1), Artículo 205. <https://doi.org/10.3390/app16010205>
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- He, Y., Trigoni, N., & Markham, A. (2021). *SoundDet: Polyphonic moving sound event detection and localization from raw waveform* [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2106.06969>
- Heinicke, S., Kalan, A. K., Wagner, O. J., Mundry, R., Lukashevich, H., & Kuhl, H. S. (2015). Assessing the performance of a semi-automated acoustic monitoring system for primates. *Methods in Ecology and Evolution*, 6(7), 753–763. <https://doi.org/10.1111/2041-210X.12384>
- Hill, A. P., Prince, P., Piña Covarrubias, E., Doncaster, C. P., Snaddon, J. L., & Rogers, A. (2018). AudioMoth: Evaluation of a smart open acoustic device for monitoring biodiversity and the environment. *Methods in Ecology and Evolution*, 9(5), 1199–1211. <https://doi.org/10.1111/2041-210X.12955>
- Hong, Z., Xie, Y., He, K., Zhou, Q., Miu, Y., Peng, H., Cao, Z., & Fan, P. (2026). Deep audio detection networks from passive acoustic monitoring of a critically endangered primate. *Integrative Zoology*, 21(4), 986–1001. <https://doi.org/10.1111/1749-4877.70019>

- Hopping, W. A., Kahl, S., & Klinck, H. (2022). *A collection of fully-annotated soundscape recordings from the Southwestern Amazon Basin* (Ver. 1) [Conjunto de datos]. Zenodo. <https://doi.org/10.5281/zenodo.7079124>
- Jin, C., Kim, M., Jang, S., & Paeng, D.-G. (2022). Semantic segmentation-based whistle extraction of Indo-Pacific bottlenose dolphin residing at the coast of Jeju island. *Ecological Indicators*, 137, 108792. <https://doi.org/10.1016/j.ecolind.2022.108792>
- K. Lisa Yang Center for Conservation Bioacoustics. (s.f.). *Raven Pro: Interactive sound analysis software* [Software]. The Cornell Lab of Ornithology. <https://www.ravensoundsoftware.com/>
- Kahl, S., Wood, C. M., Eibl, M., & Klinck, H. (2021). BirdNET: A deep learning solution for avian diversity monitoring. *Ecological Informatics*, 61, 101236. <https://doi.org/10.1016/j.ecoinf.2021.101236>
- Kalan, A. K., Mundry, R., Wagner, O. J. J., Heinicke, S., Boesch, C., & Kühl, H. S. (2015). Towards the automated detection and occupancy estimation of primates using passive acoustic monitoring. *Ecological Indicators*, 54, 217–226. <https://doi.org/10.1016/j.ecolind.2015.02.023>
- Kath, H., Serafini, P. P., Campos, I. B., Gouvêa, T. S., & Sonntag, D. (2024). Leveraging transfer learning and active learning for data annotation in passive acoustic monitoring of wildlife. *Ecological Informatics*, 82, 102710. <https://doi.org/10.1016/j.ecoinf.2024.102710>
- Kershenbaum, A., Blumstein, D. T., Roch, M. A., Akçay, Ç., Backus, G., Bee, M. A., Bohn, K., Cao, Y., Carter, G., Cašar, C., Coen, M., DeRuiter, S. L., Doyle, L., Edelman, S., Ferrer-i-Cancho, R., Freeberg, T. M., Garland, E. C., Gustison, M., Harley, H. E., ... Zamora-Gutiérrez, V. (2016). Acoustic sequences in non-human animals: A tutorial review and prospectus. *Biological Reviews*, 91(1), 13–52. <https://doi.org/10.1111/brev.12160>
- Koga, N., Bando, Y., & Imoto, K. (2024). LEAD dataset: How can labels for sound event detection vary depending on annotators? *2024 Asia Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*, 1–6. <https://doi.org/10.1109/APSIPAASC63619.2025.10848643>
- Leroy, E. C., Thomisch, K., Royer, J.-Y., Boebel, O., & Van Opzeeland, I. (2018). On the reliability of acoustic annotations and automatic detections of Antarctic blue whale calls under different acoustic conditions. *The Journal of the Acoustical Society of America*, 144(2), 740–754. <https://doi.org/10.1121/1.5049803>
- Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. En D. Fleet, T. Pajdla, B. Schiele & T. Tuytelaars (Eds.), *Computer vision – ECCV 2014* (pp. 740–755). Springer. https://doi.org/10.1007/978-3-319-10602-1_48
- Lostanlen, V., Salamon, J., Farnsworth, A., Kelling, S., & Bello, J. P. (2018). BirdVox-full-night: A dataset and benchmark for avian flight call detection. *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 266–270. <https://doi.org/10.1109/ICASSP.2018.8461410>
- Lostanlen, V., Salamon, J., Farnsworth, A., Kelling, S., & Bello, J. P. (2019). Robust sound event detection in bioacoustic sensor networks. *PLOS ONE*, 14(10), e0214168. <https://doi.org/10.1371/journal.pone.0214168>
- Meedeniya, D., Ariyaratne, I., Bandara, M., Jayasundara, R., & Perera, C. (2023). A survey on deep learning based forest environment sound classification at the edge. *ACM Computing Surveys*, 56(3), 1–36. <https://doi.org/10.1145/3618104>

- Nam, H., Kim, S.-H., Ko, B.-Y., & Park, Y.-H. (2022). Frequency dynamic convolution: Frequency-adaptive pattern recognition for sound event detection. *Proceedings of Interspeech 2022*, 2763–2767. <https://doi.org/10.21437/Interspeech.2022-10127>
- Nguyen Hong Duc, P., Torterotot, M., Samaran, F., White, P. R., Gérard, O., Adam, O., & Cazau, D. (2021). Assessing inter-annotator agreement from collaborative annotation campaign in marine bioacoustics. *Ecological Informatics*, 61, 101185. <https://doi.org/10.1016/j.ecoinf.2020.101185>
- Nolasco, I., Singh, S., Morfi, V., Lostanlen, V., Strandburg-Peshkin, A., Vidaña-Vila, E., Gill, L., Pamula, H., Whitehead, H., Kiskin, I., Jensen, F. H., Morford, J., Emmerson, M. G., Versace, E., Grout, E., Liu, H., Ghani, B., & Stowell, D. (2023). Learning to detect an animal sound from five examples. *Ecological Informatics*, 77, 102258. <https://doi.org/10.1016/j.ecoinf.2023.102258>
- Pacheco, V., Diaz, S., Graham-Angeles, L., Flores-Quispe, M., Calizaya-Mamani, G., Ruelas, D., & Sánchez-Vendizú, P. (2021). Lista actualizada de la diversidad de los mamíferos del Perú y una propuesta para su actualización. *Revista Peruana de Biología*, 28(4), e21019. <https://doi.org/10.15381/rpb.v28i4.21019>
- Pérez-Granados, C., & Schuchmann, K.-L. (2021). Passive acoustic monitoring of the diel and annual vocal behavior of the black and gold howler monkey. *American Journal of Primatology*, 83(3), e23241. <https://doi.org/10.1002/ajp.23241>
- Pham, P., Li, J., Szurley, J., & Das, S. (2018). *Eventness: Object detection on spectrograms for temporal localization of audio events* [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.1712.09668>
- Politis, A., Adavanne, S., & Virtanen, T. (2020). A dataset of reverberant spatial sound scenes with moving sources for sound event localization and detection. *Proceedings of the Detection and Classification of Acoustic Scenes and Events 2020 Workshop (DCASE2020)*, 165–169. https://dcase.community/documents/workshop2020/proceedings/DCASE2020Workshop_Politis_88.pdf
- Romero-Mujalli, D., Bergmann, T., Zimmermann, A., & Scheumann, M. (2021). Utilizing DeepSqueak for automatic detection and classification of mammalian vocalizations: A case study on primate vocalizations. *Scientific Reports*, 11, 24463. <https://doi.org/10.1038/s41598-021-03941-1>
- Ruiz, D., Ulloa, J. S., Miao, Z., Betancourt, N., Toro-Gómez, M. P., Hernández, A., Demuro, B., Barona-Cortés, E., Mendoza-Henao, A., Sierra-Ricaurte, A., Pérez-Peña, S., Dodhia, R., Arbeláez, P., & Lavista Ferres, J. M. (2026). *A strongly annotated passive acoustic dataset for tropical bird monitoring* [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2605.20578>
- Sayigh, L., Daher, M. A., Allen, J., Gordon, H., Joyce, K., Stuhlmann, C., & Tyack, P. (2016). The Watkins Marine Mammal Sound Database: An online, freely accessible resource. *Proceedings of Meetings on Acoustics*, 27, Artículo 040013. <https://doi.org/10.1121/2.0000358>
- Stowell, D. (2022). Computational bioacoustics with deep learning: A review and roadmap. *PeerJ*, 10, e13152. <https://doi.org/10.7717/peerj.13152>
- Sugai, L. S. M., Silva, T. S. F., Ribeiro, J. W., & Llusia, D. (2019). Terrestrial passive acoustic monitoring: Review and perspectives. *BioScience*, 69(1), 15–25. <https://doi.org/10.1093/biosci/biy147>
- Taylor, A. M., & Reby, D. (2010). The contribution of source-filter theory to mammal vocal communication research. *Journal of Zoology*, 280(3), 221–236. <https://doi.org/10.1111/j.1469-7998.2009.00661.x>

- Vellinga, W.-P. (2026). *Xeno-canto – Bird sounds from around the world* [Conjunto de datos]. Xeno-canto Foundation for Nature Sounds. <https://doi.org/10.15468/QV0KSN>
- Venkatesh, S., Moffat, D., & Miranda, E. R. (2022). You only hear once: A YOLO-like algorithm for audio segmentation and sound event detection. *Applied Sciences*, 12(7), 3293. <https://doi.org/10.3390/app12073293>
- Veselý, A. (2008). Problem tree: A problem structuring heuristic. *Central European Journal of Public Policy*, 2(2), 68–80.
- Wired Amazon. (s.f.). *8 Primates Project*. Rainforest Expeditions. Recuperado el 7 de octubre de 2026, de <https://www.wiredamazon.com/projects/8-primates/>
- Wood, L., Anderson, K., Gerstoft, P., Bell, R., Subbaraman, R., & Bharadia, D. (2022). *Deep learning object detection approaches to signal identification* [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2210.16173>
- Zhu, X., Su, W., Lu, L., Li, B., Wang, X., & Dai, J. (2021). Deformable DETR: Deformable transformers for end-to-end object detection. *International Conference on Learning Representations (ICLR)*.
- Zhu, Z., & Sato, Y. (2025). Sound event detection using time-frequency bounding boxes with a self-supervised audio spectrogram transformer. *Proceedings of the 10th Workshop on Detection and Classification of Acoustic Scenes and Events (DCASE 2025)*, 150–154. https://dcase.community/documents/workshop2025/proceedings/DCASE2025Workshop_Zhu_54.pdf
- Zwerts, J. A., Stephenson, P. J., Maisels, F., Rowcliffe, M., Astaras, C., Jansen, P. A., van der Waarde, J., Sterck, L. E. H. M., Verweij, P. A., Bruce, T., Brittain, S., & van Kuijk, M. (2021). Methods for wildlife monitoring in tropical forests: Comparing human observations, camera traps, and passive acoustic sensors. *Conservation Science and Practice*, 3(12), e568. <https://doi.org/10.1111/csp2.568>

Anexo A

Plan del Proyecto

A.1. Justificación y viabilidad

A.1.1. Justificación

La investigación es conveniente porque ataca directamente el cuello de botella descrito en el Capítulo 1: el análisis manual del audio recolectado. El monitoreo acústico pasivo con sensores de bajo costo permite recolectar a gran escala, pero ese potencial queda desaprovechado mientras el análisis siga siendo lento, costoso y laborioso. Un sistema de pre-anotación asistida hace que el monitoreo a gran escala resulte operativamente viable y permite que el tiempo experto se dedique a la interpretación ecológica en lugar de al procesamiento.

En el plano práctico, el proyecto reduce la infrautilización del material acústico acumulado y, con ella, la latencia entre la recolección y la generación de conocimiento. En el plano teórico, aborda dos vacíos confirmados de forma independiente por la revisión del Capítulo 3: la ausencia de modelos y de conjuntos de datos con anotación tiempo–frecuencia para primates neotropicales, y la falta de estrategias establecidas para el etiquetado jerárquico de especie y tipo de llamada sobre eventos detectados.

A.1.2. Beneficios esperados

- Reducir la carga de revisión por hora de audio analizada: menos material que inspeccionar para la misma cobertura de eventos.
- Poner en uso el audio ya grabado que hoy permanece sin anotar.
- Acortar la latencia entre la recolección y la obtención de resultados ecológicos.

- Disminuir el costo y el error humano asociados al análisis manual.
- Entregar una herramienta reutilizable, con código y pesos públicos, integrable al flujo de trabajo existente en Raven.

A.1.3. Viabilidad técnica

El proyecto no depende de ningún desarrollo previo inexistente. Las arquitecturas necesarias están disponibles y documentadas (K. He et al., 2016; Kahl et al., 2021; Nolasco et al., 2023); las herramientas son software libre de uso extendido (Tabla 1.10); los recursos de cómputo se cubren con acceso a GPU en la nube; los datos iniciales, aproximadamente 10 GB de grabaciones WAV capturadas con AudioMoth, con sus anotaciones de Raven, ya están entregados; y la metodología de trabajo (CRISP-DM) está establecida y es la que estructura el documento.

A.1.4. Viabilidad temporal

El cronograma es factible dentro del calendario académico, con dos puntos de tensión identificados. El primero es OE1: la curación y estandarización del conjunto es meticulosa y cualquier retraso ahí se propaga a todo lo que sigue. El segundo es OE2, que consume tiempo tanto de cómputo como de análisis humano en la experimentación. Juega a favor que no haya recolección de datos en campo: el material de partida ya existe.

A.1.5. Viabilidad económica

El grueso del software es de código abierto y sin costo. Los sensores ya están desplegados y su costo no recae sobre este proyecto. El desembolso real se concentra en cómputo en la nube y en servicios de almacenamiento; el resto del presupuesto es una valorización del tiempo dedicado, no un pago efectivo.

A.2. Definición y alcance del proyecto

A.2.1. Alcance

El proyecto desarrolla y evalúa un sistema de pre-anotación asistida que produce cajas tiempo–frecuencia etiquetadas sobre grabaciones de monitoreo acústico pasivo, para las ocho especies de la Tabla 1.1. El sistema debe cumplir tres tareas: localizar el

evento en tiempo y frecuencia, identificar la especie y clasificar el tipo de vocalización. Una parte central del alcance es la caracterización cuantitativa de su comportamiento operativo: el tiempo de revisión que cuesta una hora de audio con y sin la asistencia del sistema, la carga de propuestas que genera y la naturaleza de sus discrepancias. La medición cronometrada y la revisión experta de una muestra de predicciones están dentro del alcance y dependen de la disponibilidad del equipo de investigación, que es el riesgo R3 de la sección A.3.5. El proyecto recorre el ciclo completo de CRISP-DM, desde la preparación de los datos hasta el procesamiento de grabaciones nuevas.

A.2.2. Entregables y criterios de aceptación

Los entregables son los resultados esperados enumerados en la sección 1.5. La Tabla A.1 recoge el criterio de aceptación de cada uno: la condición verificable que permite declararlo terminado.

Los criterios de OE1 y OE2 se comprueban sobre artefactos del proyecto: archivos, código y modelo. Los de RE3.2 y RE3.3 exigen además una sesión con el equipo de investigación, y por eso son los expuestos al riesgo R3.

A.2.3. Exclusiones

- No se desarrolla un sistema de monitoreo en tiempo real desplegable en campo; el alcance es el procesamiento de grabaciones existentes.
- El análisis detallado de sonidos antropogénicos queda fuera del foco primario, más allá de su detección y categorización general.
- El estudio se limita a las ocho especies declaradas.
- No se desarrolla hardware de adquisición.
- No se realiza nueva recolección de datos en campo.
- **No se cubre la distribución ni el mantenimiento del conjunto de datos ni del modelo.** Las grabaciones y las anotaciones primarias pertenecen al equipo de investigación, y publicarlas, licenciarlas o sostenerlas como recurso de terceros son decisiones suyas y no de esta tesis. El repositorio que compromete RE2.5 contiene el código y los artefactos del modelo final; empaquetar, versionar para terceros o dar soporte queda fuera.

Tabla A.1
Criterios de aceptación

Entreg.	Criterio de aceptación
RE1.1	El protocolo define reglas de sintaxis para las etiquetas de especie y tipo de llamada, y especifica el tratamiento de errores, duplicados e inconsistencias de las anotaciones originales.
RE1.2	Todas las anotaciones del conjunto final cumplen las reglas de RE1.1, y se entrega un manifiesto con la asignación de cada archivo de grabación a una partición.
RE1.3	La ficha describe la estructura de directorios, incluye un diccionario de datos por columna y declara transformaciones, sesgos y limitaciones conocidas.
RE2.1	El documento justifica la arquitectura propuesta y la línea base a partir del estado del arte, y fija la configuración e hiperparámetros iniciales de ambas.
RE2.2	El <i>pipeline</i> ejecuta el ciclo completo de entrenamiento, evaluación e inferencia, e incluye instrucciones de instalación y ejecución.
RE2.3	El informe presenta las métricas de los tres ejes para la propuesta y la línea base, discute los resultados y concluye con una recomendación justificada.
RE2.4	Los pesos del modelo se cargan sin error y un script de demostración produce predicciones en el formato de tabla de selección de Raven sobre un clip de prueba.
RE2.5	El repositorio público contiene el código del <i>pipeline</i> , los pesos del modelo final y las instrucciones para reproducir la inferencia.
RE3.1	La metodología, fechada antes de la evaluación, especifica el subconjunto de prueba, el punto de operación elegido en validación y el umbral de IoU que define un verdadero positivo, fijados antes de medir, las métricas, el emparejamiento y el procedimiento de revisión de discrepancias.
RE3.2	Se reportan los minutos por hora de audio en condición manual y asistida sobre material comparable, con el orden de revisión declarado, y el tiempo de procesamiento del modelo por hora de audio.
RE3.3	El informe clasifica la muestra de predicciones de alta confianza en evento no anotado, ruido y error de encuadre, e incluye ejemplos sobre el espectrograma de aciertos, de errores comunes y de detecciones correctas ausentes de la referencia.

A.2.4. Limitaciones y restricciones

Limitaciones. La ausencia de conjuntos etiquetados a gran escala para primates neotropicales obliga a invertir un esfuerzo considerable en OE1, y la calidad de las anotaciones manuales recibidas es subjetiva y variable. El entorno acústico amazónico es denso y presenta con frecuencia baja relación señal-ruido. La variabilidad intraespecífica de las vocalizaciones y su naturaleza graduada dificultan definir categorías discretas, y las llamadas superpuestas o de baja intensidad son especialmente difíciles de modelar. Los sensores de bajo costo añaden variabilidad entre dispositivos. Por último, sin revisión experta las predicciones de alta confianza situadas en regiones sin anotación (tipo D de la Tabla 1.7) no pueden clasificarse de forma concluyente entre eventos omitidos por la referencia y falsos positivos genuinos: esta tesis las caracteriza geométricamente y las reporta como una cantidad acotada, y deja su resolución como trabajo futuro condicionado a la disponibilidad del equipo de investigación.

Restricciones. El desarrollo está sujeto al calendario académico. El proyecto depende del conjunto de datos entregado y de la verificación de su calidad. El acceso a GPU limita la escala de la experimentación, en particular la búsqueda exhaustiva de hiperparámetros. El desarrollo, la implementación y la evaluación recaen sobre el autor, con la supervisión del asesor.

A.3. Planificación del proyecto

A.3.1. Estructura de desglose del trabajo

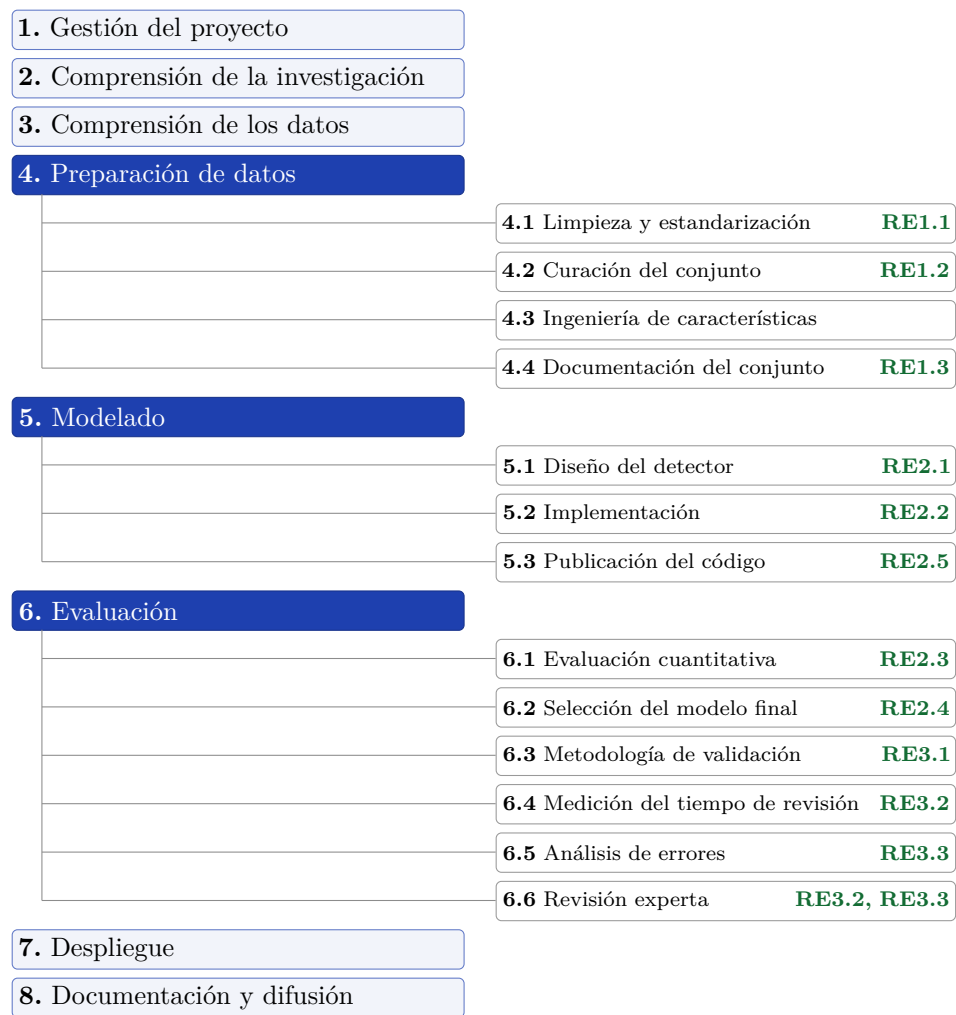
La estructura de desglose del trabajo (EDT) se organiza según las fases de CRISP-DM y los objetivos específicos. La Tabla A.2 resume los paquetes de trabajo y el entregable asociado a cada uno, y la Figura A.1 los dibuja en jerarquía: solo se abren los tres paquetes que producen resultados esperados.

Doce paquetes producen resultados esperados: cada uno de los once resultados tiene el suyo, y la revisión experta (6.6) alimenta además a RE3.2 y RE3.3. Los paquetes sin entregable propio son de soporte: la gestión y la documentación acompañan todo el proyecto, la comprensión de la investigación y de los datos y la ingeniería de características preparan el trabajo de los paquetes que sí entregan, y el despliegue aplica el modelo final a grabaciones nuevas.

Tabla A.2
EDT del proyecto

ID	Paquete	Contenido	Entregable
1	Gestión del proyecto	Planificación, seguimiento, comunicación con el asesor, gestión de riesgos y de la calidad, cierre	N/A
2	Comprensión de la investigación	Objetivos, formulación del problema de detección y criterios de éxito	N/A
3	Comprensión de los datos	Acceso al corpus, descripción del formato, análisis exploratorio y verificación de calidad	N/A
4.1	Limpieza y estandarización	Corrección de etiquetas, tratamiento de audios dañados	RE1.1
4.2	Curación del conjunto	Aplicación del protocolo, vocabulario cerrado, partición por grabación	RE1.2
4.3	Ingeniería de características	Preprocesamiento, representación tiempo-frecuencia, ventaneo, aumento de datos	N/A
4.4	Documentación del conjunto	Ficha del conjunto derivado	RE1.3
5.1	Diseño del detector	Arquitectura propuesta y línea base, con justificación	RE2.1
5.2	Implementación	<i>Pipeline</i> de entrenamiento, evaluación e inferencia	RE2.2
5.3	Publicación del código	Repositorio con código y pesos	RE2.5
6.1	Evaluación cuantitativa	Métricas por eje, comparación con la línea base, análisis de errores	RE2.3
6.2	Selección del modelo final	Modelo elegido y exportador a tabla de selección de Raven	RE2.4
6.3	Metodología de validación	Subconjunto de prueba, punto de operación, emparejamiento, métricas y protocolo de revisión experta	RE3.1
6.4	Medición del tiempo de revisión	Cronometraje de la revisión manual y asistida sobre material comparable, y costo de inferencia	RE3.2
6.5	Análisis de errores	Clasificación de las predicciones de alta confianza y ejemplos sobre el espectrograma	RE3.3
6.6	Revisión experta de la muestra	Resolución con el equipo de investigación de las predicciones sin anotación	RE3.2, RE3.3
7	Despliegue	Inferencia por lotes sobre grabaciones no vistas y generación de tablas de anotación	N/A
8	Documentación y difusión	Redacción de la tesis y preparación de la defensa	N/A

Figura A.1
EDT en árbol



A.3.2. Cronograma

El cronograma se ajusta al calendario del curso, que fija tres entregas: E1, el documento del curso anterior con su avance (24/08/26, semana 2); E2, el entregable parcial (07/10/26, semana 8); y E3, el entregable final (23/11/26, semana 15), al que sigue la exposición final en las semanas 16 y 17. La Tabla A.3 agrupa las tareas según la entrega para la que deben estar terminadas, con el resultado esperado al que contribuyen, sus fechas y su estado a la fecha de E2.

Para E1 debían estar completos los tres resultados de OE1, el *pipeline* (RE2.2) y un primer modelo evaluado: es el trabajo del curso anterior. E2 añade el cierre del preprocesamiento, la justificación de las arquitecturas (RE2.1) y el segundo modelo; E3, el resto de OE2 y todo OE3. A la fecha de E2, las cuatro tareas de OE2 previstas para E3 ya están terminadas (Capítulo 5), y la comparación incorporó una línea base que el

Tabla A.3
Cronograma por entrega

ID	Tarea	Resultado	Inicio	Fin	Estado
E1, 24/08/26 (semana 2): documento del curso anterior con su avance					
0.1	Protocolo de curación y estandarización	RE1.1	Tesis 1		Completado
0.2	Conjunto curado de las ocho especies	RE1.2	Tesis 1		Completado
0.3	División por archivo de grabación, sin fuga entre particiones	RE1.2	Tesis 1		Completado
0.4	Documentación del conjunto y análisis exploratorio	RE1.3	Tesis 1		Completado
0.5	<i>Pipeline</i> de entrenamiento, evaluación e inferencia	RE2.2	Tesis 1		Completado
0.6	Modelo 1 entrenado y evaluado: AST con Deformable DETR	RE2.3	Tesis 1		Completado
E2, 07/10/26 (semana 8): entregable parcial					
1.1	Manejo de eventos largos: ventana dual y solapamiento mínimo relajado	RE1.2	18/08	26/08	Completado
1.2	Aumento de datos: desplazamiento temporal, ganancia, perturbación de cajas y mezcla	RE1.2	26/08	02/09	Completado
1.3	IoMin junto a IoU en la evaluación	RE2.3	02/09	05/09	Completado
1.4	Reentrenamiento del modelo 1 con 1.1 a 1.3	RE2.3	05/09	12/09	Completado
2.1	Selección y justificación de las arquitecturas	RE2.1	12/09	16/09	Completado
2.2	Modelo 2: YOLO26 sobre espectrogramas, detector de una etapa	RE2.3	16/09	28/09	Completado
E3, 23/11/26 (semana 15): entregable final					
2.3	Modelo 3: Faster R-CNN con ResNet, línea base clásica	RE2.3	28/09	10/10	Completado
2.4	Informe comparativo de las arquitecturas	RE2.3	10/10	15/10	Completado
2.5	Selección y ajuste del modelo final	RE2.4	15/10	20/10	Completado
2.6	Publicación del código, los pesos y el exportador a Raven	RE2.5	20/10	22/10	Completado
3.1	Metodología de validación	RE3.1	22/10	25/10	Pendiente
3.2	Medición del tiempo de revisión manual y asistida	RE3.2	02/11	09/11	Pendiente
3.3	Análisis de errores y revisión experta de la muestra	RE3.3	09/11	14/11	Pendiente
4.1	Redacción del informe final	Cierre	14/11	20/11	Pendiente
Exposición final, semanas 16 y 17					
4.2	Preparación de la exposición	Cierre	26/11	30/11	Pendiente
4.3	Exposición final	Cierre	30/11	01/12	Pendiente
4.4	Levantamiento de observaciones y versión final del documento	Cierre	01/12	08/12	Pendiente

plan no preveía, RT-DETR-L. En total, a la fecha de E2 están completas 16 de las 23 tareas (70 %) y 8 de los 11 resultados esperados (73 %); lo pendiente es OE3 y el cierre.

Las tareas respetan las dependencias entre paquetes: la preparación de datos precede al modelado, el modelado precede a la evaluación, y la fijación del punto de operación precede a la medición del tiempo y al análisis de errores, porque ambos se realizan sobre las predicciones del modelo ya seleccionado en ese punto. La sesión de revisión con el equipo de investigación se agenda con holgura, porque alimenta a la vez la condición asistida de RE3.2 y la muestra resuelta de RE3.3.

A.3.3. Estimación de costos

Los costos se presentan en dólares estadounidenses y distinguen desembolso real de valorización: un recurso valorizado tiene precio de mercado, pero el proyecto no lo paga. Se detallan por categoría y al final se consolidan. La Tabla A.4 recoge los equipos.

Tabla A.4
Costos de equipos

Categoría	Descripción	USD
Almacenamiento externo	SSD de 512 GB para el conjunto de datos y sus derivados	150,00
Equipo de desarrollo	Laptop con GPU dedicada ^a	1 500,00
Cómputo en la nube	Instancia con GPU de 48 GiB de memoria, estimada por horas de uso	600,00
Subtotal		2 250,00

^a Equipo preexistente propiedad del tesista. No representa un desembolso del proyecto, pero sí un recurso valorizado.

De los tres equipos, solo el almacenamiento externo y el cómputo en la nube son desembolso; el equipo de desarrollo, que es la partida mayor, ya existía. La Tabla A.5 valoriza el tiempo de las personas.

Tabla A.5
Capital intelectual

Recurso	Horas	USD/h	Total USD
Tesista ^a	600	5,00	3 000,00
Asesor de tesis ^b	40	25,00	1 000,00
Subtotal			4 000,00

^a Valorización del tiempo dedicado; no implica un pago real.

^b Cubierto por la institución.

Es la categoría de mayor peso y ninguna de sus partidas es un pago: el tiempo del tesista se valoriza y el del asesor lo cubre la institución. El software, en la Tabla A.6, es casi todo de código abierto o gratuito.

Tabla A.6
Costos de software y servicios

Software o servicio	Licencia	Meses	USD/mes	Total USD
Python, uv, Pandas, NumPy	Código abierto	12	0,00	0,00
PyTorch, torchaudio, Transformers	Código abierto	12	0,00	0,00
Visual Studio Code	Código abierto	12	0,00	0,00
Zotero	Gratuito	12	0,00	0,00
Raven Lite	Licencia de estudiante	12	0,00	0,00
Entorno de cómputo con GPU	Suscripción	12	9,99	119,88
Almacenamiento en la nube	Suscripción	12	1,67	20,00
Subtotal				139,88

Solo dos servicios se pagan, ambos por suscripción mensual: el entorno de cómputo con GPU y el almacenamiento en la nube. La Tabla A.7 estima los costos indirectos de los doce meses del proyecto.

Tabla A.7
Costos indirectos

Categoría	Descripción	Meses	Total USD
Internet	Conexión para investigación y acceso a la nube	12	480,00
Electricidad	Consumo del equipo y periféricos	12	360,00
Impresiones y otros	Materiales de estudio e impresiones	12	120,00
Subtotal			960,00

La Tabla A.8 consolida las cuatro categorías.

Tabla A.8
Resumen de costos

Categoría	Total USD
Equipamiento (hardware)	2 250,00
Recursos humanos (valorización)	4 000,00
Software y servicios	139,88
Costos indirectos	960,00
Total	7 349,88

Cerca de tres cuartas partes del total son valorización: el tiempo de las personas y el equipo preexistente. El desembolso efectivo se concentra en el cómputo, el almacenamiento y los costos indirectos, como anticipaba la viabilidad económica.

A.3.4. Lista de recursos

Personas. El tesista, responsable del desarrollo, la implementación y el análisis; y el asesor, que aporta guía, supervisión y validación experta.

Equipamiento. Equipo de desarrollo con GPU dedicada, almacenamiento externo para el conjunto de datos y sus derivados, y acceso a servidores con GPU para el entrenamiento.

Herramientas. Las de la Tabla 1.10, más el acceso a Raven para la validación cualitativa de las predicciones.

A.3.5. Gestión de riesgos

La Tabla A.9 recoge los riesgos identificados con su probabilidad, impacto, severidad y las acciones previstas.

Tabla A.9
Riesgos del proyecto

Riesgo	Prob.	Sever.	Mitigación	Contingencia
R1. Calidad de los datos: anotaciones inconsistentes y audios dañados	Baja	Media	Validación cruzada de una muestra con el asesor; detección automática de archivos corruptos o vacíos	Excluir el material bajo umbral mínimo de calidad y documentar la inconsistencia como limitación
R2. Recursos de cómputo insuficientes	Baja	Media	Espectrogramas precalculados; <i>pipeline</i> de datos optimizado; planificación de las corridas largas	Reducir la complejidad de las configuraciones menos prometedoras; acotar la búsqueda de hiperparámetros; parada temprana
R3. Baja disponibilidad de revisores expertos para la validación	Media	Baja	Sesión de revisión agendada con holgura y con el material ya preparado; la muestra se dimensiona para que una sola sesión la resuelva	Reducir el tamaño de la muestra revisada y declarar de forma explícita la limitación de alcance de RE3.2 y RE3.3
R4. Falla del equipo de desarrollo	Alta	Media	Código, datos y documentos sincronizados en repositorio remoto; entorno replicable por archivo de configuración	Reinstalar el sistema, clonar el repositorio y recrear el entorno desde su especificación
R5. Incompatibilidad del entorno de ejecución	Media	Media	Requerimientos explícitos y versionados; contenedor de ejecución; versiones estables de las bibliotecas clave	Proveer un entorno preconfigurado listo para ejecutar; migrar a versiones compatibles
R6. Aumento del costo de cómputo en la nube	Baja	Baja	Desarrollo y depuración en recursos locales o gratuitos; uso eficiente de la GPU; priorización de experimentos	Reducir el alcance de la experimentación; solicitar créditos académicos; usar recursos de la universidad
R7. Retrasos por disponibilidad limitada del tesista	Alta	Alta	Cronograma con holgura basado en la EDT; bloques de tiempo protegidos; seguimiento semanal del avance	Replanificar al detectar la desviación; negociar reducción de alcance con el asesor; priorizar la ruta crítica

Anexo B

Medios de Verificación

Los capítulos de resultados contrastan cada indicador objetivamente verificable de las Tablas 1.4, 1.5 y 1.6 con el texto, las tablas y las figuras del documento. Una parte de la evidencia, sin embargo, no cabe en un documento: los metadatos del conjunto, la configuración de cada entrenamiento, los archivos de métricas o la tabla que produce la inferencia. Este anexo describe la carpeta que la reúne.

La carpeta se entrega junto con este documento, como el archivo `medios_verificacion.zip`, y está publicada en la carpeta `research/medios_verificacion/` del repositorio del proyecto.¹ Se organiza por capítulo y, dentro de cada capítulo, por resultado esperado. La carpeta de cada resultado tiene un `README.md` con tres partes: el medio de verificación y el indicador, copiados literalmente; la sección del documento que desarrolla el resultado; y una tabla que lleva cada elemento del indicador al archivo que lo comprueba. La Tabla B.1 resume el contenido de las once carpetas.

Los ocho resultados de OE1 y OE2 tienen ya su evidencia en la carpeta. Los tres de OE3 la tendrán en el entregable final, como establece el cronograma (Tabla A.3), y su carpeta adelanta qué contendrá. Dos carpetas contienen lo que el documento no puede sustituir: la de RE1.2 da acceso a los metadatos de un conjunto que no se publica, y la de RE1.3 es el `README` que nombra el medio de verificación de la ficha.

Lo que la carpeta no contiene. Las grabaciones, las tablas de Raven y el caché de ventanas. Las dos primeras pertenecen al equipo de investigación, y el caché son espectrogramas de esas mismas grabaciones. En su lugar están sus metadatos y el registro de su carga, que son lo que el indicador de RE1.2 pide comprobar. El *checkpoint*

¹https://github.com/nhrot-fc/tesis-primate/tree/main/research/medios_verificacion

Tabla B.1*Contenido de la carpeta de medios de verificación*

RE	Carpeta	Archivos	Qué comprueban
RE1.1	04_oe1_curacion/ RE1.1	sinonimos.csv, correcciones_pares.csv, vocabulario.csv	Las reglas del protocolo tal como las aplica el código, y cuántas filas toca cada una
RE1.2	04_oe1_curacion/ RE1.2	meta.json, labels.json, listas de grabaciones por partición, registro_de_carga.txt	Que el caché carga sin error, qué registra <code>meta.json</code> y que la partición es por grabación
RE1.3	04_oe1_curacion/ RE1.3	README.md	La ficha completa, ordenada según los nueve elementos del indicador
RE2.1	05_oe2_detector/ RE2.1	configuraciones/	La configuración de las cuatro corridas comparadas
RE2.2	05_oe2_detector/ RE2.2	registros/	Los puntos de entrada del <i>pipeline</i> y el registro de cada entrenamiento
RE2.3	05_oe2_detector/ RE2.3	comparacion_modelos_v3.txt y su .json, punto_de_operacion/ matriz_confusion.png	Las métricas de la comparación y el umbral de cada modelo
RE2.4	05_oe2_detector/ RE2.4	verificacion_carga.txt, SHA256_checkpoint.txt, operating_point.json, tabla de salida y transcripción	Que el <i>checkpoint</i> carga sin claves incompatibles y que la inferencia produce la tabla de Raven
RE2.5	05_oe2_detector/ RE2.5	README.md	El índice del repositorio y de su versión v0.2.0
RE3.1–RE3.3	06_oe3_utilidad	README.md	Lo que contendrá cada carpeta en el entregable final

del modelo final, que no cabe en el repositorio, está en su versión v0.2.0, con su suma SHA-256.

Cómo se genera. Ninguna cifra de la carpeta se transcribe a mano. Un programa del repositorio, `research/medios_verificacion/generar.py`, la construye a partir de los mismos archivos de los que salen las tablas de los Capítulos 4 y 5: las cifras que dejan sus cuadernos, los archivos de las corridas y las constantes del código de curación. Por eso las cifras de la carpeta y las del documento coinciden.